

Secure Cloud Storage: Modularization, Network Adversaries and Adaptive Corruptions

Jonas Janneck¹  and Doreen Riepel² 

¹ Ruhr University Bochum, Germany

`jonas.janneck@rub.de`

² CISA Helmholtz Center for Information Security, Germany

`riepel@cispa.de`

March 3, 2026

Abstract. End-to-end cloud storage solutions are deployed at large scale, yet recent works have demonstrated severe attacks against their confidentiality and integrity. Motivated by this, a first formal treatment of secure cloud storage was given at CRYPTO 2024 by Backendal, Davis, Günther, Haller and Paterson (BDGHP). They define syntax and security notions, capturing client-to-client security of cloud storage schemes with respect to a password distribution. They also give an efficient construction using the Two-Hash Diffie-Hellman (2HDH) OPRF and standard cryptographic building blocks, which they prove secure under selective corruptions in the random oracle model. However, several aspects of practical security guarantees remain open.

We extend and refine the work of BDGHP along multiple dimensions, advancing the analysis of secure cloud storage schemes. First, we prove that their construction can be proven secure against adaptive corruptions (with a slight modification), circumventing technical challenges posed by file sharing. Second, we modularize the scheme further by introducing an abstraction for the authentication procedure. This allows us to identify the concrete role of 2HDH and alternative instantiations. Third, we introduce a weaker model that captures adversaries who can arbitrarily control the network, except during registration. This allows us to prove concrete guarantees about online password guessing attacks, whereas the stronger model inherently allows for offline guessing. Finally, we formalize and prove explicit authentication, relying on the security of our new authentication abstraction and the MAC scheme, where the latter was previously not used in the security analysis.

Table of Contents

1	Introduction	3
1.1	Our Contributions	4
1.2	Technical Overview	5
2	Preliminaries	8
2.1	Notation	8
2.2	AEAD and MAC Schemes	8
2.3	Groups and Diffie-Hellman Problems	9
2.4	Password Guessing Game	10
3	Authentication Protocols	11
3.1	Definitions	11
3.2	Instantiations	13
4	Cloud Storage Schemes	15
4.1	Syntax	15
4.2	Security Definitions	16
5	Construction	20
6	Network Adversary Model	23
6.1	Definitions	23
6.2	Security	25
7	Conclusion and Discussion	27
7.1	Client-to-Client Security	27
7.2	Security against Network Adversaries	28
7.3	Further Discussion and Future Work	30
A	Additional Preliminaries	33
A.1	AEAD Security Notions	33
A.2	Relation for OW-CPA	35
A.3	Security Notion for MAC Schemes	38
B	Omitted Proofs from Section 3	38
B.1	Proof of Theorem 1	38
B.2	Proof of Theorem 2	41
B.3	Proof of Theorem 3	46
C	Theorems and Proofs from Section 5	49
C.1	Confidentiality	49
C.2	Integrity	60
D	Additional Auth Notion and Construction for External Adversaries	63
E	Theorems and Proofs from Section 6	63
E.1	Confidentiality	63
E.2	Integrity	69
E.3	Explicit Authentication	69

1 Introduction

End-to-End Encrypted (E2EE) cloud storage solutions allow a user to have the control over cryptographically protecting their data stored in the cloud. In such solutions, the user typically generates symmetric encryption keys, often derived from a (possibly low-entropy) password, and then encrypts a file before uploading it to the server. Other than meta-data, the server should not learn anything about the content of the file nor compromise its integrity. E2EE cloud storage solutions, such as those from MEGA [MEG], Nextcloud [Nex] and Apple [App], are offered at large-scale, targeting both private and corporate customers. Files should be easily accessible from multiple devices, and a user may also share them in a secure way with other users, which makes these protocols generally highly interactive and the design of secure solutions non-trivial.

ATTACKS AGAINST CLOUD STORAGE SOLUTIONS. A recent line of works [BHP23, AHMP23, HR23, ABCP24, HT24] has shown that indeed many cloud storage solutions are not as secure as the users would probably had hoped for. Even though providers were promising strong privacy, the attacks are critical in that they are practical and affect millions of users. Motivated by this, Backendal, Davis, Günther, Haller and Paterson (BDGHP) [BDG⁺24] initiate the formal study of secure cloud storage solutions from a provable security perspective¹ to better understand the threat model and how to avoid such attacks in the first place. They present a formal security model for E2EE cloud storage that in particular captures the core of these attacks.

THE MODEL AND SCHEME OF [BDG⁺24]. BDGHP are the first to formally define the syntax and expected security guarantees that an E2EE cloud storage scheme should satisfy. They specify sub-protocols that capture registration, authentication, file upload, update and download, as well as file sharing. The main difference to prior work in related areas is that operations are all modeled in a non-atomic way, which allows for arbitrary interleavings of executions of the sub-protocol between multiple users.

Their multi-user security notions are defined using code-based games and parameterized by a password distribution. They capture two flavors of client-to-client (C2C) security:

- **Confidentiality:** An adversary should not learn anything about an honest user’s file unless it is shared with another compromised user.
- **Integrity:** An adversary cannot modify an honest user’s file or create new files for them, as long as they are not shared with another compromised user.

Both properties should hold even if the adversary can control the server, capturing what we intuitively refer to when talking about end-to-end encryption. At the same time, BDGHP also capture malicious users by allowing the adversary to corrupt users. Here, we mainly distinguish two settings:

- **Selective corruptions:** The adversary has to declare the set of users that it will corrupt throughout the game in advance.
- **Adaptive corruptions:** The adversary can decide during the execution and specifically dependent on other interactions which users it will corrupt.

For cloud storage schemes, corrupting a user not only gives the adversary access to their password, but also to their files and temporary states. This complicates security proofs, especially those for adaptive corruptions. In fact, BDGHP (whose contributions focus on many other things) only prove security under selective corruptions. We will reiterate some of the difficulties that arise in the adaptive setting below.

The construction from BDGHP uses prime-order groups, more explicitly, the Two-Hash Diffie-Hellman (2HDH) oblivious pseudorandom function (OPRF) [JKK14], an authenticated encryption scheme with associated data (AEAD), a pseudorandom function (PRF) and a message authentication code (MAC). It achieves confidentiality and integrity under selective corruptions in the random oracle model, relying on standard security properties of the AEAD scheme and the PRF, as well as the advantage of an adversary in a dedicated password guessing game.

¹ For a more detailed comparison to earlier and related work on cloud storage in cryptographic contexts, we refer the reader to the end of Section 1.2 and also [BDG⁺24, Section 1.3], especially Table 1.

On a high-level, the cloud storage scheme works as follows. In the registration and in the authentication procedure, the client computes a raw secret from their password which is then used to derive an encryption key and a MAC key. The encryption key is used to encrypt/decrypt the client’s master key and the MAC key to authenticate to the server during the authentication procedure. On the next layer, the master key is used to encrypt file keys that can be potentially shared between clients and the file keys are eventually used to encrypt the files that are uploaded. The multiple layers are needed to enable client’s key rotation, sharing of files, and updating files without re-encryptions along the complete chain or for every user involved.

SELECTIVE VS. ADAPTIVE CORRUPTIONS. For practical applications, security under adaptive corruptions is desired because it captures adversarial capabilities more accurately. It is the standard goal in many cryptographic contexts, like (password-based) key exchange [BR94, BPR00], and clearly preferred for practical applications, e.g., for threshold signatures [CKM23]. In some cases, however, it is not required to explicitly prove security under adaptive corruptions. The two notions are often asymptotically equivalent for “simple” primitives, e.g., signatures, public-key or secret-key encryption. In two-party interactive protocols, such as authenticated key exchange, security proofs for the multi-user setting often boil down to analyzing different cases, where in each case we can rely on two uncorrupted user secrets (either their long-term or their ephemeral secret key).

Unfortunately, this is not the case for cloud storage schemes. One key aspect why “naive” proof strategies fail lies in the fact that corrupting one user’s state also leaks information about other users’ states. In particular, this becomes clear when considering file sharing. Confidentiality is modeled by having the adversary distinguish which of two files was encrypted. However, a file can be shared with an arbitrary number of users. If we were to guess which set of users remains uncorrupted throughout the game, this would result in an exponential security loss.

ADDITIONAL DESIRABLE SECURITY PROPERTIES. The work of BDGHP models the core properties of E2EE cloud storage. Interestingly, their construction makes use of building blocks that are not required to achieve those core properties. More specifically, the properties of 2HDH and the MAC scheme are never used in the proofs. As stated by the authors, they design the protocol in a way that already captures additional desirable properties which they only state intuitively. One of them is, since we are in the password-based setting, to capture online guessing attacks explicitly. In their model, the server is considered malicious, hence it can always brute-force passwords (offline). Ideally, we would want to capture a property similar to password-based key exchange, namely, that one online interaction is the best an (external) adversary can do to rule out one password. In this case, the (honest) server can limit attacks to a small number of tries. The MAC, on the other hand, can allow the server to explicitly authenticate a session. This feature allows the server to abort earlier for failed attempts but can be also used to notify users about a successful login allowing them to take action in case it was adversarial.

POST-QUANTUM SECURITY. The construction of BDGHP is modular except for the authentication procedure, where 2HDH is used. It seems reasonable to simply replace 2HDH with any other (post-quantum) secure OPRF. Since the proof does not rely on any properties of 2HDH, it is however unclear what exact security properties are required for this replacement to be sound. Additionally, post-quantum secure OPRFs, e.g., those recently studied in [BDFH25, HKL⁺25], are rather cost-intense. If we require a weaker property, we can possibly construct a more efficient scheme.

1.1 Our Contributions

We build upon the work of BDGHP as follows:

- **Security under Adaptive Corruptions:** We prove that their construction achieves security under adaptive corruptions. While we make a slight modification which enables an easier proof, we also argue why our techniques should generally apply to the original construction.
- **Modularization:** To capture the properties of 2HDH, we provide an additional abstraction which we call an authentication protocol (Auth) and security notions. The name comes from the fact that it modularizes

Table 1. Overview of theorems for the cloud storage construction. The table shows which security notions are fulfilled under which assumptions on the underlying building blocks. All notions consider adaptive corruptions. OW-CPA is a new notion, its relation to standard notions is given in Appendix A.1.

Notion	Auth	AEAD	MAC
Theorem 10, Conf (Figure 9)	INS-OW	INT-CTXT + OW-CPA + IND-CCA	—
Theorem 11, Int (Figure 10)	INS-OW	INT-CTXT + OW-CPA	—
Theorem 12, Net-Conf (Figure 13)	OUT-OW	INT-CTXT + OW-CPA + IND-CCA	—
Theorem 13, Net-Int (Figure 14)	OUT-OW	INT-CTXT + OW-CPA	—
Theorem 14, Exp-Auth (Figure 15)	OUT-OW	—	SUF

the authentication procedure. This allows us to identify the exact role of 2HDH, and for which properties simpler constructions are sufficient, providing a path towards post-quantum secure cloud storage.

- **Security against Network Adversaries:** We formalize confidentiality and integrity for active adversaries that control the network arbitrarily, except that we assume secure channels in the registration process. This model is stronger than the notion of client-to-server (C2S) security sketched by BDGHP, but weaker than C2C security. It allows to make a statement about online password guessing and crucially relies on the stronger authentication property, as achieved by 2HDH.
- **Explicit Authentication:** We give a security game that captures explicit (client) authentication which allows the server to detect malicious login attempts early, so it can abort in practice in case of a failed attempt or notify the user in case of a successful attempt. We prove that the property is achieved by the construction of BDGHP, relying on the security of the MAC scheme, which explains its purpose in the design.

Along the way, we make some other minor contributions. On a more conceptual level, we make notational simplifications to the pseudocode description of the model. Finally, we get tighter bounds for the integrity proof.

1.2 Technical Overview

We now want to give a more detailed overview of our main contributions, in particular, about our proof techniques and challenges that we overcome, and the motivation for our new definitions. In Table 1, we provide an overview of the theorems that we prove and which assumptions we rely on.

ADAPTIVE SECURITY. During account registration, the game needs to commit to a ciphertext that encrypts the user’s master key. If the user is later compromised, the ciphertext can be decrypted by the adversary and the master key is revealed. If the user is not compromised and also owner of a challenge file (or a user that the challenge file is shared to), the master key must be replaced by a random key for the remainder of the proof. Therefore, the game must decide in the beginning, i.e., during the call to the account registration oracle, if they replace it by random or not. This is typically referred to as the commitment problem. In the setting with selective corruptions, the game (or the reduction) will have all the relevant information for a correct simulation. The problem arises in the adaptive setting and due to the fact that it is not enough to simply guess one uncorrupted user because of potentially shared files. Therefore, we need to address the problem differently.

Our approach borrows ideas from constructing and proving tight security of authenticated key exchange protocols, e.g., [GJ18, JKRS21]. Those works solve the commitment problem in the context of public-key encryption, whereas here, we are concerned with symmetric-key primitives. The ideas are however very much related when working in idealized models, such as the random oracle model (ROM), cf. also [Nie02, JT20, Jae23]. At this point it is worth to emphasize that the scheme from BDGHP already relies on the ROM. We minimally change the scheme, as we describe below, but we argue that the same proof technique can also work for the original construction.

Intuitively, the encryption scheme that we want to use to encrypt the master key needs to be non-committing. When the encryption happens, the game samples a random ciphertext. Then, later, the game can “explain” how this ciphertext was produced by programming the random oracle accordingly. We can imagine two ways of constructing such an encryption scheme for the cloud storage scheme.

- By extending the message size of the encryption scheme, we can build a non-committing encryption scheme Enc from a scheme Enc' and a random oracle H as follows: We first split the message m into $m' \parallel r$ where m' has the size of messages before the extension and some r some sufficiently large randomness. Then we compute $m^* = m' \oplus H(r)$ and encrypt $m^* \parallel r$ under Enc' . Note that this slightly increases the ciphertext size. The resulting scheme is non-committing because m can be chosen at the moment the random oracle is queried on r . For this to work, r and therefore the original message m must have sufficient entropy. This is satisfied by BDGHP because they encrypt uniformly random keys. Hence, we can instantiate each key encryption in BDGHP by the modified encryption described here and therefore still rely on the original scheme.
- There is a simpler way that also exploits the fact that the encryption operations in the cloud storage scheme do not handle arbitrary messages but mostly uniformly random messages.² Instead of choosing the master key uniformly at random, we choose a random `seed` and compute the master key as $H(\text{seed})$. Then we encrypt `seed` instead of the master key. This approach does not change the ciphertext size but slightly modifies the scheme.

In our analysis, we use the second approach. This way, the game does not need to commit to the exact value of $H(\text{seed})$ at the beginning, as long as `seed` is not queried to the random oracle. We decide against the first approach and instead change the scheme from BDGHP scheme slightly to avoid confusion for potential implementations. The first approach requires the AEAD to be non-committing which is usually not the case in practice³ while the second approach works with an arbitrary (committing) AEAD and explicitly employs the non-committing technique in the scheme.

AUTHENTICATION PROTOCOLS. Instead of directly using 2HDH over prime-order groups, we modularize the construction further. 2HDH is a concrete instantiation of an oblivious pseudorandom function (OPRF). Here, its main purpose is to authenticate the user: If the user holds the correct password, then by evaluating the OPRF, they can compute their raw secret and therefore their master key.

As far as we are aware, one cannot simply plug in “standard” properties of an OPRF⁴, which is why we provide a new abstraction level which we call (two-message) authentication protocols, or short **Auth**. Such protocols are executed between a client and a server, each sending exactly one message. Both client and server hold a secret (whose distributions are part of the parameters of the scheme) and at the end, the client computes a key. In terms of security, we want this key to be hard to compute, which we formalize as a one-wayness property.

We consider two different notions of one-wayness, one against insider and one against outsider adversaries, which we call **INS-OW** and **OUT-OW**, respectively. We rely on **INS-OW** to prove the confidentiality notion of BDGHP. It turns out that a very simple construction using only a random oracle, which we call **SHash**, is already sufficient to achieve this notion⁵. We will however explain the role of 2HDH in more detail below, when talking about network adversaries.

NETWORK ADVERSARIES. In the C2C model from BDGHP, the adversary controls the server. While this is definitely an attack scenario that we want to capture, in practice, we may also be interested in a weaker setting, namely, what kind of security we can still achieve when the adversary (only) controls the network. One property that can be captured this way is impersonation attempts, e.g., the adversary could try to impersonate a client towards the server or vice versa. This allows us to distinguish between online and offline

² This does not hold for the file encryption but this operation does not need to be non-committing.

³ Additionally, there are good reasons to not rely on non-committing AEADs [DGRW18, LGR21]. We do not see how similar attacks good be applied to our second approach.

⁴ Very recently, Friedrichs, Lehmann and Özbay [FL25] have provided a detailed study on game-based security notions for OPRFs. However, none of them seems directly applicable to our setting.

⁵ This is not surprising since BDGHP does not rely on the security of 2HDH at all.

Table 2. Comparison between different cloud storage security models, which Auth notion is required to achieve them, and if for an Auth instantiation password guesses can be made offline (✗) or need an online query per guess (✓). THash refers to a intermediate construction using client-sided and server-sided hashing (see further Appendix D).

Model	Auth notion	Online queries?		
		SHash	THash	2HDH
E2EE (C2C) (defined in [BDG ⁺ 24])	INS-OW	✗	✗	✗
Network Adversary (Section 6.1)	OUT-OW	✗	✗	✓
External Adversary (C2S) (sketch in [BDG ⁺ 24])	Weak-OW	✗	✓	✓

password guessing attacks. Note that in the malicious server model, offline password guessing cannot be avoided.

We formalize such a model, which we call the network adversary model, and define the two properties of confidentiality (Net-Conf) and integrity (Net-Int). We only make one exception: namely, we need to assume that the registration is done via secure channels, similar to what asymmetric password-authenticated key exchange protocols [BM93] require. This seems generally unavoidable for such a model. BDGHP actually provides a sketch for (a variant of) such a model, generally assuming secure channels between clients and server. This is weaker than what we described for our network adversary. Informally, we can separate the different models instantiating the cloud storage scheme with different constructions of our Auth protocol. We give an overview in Table 2 including three possible instantiations for Auth, where Weak-OW is a weaker version of OUT-OW where the adversary does not get access to any client oracles, i.e. can only query the response oracle. It is even simpler to instantiate this notion to achieve online queries per password guessing attempts.

Eventually, we prove security of the cloud storage scheme relying on OUT-OW for Auth. When instantiated with 2HDH, we get indeed better bounds regarding password guessing, additionally assuming a variant of the one-more CDH problem.

EXPLICIT AUTHENTICATION. We additionally formalize a third property in the network adversary setting, namely that of explicit (client) authentication. In the C2C setting, this notion does not make much sense because the server is malicious anyway. However, in combination with the previous property, this allows a server to actively detect online password guessing. This can be very useful in practice because the server can then limit the number of failed login attempts but can also notify clients about successful login attempts. To prove this property for the original cloud storage construction of BDGHP, we additionally require strong unforgeability of the MAC scheme.

ADDITIONAL RELATED AND FUTURE WORK. Earlier works have studied remote file sharing systems [GSMB03], availability [BCQ⁺13] and metadata hiding [CP20, CHGY22] for cloud services, compelled access security [TMRM18] or end-to-end encrypted web applications [PSV⁺14]. Importantly, their focus is more on advanced properties and none of these have looked at non-atomic operations. Useful in the context of secure cloud storage are also more fundamental cryptographic concepts like (oblivious) key management [JKR19, CLTY22, DHL22], generally password-based or password-hardened encryption [LESC17, LER⁺18, BS23] or updatable encryption [BLMR13].

Very recently and probably closest to our work and BDGHP, git services were also analyzed with respect to their practical E2EE [LSTY25]. They tackle additional problems because the effort of changing files should be proportional to the editing difference. Similar to BDGHP, they do not consider adaptive corruptions or stronger properties in weaker models.

For future work, we want to further study the relation of our authentication primitive to OPRFs. Since those are a well-studied cryptographic primitive, achieving post-quantum secure instantiations via OPRFs is the most straight-forward path. Our scheme heavily relies on random oracles, so a natural question is whether one can achieve security in the quantum random oracle model. Here, we believe that techniques for

tighter security for key exchange protocols [PWZ23, Jae26] can be applicable since they need to solve similar commitment problems.

2 Preliminaries

2.1 Notation

SETS AND ALGORITHMS. Unless explicitly stated otherwise, all algorithms are assumed to be probabilistic polynomial time (PPT). For such an algorithm $\mathcal{A}$ invoked on input $(x_1, \dots)$, the notation $(y_1, \dots) \xleftarrow{\$} \mathcal{A}(x_1, \dots)$ denotes that the output tuple $(y_1, \dots)$ is produced under the internal randomness of $\mathcal{A}$. Oracle access is denoted by $\mathcal{A}^O$, signifying that $\mathcal{A}$ may query the oracle O as a subroutine. We write **return** to denote the return instruction of an oracle or internal subroutine, and **output** for the return of the main procedure, e.g., a game or an adversary’s final output. For a probabilistic algorithm $\mathcal{A}$, the notation $x \in \mathcal{A}$ asserts that x belongs to the support of the output distribution of $\mathcal{A}$, i.e., there exists some random tape under which $\mathcal{A}$ outputs x with nonzero probability. If the special symbol $\perp$ is input to an algorithm, we generally define the output of the algorithm to be always $\perp$ as well. Finally, for any Boolean predicate $\mathbb{B}$, we define $\llbracket \mathbb{B} \rrbracket \in \{0, 1\}$ to be the indicator of its truth value: it equals 1 if $\mathbb{B}$ evaluates to true, and 0 otherwise. For a vector $x = (x_1, \dots, x_n)$ we use the function **append** which is defined as $x.\text{append}(y) := (x_1, \dots, x_n, y)$. The min-entropy of a distribution $\mathcal{D}$ is denoted by $H_\infty(\mathcal{D})$. We use $T[x]$ to denote a table entry in table T for key x and assume that empty entries are $\perp$. For a set X , $\mathcal{U}(X)$ denotes the uniform distribution over X .

SECURITY GAMES. We use standard code-based security games [BR06]. A *game* G is a probability experiment in which an adversary $\mathcal{A}$ interacts with an implicit challenger that answers oracle queries issued by $\mathcal{A}$. The game G has one *main procedure* and an arbitrary amount of additional *oracle procedures* which describe how these oracle queries are answered. We denote the (binary) output b of game G between a challenger and an adversary $\mathcal{A}$ as $G(\mathcal{A}) \Rightarrow b$. $\mathcal{A}$ is said to *win* G if $G(\mathcal{A}) \Rightarrow 1$, or shortly $G \Rightarrow 1$. Unless otherwise stated, the randomness in the probability term $\Pr[G(\mathcal{A}) \Rightarrow 1]$ is over all the random coins in game G and adversary $\mathcal{A}$. To provide a cleaner description and avoid repetitions, we sometimes refer to procedures of different games. To call the oracle procedure **ORACLE** of game G on input x , we shortly write $G.\text{ORACLE}(x)$. If a game is aborted, the output is 0. For our analysis we rely on the commonly used main difference lemma [BR06].

In the main body of the paper, we sometimes use informal statements of the form “if Scheme S is secure, then Scheme S' [built from S] is secure”. Such statements are only meaningful when having an explicit security parameter and should be interpreted in that context. What we actually mean is “for any adversary $\mathcal{A}$ against S' , there exists an adversary $\mathcal{B}$ against S such that the advantage of $\mathcal{A}$ is bounded by that of $\mathcal{B}$ ”, specifying a concrete bound B and running time t . For every informal theorem we give such a concrete statement in the appendix.

RANDOM ORACLES. We use the random oracle model [BR93] and let a scheme S specify a set $\mathcal{OS}$ of functions, called the oracle space. Hence, we also need to define primitives with respect to an oracle space. A security game samples a function $H \xleftarrow{\$} \mathcal{OS}$ at random and provides a random oracle RO to the adversary which on input x returns $H(x)$. To support several oracles, the oracle space might be defined as a cartesian product, e.g. $\mathcal{OS} := \mathcal{OS}_1 \times \mathcal{OS}_2$. The adversary gets access to all sub oracles, e.g. RO_1 for $H_1 \xleftarrow{\$} \mathcal{OS}_1$ and RO_2 for $H_2 \xleftarrow{\$} \mathcal{OS}_2$. Since the random oracle is always defined as described here, we do not necessarily define it for each security notion individually. If an algorithm $A(x, \dots)$ has access to random oracle H , we write $A[H](x, \dots)$. In case the oracle space is the empty set, we might ignore it.

2.2 AEAD and MAC Schemes

Definition 1 (AEAD). A (nonce-based) authenticated encryption scheme with associated data AEAD is a tuple (Enc, Dec) along with $kl, nl \in \mathbb{N}$ defined as follows:

$\text{ct} \leftarrow \text{Enc}(k, n, m, \text{ad})$: Given a key $k \in \{0, 1\}^{kl}$, a nonce $n \in \{0, 1\}^{nl}$, a message $m \in \{0, 1\}^*$, and associated data $\text{ad} \in \{0, 1\}^*$, the deterministic encryption algorithm outputs a ciphertext $\text{ct} \in \{0, 1\}^*$.

$m \leftarrow \text{Dec}(k, n, \text{ct}, \text{ad})$: Given a key $k \in \{0, 1\}^{kl}$, a nonce $n \in \{0, 1\}^{nl}$, a ciphertext $\text{ct} \in \{0, 1\}^*$, and associated data $\text{ad} \in \{0, 1\}^*$, the deterministic decryption algorithm outputs a message $m \in \{0, 1\}^{ml}$.

An AEAD is correct if for all $k \in \{0, 1\}^{kl}$, $n \in \{0, 1\}^{nl}$, $m \in \{0, 1\}^*$, $\text{ad} \in \{0, 1\}^*$ it holds that

$$\text{Dec}(k, n, \text{Enc}(k, n, m, \text{ad}), \text{ad}) = m.$$

In Appendix A.1, we define several (multi-user) security notions for AEAD schemes that we will use throughout the paper. We also show their relation to standard notions in Appendix A.2. All the used notions can be reduced from standard notions [JSSOW17].

Definition 2 (MAC). A message authentication code MAC is a tuple (Tag, Ver) along with $kl, tl \in \mathbb{N}$ defined as follows:

$\text{tag} \leftarrow \text{Tag}(k, m)$: Given a key $k \in \{0, 1\}^{kl}$ and a message $m \in \{0, 1\}^*$ the deterministic tag algorithm outputs a tag $\text{tag} \in \{0, 1\}^{tl}$.

$b \leftarrow \text{Ver}(k, m, \text{tag})$: Given a key $k \in \{0, 1\}^{kl}$, a message $m \in \{0, 1\}^*$, and a tag $\text{tag} \in \{0, 1\}^{tl}$ the deterministic verification algorithm outputs a bit $b \in \{0, 1\}$.

A MAC is correct if for all $k \in \{0, 1\}^{kl}$ and $m \in \{0, 1\}^*$ it holds that

$$\text{Ver}(k, m, \text{Tag}(k, m)) = 1.$$

Strong unforgeability for a MAC is defined in Appendix A.3.

2.3 Groups and Diffie-Hellman Problems

Throughout the paper, let $(\mathbb{G}, q, g)$ be the description of a cyclic group of prime order q with generator g . We will use two assumptions to prove security of our group-based authentication scheme which is based on the 2HDH oblivious pseudorandom function. First, we will need (a variant of) the one-more computational Diffie-Hellman (OMCDH) problem, also called chosen-target OMCDH [Bol03]. We adopt this name and define the security game CT-OMCDH in the multi-user setting in Figure 1.

Definition 3 (CT-OMCDH). The chosen-target one-more Diffie-Hellman problem is defined via the game in Figure 1. We define the advantage of an adversary $\mathcal{A}$ as

$$\text{Adv}_{\mathbb{G}, q, g}^{Q\text{-CT-OMCDH}}(\mathcal{A}) := \Pr[Q\text{-CT-OMCDH}_{\mathbb{G}, q, g}(\mathcal{A}) \Rightarrow 1]$$

with $Q = (n, k, Q_{\text{COMP}}, Q_{\text{EXP}}, Q_{\text{CHECK}})$.

Remark 1 (Comparison to existing definitions). Compared to the more standard definition of a one-more assumption, e.g., the one analyzed in the generic group model by [BFP21], where the adversary gets an oracle that outputs the CDH solution to arbitrary group elements as input, the adversary in our game is more restricted. In particular, the EXP oracle is only defined with respect to exponents chosen by the game, making it falsifiable. On the other hand, our definition is multi-user, making it seemingly stronger. We note that our definition is similar to the original definition of chosen-target CDH in [Bol03] and also the one used in the characterization in [JX26], where it is simply called OMDH. Their definitions are captured as $(1, k, 0, Q_{\text{EXP}}, 0)$ -CT-OMCDH, where there is only a single user (and therefore no COMP oracle) and no CHECK oracle, where the latter acts as a DDH oracle in our setting, the same way it is also used in the strong CDH assumption [ABR01]. Additionally, by guessing the index i^* , we can show that the single-user variant (i.e., where $(n, Q_{\text{COMP}}) = (1, 0)$) non-tightly implies the multi-user one, therefore the main difference to previous assumption lies in the additional CHECK oracle, which typically makes no difference in the algebraic group model, see [BFL20].

Game $Q\text{-CT-OMCDH}_{\mathbb{G},q,g}(\mathcal{A})$	Oracle $\text{EXP}(i \in [n], \alpha)$
00 $\mathcal{S}_{\text{cmp}} \leftarrow \emptyset$	14 ctr_i++
01 for $i \in [n]$	15 return α^{v_i}
02 $v_i \xleftarrow{\$} \mathbb{Z}_q$	
03 $V_i \leftarrow g^{v_i}$	Oracle $\text{CHECK}(i \in [n], \alpha, Z)$
04 $\text{ctr}_i \leftarrow 0$	16 return $\llbracket \alpha^{v_i} = Z \rrbracket$
05 for $j \in [k]$	
06 $u_j \xleftarrow{\$} \mathbb{Z}_q$	Oracle $\text{COMP}(i \in [n])$
07 $U_j \leftarrow g^{u_j}$	17 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$
08 $(i^*, \mathcal{Z}) \xleftarrow{\$} \mathcal{A}^{\text{EXP,CHECK}}(V_1, \dots, V_n, U_1, \dots, U_k)$	18 return v_{i^*}
09 if $ \mathcal{Z} \leq \text{ctr}_{i^*} \vee i^* \in \mathcal{S}_{\text{cmp}}$ return 0	
10 for $(U, Z) \in \mathcal{Z}$	
11 if $\nexists j : U = U_j$ return 0	
12 if $Z \neq U^{v_{i^*}}$ return 0	
13 return 1	

Fig. 1. Game defining the CT-OMCDH problem for a group $(\mathbb{G}, p, g)$ with $Q = (n, k, Q_{\text{COMP}}, Q_{\text{EXP}}, Q_{\text{CHECK}})$ and adversary $\mathcal{A}$ making at most Q_{O} queries to oracle O .

Game $Q\text{-CB-InvDH}_{\mathbb{G},q,g}(\mathcal{A})$	Oracle $\text{CHECK}(i \in [n], j \in [k], \beta, Z)$
00 for $i \in [n]$	08 return $\llbracket \beta^{u_j/v_i} = Z \rrbracket$
01 $v_i \xleftarrow{\$} \mathbb{Z}_q$	
02 $V_i \leftarrow g^{v_i}$	
03 for $j \in [k]$	
04 $u_j \xleftarrow{\$} \mathbb{Z}_q$	
05 $U_j \leftarrow g^{u_j}$	
06 $(i^*, j^*, \beta, Z) \xleftarrow{\$} \mathcal{A}^{\text{CHECK}}(V_1, \dots, V_n, U_1, \dots, U_k)$	
07 return $\llbracket \beta^{u_{j^*}/v_{i^*}} = Z \rrbracket$	

Fig. 2. Game defining the CB-InvDH problem for a group $(\mathbb{G}, p, g)$ with $Q = (n, k, Q_{\text{CHECK}})$ and adversary $\mathcal{A}$ making at most Q_{CHECK} queries to oracle CHECK .

The second problem that we will need is what we call chosen-basis inverse Diffie-Hellman CB-InvDH. We again define a multi-user variant of the problem, as shown in Figure 2. The game is non-interactive, except for a CHECK oracle, similar to the previous game.

Definition 4 (CB-InvDH). *The chosen-basis inverse Diffie-Hellman problem is defined via the game in Figure 2. We define the advantage of an adversary $\mathcal{A}$ as*

$$\text{Adv}_{\mathbb{G},q,g}^{Q\text{-CB-InvDH}}(\mathcal{A}) := \Pr[Q\text{-CB-InvDH}_{\mathbb{G},q,g}(\mathcal{A}) \Rightarrow 1]$$

with $Q = (n, k, Q_{\text{CHECK}})$.

Remark 2. We state the problem in the multi-user setting which is better suited for our use case. Since there are no corruptions, we can show that $(1, 1, Q_{\text{CHECK}})\text{-CB-InvDH}$ tightly implies $(n, k, Q_{\text{CHECK}})\text{-CB-InvDH}$, where the only additional overhead is rerandomizing the challenges via exponentiation and removing those randomizers for CHECK queries and the final solution. We are not aware of other applications of the exact same assumption with the chosen basis, however, it is conceptually similar to the simultaneous Diffie-Hellman assumption [PW17], which has been analyzed in the GGM in the same work.

2.4 Password Guessing Game

In the application of the cloud storage scheme, the client secret distribution will be a password distribution and the security should depend on the advantage of an adversary guessing passwords correctly. We capture this, similar to [BDG⁺24], via the game in Figure 3, except that we use the variant where the adversary can corrupt passwords adaptively.

Definition 5 (Password Guessing). Password guessing for some distribution $\mathcal{PW}$ is defined via the game in Figure 3. We define the advantage of an adversary $\mathcal{A}$ against distribution $\mathcal{PW}$ as

$$\text{Adv}_{\mathcal{PW}}^{Q\text{-PW-Guess}}(\mathcal{A}) := \Pr[Q\text{-PW-Guess}_{\mathcal{PW}}(\mathcal{A}) \Rightarrow 1]$$

with $Q = (n, Q_{\text{TEST}}, Q_{\text{COMP}})$.

Game $Q\text{-PW-Guess}_{\mathcal{PW}}(\mathcal{A})$	Oracle $\text{TEST}(i \in [n], \text{pw})$
00 $\mathcal{S}_{\text{cmp}} \leftarrow \emptyset$	05 if $\text{pw} = \text{pw}_i \wedge i \notin \mathcal{S}_{\text{cmp}}$
01 win $\leftarrow$ false	06 win $\leftarrow$ true
02 $(\text{pw}_1, \dots, \text{pw}_n) \xleftarrow{\$} \mathcal{PW}$	07 return $\llbracket \text{pw} = \text{pw}_i \rrbracket$
03 $\mathcal{A}^{\text{TEST, COMP}}$	
04 return win	
	Oracle COMP ($i \in [n]$)
	08 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$
	09 return pw_i

Fig. 3. Game defining password guessing for a distribution $\mathcal{PW}$ with $Q = (n, Q_{\text{TEST}}, Q_{\text{COMP}})$ and adversary $\mathcal{A}$ making at most Q_{O} queries to oracle $\mathcal{O}$.

3 Authentication Protocols

In this section, we define authentication protocols as an abstract building block to construct cloud storage schemes. This allows us to take a modular approach analyzing different cloud storage security models and their implications for the authentication primitive. We also present three instantiations that can be chosen to plug into the generic cloud storage scheme depending on the security model one wants to achieve.

3.1 Definitions

Definition 6 (Authentication Protocol). A two-message authentication protocol Auth is a tuple $(\mathcal{OS}, \mathcal{D}_S, Y, \text{Init}, \text{Resp}, \text{Fin}, \text{Full})$ where $\mathcal{OS}$ is the oracle space, $\mathcal{D}_S$ a distribution, Y (the output space) is a set. For $H \in \mathcal{OS}$, the remaining algorithms are defined as follows:

$(\text{auth}_1, \text{st}) \xleftarrow{\$} \text{Init}[H](x_C, \text{aux})$: Given a client secret $x_C \in \{0, 1\}^*$ and some auxiliary information $\text{aux} \in \{0, 1\}^*$, the initialization algorithm outputs a first authentication message auth_1 and a state st .
 $\text{auth}_2 \leftarrow \text{Resp}[H](\text{auth}_1, x_S)$: Given a first authentication message auth_1 and a server's secret $x_S \in \text{sup}(\mathcal{D}_S)$, the deterministic⁶ response algorithm outputs a second authentication message.
 $y \leftarrow \text{Fin}[H](\text{auth}_2, x_C, \text{st})$: Given a second authentication message auth_2 , a client secret $x_C \in \{0, 1\}^*$, and a state st , the deterministic finalization algorithm outputs some $y \in Y$.
 $y \leftarrow \text{Full}[H](x_C, \text{aux}, x_S)$: Given a client secret $x_C \in \{0, 1\}^*$, auxiliary information $\text{aux} \in \{0, 1\}^*$, and a server secret $x_S \in \text{sup}(\mathcal{D}_S)$, the deterministic full execution algorithm outputs some $y \in Y$.

Auth has correctness error

$$\delta_{\text{Auth}} := \max \Pr \left[\text{Full}[H](x_C, \text{aux}, x_S) \neq y \mid \begin{array}{l} (\text{auth}_1, \text{st}) \xleftarrow{\$} \text{Init}[H](x_C, \text{aux}) \\ \text{auth}_2 \leftarrow \text{Resp}[H](\text{auth}_1, x_S) \\ y \leftarrow \text{Fin}[H](\text{auth}_2, x_C, \text{st}) \end{array} \right],$$

where the maximum is taken over $H \in \mathcal{OS}$, $x_C \in \{0, 1\}^*$, $x_S \in \text{sup}(\mathcal{D}_S)$ and $\text{aux} \in \{0, 1\}^*$.

⁶ For our instantiations, it is sufficient to have deterministic response algorithms. However, one could also generalize this to probabilistic algorithms.

Game $Q\text{-INS-OW}_{\text{Auth}, \mathcal{D}_C}(\mathcal{A})$ <pre> 00 $H \xleftarrow{\\$} \mathcal{OS}$ 01 $\mathcal{S}_{\text{cmp}} \leftarrow \emptyset$ 02 $n_r \leftarrow 0$ 03 $(x_{C,1}, \dots, x_{C,Q_{\text{REG}}}) \xleftarrow{\\$} \mathcal{D}_C$ 04 for $i \in [Q_{\text{REG}}]$ 05 $x_{S,i} \xleftarrow{\\$} \mathcal{D}_S$ 06 $(y^*, \text{aux}^*) \xleftarrow{\\$} \mathcal{A}^{\text{REG,INIT,FIN,COMP,CHECK,RO}}$ 07 if $T_{\text{aux}}[\text{aux}^*] = \perp$ return 0 08 $i^* \leftarrow T_{\text{aux}}[\text{aux}^*]$ 09 return $\llbracket y^* = \text{Full}[H](x_{C,i^*}, \text{aux}^*, x_{S,i^*}) \wedge i^* \notin \mathcal{S}_{\text{cmp}} \rrbracket$ </pre> Oracle $\text{REG}(\text{aux})$ <pre> 10 require $T_{\text{aux}}[\text{aux}] = \perp$ $\llbracket \text{enforce unique aux} \rrbracket$ 11 n_r++ 12 $T_{\text{aux}}[\text{aux}] \leftarrow n_r$ 13 return x_{S,n_r} </pre> Oracle $\text{INIT}(\text{aux}, j)$ <pre> 14 require $T_{\text{aux}}[\text{aux}] \neq \perp$ 15 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 16 $(\text{auth}_1, \text{st}) \xleftarrow{\\$} \text{Init}[H](x_{C,i}, \text{aux})$ 17 $T[i, j] \leftarrow (\text{st}, \text{aux})$ 18 return auth_1 </pre>	Oracle $\text{FIN}(\text{auth}_2, i \in [n_r], j)$ <pre> 19 if $T[i, j] = \perp$ 20 return $\perp$ 21 $(\text{st}, \text{aux}) \leftarrow T[i, j]$ 22 $T[i, j] \leftarrow \perp$ $\llbracket \text{no state reuse} \rrbracket$ 23 $y \leftarrow \text{Fin}[H](\text{auth}_2, x_{C,i}, \text{st})$ 24 if $y = \text{Full}[H](x_{C,i}, \text{aux}, x_{S,i})$ 25 return $\top$ 26 return y </pre> Oracle $\text{COMP}(i \in [n_r])$ <pre> 27 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 28 return $x_{C,i}$ </pre> Oracle $\text{CHECK}(y, \text{aux})$ <pre> 29 if $T_{\text{aux}}[\text{aux}] = \perp$ return 0 30 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 31 return $\llbracket y = \text{Full}[H](x_{C,i}, \text{aux}, x_{S,i}) \rrbracket$ </pre> Oracle $\text{RO}(x)$ <pre> 32 return $H(x)$ </pre>
--	--

Fig. 4. Game defining INS-OW security for a two-message authentication scheme $\text{Auth} := (\mathcal{OS}, \mathcal{D}_S, Y, \text{Init}, \text{Resp}, \text{Fin}, \text{Full})$ with $Q = (Q_{\text{REG}}, Q_{\text{COMP}}, Q_{\text{INIT}}, Q_{\text{FIN}}, Q_{\text{CHECK}}, Q_{\text{RO}})$ and adversary $\mathcal{A}$ making at most Q_O queries to oracle O . Differences to OUT-OW (Figure 5) are highlighted in blue.

We define two one-wayness security notions for the Auth primitive. It should be hard to recover the correct output of an Auth session between a client and a server but it should be easy to recognize it (CHECK oracle). This property should be fulfilled under adaptive corruptions. In the stronger INS-OW notion, the adversary is an insider attacker having access to all the server secrets while in the OUT-OW they do not have any additional information. It is sufficient for the application to require security for unique auxiliary information per user since this will be the user identifier.

Definition 7 (One-Wayness). *Output one-wayness is defined via the game in Figure 4 for insider adversaries and in Figure 5 for outsider adversaries. We define the advantage of an adversary $\mathcal{A}$ and a client's secrets distribution $\mathcal{D}_C$ as*

$$\text{Adv}_{\text{Auth}, \mathcal{D}_C}^{Q\text{-INS-OW}}(\mathcal{A}) := \Pr [Q\text{-INS-OW}_{\text{Auth}, \mathcal{D}_C}(\mathcal{A}) \Rightarrow 1]$$

with $Q = (Q_{\text{REG}}, Q_{\text{COMP}}, Q_{\text{INIT}}, Q_{\text{FIN}}, Q_{\text{CHECK}}, Q_{\text{RO}})$ and

$$\text{Adv}_{\text{Auth}, \mathcal{D}_C}^{Q\text{-OUT-OW}}(\mathcal{A}) := \Pr [Q\text{-OUT-OW}_{\text{Auth}, \mathcal{D}_C}(\mathcal{A}) \Rightarrow 1]$$

with $Q = (Q_{\text{REG}}, Q_{\text{COMP}}, Q_{\text{INIT}}, Q_{\text{RESP}}, Q_{\text{FIN}}, Q_{\text{CHECK}}, Q_{\text{RO}})$.

Remark 3 (Relation to OPRFs). There is an obvious syntactic similarity to oblivious pseudorandom functions (OPRFs). However, a formal proof that relates the two primitives requires more care. In this work, we are primarily concerned with the exact requirements for the application and its (tight) security proof, which is why we consider (one-way) multi-user definitions with corruptions. We are not aware that such notions exist for OPRFs. Therefore, we think that identifying the exact relation is an important open question, especially for post-quantum secure instantiations.

Game $Q\text{-OUT-OW}_{\text{Auth}, \mathcal{D}_C}(\mathcal{A})$ 00 $H \xleftarrow{\$} \mathcal{OS}$ 01 $\mathcal{S}_{\text{cmp}} \leftarrow \emptyset$ 02 $n_r \leftarrow 0$ 03 $(x_{C,1}, \dots, x_{C,Q_{\text{REG}}}) \xleftarrow{\$} \mathcal{D}_C$ 04 for $i \in [Q_{\text{REG}}]$ 05 $x_{S,i} \xleftarrow{\$} \mathcal{D}_S$ 06 $(y^*, \text{aux}^*) \xleftarrow{\$} \mathcal{A}^{\text{REG,INIT,RESP,FIN,COMP,CHECK,RO}}$ 07 if $T_{\text{aux}}[\text{aux}^*] = \perp$ return 0 08 $i^* \leftarrow T_{\text{aux}}[\text{aux}^*]$ 09 return $\llbracket y^* = \text{Full}[H](x_{C,i^*}, \text{aux}^*, x_{S,i^*}) \wedge i^* \notin \mathcal{S}_{\text{cmp}} \rrbracket$ Oracle $\text{REG}(\text{aux})$ 10 require $T_{\text{aux}}[\text{aux}] = \perp$ $\llbracket \text{enforce unique aux} \rrbracket$ 11 n_r++ 12 $T_{\text{aux}}[\text{aux}] \leftarrow n_r$ Oracle $\text{INIT}(\text{aux}, j)$ 13 require $T_{\text{aux}}[\text{aux}] \neq \perp$ 14 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 15 $(\text{auth}_1, \text{st}) \xleftarrow{\$} \text{Init}[H](x_{C,i}, \text{aux})$ 16 $T[i, j] \leftarrow (\text{st}, \text{aux})$ 17 return auth_1 Oracle $\text{RESP}(\text{auth}_1, i \in [n_r])$ 18 $\text{auth}_2 \leftarrow \text{Resp}[H](\text{auth}_1, x_{S,i})$ 19 return auth_2	Oracle $\text{FIN}(\text{auth}_2, i \in [n_r], j)$ 20 if $T[i, j] = \perp$ 21 return $\perp$ 22 $(\text{st}, \text{aux}) \leftarrow T[i, j]$ 23 $T[i, j] \leftarrow \perp$ $\llbracket \text{no state reuse} \rrbracket$ 24 $y \leftarrow \text{Fin}[H](\text{auth}_2, x_{C,i}, \text{st})$ 25 if $y = \text{Full}[H](x_{C,i}, \text{aux}, x_{S,i})$ 26 return $\top$ 27 return y Oracle $\text{COMP}(i \in [n_r])$ 28 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 29 return $(x_{C,i}, \text{aux}_i)$ Oracle $\text{CHECK}(y, \text{aux})$ 30 if $T_{\text{aux}}[\text{aux}] = \perp$ return 0 31 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 32 return $\llbracket y = \text{Full}[H](x_{C,i}, \text{aux}, x_{S,i}) \rrbracket$ Oracle $\text{RO}(x)$ 33 return $H(x)$
--	---

Fig. 5. Game defining OUT-OW security for a two-message authentication scheme $\text{Auth} := (\mathcal{OS}, \mathcal{D}_S, Y, \text{Init}, \text{Resp}, \text{Fin}, \text{Full})$ with $Q = (Q_{\text{REG}}, Q_{\text{COMP}}, Q_{\text{INIT}}, Q_{\text{RESP}}, Q_{\text{FIN}}, Q_{\text{CHECK}}, Q_{\text{RO}})$ and adversary $\mathcal{A}$ making at most Q_0 queries to oracle O. Differences to INS-OW (Figure 4) are highlighted in blue.

3.2 Instantiations

We provide two instantiations here, and a third one for demonstrative purposes (cf. Table 2) in Appendix D. 2HDH. This construction is parametrized by a group $\mathbb{G}$ of prime order q and generator g , and a output size kl . We then define

$$2\text{HDH}[\mathbb{G}, q, g, \text{kl}] := (\mathcal{OS}, \mathcal{U}(\mathbb{Z}_q), \{0, 1\}^{\text{kl}}, \text{Init}, \text{Resp}, \text{Fin}, \text{Full}),$$

for oracle space $\mathcal{OS} := \{\{0, 1\}^* \rightarrow \mathbb{G}\} \times \{\{0, 1\}^* \rightarrow \{0, 1\}^{\text{kl}}\}$ and the remaining algorithms as defined in Figure 6.

Lemma 1 (Correctness). *It holds that $\delta_{2\text{HDH}[\mathbb{G}, q, g, \text{kl}]} = 0$.*

Theorem 1 (INS-OW of 2HDH). *For any distribution $\mathcal{D}_C$ and adversary $\mathcal{A}$ against INS-OW of $2\text{HDH} := 2\text{HDH}[\mathbb{G}, q, g, \text{kl}]$ there exists an PW-Guess adversary $\mathcal{B}$ against $\mathcal{D}_C$ with $t_{\mathcal{A}} \approx t_{\mathcal{B}}$ such that*

$$\text{Adv}_{2\text{HDH}, \mathcal{D}_C}^{Q\text{-INS-OW}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{D}_C}^{(Q_{\text{REG}}, Q_{\text{RO}}, Q_{\text{COMP}})\text{-PW-Guess}}(\mathcal{B}) + \frac{Q_{\text{CHECK}} + 1}{2^{\text{kl}} - Q_{\text{CHECK}}}$$

with $Q = (Q_{\text{REG}}, Q_{\text{COMP}}, Q_{\text{INIT}}, Q_{\text{FIN}}, Q_{\text{CHECK}}, Q_{\text{RO}})$.

The proof is given in Appendix B.1.

2HDH.Init (x_C, aux) 00 $r \xleftarrow{\$} \mathbb{Z}_q$ 01 $\alpha \leftarrow H_1(x_C, \text{aux})^r$ 02 return ($\alpha, (r, \text{aux})$)	2HDH.Resp (auth_1, x_S) 07 if $\text{auth}_1 \notin \mathbb{G}$ 08 return $\perp$ 09 $\beta \leftarrow \text{auth}_1^{x_S}$ 10 return β
2HDH.Fin ($\text{auth}_2, x_C, (r, \text{aux})$) 03 if $\text{auth}_2 \notin \mathbb{G}$ return $\perp$ 04 $\gamma \leftarrow \text{auth}_2^{1/r}$ 05 $y \leftarrow H_2(x_C, \text{aux}, \gamma)$ 06 return y	2HDH.Full (x_C, aux, x_S) 11 $y \leftarrow H_2(x_C, \text{aux}, H_1(x_C, \text{aux})^{x_S})$ 12 return y
SHash.Init (x_C, aux) 13 $\text{auth}_1 \leftarrow \text{aux}$ 14 return ($\text{auth}_1, \varepsilon$)	SHash.Resp (auth_1, x_S) 17 return x_S
SHash.Fin ($\text{auth}_2, x_C, \text{st}$) 15 $y \leftarrow H(x_C, \text{aux}, \text{auth}_2)$ 16 return y	SHash.Full (x_C, aux, x_S) 18 $y \leftarrow H(x_C, \text{aux}, x_S)$ 19 return y

Fig. 6. Top: Diffie-Hellman based authentication scheme $2\text{HDH} := (\mathcal{OS}, \mathcal{U}(\mathbb{Z}_q), \{0, 1\}^{\text{kl}}, \text{Init}, \text{Resp}, \text{Fin}, \text{Full})$. **Bottom:** Hash-based scheme $\text{SHash} := (\mathcal{OS}, \mathcal{D}_S, \{0, 1\}^{\text{kl}}, \text{Init}, \text{Resp}, \text{Fin}, \text{Full})$, where $\mathcal{D}_S$ is an arbitrary distribution.

Theorem 2 (OUT-OW of 2HDH). *For any distribution $\mathcal{D}_C$ and adversary $\mathcal{A}$ against OUT-OW of $2\text{HDH} := 2\text{HDH}[\mathbb{G}, q, g, \text{kl}]$, there exist a CT-OMCDH adversary $\mathcal{B}$ against $(\mathbb{G}, q, g)$, a CB-InvDH adversary $\mathcal{C}$ against $(\mathbb{G}, q, g)$ and a PW-Guess adversary $\mathcal{D}$ against $\mathcal{D}_C$ with $t_{\mathcal{A}} \approx t_{\mathcal{B}} \approx t_{\mathcal{C}} - Q_{\text{FIN}} Q_{\text{RO}} \approx t_{\mathcal{D}}$ such that*

$$\begin{aligned} \text{Adv}_{2\text{HDH}, \mathcal{D}_C}^{Q\text{-OUT-OW}}(\mathcal{A}) &\leq \text{Adv}_{\mathbb{G}, q, g}^{(Q_{\text{REG}}, Q_{\text{RO}}, Q_{\text{COMP}}, Q_{\text{RESP}}, Q_{\text{RO}} + Q_{\text{CHECK}})\text{-CT-OMCDH}}(\mathcal{B}) + \text{Adv}_{\mathbb{G}, q, g}^{(Q_{\text{INIT}}, Q_{\text{RO}}, 2Q_{\text{FIN}} Q_{\text{RO}})\text{-CB-InvDH}}(\mathcal{C}) \\ &\quad + \text{Adv}_{\mathcal{D}_C}^{(Q_{\text{REG}}, Q_{\text{RESP}}, Q_{\text{COMP}})\text{-PW-Guess}}(\mathcal{D}) + \frac{Q_{\text{CHECK}} + 1}{2^{\text{kl}} - Q_{\text{CHECK}}} + \frac{Q_{\text{RO}}^2}{2^{\text{kl}}} \end{aligned}$$

with $Q = (Q_{\text{REG}}, Q_{\text{COMP}}, Q_{\text{INIT}}, Q_{\text{RESP}}, Q_{\text{FIN}}, Q_{\text{CHECK}}, Q_{\text{RO}})$.

The proof is given in Appendix B.2. We note that for outsider security, we still expect that the adversary can simply guess the client secret, as reflected in the password guessing advantage of the bound. However, we can now limit the number of guesses to online interactions, i.e. RESP queries to the server. For each query, the adversary can check whether they guessed correctly. The CT-OMCDH assumption then ensures that no further guess can be made, and the CB-InvDH assumption ensures that interactions with the client itself cannot help for additional guesses.

SIMPLE HASH. This construction is parametrized by a distribution $\mathcal{D}_S$ and an output size kl . We then define

$$\text{SHash}[\mathcal{D}_S, \text{kl}] := (\mathcal{OS}, \mathcal{D}_S, \{0, 1\}^{\text{kl}}, \text{Init}, \text{Resp}, \text{Fin}, \text{Full}),$$

for oracle space $\mathcal{OS} := \{\{0, 1\}^* \rightarrow \{0, 1\}^{\text{kl}}\}$ and the remaining algorithms as defined in Figure 6.

Lemma 2 (Correctness). *It holds that $\delta_{\text{SHash}[\mathcal{D}_S, \text{kl}]} = 0$.*

Theorem 3 (INS-OW of SHash). *For any distribution $\mathcal{D}_C$ and adversary $\mathcal{A}$ against INS-OW of $\text{SHash} := \text{SHash}[\mathcal{D}_S, \text{kl}]$, there exists a PW-Guess adversary $\mathcal{B}$ against $\mathcal{D}_C$ with $t_{\mathcal{A}} \approx t_{\mathcal{B}}$ such that*

$$\text{Adv}_{\text{SHash}, \mathcal{D}_C}^{Q\text{-INS-OW}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{D}_C}^{(Q_{\text{REG}}, Q_{\text{RO}}, Q_{\text{COMP}})\text{-PW-Guess}}(\mathcal{B}) + \frac{Q_{\text{CHECK}} + 1}{2^{\text{kl}} - Q_{\text{CHECK}}}$$

with $Q = (Q_{\text{REG}}, Q_{\text{COMP}}, Q_{\text{INIT}}, Q_{\text{FIN}}, Q_{\text{CHECK}}, Q_{\text{RO}})$.

The proof is given in Appendix B.3.

4 Cloud Storage Schemes

In this section we define a cloud storage according to [BDG⁺24]. The notation is mainly taken from the same work but we introduce small conceptual changes in the security notion to clarify the model.

4.1 Syntax

The definitions in this section are taken from [BDG⁺24].

CLOUD STORAGE SCHEME. A cloud storage scheme $\mathcal{CS}$ is a tuple of an oracle space $\mathcal{OS}$ and seven two-party protocols:

$$\mathcal{CS} = (\mathcal{OS}, \Pi_{\text{areg}}, \Pi_{\text{auth}}, \Pi_{\text{put}}, \Pi_{\text{upd}}, \Pi_{\text{get}}, \Pi_{\text{share}}, \Pi_{\text{accept}}) .$$

Table 3. Input-output semantics of the subprotocols in the cloud storage system $\mathcal{CS}$ from a client’s perspective.

Protocol	Input	Output	Description
Π_{areg}	aid, pw	ε	Account registration
Π_{auth}	aid, pw	ε	Authentication and start of a new session
Π_{put}	fid, f	ε	File upload
Π_{upd}	fid, f	ε	File update
Π_{get}	fid	f	File download
Π_{share}	fid, raid	oob	Out-of-band file sharing
Π_{accept}	fid, oob	ε	Accept shared file

PROTOCOL EXECUTION. The execution of a two-party protocol Π proceeds in $\Pi.SN \in \mathbb{N}$ sequential steps, alternating between the client and the server. Each step r is denoted as $\Pi^{C:r}$ or $\Pi^{S:r}$ depending on the client (C) or the server (S) executing it. The syntax for a protocol step is

$$(\text{st}_P, \text{st}_P^{\text{tmp}}, \text{out}_P, \text{m}_{\text{out}}) \leftarrow \Pi^{P:r}[\mathcal{H}](\text{st}_P, \text{st}_P^{\text{tmp}}, [\text{in}_P,]\text{m}_{\text{in}}),$$

where $\mathcal{H} \in \mathcal{OS}$, st_P is the persistent state of party P , st_P^{tmp} is their temporary state (only for that particular protocol execution), in_P is the local input of party P which is only used for the first step $r = 1$ ⁷, m_{in} and m_{out} denote the input and output message of the protocol, and out_P is the local output of party P . If inputs/outputs are not used, they are set to the empty string ε . This is for example the case for m_{in} for the first client step of an execution or for out_P for every non-terminating step of the protocol. By assumption, a client C always initiates a protocol execution, i.e. sends the first message.

TERMINATION. Return statements of a party $P \in \{C, S\}$ are captured by the two operations success_P and fail_P defined as

$$\begin{aligned} \text{success}_P(\text{m}_P) : \text{out}_P.\text{decision} &\leftarrow \text{true}; \text{ return } (\text{st}_P, \text{st}_P^{\text{tmp}}, \text{out}_P, \text{m}_P), \\ \text{fail}_P : \text{out}_P.\text{decision} &\leftarrow \text{false}; \text{ return } (\text{st}_P, \text{st}_P^{\text{tmp}}, \text{out}_P, \perp). \end{aligned}$$

Further, $\text{success}_P(\top)$ returns $\top$ and failing sub procedures lead to a failing main procedure.

FULL EXECUTION AND CORRECTNESS. In Figure 7, we recall the full, i.e. atomic, execution of a two-party protocol where the protocol between client and server is executed with the corrected messages in the correct order from [BDG⁺24]. This in particular used to define correctness.

⁷ Brackets “[...]” indicate that the input is optional; in this case, only for the first step.


```

Procedure Exec $_{\Pi[H]}$ (stC, stS, inc, ins):
00 (stC, stCtmp, outC, m[1])  $\xleftarrow{\$}$   $\Pi^{(C:1)}[H](st_C, \varepsilon, inc, \varepsilon)$ 
01 (stS, stStmp, outS, m[2])  $\xleftarrow{\$}$   $\Pi^{(S:1)}[H](st_S, \varepsilon, ins, m[1])$ 
02 for  $i = 3, \dots, \Pi.SN$ :
03    $r \leftarrow \lceil i/2 \rceil$ 
04   if  $i$  odd:  $P \leftarrow C$  else  $P \leftarrow S$ 
05   if outP.decision =  $\perp$ 
06     (stP, stPtmp, outP, m[i])  $\xleftarrow{\$}$   $\Pi^{(P:r)}[H](st_P, st_P^{tmp}, m[i-1])$ 
07 return (m, stC, stS, outC, outS)

```

Fig. 7. Execution of protocol $\Pi[H]$ between client C and server S .

Definition 8 (Corr). *Correctness for a cloud storage scheme CS is defined via the game in Figure 8. We define the advantage of an adversary $\mathcal{A}$ as*

$$\text{Adv}_{CS}^{Q\text{-Corr}}(\mathcal{A}) := \Pr[Q\text{-Corr}_{CS}(\mathcal{A}) \Rightarrow 1],$$

for $Q = (Q_{\text{REG}}, Q_{\text{AUTH}}, Q_{\text{PUT}}, Q_{\text{UPD}}, Q_{\text{GET}}, Q_{\text{SHARE}}, Q_{\text{ACPT}}, Q_{\text{RO}})$.

4.2 Security Definitions

We define confidentiality and integrity for cloud storage schemes against *adaptive* compromises. Compared to [BDG⁺24], we make small changes in notation. Our games are given in Figures 9 and 10.

Instead of using indices for users, we only act on the level of account identifiers aid which were also used in [BDG⁺24]. This is closer to practice since in practice only the account identifier is used as a handle, e.g. the e-mail address or the user name of a client. Note that we still allow for several registration attempts but after the registration succeeds the account identifier must be unique. Not using indices allows us to drop the set $\mathcal{S}_{\text{hon}}$ since we track correctly registered aid 's and therefore it is sufficient to keep track of the accounts that were compromised.

The input interface of oracles is also changed compared to [BDG⁺24]. Instead of using a generic input inc , we clarify what the input should include by making the components explicit which provides an easier understanding of the notion.

We change the confidentiality from [BDG⁺24] by marking all challenge file owners as challenge owners in the accept oracle (Line 74).⁸ In BDGHP, no owners are added in the accept oracle but only in the share oracle which allows an adversary to first share a (non-challenge) file with a compromised user, then register it as a challenge file by updating the content, and finally accept it by the compromised user. In such a case, the compromised user was not registered as a challenge file owner and the adversary would win the game.

Lastly, for the integrity notion we capture the winning condition explicitly by requiring the output file to be not $\perp$. In the notion of [BDG⁺24], this is only implicitly captured via the decision property. We note that this may be problematic since the property is specified by the protocol itself. A badly designed protocol can always set the variable to **true**, bypassing the intentions of the security notion.⁹

Later, in Section 6, we also define a new security notion that captures security guarantees for honest servers.

⁸ This change as well as the following description of the possible attack against BDGHP's security model was pointed out by a EUROCRYPT'26 reviewer.

⁹ While such a protocol would not be considered correct, we still think that it makes definitions less intuitive. More concretely, the correctness game of [BDG⁺24] would never check whether an uploaded file cannot be retrieved (line 20 in Figure 3 in the ePrint version), but only whether the user retrieved an unexpected file (line 21). Since $\perp$ counts as an unexpected file, this works. However, we believe that such a subtlety could be avoided by making the decision bit dependent on some event, as we do for the integrity notion by requiring the file to be not $\perp$.

Game $Q\text{-Corr}_{CS}(\mathcal{A})$ <pre> 00 $H \xleftarrow{\\$} \mathcal{O}_S$ 01 $\text{win} \leftarrow \text{false}$ 02 $n_c \leftarrow 0$ // session counter 03 $\mathcal{A}^{\mathcal{O}}$ 04 return win </pre> Procedure $\text{RunProt}(\text{prot}, i_c, \text{inc}, \text{ins})$ <pre> 05 if $\text{prot} \in \{\text{areg}, \text{auth}\}$ 06 $\text{st}_C \leftarrow \varepsilon$ 07 else 08 $(\text{aid}, \text{st}_C) \leftarrow T_{\text{st}_C}[i_c]$ 09 $(m, \text{st}_C, \text{st}_S, \text{out}_C, \text{outs}) \xleftarrow{\\$} \text{Execn}_{\text{prot}}(\text{st}_C, \text{st}_S, \text{inc}, \text{ins})$ 10 if $\text{out}_C.\text{decision} \wedge \text{outs}.\text{decision}$ 11 if $\text{prot} = \text{areg}$ 12 $T_{\text{uaid}}[\text{inc}.\text{aid}] \leftarrow \text{true}$ 13 if $\text{prot} = \text{auth}$ 14 $T_{\text{st}_C}[i_c] \leftarrow (\text{inc}.\text{aid}, \text{st}_C)$ 15 if $\text{prot} \notin \{\text{areg}, \text{auth}\}$ 16 $T_{\text{st}_C}[i_c] \leftarrow (\text{aid}, \text{st}_C)$ 17 return $(m, \text{st}_C, \text{st}_S, \text{out}_C, \text{outs})$ </pre> Oracle $\text{REG}(\text{inc}, \text{ins})$ <pre> 18 require $T_{\text{uaid}}[\text{inc}.\text{aid}] = \perp$ 19 $T_{\text{pw}}[\text{inc}.\text{aid}] \leftarrow \text{inc}.\text{pw}$ 20 return $\text{RunProt}(\text{areg}, \varepsilon, \text{inc}, \text{ins})$ </pre> Oracle $\text{AUTH}(\text{inc}, \text{ins})$ <pre> 21 require $T_{\text{uaid}}[\text{inc}.\text{aid}] = \text{true}$ 22 n_c++ 23 $i_c \leftarrow n_c$ 24 return $\text{RunProt}(\text{auth}, i_c, \text{inc}, \text{ins})$ </pre> Oracle $\text{PUT}(i_c \in [n_c], \text{inc}, \text{ins})$ <pre> 25 require $T_f[\text{inc}.\text{fid}] = \perp$ 26 $(m, \text{st}_C, \text{st}_S, \text{out}_C, \text{outs}) \xleftarrow{\\$} \text{RunProt}(\text{put}, i_c, \text{inc}, \text{ins})$ 27 if $\text{outs}.\text{decision}$ 28 $U \leftarrow \{T_{\text{st}_C}[i_c].\text{aid}\}$ 29 $F \leftarrow \text{inc}.\text{file}$ 30 $T_f[\text{inc}.\text{fid}] \leftarrow (U, F)$ 31 return $(m, \text{st}_C, \text{st}_S, \text{out}_C, \text{outs})$ </pre>	Oracle $\text{UPD}(i_c \in [n_c], \text{fid}, \text{file})$ <pre> 32 require $i_c \in T_{\text{st}_C} \wedge T_{\text{st}_C}[\text{inc}.\text{aid}] \in T_f[\text{inc}.\text{fid}].U$ 33 $(m, \text{st}_C, \text{st}_S, \text{out}_C, \text{outs}) \xleftarrow{\\$} \text{RunProt}(\text{update}, i_c, \text{inc}, \text{ins})$ 34 if $\text{outs}.\text{decision}$ 35 $T_f[\text{inc}.\text{fid}].F \leftarrow \text{inc}.\text{file}$ 36 return $(m, \text{st}_C, \text{st}_S, \text{out}_C, \text{outs})$ </pre> Oracle $\text{GET}(i_c \in [n_c], \text{inc}, \text{ins})$ <pre> 37 require $T_{\text{st}_C}[i_c].\text{aid} \in T_f[\text{inc}.\text{fid}].U$ 38 $(m, \text{st}_C, \text{st}_S, \text{out}_C, \text{outs}) \xleftarrow{\\$} \text{RunProt}(\text{get}, i_c, \text{inc}, \text{ins})$ 39 if $\text{out}_C.\text{decision} \wedge \text{out}_C.\text{file} \neq T_f[\text{inc}.\text{fid}].F$ 40 $\text{win} \leftarrow \text{true}$ // get wrong file 41 if $\text{out}_C.\text{decision} = \text{false} \wedge \text{inc}.\text{fid} \in T_f[\text{inc}.\text{fid}]$ 42 $\text{win} \leftarrow \text{true}$ // do not get file 43 return $(m, \text{st}_C, \text{st}_S, \text{out}_C, \text{outs})$ </pre> Oracle $\text{SHARE}(i_c \in [n_c], \text{inc}, \text{ins})$ <pre> 44 require $T_{\text{st}_C}[i_c].\text{aid} \in T_f[\text{inc}.\text{fid}].U$ 45 $(m, \text{st}_C, \text{st}_S, \text{out}_C, \text{outs}) \xleftarrow{\\$} \text{RunProt}(\text{share}, i_c, \text{inc}, \text{ins})$ 46 if $\text{out}_C.\text{decision}$ 47 $\mathcal{S}_{\text{shr}} \leftarrow \mathcal{S}_{\text{shr}} \cup \{(\text{inc}.\text{raid}, \text{inc}.\text{fid})\}$ 48 $T_{\text{oob}}[(\text{inc}.\text{raid}, \text{inc}.\text{fid})] \leftarrow \text{out}_C.\text{oob}$ 49 return $(m, \text{st}_C, \text{st}_S, \text{out}_C, \text{outs})$ </pre> Oracle $\text{ACPT}(i_c \in [n_c], \text{inc}, \text{ins})$ <pre> 50 require $T_{\text{oob}}[(T_{\text{st}_C}[i_c].\text{aid}, \text{inc}.\text{fid})] \neq \perp$ 51 $\text{inc}.\text{oob} \leftarrow T_{\text{oob}}[(T_{\text{st}_C}[i_c].\text{aid}, \text{inc}.\text{fid})]$ 52 $(m, \text{st}_C, \text{st}_S, \text{out}_C, \text{outs}) \xleftarrow{\\$} \text{RunProt}(\text{accept}, i_c, \text{inc}, \text{ins})$ 53 if $\text{outs}.\text{decision}$ 54 $T_f[\text{inc}.\text{fid}].U \leftarrow T_f[\text{inc}.\text{fid}].U \cup \{T_{\text{st}_C}[i_c].\text{inc}.\text{aid}\}$ 55 if $\text{out}_C.\text{decision}$ 56 $T_{\text{oob}}[(T_{\text{st}_C}[i_c].\text{aid}, \text{inc}.\text{fid})] \leftarrow \perp$ 57 if $\text{out}_C.\text{decision} = \text{false} \wedge (T_{\text{st}_C}[i_c].\text{inc}.\text{aid}, \text{inc}.\text{fid}) \in \mathcal{S}_{\text{shr}}$ 58 $\text{win} \leftarrow \text{true}$ // does not accept shared file 59 return $(m, \text{st}_C, \text{st}_S, \text{out}_C, \text{outs})$ </pre> Oracle $\text{RO}(x)$ <pre> 60 return $H(x)$ </pre>
--	--

Fig. 8. Correctness game for a cloud storage scheme CS. Adversary $\mathcal{A}$ has access to oracles $\mathcal{O} = \{\text{REG}, \text{AUTH}, \text{PUT}, \text{UPD}, \text{GET}, \text{SHARE}, \text{ACPT}, \text{RO}\}$ making at most $Q_{\mathcal{O}}$ queries to oracle $\mathcal{O}$.

Definition 9 (Conf). Confidentiality for a cloud storage scheme CS is defined via the game in Figure 9. We define the advantage of an adversary $\mathcal{A}$ and a password distribution $\mathcal{PW}$ as

$$\text{Adv}_{CS, \mathcal{PW}}^{Q\text{-Conf}}(\mathcal{A}) := 2 \Pr [Q\text{-Conf}_{CS, \mathcal{PW}}(\mathcal{A}) \Rightarrow 1] - 1,$$

for $Q = (Q_{\text{STEP}}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1}, Q_{\text{PUT}_1}, Q_{\text{UPD}_1}, Q_{\text{GET}_1}, Q_{\text{SHARE}_1}, Q_{\text{ACPT}_1}, Q_{\text{CHALL}}, Q_{\text{COMP}}, Q_{\text{RO}})$.

Confidentiality captures that an adversary does not learn anything about honest users' files unless they are shared with a compromised party. This needs to hold even if the adversary can control the server and adaptively compromise clients. Further, protocols can be executed non-atomic and each step of the protocols can be interleaved arbitrarily.

Game $Q\text{-Conf}_{CS, \mathcal{PW}}(\mathcal{A})$ <pre> 00 $H \xleftarrow{\\$} OS$ 01 $(pw_1, \dots, pw_{Q_{REG_1}}) \xleftarrow{\\$} \mathcal{PW}$ 02 $\mathcal{S}_{cmp}, \mathcal{S}_{ch}, \mathcal{S}_{chOwn} \leftarrow \emptyset$ 03 $(n_c, n_p, n_r) \leftarrow 0$ // session, protocol execution, 04 $b \xleftarrow{\\$} \{0, 1\}$ ↪ registration counter 05 $b^* \leftarrow \mathcal{A}^O$ 06 if $\exists aid \in \mathcal{S}_{chOwn} : aid \in \mathcal{S}_{cmp} \vee T_{uaid}[aid] = \perp$ 07 return 0 08 return $\llbracket b^* = b \rrbracket$ </pre> Procedure $Step_1(\text{prot}, i_c, inc)$ <pre> 09 $(T_{stc}[i_c], st_c^{tmp}, out_c, m^*) \leftarrow \Pi_{prot}^{C:1}[H](T_{stc}[i_c], \varepsilon, inc, \varepsilon)$ 10 n_p++ 11 $i_p \leftarrow n_p$ 12 $T_p[(\text{prot}, i_p)] \leftarrow (i_c, st_c^{tmp}, inc, 2)$ 13 return $Chk(\text{prot}, i_p, out_c, m^*)$ </pre> Procedure $Chk(\text{prot}, i_p, out_c, m^*)$ <pre> 14 $(i_c, st_c^{tmp}, inc, r) \leftarrow T_p[(\text{prot}, i_p)]$ 15 $fid \leftarrow inc.fid$ 16 if $\text{prot} = \text{areg}$ 17 if $out_c.decision$ 18 $T_{uaid}[inc.aid] \leftarrow true$ // reg completed 19 else 20 $T_{uaid}[inc.aid] \leftarrow \perp$ // allow new reg attempt 21 if $\text{prot} = \text{get}$: 22 if $fid \in \mathcal{S}_{ch}$: $out_c.f \leftarrow \perp$ 23 return $(out_c.f, m^*)$ 24 if $\text{prot} = \text{share} \wedge out_c.decision$: 25 $T_{oob}[(inc.raid, fid)] \leftarrow out_c.oob$ 26 if $\text{prot} = \text{accept} \wedge out_c.decision$: 27 $T_{oob}[(T_u[i_c], fid)] \leftarrow \perp$ 28 return m^* </pre> Oracle $STEP(\text{prot}, i_p \in [n_p], m)$ <pre> 29 $(i_c, st_c^{tmp}, inc, r) \leftarrow T_p[(\text{prot}, i_p)]$ 30 require $r \leq \Pi_{prot}.SN$ 31 $(T_{stc}[i_c], st_c^{tmp}, out_c, m^*) \leftarrow \Pi_{prot}^{C:r}[H](T_{stc}[i_c], st_c^{tmp}, m)$ 32 $T_p[(\text{prot}, i_p)] \leftarrow (i_c, st_c^{tmp}, inc, r + 1)$ 33 return $Chk(\text{prot}, i_p, out_c, m^*)$ </pre> Oracle $COMP(aid)$ <pre> 34 require $T_{uaid}[aid] \neq \perp$ 35 $\mathcal{S}_{cmp} \leftarrow \mathcal{S}_{cmp} \cup \{aid\}$ 36 return $(T_{pw}[aid], \{T_{stc}[i_c] \mid T_u[i_c] = aid\}, \{T_{oob}[aid, \cdot]\})$ </pre> Oracle $CHALL(i_c \in [n_c], fid, file_0, file_1, \text{prot})$ <pre> 37 require $\text{prot} \in \{PUT_1, UPD_1\}$ 38 $\mathcal{S}_{ch} \leftarrow \mathcal{S}_{ch} \cup \{fid\}$ 39 $\mathcal{S}_{chOwn} \leftarrow \mathcal{S}_{chOwn} \cup \{T_u[i_c]\} \cup T_f[fid]$ 40 return $\text{prot}(i_c, fid, file_b)$ </pre>	Oracle $REG_1(aid)$ <pre> 41 require $T_{uaid}[aid] = \perp$ 42 n_r++ 43 $T_{pw}[aid] \leftarrow pw_{n_r}$ 44 $inc.aid \leftarrow aid$ 45 $T_{uaid}[aid] \leftarrow false$ // block aid during reg process 46 return $Step_1(\text{areg}, \varepsilon, inc)$ </pre> Oracle $AUTH_1(aid)$ <pre> 47 require $T_{uaid}[aid] = true$ 48 $(inc.aid, inc.pw) \leftarrow (aid, T_{pw}[aid])$ 49 n_c++ 50 $i_c \leftarrow n_c$ 51 $(T_u[i_c], T_{stc}[i_c]) \leftarrow (aid, \varepsilon)$ 52 return $Step_1(\text{auth}, i_c, inc)$ </pre> Oracle $PUT_1(i_c \in [n_c], fid, file)$ <pre> 53 require $T_f[fid] = \perp$ 54 $T_f[fid] \leftarrow \{T_u[i_c]\}$ 55 $(inc.fid, inc.file) \leftarrow (fid, file)$ 56 return $Step_1(\text{put}, i_c, inc)$ </pre> Oracle $UPD_1(i_c \in [n_c], fid, file)$ <pre> 57 $(inc.fid, inc.file) \leftarrow (fid, file)$ 58 return $Step_1(\text{update}, i_c, inc)$ </pre> Oracle $GET_1(i_c \in [n_c], fid)$ <pre> 59 $inc.fid \leftarrow fid$ 60 return $Step_1(\text{get}, i_c, inc)$ </pre> Oracle $SHARE_1(i_c \in [n_c], fid, raid)$ <pre> 61 $T_f[fid] \leftarrow T_f[fid] \cup \{T_u[i_c]\}$ 62 $(inc.fid, inc.raid) \leftarrow (fid, raid)$ 63 if $fid \in \mathcal{S}_{ch}$: 64 $\mathcal{S}_{chOwn} \leftarrow \mathcal{S}_{chOwn} \cup \{T_u[i_c], raid\}$ 65 return $Step_1(\text{share}, i_c, inc)$ </pre> Oracle $ACPT_1(i_c \in [n_c], fid, oob)$ <pre> 66 $aid \leftarrow T_u[i_c]$ 67 require $oob \neq \perp \vee T_{oob}[(aid, fid)] \neq \perp$ 68 $T_f[fid] \leftarrow T_f[fid] \cup \{aid\}$ 69 if $oob = \perp$: 70 $inc.oob \leftarrow T_{oob}[(aid, fid)]$ 71 else 72 $T_f[fid] \leftarrow T_f[fid] \cup \{i_{mal}\}$ 73 $inc.oob \leftarrow oob$ 74 if $fid \in \mathcal{S}_{ch}$: 75 $\mathcal{S}_{chOwn} \leftarrow \mathcal{S}_{chOwn} \cup T_f[fid]$ 76 return $Step_1(\text{accept}, i_c, inc)$ </pre> Oracle $RO(x)$ <pre> 77 return $H(x)$ </pre>
--	--

Fig. 9. Confidentiality game for a cloud storage scheme CS. Adversary $\mathcal{A}$ has access to oracles $\mathcal{O} = \{STEP, REG_1, AUTH_1, PUT_1, UPD_1, GET_1, SHARE_1, ACPT_1, CHALL, COMP, RO\}$ making at most Q_O queries to oracle O . Differences to Int (Figure 10) are highlighted in blue.

Game $Q\text{-Int}_{\text{CS}, \mathcal{PW}}(\mathcal{A})$

```

00  $H \xleftarrow{\$} \mathcal{OS}$ 
01  $(pw_1, \dots, pw_{Q_{\text{REG}_1}}) \xleftarrow{\$} \mathcal{PW}$ 
02  $\mathcal{S}_{\text{cmp}} \leftarrow \{i_{\text{mal}}\}$   $\quad \quad \quad \parallel i_{\text{mal}} \text{ is a special symbol not in } \mathbb{N}$ 
03  $(n_c, n_p, n_r) \leftarrow 0$   $\quad \quad \quad \parallel \text{session, protocol execution,}$ 
04  $\text{win} \leftarrow \text{false}$   $\quad \quad \quad \hookrightarrow \text{registration counter}$ 
05  $\mathcal{A}^{\mathcal{O}}$ 
06 return win

```

Procedure $\text{Step}_1(\text{prot}, i_c, \text{inc})$

```

07  $(T_{\text{stc}}[i_c], \text{stc}_{\text{C}}^{\text{tmp}}, \text{outc}, m^*) \leftarrow \Pi_{\text{prot}}^{C:1}[\text{H}](T_{\text{stc}}[i_c], \varepsilon, \text{inc}, \varepsilon)$ 
08  $n_p++$ 
09  $i_p \leftarrow n_p$ 
10  $T_p[(\text{prot}, i_p)] \leftarrow (i_c, \text{stc}_{\text{C}}^{\text{tmp}}, \text{inc}, 2)$ 
11 return  $\text{Chk}(\text{prot}, i_p, \text{outc}, m^*)$ 

```

Procedure $\text{Chk}(\text{prot}, i_p, \text{outc}, m^*)$

```

12  $(i_c, \text{stc}_{\text{C}}^{\text{tmp}}, \text{inc}, r) \leftarrow T_p[(\text{prot}, i_p)]$ 
13  $\text{fid} \leftarrow \text{inc.fid}$ 
14 if  $\text{prot} = \text{areg}$ 
15   if  $\text{outc.decision}$ 
16      $T_{\text{uaid}}[\text{inc.aid}] \leftarrow \text{true}$   $\quad \quad \quad \parallel \text{reg completed}$ 
17   else
18      $T_{\text{uaid}}[\text{inc.aid}] \leftarrow \perp$   $\quad \quad \quad \parallel \text{allow new reg attempt}$ 
19   if  $\text{prot} = \text{get}$ :
20     if  $T_u[i_c] \notin \mathcal{S}_{\text{cmp}} \wedge T_f[\text{fid}].U \cap \mathcal{S}_{\text{cmp}} = \emptyset \wedge \neg \text{win}$ 
21        $\text{file} \leftarrow \text{outc.file}$ 
22        $\text{win} \leftarrow [\text{outc.decision} \wedge \text{file} \neq \perp]$ 
23        $\wedge [\text{file} \notin T_f[\text{fid}].F]$ 
23     return  $(\text{outc.f}, m^*)$ 
24   if  $\text{prot} = \text{share} \wedge \text{outc.decision}$ :
25      $T_{\text{oob}}[(\text{inc.raid}, \text{fid})] \leftarrow \text{outc.oob}$ 
26   if  $\text{prot} = \text{accept} \wedge \text{outc.decision}$ :
27      $T_{\text{oob}}[(T_u[i_c], \text{fid})] \leftarrow \perp$ 
28   return  $m^*$ 

```

Oracle $\text{STEP}(\text{prot}, i_p \in [n_p], m)$

```

29  $(i_c, \text{stc}_{\text{C}}^{\text{tmp}}, \text{inc}, r) \leftarrow T_p[(\text{prot}, i_p)]$ 
30 require  $r \leq \Pi_{\text{prot}}.\text{SN}$ 
31  $(T_{\text{stc}}[i_c], \text{stc}_{\text{C}}^{\text{tmp}}, \text{outc}, m^*) \leftarrow \Pi_{\text{prot}}^{C:r}[\text{H}](T_{\text{stc}}[i_c], \text{stc}_{\text{C}}^{\text{tmp}}, m)$ 
32  $T_p[(\text{prot}, i_p)] \leftarrow (i_c, \text{stc}_{\text{C}}^{\text{tmp}}, \text{inc}, r+1)$ 
33 return  $\text{Chk}(\text{prot}, i_p, \text{outc}, m^*)$ 

```

Oracle $\text{COMP}(\text{aid})$

```

34 require  $T_{\text{uaid}}[\text{aid}] \neq \perp$ 
35  $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{\text{aid}\}$ 
36 return  $(T_{\text{pw}}[\text{aid}], \{T_{\text{stc}}[i_c] \mid T_u[i_c] = \text{aid}\}, \{T_{\text{oob}}[(\text{aid}, \cdot)]\})$ 

```

Oracle $\text{REG}_1(\text{aid})$

```

37 require  $T_{\text{uaid}}[\text{aid}] = \perp$ 
38  $n_r++$ 
39  $T_{\text{pw}}[\text{aid}] \leftarrow pw_{n_r}$ 
40  $\text{inc.aid} \leftarrow \text{aid}$ 
41  $T_{\text{uaid}}[\text{aid}] \leftarrow \text{false}$   $\quad \quad \quad \parallel \text{block aid during reg process}$ 
42 return  $\text{Step}_1(\text{areg}, \varepsilon, \text{inc})$ 

```

Oracle $\text{AUTH}_1(\text{aid})$

```

43 require  $T_{\text{uaid}}[\text{aid}] = \text{true}$ 
44  $(\text{inc.aid}, \text{inc.pw}) \leftarrow (\text{aid}, T_{\text{pw}}[\text{aid}])$ 
45  $n_c++$ 
46  $i_c \leftarrow n_c$ 
47  $(T_u[i_c], T_{\text{stc}}[i_c]) \leftarrow (\text{aid}, \varepsilon)$ 
48 return  $\text{Step}_1(\text{auth}, i_c, \text{inc})$ 

```

Oracle $\text{PUT}_1(i_c \in [n_c], \text{fid}, \text{file})$

```

49 require  $T_f[\text{fid}] = \perp$ 
50  $U \leftarrow \{T_u[i_c]\}$ 
51  $F \leftarrow \{\text{file}\}$ 
52  $T_f[\text{fid}] \leftarrow (U, F)$ 
53  $(\text{inc.fid}, \text{inc.file}) \leftarrow (\text{fid}, \text{file})$ 
54 return  $\text{Step}_1(\text{put}, i_c, \text{inc})$ 

```

Oracle $\text{UPD}_1(i_c \in [n_c], \text{fid}, \text{file})$

```

55 if  $T_u[i_c] \in T_f[\text{fid}].U$ 
56    $T_f[\text{fid}].F \leftarrow T_f[\text{fid}].F \cup \{\text{file}\}$ 
57  $(\text{inc.fid}, \text{inc.file}) \leftarrow (\text{fid}, \text{file})$ 
58 return  $\text{Step}_1(\text{update}, i_c, \text{inc})$ 

```

Oracle $\text{GET}_1(i_c \in [n_c], \text{fid})$

```

59  $\text{inc.fid} \leftarrow \text{fid}$ 
60 return  $\text{Step}_1(\text{get}, i_c, \text{inc})$ 

```

Oracle $\text{SHARE}_1(i_c \in [n_c], \text{fid}, \text{raid})$

```

61  $T_f[\text{fid}].U \leftarrow T_f[\text{fid}].U \cup \{T_u[i_c]\}$ 
62  $(\text{inc.fid}, \text{inc.raid}) \leftarrow (\text{fid}, \text{raid})$ 
63 return  $\text{Step}_1(\text{share}, i_c, \text{inc})$ 

```

Oracle $\text{ACPT}_1(i_c \in [n_c], \text{fid}, \text{oob})$

```

64  $\text{aid} \leftarrow T_u[i_c]$ 
65 require  $\text{oob} \neq \perp \vee T_{\text{oob}}[(\text{aid}, \text{fid})] \neq \perp$ 
66  $T_f[\text{fid}].U \leftarrow T_f[\text{fid}].U \cup \{\text{aid}\}$ 
67 if  $\text{oob} = \perp$ :
68    $\text{inc.oob} \leftarrow T_{\text{oob}}[(\text{aid}, \text{fid})]$ 
69 else
70    $T_f[\text{fid}].U \leftarrow T_f[\text{fid}].U \cup \{i_{\text{mal}}\}$ 
71    $\text{inc.oob} \leftarrow \text{oob}$ 
72 return  $\text{Step}_1(\text{accept}, i_c, \text{inc})$ 

```

Oracle $\text{RO}(x)$

```

73 return  $H(x)$ 

```

Fig. 10. Integrity game for a cloud storage scheme CS. Adversary $\mathcal{A}$ has access to oracles $\mathcal{O} = \{\text{STEP}, \text{REG}_1, \text{AUTH}_1, \text{PUT}_1, \text{UPD}_1, \text{GET}_1, \text{SHARE}_1, \text{ACPT}_1, \text{COMP}, \text{RO}\}$ making at most $Q_{\mathcal{O}}$ queries to oracle $\mathcal{O}$. Differences to Conf (Figure 9) are highlighted in blue.

Definition 10 (Int). *Integrity for a cloud storage scheme CS is defined via the game in Figure 10. We define the advantage of an adversary $\mathcal{A}$ as*

$$\text{Adv}_{\text{CS}, \mathcal{P}\mathcal{W}}^{Q\text{-Int}}(\mathcal{A}) := \Pr [Q\text{-Int}_{\text{CS}, \mathcal{P}\mathcal{W}}(\mathcal{A}) \Rightarrow 1],$$

for $Q = (Q_{\text{STEP}}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1}, Q_{\text{PUT}_1}, Q_{\text{UPD}_1}, Q_{\text{GET}_1}, Q_{\text{SHARE}_1}, Q_{\text{ACPT}_1}, Q_{\text{COMP}}, Q_{\text{RO}})$.

The adversary model for integrity is the same as for confidentiality. Integrity captures that an attacker cannot modify honest client's files or create new valid files as long as the files are not shared with compromised parties.

5 Construction

We define $\text{CS}[\text{Auth}, \text{AEAD}, \text{MAC}]$ scheme based on three primitives: Auth, AEAD, and MAC. Let $\text{Auth} := (\mathcal{OS}_{\text{Auth}}, \mathcal{D}_S, Y, \text{Init}, \text{Resp}, \text{Fin}, \text{Full})$. Then we define the oracle space as

$$\mathcal{OS} := \{f : \{0, 1\}^* \rightarrow \{0, 1\}^{\text{kl}}\} \times \mathcal{OS}_{\text{Auth}}$$

where kl is the key length of AEAD and the cloud storage scheme as

$$\text{CS}[\text{Auth}, \text{AEAD}, \text{MAC}] := (\mathcal{OS}, \Pi_{\text{areg}}, \Pi_{\text{auth}}, \Pi_{\text{put}}, \Pi_{\text{upd}}, \Pi_{\text{get}}, \Pi_{\text{share}}, \Pi_{\text{accept}}),$$

where the seven protocols are defined in Figures 11 and 12. By NewID , we denote a procedure which samples a new ID that was never output before.

CORRECTNESS. Correctness follows from the correctness of all underlying primitives. We only state the theorem.

Theorem 4 (Correctness). *Let AEAD and MAC be correct and Auth be δ_{Auth} -correct. Then for any adversary $\mathcal{A}$ against Corr of $\text{CS}[\text{Auth}, \text{AEAD}, \text{MAC}]$ it holds that*

$$\text{Adv}_{\text{CS}[\text{Auth}, \text{AEAD}, \text{MAC}]}^{Q_{\mathcal{A}}\text{-Corr}}(\mathcal{A}) \leq Q_{\text{AUTH}} \cdot \delta_{\text{Auth}}$$

with $Q_{\mathcal{A}} = (Q_{\text{REG}}, Q_{\text{AUTH}}, Q_{\text{PUT}}, Q_{\text{UPD}}, Q_{\text{GET}}, Q_{\text{SHARE}}, Q_{\text{ACPT}}, Q_{\text{RO}})$.

CONFIDENTIALITY AND INTEGRITY. We give informal statements below. The formal theorems with concrete bounds can be found as Theorems 10 and 11 in Appendices C.1 and C.2, respectively, where we also give the full proofs. Additionally, we discuss and provide bounds for the scheme instantiated with 2HDH in Section 7.

Theorem 5 (Confidentiality, Informal). *Let Auth be an authentication scheme that is δ_{Auth} -correct and INS-OW secure. Let AEAD be an AEAD scheme that is OW-CPA, IND-CCA and INT-CTXT secure. Let MAC be a MAC scheme. Then $\text{CS} := \text{CS}[\text{Auth}, \text{AEAD}, \text{MAC}]$ is Conf secure.*

Proof (Idea). The main path of the proof is the same as in [BDG⁺24]: we go through the key hierarchy and use that “honest keys” are indistinguishable from random (honest client's raw secrets, their encryption key, their personal master key, and eventually file keys which leads to confidential files). The difference to [BDG⁺24] is that we allow for adaptive compromises which is why we cannot commit to any keys because the reduction does not know which ciphertexts might be opened later due to a compromise. Hence, we cannot simply set keys to random. Our proof's main idea is to stay in an uncommitted state by only encrypting seeds and patching the random oracle upon compromises. While this is straightforward for a single non-committing encryption, the key hierarchy adds additional challenges because we need to make sure that random oracles are simulated consistently in each step and can be patched without unnecessary reduction losses. $\square$

$\Pi_{\text{areg}}^{C:1}[H, H_{\text{Auth}}](\varepsilon, \varepsilon, \text{inc}, \varepsilon)$ 00 $(\text{aid}, \text{pw}) \leftarrow (\text{inc}.\text{aid}, \text{inc}.\text{pw})$ 01 $k \xleftarrow{\$} \text{Auth}.\mathcal{D}_S$ 02 $\text{rw} \leftarrow \text{Auth.Full}[H_{\text{Auth}}](\text{pw}, \text{aid}, k)$ 03 $k_{\text{ek}} \leftarrow H(\text{rw}, \text{aid}, \text{"ek"})$ 04 $k_{\text{mac}} \leftarrow H(\text{rw}, \text{aid}, \text{"mac"})$ 05 $(n_{\text{mk}}, \text{seed}_{\text{mk}}) \xleftarrow{\$} \{0, 1\}^{n_l} \times \{0, 1\}^{k_l}$ 06 $k_{\text{mk}} \leftarrow H(\text{seed}_{\text{mk}}, \text{aid}, \text{"mk"})$ 07 $\text{ct}_{\text{mk}} \xleftarrow{\$} \text{AEAD}.\text{Enc}(k_{\text{ek}}, n_{\text{mk}}, \text{seed}_{\text{mk}}, (\text{aid}, \text{"mk"}))$ 08 $m_C \leftarrow (\text{aid}, k, n_{\text{mk}}, \text{ct}_{\text{mk}}, k_{\text{mac}})$ 09 $\text{success}_C(m_C)$	$\Pi_{\text{areg}}^{S:1}[H, H_{\text{Auth}}](\text{st}_S, \varepsilon, \text{ins}, m_C)$ 10 $(\text{aid}, k, n_{\text{mk}}, \text{ct}_{\text{mk}}, k_{\text{mac}}) \leftarrow m_C$ 11 $\text{st}_S.\text{acc}[\text{aid}] \leftarrow (k, n_{\text{mk}}, \text{ct}_{\text{mk}}, k_{\text{mac}})$ 12 success_S
$\Pi_{\text{auth}}^{C:1}[H, H_{\text{Auth}}](\text{st}_C, \varepsilon, \text{inc}, \varepsilon)$ 13 $(\text{aid}, \text{pw}) \leftarrow (\text{inc}.\text{aid}, \text{inc}.\text{pw})$ 14 $(\text{auth}_1, \text{st}) \xleftarrow{\$} \text{Auth.Init}[H_{\text{Auth}}](\text{pw}, \text{aid})$ 15 $m_C \leftarrow (\text{aid}, \text{auth}_1)$ 16 $\text{st}_C^{\text{tmp}} \leftarrow (\text{aid}, \text{pw}, \text{st}, m_C)$ 17 $\text{return } (\text{st}_C, \text{st}_C^{\text{tmp}}, \varepsilon, m_C)$ $\Pi_{\text{auth}}^{C:2}[H, H_{\text{Auth}}](\text{st}_C, \text{st}_C^{\text{tmp}}, m_S)$ 18 $(\text{aid}, \text{pw}, \text{st}, m'_C) \leftarrow \text{st}_C^{\text{tmp}}$ 19 $(\text{sid}, \text{auth}_2) \leftarrow m_S$ 20 $\text{rw} \leftarrow \text{Auth.Fin}[H_{\text{Auth}}](\text{auth}_2, \text{pw}, \text{aid}, \text{st})$ 21 $\text{st}_C^{\text{tmp}}.k_{\text{ek}} \leftarrow H(\text{rw}, \text{aid}, \text{"ek"})$ 22 $k_{\text{mac}} \leftarrow H(\text{rw}, \text{aid}, \text{"mac"})$ 23 $m_C \leftarrow \text{MAC}.\text{Tag}(k_{\text{mac}}, (m'_C, m_S))$ 24 $\text{return } (\text{st}_C, \text{st}_C^{\text{tmp}}, \varepsilon, m_C)$ $\Pi_{\text{auth}}^{C:3}[H, H_{\text{Auth}}](\text{st}_C, \text{st}_C^{\text{tmp}}, m_S)$ 25 $\text{aid} \leftarrow \text{st}_C^{\text{tmp}}.\text{aid}$ 26 $(n_{\text{mk}}, \text{ct}_{\text{mk}}) \leftarrow m_S$ 27 $\text{seed}_{\text{mk}} \leftarrow \text{AEAD}.\text{Dec}(\text{st}_C^{\text{tmp}}.k_{\text{ek}}, n_{\text{mk}}, \text{ct}_{\text{mk}}, (\text{aid}, \text{"mk"}))$ 28 $k_{\text{mk}} \leftarrow H(\text{seed}_{\text{mk}}, \text{aid}, \text{"mk"})$ 29 $\text{st}_C \leftarrow (\text{aid}, \text{st}_C^{\text{tmp}}.\text{sid}, k_{\text{mk}})$ 30 $\text{return } \text{success}_C$	$\Pi_{\text{auth}}^{S:1}[H, H_{\text{Auth}}](\text{st}_S, \varepsilon, \text{ins}, m_C)$ 31 $(\text{aid}, \text{auth}_1) \leftarrow m_C$ 32 if $\text{aid} \notin \text{st}_S.\text{acc}$ 33 fails 34 $\text{sid} \xleftarrow{\$} \text{NewID}$ 35 $(k, n_{\text{mk}}, \text{ct}_{\text{mk}}, k_{\text{mac}}) \leftarrow \text{st}_S.\text{acc}[\text{aid}]$ 36 $\text{auth}_2 \xleftarrow{\$} \text{Auth.Resp}[H_{\text{Auth}}](\text{auth}_1, k)$ 37 $m_S \leftarrow (\text{sid}, \text{auth}_2)$ 38 $\text{st}_S^{\text{tmp}} \leftarrow (\text{aid}, k_{\text{mac}}, n_{\text{mk}}, \text{ct}_{\text{mk}}, m_C, m_S)$ 39 return $(\text{st}_S, \text{st}_S^{\text{tmp}}, \varepsilon, m_S)$ $\Pi_{\text{auth}}^{S:2}[H, H_{\text{Auth}}](\text{st}_S, \text{st}_S^{\text{tmp}}, m_C)$ 40 $(\text{aid}, k_{\text{mac}}, n_{\text{mk}}, \text{ct}_{\text{mk}}, m'_C, m'_S) \leftarrow \text{st}_S^{\text{tmp}}$ 41 if $\text{MAC}.\text{Ver}(k_{\text{mac}}, (m'_C, m'_S), m_C) = 0$ 42 fails 43 $\text{st}_S.\text{sess}[\text{sid}] \leftarrow \text{aid}$ 44 $m_S \leftarrow (n_{\text{mk}}, \text{ct}_{\text{mk}})$ 45 $\text{success}_S(m_S)$
$\Pi_{\text{put}}^{C:1}[H, H_{\text{Auth}}](\text{st}_C, \varepsilon, \text{inc}, \varepsilon)$ 46 $(\text{fid}, \text{file}) \leftarrow \text{inc}$ 47 $(n_k, n_f) \xleftarrow{\$} \{0, 1\}^{n_l} \times \{0, 1\}^{n_l}$ 48 $\text{ad} \leftarrow (\text{st}_C.\text{aid}, \text{fid}, \text{"fk"})$ 49 $\text{seed}_k \xleftarrow{\$} \{0, 1\}^{k_l}$ 50 $\text{ct}_k \xleftarrow{\$} \text{AEAD}.\text{Enc}(\text{st}_C.k_{\text{mk}}, n_k, \text{seed}_k, \text{ad})$ 51 $k_f \xleftarrow{\$} H(\text{seed}_k, \text{fid}, \text{"fk"})$ 52 $\text{ct}_f \xleftarrow{\$} \text{AEAD}.\text{Enc}(k_f, n_f, \text{file}, (\text{fid}, \text{"file"}))$ 53 $m_C \leftarrow (\text{st}_C.\text{sid}, \text{fid}, n_k, n_f, \text{ct}_k, \text{ct}_f)$ 54 $\text{success}_C(m_C)$	$\Pi_{\text{put}}^{S:1}[H, H_{\text{Auth}}](\text{st}_S, \varepsilon, \text{ins}, m_C)$ 55 $(\text{sid}, \text{fid}, n_k, n_f, \text{ct}_k, \text{ct}_f) \leftarrow m_C$ 56 $\text{aid} \leftarrow \text{st}_S.\text{sess}[\text{sid}]$ 57 if $\text{fid} \in \text{st}_S.\text{owner}$ 58 fails 59 $\text{st}_S.\text{owner}[\text{fid}] \leftarrow \{\text{aid}\}$ 60 $\text{st}_S.\text{FK}[(\text{aid}, \text{fid})] \leftarrow (n_k, \text{ct}_k)$ 61 $\text{st}_S.\text{F}[\text{fid}] \leftarrow (n_f, \text{ct}_f)$ 62 success_S

Fig. 11. Protocols for account registration (areg), authentication (auth), and file upload (put). Our new authentication abstraction is highlighted in blue.

Theorem 6 (Integrity, Informal). *Let Auth be an authentication scheme that is δ_{Auth} -correct and INS-OW secure. Let AEAD be an AEAD scheme that is OW-CPA and INT-CTXT secure. Let MAC be a MAC scheme. Then $\text{CS} := \text{CS}[\text{Auth}, \text{AEAD}, \text{MAC}]$ is Int secure.*

Proof (Idea). The proof follows the same structure as for Theorem 10. After applying the same game changes, we are in a state where the seeds for all the file keys are uniformly random if the files are only owned by honest users. Instead of the AEAD's IND-CCA security, we can reduce to its INT-CTXT security. $\square$

$\Pi_{\text{upd}}^{C:1}[H, H_{\text{Auth}}](st_C, \varepsilon, inc, \varepsilon)$ 00 $mc \leftarrow \text{getHeader}^{C:1}(st_C, inc.fid)$ 01 $(st_C^{tmp}.fid, st_C^{tmp}.file) \leftarrow (inc.fid, inc.file)$ 02 return $(st_C, st_C^{tmp}, \varepsilon, mc)$ $\Pi_{\text{upd}}^{C:2}[H, H_{\text{Auth}}](st_C, st_C^{tmp}, ms)$ 03 $seed_k \leftarrow \text{getHeader}^{C:2}(st_C, st_C^{tmp}.fid, ms)$ 04 $k_f \leftarrow H(seed_k, st_C^{tmp}.fid, "fk")$ 05 $n \xleftarrow{\$} \{0, 1\}^{nl}$ 06 $ctr \xleftarrow{\$} \text{AEAD.Enc}(k_f, n, st_C^{tmp}.file, (st_C^{tmp}.fid, "file"))$ 07 $mc \leftarrow (st_C.sid, st_C^{tmp}.fid, n, ctr)$ 08 success $_C(mc)$	$\Pi_{\text{upd}}^{S:1}[H, H_{\text{Auth}}](st_S, \varepsilon, ins, mc)$ 09 $ms \leftarrow \text{getHeader}^S(st_S, mc)$ 10 return $(st_S, \varepsilon, \varepsilon, ms)$ $\Pi_{\text{upd}}^{S:2}[H, H_{\text{Auth}}](st_S, st_S^{tmp}, mc)$ 11 $(sid, fid, n, ctr) \leftarrow mc$ 12 $aid \leftarrow st_S.sess[sid]$ 13 if $aid \notin st_S.owner[fid]$ 14 return $\perp$ 15 $F[fid] \leftarrow (n, ctr)$ 16 success $_S$
$\Pi_{\text{get}}^{C:1}[H, H_{\text{Auth}}](st_C, \varepsilon, inc, \varepsilon)$ 17 $mc \leftarrow (st_C.sid, inc.fid)$ 18 $st_C^{tmp}.fid \leftarrow inc.fid$ 19 return $(st_C, \varepsilon, \varepsilon, mc)$ $\Pi_{\text{get}}^{C:2}[H, H_{\text{Auth}}](st_C, st_C^{tmp}, ms)$ 20 $((n_k, ct_k), (n_f, ctr)) \leftarrow ms$ 21 $ad \leftarrow (st_C.aid, st_C^{tmp}.fid, "fk")$ 22 $seed_k \leftarrow \text{AEAD.Dec}(st_C.k_{mk}, n_k, ct_k, ad)$ 23 $k_f \leftarrow H(seed_k, st_C.fid, "fk")$ 24 $out_C.file \leftarrow \text{AEAD.Dec}(k_f, n_f, ctr, (st_C^{tmp}.fid, "file"))$ 25 success $_C$	$\Pi_{\text{get}}^{S:1}[H, H_{\text{Auth}}](st_S, \varepsilon, ins, mc)$ 26 $(sid, fid) \leftarrow mc$ 27 $aid \leftarrow st_S.sess[sid]$ 28 if $aid \notin st_S.owner[fid]$ 29 return $\perp$ 30 $ms \leftarrow (FK[(aid, fid)], F[fid])$ 31 success $_S(ms)$
$\Pi_{\text{share}}^{C:1}[H, H_{\text{Auth}}](st_C, \varepsilon, inc, \varepsilon)$ 32 $fid \leftarrow inc.fid$ 33 $m' \leftarrow \text{getHeader}^{C:1}(st_C, fid)$ 34 $st_C^{tmp}.fid \leftarrow fid$ 35 $mc \leftarrow (m', inc.raid, fid)$ 36 return $(st_C, st_C^{tmp}, \varepsilon, mc)$ $\Pi_{\text{share}}^{C:2}[H, H_{\text{Auth}}](st_C, st_C^{tmp}, ms)$ 37 $seed_k \leftarrow \text{getHeader}^{C:2}(st_C, st_C^{tmp}.fid, ms)$ 38 $out_C.oob \leftarrow seed_k$ 39 success $_C$	$\Pi_{\text{share}}^{S:1}[H, H_{\text{Auth}}](st_S, \varepsilon, ins, mc)$ 40 $(m', raid, fid) \leftarrow mc$ 41 $ms \leftarrow \text{getHeader}^S(st_S, m')$ 42 $st_S.owner[fid] \leftarrow st_S.owner[fid] \cup \{raid\}$ 43 success $_S(ms)$
$\Pi_{\text{accept}}^{C:1}[H, H_{\text{Auth}}](st_C, \varepsilon, inc, \varepsilon)$ 44 $seed_k \leftarrow inc.oob$ 45 $n \xleftarrow{\$} \{0, 1\}^{nl}$ 46 $ad \leftarrow (st_C.aid, inc.fid, "fk")$ 47 $ct_k \xleftarrow{\$} \text{AEAD.Enc}(st_C.k_{mk}, n, seed_k, ad)$ 48 $mc \leftarrow (st_C.sid, inc.fid, n, ct_k)$ 49 success $_C(mc)$	$\Pi_{\text{accept}}^{S:1}[H, H_{\text{Auth}}](st_S, \varepsilon, ins, mc)$ 50 $(sid, fid, n, ct_k) \leftarrow mc$ 51 $aid \leftarrow st_S.sess[sid]$ 52 if $aid \notin st_S.owner[fid]$ 53 return $\perp$ 54 $FK[(aid, fid)] \leftarrow (n, ct_k)$ 55 success $_S$
$\text{getHeader}^{C:1}(st_C, fid)$ 56 $mc \leftarrow (st_C.sid, fid)$ 57 return mc $\text{getHeader}^{C:2}(st_C, fid, ms)$ 58 $(n_k, ct_k) \leftarrow ms$ 59 $ad \leftarrow (st_C.aid, fid, "fk")$ 60 $seed_k \leftarrow \text{AEAD.Dec}(st_C.k_{mk}, n_k, ct_k, ad)$ 61 return $seed_k$	$\text{getHeader}^S(st_S, mc)$ 62 $(sid, fid) \leftarrow mc$ 63 $aid \leftarrow st_S.sess[sid]$ 64 if $aid \notin st_S.owner[fid]$ 65 return $\perp$ 66 $ms \leftarrow (FK[(aid, fid)])$ 67 return ms

Fig. 12. Protocols for file update (upd), download (get), sharing (share), accepting (accept), and helper functions.

6 Network Adversary Model

The security model from [BDG⁺24] which we considered so far allows for malicious servers, also denoted as C2C (client-to-client) security. While this is a relevant model, it is also quite strong. For example, if the adversary controls the server, we cannot circumvent offline password guessing attacks since the server has all information except the password. This is reflected in the theorem bounds via the INS-OW term for Auth which is reduced to offline password guessing.

In addition to the malicious server model, [BDG⁺24] also discusses a weaker model against “external adversaries” where the adversary can still compromise clients but does not control the server anymore; denoted as C2S (client-to-server). While this model is weaker, it allows for better security bounds or advanced security guarantees. An example is the aforementioned problem with offline password guessing which can be avoided if the server cannot be compromised.

THE PROPOSED MODEL FROM [BDG⁺24]. For the weaker model, the authors propose that the server is honest but adversaries can compromise clients, and that there are secure channels between clients and server. Overall, this should give better security in the sense that an adversary might now require online queries for password guessing which is why they use 2HDH in their construction. Since we introduced the Auth primitive to abstract the authentication procedure, we aim to identify the correct security notion that is sufficient for the given setting, and can then think about appropriate instantiations. Considering the notions in Section 3, we can see that INS-OW as needed for the E2EE case is stronger than what we need, which was expected. The additionally defined notion OUT-OW is still too strong because for secure channels we do not need an INIT or FIN oracle. This is because compromised users can still only communicate with the server and not interfere or see the communication between honest clients and the server. For the sake of completeness, we present such an Auth notion in Appendix D along with an instantiation much simpler than 2HDH.

A NEW MODEL. We target a stronger model than the one just discussed. Instead of secure channels between clients and users, we assume an adversary controlling the whole network with one exception: adversaries are not allowed to see the communication of client and server during the account registration. This is similar to the setting of asymmetric password-authenticated key exchange protocols [BM93]. If we would allow that, an adversary could simply simulate the server and the model would not be different to the original C2C model.

This model lies in between that model and the weaker C2S model but allows for relying on online queries for password guesses. In detail, it is sufficient to rely on OUT-OW security of the Auth primitive which can be instantiated by 2HDH based on online queries.

EXPLICIT AUTHENTICATION. Another property of interest in the network model is *explicit authentication*. In the C2C setting, it does not make sense to formalize such a notion because only clients authenticate to the server and the server can be malicious. So why does it make sense to require explicit authentication in the network setting? The reason is that it is possible to reduce the security to online authentication queries for password guesses. In practice, that can be very useful because online queries can be easily restricted. A query to the auth oracle means that a user tries to log into an account. Failed attempts can only be recognized by an explicit authentication procedure. The same holds for successful attempts which can be useful in practical applications to notify clients via a different channel such that they can take action if the login was malicious.

6.1 Definitions

We give our new security definitions, strengthening external adversaries to a network adversary.

Definition 11 (Net-Conf). *Confidentiality for a cloud storage scheme CS is defined via the game in Figure 13. We define the advantage of an adversary $\mathcal{A}$ as*

$$\text{Adv}_{\text{CS}, \mathcal{PW}}^{\text{Q-Net-Conf}}(\mathcal{A}) := 2 \Pr [Q\text{-Net-Conf}_{\text{CS}, \mathcal{PW}}(\mathcal{A}) \Rightarrow 1] - 1,$$

for $Q = (Q_{\text{STEP}}, Q_{\text{STEPS}_1}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1}, Q_{\text{PUT}_1}, Q_{\text{UPD}_1}, Q_{\text{GET}_1}, Q_{\text{SHARE}_1}, Q_{\text{ACPT}_1}, Q_{\text{CHALL}}, Q_{\text{COMP}}, Q_{\text{RO}})$.

Game $Q\text{-Net-Conf}_{CS, \mathcal{PW}}(\mathcal{A})$ 00 $H \xleftarrow{\$} \mathcal{OS}$ 01 $(\text{pw}_1, \dots, \text{pw}_{Q_{\text{REG}_1}}) \xleftarrow{\$} \mathcal{PW}$ 02 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_{\text{ch}}, \mathcal{S}_{\text{chOwn}} \leftarrow \emptyset$ 03 $(n_c, n_p, n_r) \leftarrow (0, 0, 0) \quad \text{\textit{\textbackslash session, protocol execution,}}$ 04 $b \xleftarrow{\$} \{0, 1\} \quad \text{\textit{\textbackslash registration counter}}$ 05 $b^* \leftarrow \mathcal{A}^\mathcal{O}$ 06 if $\exists \text{aid} \in \mathcal{S}_{\text{chOwn}} : \text{aid} \in \mathcal{S}_{\text{cmp}} \vee T_{\text{uaid}}[\text{aid}] = \perp$ 07 return 0 08 return $\llbracket b^* = b \rrbracket$ Procedure $\text{Chk}(i_p, \text{out}_c, m^*)$ 09 $(i_c, \text{st}_c^{\text{tmp}}, \text{inc}, r) \leftarrow T_p[(\text{prot}, C, i_p)]$ 10 $\text{fid} \leftarrow \text{inc.fid}$ 11 if $\text{prot} = \text{areg}$ 12 if $\text{out}_c.\text{decision}$ 13 $T_{\text{uaid}}[\text{inc.aid}] \leftarrow \text{true} \quad \text{\textit{\textbackslash reg completed}}$ 14 else 15 $T_{\text{uaid}}[\text{inc.aid}] \leftarrow \perp \quad \text{\textit{\textbackslash allow new reg attempt}}$ 16 $T_{\text{areg}}[i_p] \leftarrow m^*$ 17 return $ m^* \quad \text{\textit{\textbackslash registration via secure channel}}$ 18 if $\text{prot} = \text{get}$: 19 if $\text{fid} \in \mathcal{S}_{\text{ch}}$ 20 $\text{out}_c.f \leftarrow \perp$ 21 return $(\text{out}_c.f, m^*)$ 22 if $\text{prot} = \text{share} \wedge \text{out}_c.\text{decision}$: 23 $T_{\text{oob}}[(\text{inc.raid}, \text{fid})] \leftarrow \text{out}_c.\text{oob}$ 24 if $\text{prot} = \text{accept} \wedge \text{out}_c.\text{decision}$: 25 $T_{\text{oob}}[(T_u[i_c], \text{fid})] \leftarrow \perp$ 26 return m^*	Oracle $\text{STEP}(\text{prot}, P \in \{C, S\}, i_p \in [n_p], m)$ 27 require $T_p[(\text{prot}, P, i_p)] \neq \perp$ 28 $(i_c, \text{st}_c^{\text{tmp}}, \text{in}, r) \leftarrow T_p[(\text{prot}, P, i_p)]$ 29 require $r \leq \Pi_{\text{prot}}.\text{SN}$ 30 if $P = C$ 31 $(T_{\text{stc}}[i_c], \text{st}_c^{\text{tmp}}, \text{out}_c, m^*) \leftarrow \Pi_{\text{prot}}^{C:r}[\text{H}](T_{\text{stc}}[i_c], \text{st}_c^{\text{tmp}}, m)$ 32 else 33 $(\text{st}_s, \text{st}_s^{\text{tmp}}, \text{out}_s, m^*) \leftarrow \Pi_{\text{prot}}^{S:r}[\text{H}](\text{st}_s, \text{st}_s^{\text{tmp}}, m)$ 34 $T_p[(\text{prot}, P, i_p)] \leftarrow (i_c, \text{st}_c^{\text{tmp}}, \text{in}, r + 1)$ 35 if $P = S$ 36 if $\text{prot} = \text{areg}$ 37 return $ m^* \quad \text{\textit{\textbackslash registration via secure channel}}$ 38 return m^* 39 return $\text{Chk}(\text{prot}, i_p, \text{out}_c, m^*)$ Oracle $\text{STEPS}_1(\text{prot}, i_p \in [n_p], m, \text{ins})$ 40 if $\text{prot} = \text{areg}$ 41 $m \leftarrow T_{\text{areg}}[i_p]$ 42 $(\text{st}_s, \text{st}_s^{\text{tmp}}, \text{out}_s, m^*) \leftarrow \Pi_{\text{prot}}^{S:1}[\text{H}](\text{st}_s, \varepsilon, \text{ins}, m)$ 43 $T_p[(\text{prot}, S, i_p)] \leftarrow (\varepsilon, \text{st}_s^{\text{tmp}}, \text{ins}, 2)$ 44 if $\text{prot} = \text{areg}$ 45 return $ m^* \quad \text{\textit{\textbackslash registration via secure channel}}$ 46 return m^*
--	--

Fig. 13. Network adversary confidentiality game for a cloud storage scheme CS. Adversary $\mathcal{A}$ has access to oracles $\mathcal{O} = \{\text{STEP}, \text{STEPS}_1, \text{REG}_1, \text{AUTH}_1, \text{PUT}_1, \text{UPD}_1, \text{GET}_1, \text{SHARE}_1, \text{ACPT}_1, \text{CHALL}, \text{COMP}, \text{RO}\}$ making at most $Q_{\mathcal{O}}$ queries to oracle $\mathcal{O}$. Oracles not shown here as well as the procedure Step_1 are defined as in Figure 9. Differences to Conf (Figure 9) are highlighted in blue.

In addition to the original C2C model from [BDG⁺24], we need to model that the adversary does not have control of server but can control the network. This means they can control all the communication between honest clients and servers which is modeled by providing the adversary not only oracles executing client's protocol's steps but also server's protocol's steps. In more detail, the adversary gets an additional oracle STEPS_1 to execute the server's first step of a protocol for some client message. The usual STEP can now also be executed for the server which was not necessary before. As mentioned in the model description, we do not want to allow the adversary to see or interfere during the registration procedure because this would not distinguish the model from the C2C model of [BDG⁺24]. This is modeled by not returning the real protocol messages when executing the registration protocol but only outputting the length which is usually leaked for secure channels. The winning condition for an adversary does not change ε compared to the original Conf notion (Figure 9).

Definition 12 (Net-Int). *Integrity for a cloud storage scheme CS is defined via the game in Figure 14. We define the advantage of an adversary $\mathcal{A}$ as*

$$\text{Adv}_{CS, \mathcal{PW}}^{Q\text{-Net-Int}}(\mathcal{A}) := \Pr [Q\text{-Net-Int}_{CS, \mathcal{PW}}(\mathcal{A}) \Rightarrow 1],$$

for $Q = (Q_{\text{STEP}}, Q_{\text{STEPS}_1}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1}, Q_{\text{PUT}_1}, Q_{\text{UPD}_1}, Q_{\text{GET}_1}, Q_{\text{SHARE}_1}, Q_{\text{ACPT}_1}, Q_{\text{COMP}}, Q_{\text{RO}})$.

The adversarial scenario is modeled in the same way as in Net-Conf and the winning condition is analogously taken from Int (Figure 10).

Game $Q\text{-Net-Int}_{CS, \mathcal{PW}}(\mathcal{A})$ 00 $H \xleftarrow{\$} \mathcal{OS}$ 01 $(pw_1, \dots, pw_{Q_{REG_1}}) \xleftarrow{\$} \mathcal{PW}$ 02 $\mathcal{S}_{cmp} \leftarrow \{i_{mal}\}$ 03 $(n_c, n_p, n_r) \leftarrow (0, 0, 0) \quad \text{\texttt{\textbackslash session, protocol execution,}}$ 04 $win \leftarrow false \quad \text{\texttt{\textbackslash registration counter}}$ 05 $\mathcal{A}^{\mathcal{O}}$ 06 return win Procedure $Chk(prot, i_p, out_c, m^*)$ 07 $(i_c, st_c^{tmp}, inc, r) \leftarrow T_p[(prot, i_p)]$ 08 $fid \leftarrow inc.fid$ 09 if $prot = areg$ 10 if $out_c.decision$ 11 $T_{uid}[inc.aid] \leftarrow true \quad \text{\texttt{\textbackslash reg completed}}$ 12 else 13 $T_{uid}[inc.aid] \leftarrow \perp \quad \text{\texttt{\textbackslash allow new reg attempt}}$ 14 $T_{areg}[i_p] \leftarrow m^*$ 15 return $ m^* \quad \text{\texttt{\textbackslash registration via secure channel}}$ 16 if $prot = get$: 17 if $T_u[i_c] \notin \mathcal{S}_{cmp} \wedge T_f[fid].U \cap \mathcal{S}_{cmp} = \emptyset \wedge \neg win$ 18 $file \leftarrow out_c.file$ 19 $win \leftarrow [out_c.decision \wedge file \neq \perp]$ 20 $\wedge [file \notin T_f[fid].F]$ 21 return $(out_c.f, m^*)$ 22 if $prot = share \wedge out_c.decision$: 23 $T_{oob}[(inc.raid, fid)] \leftarrow out_c.oob$ 24 if $prot = accept \wedge out_c.decision$: 25 $T_{oob}[(T_u[i_c], fid)] \leftarrow \perp$ 26 return m^*	Oracle $STEP(prot, P \in \{C, S\}, i_p \in [n_p], m)$ 26 require $T_p[(prot, P, i_p)] \neq \perp$ 27 $(i_c, st^{tmp}, in, r) \leftarrow T_p[(prot, P, i_p)]$ 28 require $r \leq \Pi_{prot}.SN$ 29 if $P = C$ 30 $(T_{stc}[i_c], st_c^{tmp}, out_c, m^*) \leftarrow \Pi_{prot}^{C:r}[H](T_{stc}[i_c], st^{tmp}, m)$ 31 else 32 $(st_s, st^{tmp}, out, m^*) \leftarrow \Pi_{prot}^{S:r}[H](st_s, st^{tmp}, m)$ 33 $T_p[(prot, P, i_p)] \leftarrow (i_c, st^{tmp}, in, r + 1)$ 34 if $P = S$ 35 if $prot = areg$ 36 return $ m^* \quad \text{\texttt{\textbackslash registration via secure channel}}$ 37 return m^* 38 return $Chk(prot, i_p, out_c, m^*)$ Oracle $STEPS_1(prot, i_p \in [n_p], m, ins)$ 39 if $prot = areg$ 40 $m \leftarrow T_{areg}[i_p]$ 41 $(st_s, st^{tmp}, out, m^*) \leftarrow \Pi_{prot}^{S:1}[H](st_s, \varepsilon, ins, m)$ 42 $T_p[(prot, S, i_p)] \leftarrow (\varepsilon, st^{tmp}, ins, 2)$ 43 if $prot = areg$ 44 return $ m^* \quad \text{\texttt{\textbackslash registration via secure channel}}$ 45 return m^*
---	--

Fig. 14. Network adversary integrity game for a cloud storage scheme CS. Adversary $\mathcal{A}$ has access to oracles $\mathcal{O} = \{STEP, STEPS_1, REG_1, AUTH_1, PUT_1, UPD_1, GET_1, SHARE_1, ACPT_1, COMP, RO\}$ making at most Q_O queries to oracle $\mathcal{O}$. Oracles not shown here as well as the procedure $Step_1$ are defined as in Figure 10. Differences to Int (Figure 10) are highlighted in blue.

Definition 13 (Exp-Auth). *Explicit authentication for a cloud storage scheme CS is defined via the game in Figure 15. We define the advantage of an adversary $\mathcal{A}$ as*

$$\text{Adv}_{CS}^{Q\text{-Exp-Auth}}(\mathcal{A}) := \Pr[Q\text{-Exp-Auth}_{CS}(\mathcal{A}) \Rightarrow 1],$$

for $Q = (Q_{STEP}, Q_{STEPS_1}, Q_{REG_1}, Q_{AUTH_1}, Q_{PUT_1}, Q_{UPD_1}, Q_{GET_1}, Q_{SHARE_1}, Q_{ACPT_1}, Q_{COMP}, Q_{RO})$.

The adversarial model is the same as for the previous notions and the adversary has access to the same oracles. The only possibility for the adversary to win is if an execution of the authentication protocol succeeds for an uncompromised user and the corresponding message transcript for that authentication session was not just forwarded from the honest client and server executions. The last condition is a triviality condition because the adversary has access to the whole network and can therefore decide to forward messages from honest clients and the server. These messages will obviously lead to a successful protocol execution.

6.2 Security

Similar to the previous section, we state the informal theorems here. The formal theorems can be found as Theorems 12 to 14 in Appendix E, where we also give the full proofs.

Theorem 7 (Net-Conf, Informal). *Let Auth be an authentication scheme that is δ_{Auth} -correct and OUT-OW secure. Let AEAD be an AEAD scheme that is OW-CPA, IND-CCA and INT-CTXT secure. Let MAC be a MAC scheme. Then $CS := CS[Auth, AEAD, MAC]$ is Int secure.*

Proof (Sketch). Applying OUT-OW in the same way as for the proofs of Theorem 12 and Theorem 13. This allows us to have a uniformly random MAC key for the following reduction to SUF. An additional challenge to reduce the security of the MAC is that during the registration procedure, the client sends the MAC key to the server. We can simulate this step because the security model assumes a secure channel during registration and the length of the MAC key is fixed. If the adversary wins the Exp-Auth game, it must be with a new message or tag for the MAC because just relaying transcripts from the client/server interaction does not yield a valid winning condition. $\square$

7 Conclusion and Discussion

We have presented several security definitions and proofs for the cloud storage scheme. Here, we want to summarize and discuss the results and implications for the scheme when instantiated with 2HDH as the authentication protocol, as originally suggested by [BDG⁺24], plus the changes we employed to achieve security in the presence of adaptive corruptions.

For the overview in Sections 7.1 and 7.2, we want to minimize the number of variables. In particular, we set the number of users to $N := Q_{\text{REG}_1}$ (the number of initial registration queries) and the number of adaptive corruptions to $C := Q_{\text{COMP}} \leq N$. We unify the remaining queries, those to all the other oracles except for the random oracle(s), as *online queries*, i.e. $Q_{\text{ON}} := \max\{Q_{\text{STEP}}, Q_{\text{AUTH}_1}, Q_{\text{PUT}_1}, Q_{\text{UPD}_1}, Q_{\text{GET}_1}, Q_{\text{SHARE}_1}, Q_{\text{ACPT}_1}, Q_{\text{CHALL}}\}$, and the random oracle queries as *offline queries*, i.e. $Q_{\text{OFF}} := Q_{\text{RO}}$. We give an interpretation of the assumptions in the two attacker models, and we then also provide a more general discussion in Section 7.3.

7.1 Client-to-Client Security

The following corollaries are obtained by combining Theorem 1 and Lemma 3 with Theorems 10 and 11, respectively, where we fold together adversaries against the same AEAD notions and upper bound the number of queries accordingly. We provide an interpretation of these bounds below.

Corollary 1 (Confidentiality). *For any password distribution $\mathcal{PW}$ and adversary $\mathcal{A}$ against the Conf security of $\text{CS} := \text{CS}[2\text{HDH}, \text{AEAD}, \text{MAC}]$, there exist a PW-Guess adversary $\mathcal{B}$ against $\mathcal{PW}$, an INT-CTXT adversary $\mathcal{C}$ against AEAD, an IND-PAS adversary $\mathcal{D}$ against AEAD, and an IND-CCA adversary $\mathcal{E}$ against AEAD with $t_{\mathcal{A}} \approx t_{\mathcal{B}} \approx t_{\mathcal{C}} - C \cdot Q_{\text{ON}} \approx t_{\mathcal{D}} - C \cdot Q_{\text{ON}} \approx t_{\mathcal{E}}$ such that*

$$\begin{aligned} \text{Adv}_{\text{CS}, \mathcal{PW}}^{Q_{\mathcal{A}}\text{-Conf}}(\mathcal{A}) &\leq 2 \cdot \text{Adv}_{\mathcal{PW}}^{Q_{\mathcal{B}}\text{-PW-Guess}}(\mathcal{B}) + 4 \cdot \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{C}}\text{-INT-CTXT}}(\mathcal{C}) + 4 \cdot \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{D}}\text{-IND-PAS}}(\mathcal{D}) + \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{E}}\text{-IND-CCA}}(\mathcal{E}) \\ &\quad + \frac{Q_{\text{ON}}^2}{2^{nl-3}} + \frac{Q_{\text{OFF}}}{2^{kl-4}}, \end{aligned}$$

with $Q_{\mathcal{A}} = (Q_{\text{ON}}, N, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, C, Q_{\text{OFF}})$ and

$$\begin{aligned} Q_{\mathcal{B}} &= (N, Q_{\text{OFF}}, C), \\ Q_{\mathcal{C}} &= (N, C, 3Q_{\text{ON}}, 4Q_{\text{ON}}), \\ Q_{\mathcal{D}} &= (N, C, 4Q_{\text{ON}}), \\ Q_{\mathcal{E}} &= (2Q_{\text{ON}}, 2Q_{\text{ON}}, 2Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}). \end{aligned}$$

Corollary 2 (Integrity). *For any password distribution $\mathcal{PW}$ and adversary $\mathcal{A}$ against the Int security of $\text{CS} := \text{CS}[2\text{HDH}, \text{AEAD}, \text{MAC}]$, there exist a PW-Guess adversary $\mathcal{B}$ against $\mathcal{PW}$, an INT-CTXT adversary $\mathcal{C}$ against AEAD, and an IND-PAS adversary $\mathcal{D}$ against AEAD with $t_{\mathcal{A}} \approx t_{\mathcal{B}} \approx t_{\mathcal{C}} - C \cdot Q_{\text{ON}} \approx t_{\mathcal{D}} - C \cdot Q_{\text{ON}}$ such that*

$$\text{Adv}_{\text{CS}, \mathcal{PW}}^{Q_{\mathcal{A}}\text{-Int}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{PW}}^{Q_{\mathcal{B}}\text{-PW-Guess}}(\mathcal{B}) + 3 \cdot \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{C}}\text{-INT-CTXT}}(\mathcal{C}) + 2 \cdot \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{D}}\text{-IND-PAS}}(\mathcal{D}) + \frac{Q_{\text{ON}}^2}{2^{nl-2}} + \frac{Q_{\text{OFF}}}{2^{kl-3}}$$

with $Q_A = (Q_{ON}, N, Q_{ON}, Q_{ON}, Q_{ON}, Q_{ON}, Q_{ON}, Q_{ON}, C, Q_{OFF})$ and

$$\begin{aligned} Q_B &= (N, Q_{OFF}, C), \\ Q_C &= (Q_{ON}, C, 2Q_{ON}, 3Q_{ON}), \\ Q_D &= (N, C, 3Q_{ON}). \end{aligned}$$

INTERPRETATION. Apart from statistical terms, the above stated theorems use multi-user AEAD notions and the password guessing advantage with adaptive compromises. For the latter, we have highlighted the number of guesses in gray. Observe that this number is linear in the number of offline queries, which can be quite large and therefore imply a significant security loss when considering low-entropy password distributions. This is, however, expected in this adversary model since the adversary controls the server and can therefore brute-force passwords offline.

With regard to the AEAD notions, we stress that they are all standard notions that are lifted to the multi-user setting. It is worth noting that IND-PAS is the potentially less standard multi-bit guessing model, cf. also [JSSOW17], whereas the IND-CCA notion is in the single-bit guessing model. This makes sense for the proof strategy and is the main reason how we can achieve security in the adaptive compromise setting. The IND-PAS security is used for the encrypted master keys (more concretely, the seed), whereas IND-CCA security is used for files. Clearly, it must be disallowed in the confidentiality game to compromise the user and ask for a challenge. However, we need to replace the master key by a random key independently of whether this key will be corrupted later or whether a file key (derived from the master key) will later be used to encrypt a challenge file. Both IND-PAS and IND-CCA can be shown to be implied non-tightly by the respective single-user variant. In terms of tightness, [JSSOW17] shows that a linear loss is inherent. However, since our games are parameterized by compromises, we can apply the result from [BRTZ24] to get a loss linear in the number of compromises for the IND-PAS advantage. Unfortunately, this does not work for the IND-CCA advantage in a straight-forward way. Modifying the proof in a non-trivial way, we conjecture however that it is possible here as well.

7.2 Security against Network Adversaries

The following corollaries are obtained by combining Theorem 2 and Lemma 3 with Theorems 12 to 14, respectively, where we fold together adversaries against the same AEAD notions and upper bound the number of queries accordingly. We provide an interpretation of these bounds below.

Corollary 3 (Net-Conf). *For any password distribution $\mathcal{PW}$ and adversary $\mathcal{A}$ against the Net-Conf security of $\text{CS} := \text{CS}[\text{2HDH}, \text{AEAD}, \text{MAC}]$, there exist a PW-Guess adversary $\mathcal{B}_1$ against $\mathcal{PW}$, a CT-OMCDH adversary $\mathcal{B}_2$ against $(\mathbb{G}, q, g)$, a CB-InvDH adversary $\mathcal{B}_3$ against $(\mathbb{G}, q, g)$, an INT-CTXT adversary $\mathcal{C}_1$ against AEAD, an OW-CPA adversary $\mathcal{D}$ against AEAD, and an IND-CCA adversary $\mathcal{E}$ against AEAD with $t_A \approx t_{B_1} \approx t_{B_2} \approx t_{B_3} - Q_{ON} \cdot Q_{OFF} \approx t_C - C \cdot Q_{ON} \approx t_D - C \cdot Q_{ON} \approx t_E$ such that*

$$\begin{aligned} \text{Adv}_{\text{CS}, \mathcal{PW}}^{Q_A\text{-Net-Conf}}(\mathcal{A}) &\leq 2 \cdot \text{Adv}_{\mathcal{PW}}^{Q_{B_1}\text{-PW-Guess}}(\mathcal{B}_1) + 2 \cdot \text{Adv}_{\mathbb{G}, q, g}^{Q_{B_2}\text{-CT-OMCDH}}(\mathcal{B}_2) + 2 \cdot \text{Adv}_{\mathbb{G}, q, g}^{Q_{B_3}\text{-CB-InvDH}}(\mathcal{B}_3) \\ &\quad + 4 \cdot \text{Adv}_{\text{AEAD}}^{Q_C\text{-INT-CTXT}}(\mathcal{C}) + 4 \cdot \text{Adv}_{\text{AEAD}}^{Q_D\text{-IND-PAS}}(\mathcal{D}) + \text{Adv}_{\text{AEAD}}^{Q_E\text{-IND-CCA}}(\mathcal{E}) \\ &\quad + \frac{Q_{ON}^2}{2^{nl-2}} + \frac{Q_{OFF}}{2^{kl-4}} \end{aligned}$$

with $Q_A = (Q_{\text{ON}}, N, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, C, Q_{\text{OFF}})$ and

$$\begin{aligned} Q_{B_1} &= (N, Q_{\text{ON}}, C) \\ Q_{B_2} &= (N, Q_{\text{OFF}}, C, Q_{\text{ON}}, 3Q_{\text{OFF}}) \\ Q_{B_3} &= (Q_{\text{ON}}, Q_{\text{OFF}}, 2Q_{\text{ON}}Q_{\text{OFF}}) \\ Q_C &= (N, C, 3Q_{\text{ON}}, 4Q_{\text{ON}}), \\ Q_D &= (N, C, 4Q_{\text{ON}}), \\ Q_E &= (2Q_{\text{ON}}, 2Q_{\text{ON}}, 2Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}). \end{aligned}$$

Corollary 4 (Net-Int). *For any password distribution PW and adversary $\mathcal{A}$ against the Net-Int security of $\text{CS} := \text{CS}[2\text{HDH}, \text{AEAD}, \text{MAC}]$, there exist a PW-Guess adversary $\mathcal{B}_1$ against PW , a CT-OMCDH adversary $\mathcal{B}_2$ against $(\mathbb{G}, q, g)$, a CB-InvDH adversary $\mathcal{B}_3$ against $(\mathbb{G}, q, g)$, an INT-CTXT adversary $\mathcal{C}$ against AEAD, and an IND-PAS adversary $\mathcal{D}$ against AEAD with $t_A \approx t_{B_1} \approx t_{B_2} \approx t_{B_3} - Q_{\text{ON}} \cdot Q_{\text{OFF}} \approx t_C - C \cdot Q_{\text{ON}} \approx t_D - C \cdot Q_{\text{ON}}$ such that*

$$\begin{aligned} \text{Adv}_{\text{CS}, PW}^{Q_A\text{-Net-Int}}(\mathcal{A}) &\leq \text{Adv}_{PW}^{Q_{B_1}\text{-PW-Guess}}(\mathcal{B}_1) + \text{Adv}_{\mathbb{G}, q, g}^{Q_{B_2}\text{-CT-OMCDH}}(\mathcal{B}_2) + \text{Adv}_{\mathbb{G}, q, g}^{Q_{B_3}\text{-CB-InvDH}}(\mathcal{B}_3) \\ &\quad + \text{Adv}_{\text{AEAD}}^{Q_C\text{-INT-CTXT}}(\mathcal{C}) + \text{Adv}_{\text{AEAD}}^{Q_D\text{-IND-PAS}}(\mathcal{D}) + \frac{Q_{\text{ON}}^2}{2^{nl-2}} + \frac{Q_{\text{OFF}}}{2^{kl-3}} \end{aligned}$$

with $Q_A = (Q_{\text{ON}}, Q_{\text{ON}}, N, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, C, Q_{\text{OFF}})$ and

$$\begin{aligned} Q_{B_1} &= (N, Q_{\text{ON}}, C) \\ Q_{B_2} &= (N, Q_{\text{OFF}}, C, Q_{\text{ON}}, 3Q_{\text{OFF}}) \\ Q_{B_3} &= (Q_{\text{ON}}, Q_{\text{OFF}}, 2Q_{\text{ON}}Q_{\text{OFF}}) \\ Q_C &= (Q_{\text{ON}}, C, 2Q_{\text{ON}}, 3Q_{\text{ON}}), \\ Q_D &= (N, C, 3Q_{\text{ON}}). \end{aligned}$$

Corollary 5 (Explicit Authentication). *For any password distribution PW and adversary $\mathcal{A}$ against the Exp-Auth security of $\text{CS} := \text{CS}[2\text{HDH}, \text{AEAD}, \text{MAC}]$, there exist a PW-Guess adversary $\mathcal{B}_1$ against PW , a CT-OMCDH adversary $\mathcal{B}_2$ against $(\mathbb{G}, q, g)$, a CB-InvDH adversary $\mathcal{B}_3$ against $(\mathbb{G}, q, g)$, and an SUF adversary $\mathcal{C}$ against MAC with $t_A \approx t_{B_1} \approx t_{B_2} \approx t_{B_3} - Q_{\text{ON}} \cdot Q_{\text{OFF}} \approx t_C$ such that*

$$\begin{aligned} \text{Adv}_{\text{CS}, PW}^{Q_A\text{-Exp-Auth}}(\mathcal{A}) &\leq \text{Adv}_{PW}^{Q_{B_1}\text{-PW-Guess}}(\mathcal{B}_1) + \text{Adv}_{\mathbb{G}, q, g}^{Q_{B_2}\text{-CT-OMCDH}}(\mathcal{B}_2) + \text{Adv}_{\mathbb{G}, q, g}^{Q_{B_3}\text{-CB-InvDH}}(\mathcal{B}_3) \\ &\quad + \text{Adv}_{\text{MAC}}^{Q_C\text{-SUF}}(\mathcal{C}) + \frac{Q_{\text{OFF}}}{2^{kl-3}} \end{aligned}$$

with $Q_A = (Q_{\text{ON}}, Q_{\text{ON}}, N, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, Q_{\text{ON}}, C, Q_{\text{OFF}})$ and

$$\begin{aligned} Q_{B_1} &= (N, Q_{\text{ON}}, C) \\ Q_{B_2} &= (N, Q_{\text{OFF}}, C, Q_{\text{ON}}, 3Q_{\text{OFF}}) \\ Q_{B_3} &= (Q_{\text{ON}}, Q_{\text{OFF}}, 2Q_{\text{ON}}Q_{\text{OFF}}) \\ Q_C &= (N, C, Q_{\text{ON}}, Q_{\text{ON}}). \end{aligned}$$

INTERPRETATION. The above statements show that in this weaker adversarial model, we can achieve better bounds for the password guessing advantage. The number of password guesses (highlighted in gray) now depends on online queries and, more specifically, only on the number of online authentication attempts. This is clearly desirable in practice and is state-of-the-art for other password-based primitives like password authenticated key exchange [BPR00]. We note that this property is due to the use of 2HDH. In a sense, we

trade offline password guessing for additional computational assumptions. We also highlight the number of EXP queries for the CT-OMCDH advantage since those can affect the concrete security, cf. the hierarchy of OMDH in the AGM [JX26]. While CB-InvDH is also a non-standard assumption, it does not have corruptions or a “one-more” oracle, therefore there is no additional loss to be expected against generic attacks (see also Remark 2).

7.3 Further Discussion and Future Work

We want to highlight that our new definitions for network adversaries are not meant to replace the original definitions of [BDG⁺24]. Our definitions themselves are not sufficient because they do not capture the scenario of a malicious server. For real-world systems, we would therefore recommend that they satisfy security in both adversary models.

It might be beneficial to present a combined notion for each property and prove that the two notions imply the combined one. However, given that the area is still early-stage, small changes would require to reprove that result. Therefore, we motivate more research on what are the actual requirements and whether the current security definitions still need slight adaptations.

Our contributions also provide the first step towards a post-quantum secure construction in the network adversary setting. We hope that our abstraction of the authentication protocol will allow for instantiations using post-quantum secure OPRFs (or variants thereof), which we leave as an interesting open problem. Similarly, there is currently no proof in the standard model. We do not see any reason why a standard-model proof should not work in the setting with selective corruptions. However, in the adaptive corruption setting, this seems much more challenging and a more promising direction could be a proof in the quantum-accessible random oracle model.

Acknowledgments

We thank the anonymous reviewers for their careful reading of this work and for pointing out several points that helped improve the paper. In particular, the argument for explicit authentication and the subtleties in the security model of BDGHP. Jonas Janneck was supported by the European Union (ERC AdG REWORC - 101054911).

References

- ABCP24. Martin R. Albrecht, Matilda Backendal, Daniele Coppola, and Kenneth G. Paterson. Share with care: Breaking E2EE in nextcloud. In *2024 IEEE European Symposium on Security and Privacy*, pages 828–840. IEEE Computer Society Press, July 2024. doi:10.1109/EuroSP60621.2024.00051. (Cited on p. 3).
- ABR01. Michel Abdalla, Mihir Bellare, and Phillip Rogaway. The oracle Diffie-Hellman assumptions and an analysis of DHIES. In David Naccache, editor, *CT-RSA 2001*, volume 2020 of *LNCS*, pages 143–158. Springer, Berlin, Heidelberg, April 2001. doi:10.1007/3-540-45353-9_12. (Cited on p. 9).
- AHMP23. Martin R. Albrecht, Miro Haller, Lenka Mareková, and Kenneth G. Paterson. Caveat implementor! Key recovery attacks on MEGA. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 190–218. Springer, Cham, April 2023. doi:10.1007/978-3-031-30589-4_7. (Cited on p. 3).
- App. Apple. Advanced data protection for icloud. <https://support.apple.com/guide/security/advance-d-data-protection-for-icloud-sec973254c5f/web>. Accessed: 2025-09-30. (Cited on p. 3).
- BCQ⁺13. Alysson Bessani, Miguel Correia, Bruno Quaresma, Fernando André, and Paulo Sousa. Depsky: Dependable and secure storage in a cloud-of-clouds. *ACM Trans. Storage*, 9(4), November 2013. doi:10.1145/2535929. (Cited on p. 7).
- BDFH25. Ward Beullens, Lucas Dodgson, Sebastian H. Faller, and Julia Hesse. The 2Hash OPRF framework and efficient post-quantum instantiations. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VIII*, volume 15608 of *LNCS*, pages 332–362. Springer, Cham, May 2025. doi:10.1007/978-3-031-91101-9_12. (Cited on p. 4).

- BDG⁺24. Matilda Backendal, Hannah Davis, Felix Günther, Miro Haller, and Kenneth G. Paterson. A formal treatment of end-to-end encrypted cloud storage. In Leonid Reyzin and Douglas Stebila, editors, *CRYPTO 2024, Part II*, volume 14921 of *LNCS*, pages 40–74. Springer, Cham, August 2024. doi:10.1007/978-3-031-68379-4_2. (Cited on pp. 3, 7, 10, 15, 16, 20, 23, 24, 27, 30, and 63).
- BFL20. Balthazar Bauer, Georg Fuchsbauer, and Julian Loss. A classification of computational assumptions in the algebraic group model. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 121–151. Springer, Cham, August 2020. doi:10.1007/978-3-030-56880-1_5. (Cited on p. 9).
- BFP21. Balthazar Bauer, Georg Fuchsbauer, and Antoine Plouviez. The one-more discrete logarithm assumption in the generic group model. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part IV*, volume 13093 of *LNCS*, pages 587–617. Springer, Cham, December 2021. doi:10.1007/978-3-030-92068-5_20. (Cited on p. 9).
- BHP23. Matilda Backendal, Miro Haller, and Kenneth G. Paterson. MEGA: Malleable encryption goes awry. In *2023 IEEE Symposium on Security and Privacy*, pages 146–163. IEEE Computer Society Press, May 2023. doi:10.1109/SP46215.2023.10179290. (Cited on p. 3).
- BLMR13. Dan Boneh, Kevin Lewi, Hart William Montgomery, and Ananth Raghunathan. Key homomorphic PRFs and their applications. In Ran Canetti and Juan A. Garay, editors, *CRYPTO 2013, Part I*, volume 8042 of *LNCS*, pages 410–428. Springer, Berlin, Heidelberg, August 2013. doi:10.1007/978-3-642-40041-4_23. (Cited on p. 7).
- BM93. Steven M. Bellovin and Michael Merritt. Augmented encrypted key exchange: A password-based protocol secure against dictionary attacks and password file compromise. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93*, pages 244–250. ACM Press, November 1993. doi:10.1145/168588.168618. (Cited on pp. 7 and 23).
- Bol03. Alexandra Boldyreva. Threshold signatures, multisignatures and blind signatures based on the gap-Diffie-Hellman-group signature scheme. In Yvo Desmedt, editor, *PKC 2003*, volume 2567 of *LNCS*, pages 31–46. Springer, Berlin, Heidelberg, January 2003. doi:10.1007/3-540-36288-6_3. (Cited on p. 9).
- BPR00. Mihir Bellare, David Pointcheval, and Phillip Rogaway. Authenticated key exchange secure against dictionary attacks. In Bart Preneel, editor, *EUROCRYPT 2000*, volume 1807 of *LNCS*, pages 139–155. Springer, Berlin, Heidelberg, May 2000. doi:10.1007/3-540-45539-6_11. (Cited on pp. 4 and 29).
- BR93. Mihir Bellare and Phillip Rogaway. Random oracles are practical: A paradigm for designing efficient protocols. In Dorothy E. Denning, Raymond Pyle, Ravi Ganesan, Ravi S. Sandhu, and Victoria Ashby, editors, *ACM CCS 93*, pages 62–73. ACM Press, November 1993. doi:10.1145/168588.168596. (Cited on p. 8).
- BR94. Mihir Bellare and Phillip Rogaway. Entity authentication and key distribution. In Douglas R. Stinson, editor, *CRYPTO’93*, volume 773 of *LNCS*, pages 232–249. Springer, Berlin, Heidelberg, August 1994. doi:10.1007/3-540-48329-2_21. (Cited on p. 4).
- BR06. Mihir Bellare and Phillip Rogaway. The security of triple encryption and a framework for code-based game-playing proofs. In Serge Vaudenay, editor, *EUROCRYPT 2006*, volume 4004 of *LNCS*, pages 409–426. Springer, Berlin, Heidelberg, May / June 2006. doi:10.1007/11761679_25. (Cited on p. 8).
- BRTZ24. Mihir Bellare, Doreen Riepel, Stefano Tessaro, and Yizhao Zhang. Count corruptions, not users: Improved tightness for signatures, encryption and authenticated key exchange. In Kai-Min Chung and Yu Sasaki, editors, *ASIACRYPT 2024, Part II*, volume 15485 of *LNCS*, pages 326–360. Springer, Singapore, December 2024. doi:10.1007/978-981-96-0888-1_11. (Cited on p. 28).
- BS23. Mihir Bellare and Laura Shea. Flexible password-based encryption: Securing cloud storage and provably resisting partitioning-oracle attacks. In Mike Rosulek, editor, *CT-RSA 2023*, volume 13871 of *LNCS*, pages 594–621. Springer, Cham, April 2023. doi:10.1007/978-3-031-30872-7_23. (Cited on p. 7).
- CHGY22. Weikeng Chen, Thang Hoang, Jorge Guajardo, and Attila A. Yavuz. Titanium: A metadata-hiding file-sharing system with malicious security. In *NDSS 2022*. The Internet Society, April 2022. (Cited on p. 7).
- CKM23. Elizabeth C. Crites, Chelsea Komlo, and Mary Maller. Fully adaptive Schnorr threshold signatures. In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part I*, volume 14081 of *LNCS*, pages 678–709. Springer, Cham, August 2023. doi:10.1007/978-3-031-38557-5_22. (Cited on p. 4).
- CLTY22. Long Chen, Ya-Nan Li, Qiang Tang, and Moti Yung. End-to-same-end encryption: Modularly augmenting an app with an efficient, portable, and blind cloud storage. In Kevin R. B. Butler and Kurt Thomas, editors, *USENIX Security 2022*, pages 2353–2370. USENIX Association, August 2022. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/chen-long>. (Cited on p. 7).

- CP20. Weikeng Chen and Raluca Ada Popa. Metal: A metadata-hiding file-sharing system. In *NDSS 2020*. The Internet Society, February 2020. [doi:10.14722/ndss.2020.24095](https://doi.org/10.14722/ndss.2020.24095). (Cited on p. 7).
- DGRW18. Yevgeniy Dodis, Paul Grubbs, Thomas Ristenpart, and Joanne Woodage. Fast message franking: From invisible salamanders to encryption. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part I*, volume 10991 of *LNCS*, pages 155–186. Springer, Cham, August 2018. [doi:10.1007/978-3-319-96884-1_6](https://doi.org/10.1007/978-3-319-96884-1_6). (Cited on p. 6).
- DHL22. Poulami Das, Julia Hesse, and Anja Lehmann. DPaSE: Distributed password-authenticated symmetric-key encryption, or how to get many keys from one password. In Yuji Suga, Kouichi Sakurai, Xuhua Ding, and Kazue Sako, editors, *ASIACCS 22*, pages 682–696. ACM Press, May / June 2022. [doi:10.1145/3488932.3517389](https://doi.org/10.1145/3488932.3517389). (Cited on p. 7).
- FL25. Karla Friedrichs, Anja Lehmann, and Cavit Özbay. Game changer: A modular framework for OPRF security. In *ASIACRYPT 2025*, *LNCS*. Springer, Singapore, December 2025. (Cited on p. 6).
- GJ18. Kristian Gjøsteen and Tibor Jager. Practical and tightly-secure digital signatures and authenticated key exchange. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 95–125. Springer, Cham, August 2018. [doi:10.1007/978-3-319-96881-0_4](https://doi.org/10.1007/978-3-319-96881-0_4). (Cited on p. 5).
- GSMB03. Eu-Jin Goh, Hovav Shacham, Nagendra Modadugu, and Dan Boneh. SiRiUS: Securing remote untrusted storage. In *NDSS 2003*. The Internet Society, February 2003. (Cited on p. 7).
- HKL⁺25. Lena Heimberger, Daniel Kales, Riccardo Lolato, Omid Mir, Sebastian Ramacher, and Christian Reiberger. Leap: A fast, lattice-based OPRF with application to private set intersection. In Serge Fehr and Pierre-Alain Fouque, editors, *EUROCRYPT 2025, Part VII*, volume 15607 of *LNCS*, pages 254–283. Springer, Cham, May 2025. [doi:10.1007/978-3-031-91098-2_10](https://doi.org/10.1007/978-3-031-91098-2_10). (Cited on p. 4).
- HR23. Nadia Heninger and Keegan Ryan. The hidden number problem with small unknown multipliers: Cryptanalyzing MEGA in six queries and other applications. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 147–176. Springer, Cham, May 2023. [doi:10.1007/978-3-031-31368-4_6](https://doi.org/10.1007/978-3-031-31368-4_6). (Cited on p. 3).
- HT24. Jonas Hofmann and Kien Tuong Truong. End-to-end encrypted cloud storage in the wild: A broken ecosystem. In Bo Luo, Xiaojing Liao, Jun Xu, Engin Kirda, and David Lie, editors, *ACM CCS 2024*, pages 3988–4001. ACM Press, October 2024. [doi:10.1145/3658644.3690309](https://doi.org/10.1145/3658644.3690309). (Cited on p. 3).
- Jae23. Joseph Jaeger. Let attackers program ideal models: Modularity and composability for adaptive compromise. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part III*, volume 14006 of *LNCS*, pages 101–131. Springer, Cham, April 2023. [doi:10.1007/978-3-031-30620-4_4](https://doi.org/10.1007/978-3-031-30620-4_4). (Cited on p. 5).
- Jae26. Joseph Jaeger. Nonadaptive one-way to hiding implies adaptive quantum reprogramming. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology – ASIACRYPT 2025*, pages 287–319, Singapore, 2026. Springer Nature Singapore. (Cited on p. 8).
- JKK14. Stanislaw Jarecki, Aggelos Kiayias, and Hugo Krawczyk. Round-optimal password-protected secret sharing and T-PAKE in the password-only model. In Palash Sarkar and Tetsu Iwata, editors, *ASIACRYPT 2014, Part II*, volume 8874 of *LNCS*, pages 233–253. Springer, Berlin, Heidelberg, December 2014. [doi:10.1007/978-3-662-45608-8_13](https://doi.org/10.1007/978-3-662-45608-8_13). (Cited on p. 3).
- JKR19. Stanislaw Jarecki, Hugo Krawczyk, and Jason K. Resch. Updatable oblivious key management for storage systems. In Lorenzo Cavallaro, Johannes Kinder, XiaoFeng Wang, and Jonathan Katz, editors, *ACM CCS 2019*, pages 379–393. ACM Press, November 2019. [doi:10.1145/3319535.3363196](https://doi.org/10.1145/3319535.3363196). (Cited on p. 7).
- JKRS21. Tibor Jager, Eike Kiltz, Doreen Riepel, and Sven Schäge. Tightly-secure authenticated key exchange, revisited. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 117–146. Springer, Cham, October 2021. [doi:10.1007/978-3-030-77870-5_5](https://doi.org/10.1007/978-3-030-77870-5_5). (Cited on p. 5).
- JSSOW17. Tibor Jager, Martijn Stam, Ryan Stanley-Oakes, and Bogdan Warinschi. Multi-key authenticated encryption with corruptions: Reductions are lossy. In Yael Kalai and Leonid Reyzin, editors, *TCC 2017, Part I*, volume 10677 of *LNCS*, pages 409–441. Springer, Cham, November 2017. [doi:10.1007/978-3-319-70500-2_14](https://doi.org/10.1007/978-3-319-70500-2_14). (Cited on pp. 9, 28, and 33).
- JT20. Joseph Jaeger and Nirvan Tyagi. Handling adaptive compromise for practical encryption schemes. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part I*, volume 12170 of *LNCS*, pages 3–32. Springer, Cham, August 2020. [doi:10.1007/978-3-030-56784-2_1](https://doi.org/10.1007/978-3-030-56784-2_1). (Cited on p. 5).
- JX26. Jake Januzelli and Jiayu Xu. A complete characterization of one-more assumptions in the algebraic group model. In Goichiro Hanaoka and Bo-Yin Yang, editors, *Advances in Cryptology – ASIACRYPT 2025*, pages 206–238, Singapore, 2026. Springer Nature Singapore. (Cited on pp. 9 and 30).

- LER⁺18. Russell W. F. Lai, Christoph Egger, Manuel Reinert, Sherman S. M. Chow, Matteo Maffei, and Dominique Schröder. Simple password-hardened encryption services. In William Enck and Adrienne Porter Felt, editors, *USENIX Security 2018*, pages 1405–1421. USENIX Association, August 2018. URL: <https://www.usenix.org/conference/usenixsecurity18/presentation/lai>. (Cited on p. 7).
- LESC17. Russell W. F. Lai, Christoph Egger, Dominique Schröder, and Sherman S. M. Chow. Phoenix: Rebirth of a cryptographic password-hardening service. In Engin Kirda and Thomas Ristenpart, editors, *USENIX Security 2017*, pages 899–916. USENIX Association, August 2017. URL: <https://www.usenix.org/conference/usenixsecurity17/technical-sessions/presentation/lai>. (Cited on p. 7).
- LGR21. Julia Len, Paul Grubbs, and Thomas Ristenpart. Partitioning oracle attacks. In Michael Bailey and Rachel Greenstadt, editors, *USENIX Security 2021*, pages 195–212. USENIX Association, August 2021. URL: <https://www.usenix.org/conference/usenixsecurity21/presentation/len>. (Cited on p. 6).
- LSTY25. Ya-Nan Li, Yaqing Song, Qiang Tang, and Moti Yung. End-to-end encrypted git services. In *ACM CCS 2025*. ACM Press, November 2025. (Cited on p. 7).
- MEG. MEGA. <https://mega.io>. Accessed: 2025-09-30. (Cited on p. 3).
- Nex. Nextcloud. <https://nextcloud.com>. Accessed: 2025-09-30. (Cited on p. 3).
- Nie02. Jesper Buus Nielsen. Separating random oracle proofs from complexity theoretic proofs: The non-committing encryption case. In Moti Yung, editor, *CRYPTO 2002*, volume 2442 of *LNCS*, pages 111–126. Springer, Berlin, Heidelberg, August 2002. doi:10.1007/3-540-45708-9_8. (Cited on p. 5).
- PSV⁺14. Raluca Ada Popa, Emily Stark, Steven Valdez, Jonas Helfer, Nikolai Zeldovich, and Hari Balakrishnan. Building web applications on top of encrypted data using mylar. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*, pages 157–172, Seattle, WA, April 2014. USENIX Association. URL: <https://www.usenix.org/conference/nsdi14/technical-sessions/presentation/popa>. (Cited on p. 7).
- PW17. David Pointcheval and Guilin Wang. VTBPEKE: Verifier-based two-basis password exponential key exchange. In Ramesh Karri, Ozgur Sinanoglu, Ahmad-Reza Sadeghi, and Xun Yi, editors, *ASIACCS 17*, pages 301–312. ACM Press, April 2017. doi:10.1145/3052973.3053026. (Cited on p. 10).
- PWZ23. Jiaxin Pan, Benedikt Wagner, and Runzhi Zeng. Tighter security for generic authenticated key exchange in the QROM. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part IV*, volume 14441 of *LNCS*, pages 401–433. Springer, Singapore, December 2023. doi:10.1007/978-981-99-8730-6_13. (Cited on p. 8).
- TMRM18. Nirvan Tyagi, Muhammad Haris Mughees, Thomas Ristenpart, and Ian Miers. BurnBox: Self-revocable encryption in a world of compelled access. In William Enck and Adrienne Porter Felt, editors, *USENIX Security 2018*, pages 445–461. USENIX Association, August 2018. URL: <https://www.usenix.org/conference/usenixsecurity18/presentation/tyagi>. (Cited on p. 7).

A Additional Preliminaries

A.1 AEAD Security Notions

The following multi-user multi-bit security notion is taken from [JSSOW17] where it is called LRIND-PAS^{NR,n}. This stands for left-or-right indistinguishability under passive attacks for adversaries without nonce-reuse. They are defined in a multi-bit setting.

Definition 14 (IND-PAS). *Indistinguishability under chosen passive attacks for an AEAD is defined via the game in Figure 16. We define the advantage of an adversary $\mathcal{A}$ as*

$$\text{Adv}_{\text{AEAD}}^{Q\text{-IND-PAS}}(\mathcal{A}) := 2 \Pr[Q\text{-IND-PAS}_{\text{AEAD}}(\mathcal{A}) \Rightarrow 1] - 1 ,$$

where $Q = (n, Q_{\text{COMP}}, Q_{\text{CHL}})$.

For our proofs we actually need two other variants. First, a standard IND variant in the single-bit setting with a separate challenge oracle. Second, a one-wayness notion, as captured in Definition 16. The relations can be found in Appendix A.2.

Game $Q\text{-IND-PAS}_{\text{AEAD}}(\mathcal{A})$ 00 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_n \leftarrow \emptyset$ 01 $\mathbf{b}_1, \dots, \mathbf{b}_n \xleftarrow{\$} \{0, 1\}$ 02 for $i \in [n]$ 03 $\mathbf{k}_i \xleftarrow{\$} \{0, 1\}^{\text{kl}}$ 04 $(i, \mathbf{b}') \xleftarrow{\$} \mathcal{A}^{\text{CHL, COMP}}$ 05 return $\llbracket \mathbf{b}_i = \mathbf{b}' \wedge i \notin \mathcal{S}_{\text{cmp}} \rrbracket$ Oracle $\text{COMP}(i \in [n])$ 06 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 07 return $\mathbf{k}_i$	Oracle $\text{CHL}(i \in [n], n, \mathbf{m}_0, \mathbf{m}_1, \text{ad})$ 08 if $(i, n) \in \mathcal{S}_n$ 09 return $\perp$ 10 $\mathcal{S}_n \leftarrow \mathcal{S}_n \cup \{(i, n)\}$ 11 $\text{ct} \leftarrow \text{Enc}(\mathbf{k}_i, n, \mathbf{m}_{\mathbf{b}_i}, \text{ad})$ 12 return ct
---	--

Fig. 16. Game defining IND-PAS security for an AEAD scheme $\text{AEAD} := (\text{Enc}, \text{Dec})$ with adversary $\mathcal{A}$ making at most Q_{COMP} queries to COMP and at most Q_{CHL} queries to CHL.

Definition 15 (IND-CCA). *Indistinguishability under chosen ciphertext attacks for an AEAD is defined via the game in Figure 17. We define the advantage of an adversary $\mathcal{A}$ as*

$$\text{Adv}_{\text{AEAD}}^{Q\text{-IND-CCA}}(\mathcal{A}) := 2 \Pr [Q\text{-IND-CCA}_{\text{AEAD}}(\mathcal{A}) \Rightarrow 1] - 1,$$

where $Q = (n, Q_{\text{COMP}}, Q_{\text{ENC}}, Q_{\text{DEC}}, Q_{\text{CHL}})$.

Game $Q\text{-IND-CCA}_{\text{AEAD}}(\mathcal{A})$ 00 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_n, \mathcal{S}_{\text{chl}} \leftarrow \emptyset$ 01 $\mathbf{b} \xleftarrow{\$} \{0, 1\}$ 02 for $i \in [n]$ 03 $\mathbf{k}_i \xleftarrow{\$} \{0, 1\}^{\text{kl}}$ 04 $\mathbf{b}' \xleftarrow{\$} \mathcal{A}^{\text{ENC, DEC, CHL, COMP}}$ 05 return $\llbracket \mathbf{b} = \mathbf{b}' \wedge \mathcal{S}_{\text{chl}} \cap \mathcal{S}_{\text{cmp}} = \emptyset \rrbracket$ Oracle $\text{CHL}(i \in [n], n, \mathbf{m}_0, \mathbf{m}_1, \text{ad})$ 06 if $(i, n) \in \mathcal{S}_n$ 07 return $\perp$ 08 $\mathcal{S}_n \leftarrow \mathcal{S}_n \cup \{(i, n)\}$ 09 $\mathcal{S}_{\text{chl}} \leftarrow \mathcal{S}_{\text{chl}} \cup \{i\}$ 10 $\text{ct} \leftarrow \text{Enc}(\mathbf{k}_i, n, \mathbf{m}_{\mathbf{b}}, \text{ad})$ 11 $\text{T}_{\text{Enc}}[i, n, \text{ct}, \text{ad}] \leftarrow \mathbf{m}$ 12 return ct	Oracle $\text{ENC}(i \in [n], n, \mathbf{m}, \text{ad})$ 13 if $(i, n) \in \mathcal{S}_n$ 14 return $\perp$ 15 $\mathcal{S}_n \leftarrow \mathcal{S}_n \cup \{(i, n)\}$ 16 $\text{ct} \leftarrow \text{Enc}(\mathbf{k}_i, n, \mathbf{m}, \text{ad})$ 17 $\text{T}_{\text{Enc}}[i, n, \text{ct}, \text{ad}] \leftarrow \mathbf{m}$ 18 return ct Oracle $\text{DEC}(i \in [n], n, \text{ct}, \text{ad})$ 19 if $\text{T}_{\text{Enc}}[i, n, \text{ct}, \text{ad}] \neq \perp$ 20 return $\perp$ 21 $\mathbf{m} \leftarrow \text{Dec}(\mathbf{k}_i, n, \text{ct}, \text{ad})$ 22 return $\mathbf{m}$ Oracle $\text{COMP}(i \in [n])$ 23 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 24 return $\mathbf{k}_i$
---	--

Fig. 17. Game defining IND-CCA security for an AEAD scheme $\text{AEAD} := (\text{Enc}, \text{Dec})$ with $Q = (n, Q_{\text{COMP}}, Q_{\text{ENC}}, Q_{\text{DEC}}, Q_{\text{CHL}})$ and adversary $\mathcal{A}$ making at most Q_{O} queries to oracle O.

Definition 16 (OW-CPA). *One-wayness under chosen plaintext attacks for an AEAD and a message length ml is defined via the game in Figure 18. We define the advantage of an adversary $\mathcal{A}$ as*

$$\text{Adv}_{\text{AEAD, ml}}^{Q\text{-OW-CPA}}(\mathcal{A}) := \Pr [Q\text{-OW-CPA}_{\text{AEAD, ml}}(\mathcal{A}) \Rightarrow 1],$$

where $Q = (n, Q_{\text{COMP}}, Q_{\text{ENC}}, Q_{\text{CHL}}, Q_{\text{CHECK}})$.

Definition 17 (INT-CTXT). *Ciphertext integrity is defined via the game in Figure 19. We define the advantage of an adversary $\mathcal{A}$ as*

$$\text{Adv}_{\text{AEAD}}^{Q\text{-INT-CTXT}}(\mathcal{A}) := \Pr [Q\text{-INT-CTXT}_{\text{AEAD}}(\mathcal{A}) \Rightarrow 1],$$

Game $Q\text{-OW-CPA}_{\text{AEAD}, \text{ml}}(\mathcal{A})$ 00 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_n \leftarrow \emptyset$ 01 for $i \in [n]$ 02 $k_i \xleftarrow{\$} \{0, 1\}^{\text{kl}}$ 03 $(i^*, n^*, \text{ct}^*, \text{ad}^*, m^*) \xleftarrow{\$} \mathcal{A}^{\text{ENC}, \text{CHL}, \text{COMP}, \text{CHECK}}$ 04 return $\llbracket m^* \neq \perp \wedge m^* = \text{T}_{\text{CHL}}[i^*, n^*, \text{ct}^*, \text{ad}^*] \rrbracket$ $\wedge \llbracket i^* \notin \mathcal{S}_{\text{cmp}} \rrbracket$ Oracle $\text{CHL}(i \in [n], n, j, \text{ad})$ 05 if $(i, n) \in \mathcal{S}_n$ 06 return $\perp$ 07 $\mathcal{S}_n \leftarrow \mathcal{S}_n \cup \{(i, n)\}$ 08 if $\text{T}_m[j] = \perp$ 09 $\text{T}_m[j] \xleftarrow{\$} \{0, 1\}^{\text{ml}}$ 10 $m \leftarrow \text{T}_m[j]$ 11 $\text{ct} \leftarrow \text{Enc}(k_i, n, m, \text{ad})$ 12 $\text{T}_{\text{CHL}}[i, n, \text{ct}, \text{ad}] \leftarrow m$ 13 return ct	Oracle $\text{ENC}(i \in [n], n, m, \text{ad})$ 14 if $(i, n) \in \mathcal{S}_n$ 15 return $\perp$ 16 $\mathcal{S}_n \leftarrow \mathcal{S}_n \cup \{(i, n)\}$ 17 $\text{ct} \leftarrow \text{Enc}(k_i, n, m, \text{ad})$ 18 return ct Oracle $\text{COMP}(i \in [n])$ 19 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 20 return k_i Oracle $\text{CHECK}(i \in [n], n, \text{ct}, \text{ad}, m)$ 21 if $\text{T}_{\text{CHL}}[i, n, \text{ct}, \text{ad}] = \perp$ 22 return $\perp$ 23 return $\llbracket m = \text{Dec}(k_i, n, \text{ct}, \text{ad}) \rrbracket$
---	--

Fig. 18. Game defining OW-CPA security for an AEAD scheme $\text{AEAD} := (\text{Enc}, \text{Dec})$ and message length ml with $Q = (n, Q_{\text{COMP}}, Q_{\text{ENC}}, Q_{\text{CHL}}, Q_{\text{CHECK}})$ and adversary $\mathcal{A}$ making at most Q_0 queries to oracle $\mathcal{O}$.

Game $Q\text{-INT-CTXT}_{\text{AEAD}}(\mathcal{A})$ 00 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_n \leftarrow \emptyset$ 01 for $i \in [n]$ 02 $k_i \xleftarrow{\$} \{0, 1\}^{\text{kl}}$ 03 $(i^*, n^*, \text{ct}^*, \text{ad}^*) \xleftarrow{\$} \mathcal{A}^{\text{ENC}, \text{COMP}, \text{CHECK}}$ 04 return $\llbracket \text{Dec}(k_{i^*}, n^*, \text{ct}^*, \text{ad}^*) \neq \perp \rrbracket$ 05 $\wedge \llbracket \text{T}_{\text{ENC}}[i^*, n^*, \text{ct}^*, \text{ad}^*] = \perp \wedge i^* \notin \mathcal{S}_{\text{cmp}} \rrbracket$ Oracle $\text{COMP}(i \in [n])$ 06 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 07 return k_i	Oracle $\text{ENC}(i \in [n], n, m, \text{ad})$ 08 if $(i, n) \in \mathcal{S}_n$ 09 return $\perp$ 10 $\mathcal{S}_n \leftarrow \mathcal{S}_n \cup \{(i, n)\}$ 11 $\text{ct} \leftarrow \text{Enc}(k_i, n, m, \text{ad})$ 12 $\text{T}_{\text{ENC}}[i, n, \text{ct}, \text{ad}] \leftarrow 1$ 13 return ct Oracle $\text{CHECK}(i \in [n], n, \text{ct}, \text{ad})$ 14 return $\llbracket \text{Dec}(k_i, n, \text{ct}, \text{ad}) \neq \perp \rrbracket$
--	---

Fig. 19. Game defining INT-CTXT security for an AEAD scheme $\text{AEAD} := (\text{Enc}, \text{Dec})$ with $Q = (n, Q_{\text{COMP}}, Q_{\text{ENC}}, Q_{\text{CHECK}})$ and adversary $\mathcal{A}$ making at most Q_0 queries to oracle $\mathcal{O}$.

for $Q = (n, Q_{\text{COMP}}, Q_{\text{ENC}}, Q_{\text{CHECK}})$.

A.2 Relation for OW-CPA

Lemma 3 ($\text{IND-PAS} \Rightarrow \text{OW-CPA}$). *If AEAD is correct, then for any adversary $\mathcal{A}$ against OW-CPA there exists an adversary $\mathcal{B}$ against IND-PAS with $t_{\mathcal{A}} \approx t_{\mathcal{B}}$ such that*

$$\text{Adv}_{\text{AEAD}, \text{ml}}^{Q\text{-OW-CPA}}(\mathcal{A}) \leq \text{Adv}_{\text{AEAD}}^{(n, Q_{\text{COMP}}, Q_{\text{ENC}} + Q_{\text{CHL}})\text{-IND-PAS}}(\mathcal{B}) + \frac{Q_{\text{CHL}} + Q_{\text{CHECK}}}{2^{\text{ml}}},$$

with $Q = (n, Q_{\text{COMP}}, Q_{\text{ENC}}, Q_{\text{CHL}}, Q_{\text{CHECK}})$.

Proof. To prove the statement, we depict a sequence of games in Figure 20.

Game G_0 . This is the $Q\text{-OW-CPA}$ game for $Q = (n, Q_{\text{COMP}}, Q_{\text{ENC}}, Q_{\text{CHL}}, Q_{\text{CHECK}})$:

$$\Pr[G_0(\mathcal{A}) \Rightarrow 1] = \text{Adv}_{\text{AEAD}, \text{ml}}^{Q\text{-OW-CPA}}(\mathcal{A}).$$

Game $G_0 - G_4$ 00 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_n \leftarrow \emptyset$ 01 for $i \in [n]$ 02 $k_i \xleftarrow{\$} \{0, 1\}^{\text{kl}}$ 03 $(i^*, n^*, \text{ct}^*, \text{ad}^*, m^*) \xleftarrow{\$} \mathcal{A}^{\text{ENC}, \text{CHL}, \text{COMP}, \text{CHECK}}$ 04 $m \leftarrow \text{T}_{\text{CHL}}[i^*, n^*, \text{ct}^*, \text{ad}^*]$ 05 $(m, \cdot) \leftarrow \text{T}_{\text{CHL}}[i^*, n^*, \text{ct}^*, \text{ad}^*]$ 06 return $\llbracket m^* \neq \perp \wedge m^* = m \wedge i^* \notin \mathcal{S}_{\text{cmp}} \rrbracket$ Oracle $\text{CHL}(i \in [n], n, j, \text{ad})$ 07 if $(i, n) \in \mathcal{S}_n$ 08 return $\perp$ 09 $\mathcal{S}_n \leftarrow \mathcal{S}_n \cup \{(i, n)\}$ 10 if $\text{T}_m[j] = \perp$ 11 $m \xleftarrow{\$} \{0, 1\}^{\text{ml}}$ 12 $\text{T}_m[j] \leftarrow m$ 13 $m_1 \xleftarrow{\$} \{0, 1\}^{\text{ml}}$ 14 $\text{T}_m[j] \leftarrow (m, m_1)$ 15 if $m = m_1$ 16 abort 17 $\text{ct} \leftarrow \text{Enc}(k_i, n, m, \text{ad})$ 18 $\text{T}_{\text{ENC}}[i, n, \text{ct}, \text{ad}] \leftarrow m$ 19 $\text{T}_{\text{CHL}}[i, n, \text{ct}, \text{ad}] \leftarrow \text{T}_m[j]$ 20 return ct	Oracle $\text{ENC}(i \in [n], n, m, \text{ad})$ 21 if $(i, n) \in \mathcal{S}_n$ 22 return $\perp$ 23 $\mathcal{S}_n \leftarrow \mathcal{S}_n \cup \{(i, n)\}$ 24 $\text{ct} \leftarrow \text{Enc}(k_i, n, m, \text{ad})$ 25 $\text{T}_{\text{ENC}}[i, n, \text{ct}, \text{ad}] \leftarrow m$ 26 return ct Oracle $\text{COMP}(i \in [n])$ 27 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 28 return k_i Oracle $\text{CHECK}(i \in [n], n, \text{ct}, \text{ad}, m)$ 29 if $\text{T}_{\text{CHL}}[i, n, \text{ct}, \text{ad}] = \perp$ 30 return $\perp$ 31 $(m', m_1) \leftarrow \text{T}_{\text{CHL}}[i, n, \text{ct}, \text{ad}]$ 32 if $m = m_1$ 33 abort 34 return $\llbracket m = m' \rrbracket$ 35 return $\llbracket m = \text{Dec}(k_i, n, \text{ct}, \text{ad}) \rrbracket$
--	---

Fig. 20. Games $G_0 - G_4$ for the proof of Lemma 3.

Game G_1 . This game is the same as the previous one except that the challenger is choosing an additional message m_1 in the challenge oracle. This message is drawn uniformly at random from the message space. Further, the new message is stored together with the old one in table T_{CHL} . Since this conceptual change does not influence the winning probability of the, we obtain

$$\Pr[G_0(\mathcal{A}) \Rightarrow 1] = \Pr[G_1(\mathcal{A}) \Rightarrow 1].$$

Game G_2 . This is the same game as before except that in the check oracle the challenger does not compute the real decryption of the input ciphertext but checks if the input message m equals the message stored in table T_{CHL} , i.e. the message that was encrypted in the challenge oracle. Note that the check oracle can only be queried on elements corresponding to a previous challenge query. Due to the assumption of (perfect) correctness of AEAD, this change is only conceptual:

$$\Pr[G_1(\mathcal{A}) \Rightarrow 1] = \Pr[G_2(\mathcal{A}) \Rightarrow 1].$$

Game G_3 . This game is the same as the previous one except that it aborts if the two messages chosen in the challenge oracle are the same. The collision probability for one query is $1/2^{\text{ml}}$ resulting in an overall bound of

$$\Pr[G_2(\mathcal{A}) \Rightarrow 1] - \Pr[G_3(\mathcal{A}) \Rightarrow 1] \leq \frac{Q_{\text{CHL}}}{2^{\text{ml}}}.$$

Game G_4 . This game is the same as the previous one except that it aborts if the check oracle is queried on the second message stored in T_{CHL} , i.e. m_1 .

Claim. It holds that

$$\Pr[G_2(\mathcal{A}) \Rightarrow 1] - \Pr[G_4(\mathcal{A}) \Rightarrow 1] \leq \frac{Q_{\text{CHECK}}}{2^{\text{ml}}}.$$

Adversary $\mathcal{B}^{\text{ENC}_{\mathcal{B}}, \text{COMP}_{\mathcal{B}}}$ 00 $(i^*, n^*, \text{ct}^*, \text{ad}^*, m^*) \xleftarrow{\$} \mathcal{A}^{\text{ENC}, \text{DEC}, \text{CHL}, \text{COMP}, \text{CHECK}}$ 01 $(m_0, m_1) \leftarrow T_{\text{Chl}}[i^*, n^*, \text{ct}^*, \text{ad}^*]$ 02 output $(i^*, \llbracket m^* = m_1 \rrbracket)$ Oracle $\text{CHL}(i \in [n], n, j, \text{ad})$ 03 if $T_m[j] = \perp$ 04 $(m_0, m_1) \xleftarrow{\$} \{0, 1\}^{\text{ml}} \times \{0, 1\}^{\text{ml}}$ 05 $T_m[j] \leftarrow (m_0, m_1)$ 06 if $m_0 = m_1$ 07 abort 08 $\text{ct} \leftarrow \text{ENC}_{\mathcal{B}}(i, n, m_0, m_1, \text{ad})$ 09 $T_{\text{Chl}}[i, n, \text{ct}, \text{ad}] \leftarrow T_m[j]$ 10 return ct	Oracle $\text{ENC}(i \in [n], n, m, \text{ad})$ 11 $\text{ct} \leftarrow \text{ENC}_{\mathcal{B}}(i, n, m, m, \text{ad})$ 12 return ct Oracle $\text{COMP}(i \in [n])$ 13 return $\text{COMP}_{\mathcal{B}}(i)$ Oracle $\text{CHECK}(i \in [n], n, \text{ct}, \text{ad}, m)$ 14 if $T_{\text{Chl}}[i, n, \text{ct}, \text{ad}] = \perp$ 15 return $\perp$ 16 $(m_0, m_1) \leftarrow T_{\text{Chl}}[i, n, \text{ct}, \text{ad}]$ 17 if $\exists b : m = m_b$ 18 output (i, b) 19 return 0
---	---

Fig. 21. Adversary $\mathcal{B}$ against IND-PAS having access to oracles $\text{ENC}_{\mathcal{B}}$ and $\text{COMP}_{\mathcal{B}}$ simulating $\mathcal{G}_4$ for adversary $\mathcal{A}$.

Proof (Claim). We need to upper bound the probability of an abort. Note that the second message, i.e. m_1 , is never encrypted nor used anywhere in the game except from being stored in T_{Chl} . This means that adversary $\mathcal{A}$ cannot have any knowledge about it and the best chances are to guess the correct message which yields a probability of at most $1/2^{\text{ml}}$. Considering Q_{CHECK} queries to CHECK yields the claimed bound. $\square$

Reduction to IND-PAS.

Claim. There exists an IND-PAS adversary $\mathcal{B}$ such that

$$\Pr[\mathcal{G}_4(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{\text{AEAD}}^{(n, Q_{\text{COMP}}, Q_{\text{ENC}} + Q_{\text{CHL}})\text{-IND-PAS}}(\mathcal{B}).$$

Proof (Claim). The reduction $\mathcal{B}$ is presented in Figure 21. Simulating $\mathcal{G}_4$ for adversary $\mathcal{A}$ is straightforward because the reduction can use their own encryption oracle. Note that the “real” encryption oracle can be simulated by calling $\mathcal{B}$ ’s (left-or-right) encryption oracle on $m_0 = m_1$. Further, $\mathcal{B}$ terminates and outputs either in the check oracle in case $\mathcal{A}$ is querying the oracle on a message stored in T_{Chl} for the corresponding query or after $\mathcal{A}$ returned their solution (Line 02).

Case 1: $\mathcal{B}$ outputs in CHECK: If $\mathcal{A}$ ’s query to the check oracle does not include the challenge message but the second message stored in T_{Chl} , $\mathcal{A}$ ’s game would abort due to the changes introduced in $\mathcal{G}_4$. Hence, the only possibility for $\mathcal{A}$ to win is to query the check oracle on message m where m was actually encrypted during the challenge. This leads to $\mathcal{B}$ winning their indistinguishability game since they choose the bit corresponding to the message queried by $\mathcal{A}$.

Case 2: $\mathcal{B}$ outputs after $\mathcal{A}$ terminated: If $\mathcal{A}$ wins their OW-CPA game, the output message m^* was the message used in the challenge oracle, i.e. ct^* is an encryption of m^* . If $m^* = m_1$, $\mathcal{B}$ is playing their IND-PAS game with $b_j = 1$. Hence, they also win their game.

In both cases it holds $\Pr[\mathcal{A} \text{ wins}] \leq \Pr[\mathcal{B} \text{ wins}]$ which concludes the proof by counting the oracle queries for $\mathcal{B}$. $\square$

Collecting the probabilities completes the proof of Lemma 3. $\square$

Game $Q\text{-SUF}_{\text{MAC}}(\mathcal{A})$ 00 for $i \in [n]$ 01 $k_i \xleftarrow{\$} \{0, 1\}^{\text{kl}}$ 02 win $\leftarrow \text{false}$ 03 $\mathcal{A}^{\text{TAG, VER, COMP}}$ 04 return win Oracle $\text{TAG}(i \in [n], m)$ 05 tag $\leftarrow \text{Tag}(k_i, m)$ 06 $\mathcal{S}_{\text{tag}} \leftarrow \mathcal{S}_{\text{tag}} \cup \{(i, m, \text{tag})\}$ 07 return tag	Oracle $\text{VER}(i \in [n], m, \text{tag})$ 08 $b \leftarrow \text{Ver}(k_i, m, \text{tag})$ 09 if $b \wedge i \notin \mathcal{S}_{\text{cmp}} \wedge (i, m, \text{tag}) \notin \mathcal{S}_{\text{tag}}$ 10 win $\leftarrow \text{true}$ 11 return b Oracle $\text{COMP}(i \in [n])$ 12 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 13 return k_i
---	---

Fig. 22. Game defining SUF security for a MAC scheme $\text{MAC} := (\text{Tag}, \text{Ver})$ with $Q = (n, Q_{\text{COMP}}, Q_{\text{TAG}}, Q_{\text{VER}})$ and adversary $\mathcal{A}$ making at most Q_{O} queries to oracle O .

A.3 Security Notion for MAC Schemes

Definition 18 (SUF). *Unforgeability for a MAC is defined via the game in Figure 22. We define the advantage of an adversary $\mathcal{A}$ as*

$$\text{Adv}_{\text{MAC}}^{Q\text{-SUF}}(\mathcal{A}) := \Pr[Q\text{-SUF}_{\text{MAC}}(\mathcal{A}) \Rightarrow 1] ,$$

where $Q = (n, Q_{\text{COMP}}, Q_{\text{TAG}}, Q_{\text{VER}})$.

B Omitted Proofs from Section 3

B.1 Proof of Theorem 1

Theorem 1 (INS-OW of 2HDH). *For any distribution $\mathcal{D}_{\text{C}}$ and adversary $\mathcal{A}$ against INS-OW of 2HDH $:= 2\text{HDH}[\mathbb{G}, q, g, \text{kl}]$ there exists an PW-Guess adversary $\mathcal{B}$ against $\mathcal{D}_{\text{C}}$ with $t_{\mathcal{A}} \approx t_{\mathcal{B}}$ such that*

$$\text{Adv}_{2\text{HDH}, \mathcal{D}_{\text{C}}}^{Q\text{-INS-OW}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{D}_{\text{C}}}^{(Q_{\text{REG}}, Q_{\text{RO}}, Q_{\text{COMP}})\text{-PW-Guess}}(\mathcal{B}) + \frac{Q_{\text{CHECK}} + 1}{2^{\text{kl}} - Q_{\text{CHECK}}}$$

with $Q = (Q_{\text{REG}}, Q_{\text{COMP}}, Q_{\text{INIT}}, Q_{\text{FIN}}, Q_{\text{CHECK}}, Q_{\text{RO}})$.

Proof. We use the sequence of games described in Figure 23.

Game $\mathcal{G}_0$. This is the INS-OW game for 2HDH and distribution $\mathcal{D}_{\text{C}}$.

Game $\mathcal{G}_1$. This is the same game as the previous one except that it introduces several conceptual changes. We introduce two new random oracles, RO'_1 and RO'_2 which behave exactly the same as RO_1 and RO_2 with the only difference that they maintain additional tables T'_1 and T'_2 which indicate that the query was made via the new oracle. The adversary still has access via the original random oracles but the new random oracles are called from all other oracle queries in the game. Hence, we mark if a query was made directly by the attacker or internally. Since the oracles still maintain the same list to answer queries, the change is only conceptual and it holds that

$$\Pr[\mathcal{G}_0(\mathcal{A}) \Rightarrow 1] = \Pr[\mathcal{G}_1(\mathcal{A}) \Rightarrow 1] .$$

Game $\mathcal{G}_2$. This is the same game as the previous one except that it aborts if random oracles RO_1 or RO_2 are queried on a registered and uncompromised auxiliary input aux and the input x is the correct client secret for that aux . The game further aborts in the registration oracle if there already exists a previous random oracle query on the correct client secret and the registered aux .

Game $G_0 - G_2$ <pre> 00 $\mathcal{S}_{\text{cmp}} \leftarrow \emptyset$ 01 $n_r \leftarrow 0$ 02 $(x_{C,1}, \dots, x_{C,Q_{\text{REG}}}) \xleftarrow{\\$} \mathcal{D}_C$ 03 for $i \in [Q_{\text{REG}}]$ 04 $x_{S,i} \xleftarrow{\\$} \mathbb{Z}_q$ 05 $(y^*, \text{aux}^*) \xleftarrow{\\$} \mathcal{A}^{\text{REG,INIT,FIN,CHL,RO}_1,\text{RO}_2}$ 06 if $T_{\text{aux}}[\text{aux}^*] = \perp$ return 0 07 $i^* \leftarrow T_{\text{aux}}[\text{aux}^*]$ 08 if $i^* \in \mathcal{S}_{\text{cmp}}$ return 0 09 return $\llbracket y^* = \text{RO}_2(x_{C,i^*}, \text{aux}^*, \text{RO}_1(x_{C,i^*}, \text{aux}^*)^{x_{S,i^*}}) \rrbracket \quad \parallel G_0$ 10 return $\llbracket y^* = \text{RO}'_2(x_{C,i^*}, \text{aux}^*, \text{RO}'_1(x_{C,i^*}, \text{aux}^*)^{x_{S,i^*}}) \rrbracket \quad \parallel G_1-G_2$ Oracle REG(aux) 11 require $T_{\text{aux}}[\text{aux}] = \perp$ 12 n_r++ 13 $T_{\text{aux}}[\text{aux}] \leftarrow n_r$ 14 if $(x_{C,n_r}, \text{aux}) \in T_1 \vee (x_{C,n_r}, \text{aux}, \cdot) \in T_2$ 15 abort 16 return x_{S,n_r} Oracle INIT(aux, j) 17 require $T_{\text{aux}}[\text{aux}] \neq \perp$ 18 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 19 $r \xleftarrow{\\$} \mathbb{Z}_q$ 20 $\alpha \leftarrow \text{RO}_1(x_{C,i}, \text{aux})^r$ 21 $\alpha \leftarrow \text{RO}'_1(x_{C,i}, \text{aux})^r$ 22 $T[i, j] \leftarrow (r, \text{aux})$ 23 return α Oracle FIN($\beta, i \in [n_r], j$) 24 if $T[i, j] = \perp$ return $\perp$ 25 $(r, \text{aux}) \leftarrow T[i, j]$ 26 $T[i, j] \leftarrow \perp$ 27 if $\beta \notin \mathbb{G}$ return $\perp$ 28 $\gamma \leftarrow \beta^{1/r}$ 29 $y \leftarrow \text{RO}_2(x_{C,i}, \text{aux}, \gamma)$ 30 $y \leftarrow \text{RO}'_2(x_{C,i}, \text{aux}, \gamma)$ 31 if $y = \text{RO}_2(x_{C,i}, \text{aux}, \text{RO}_1(x_{C,i}, \text{aux})^{x_{S,i}})$ return $\top$ 32 if $y = \text{RO}'_2(x_{C,i}, \text{aux}, \text{RO}'_1(x_{C,i}, \text{aux})^{x_{S,i}})$ return $\top$ 33 return y </pre>	Oracle COMP($i \in [n_r]$) <pre> 34 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 35 return $x_{C,i}$ Oracle RO₁(x, aux) 36 if $T_{\text{aux}}[\text{aux}] \neq \perp \wedge T_{\text{aux}}[\text{aux}] \notin \mathcal{S}_{\text{cmp}} \wedge x = x_{C,i}$ 37 abort 38 if $(x, \text{aux}) \notin T_1$ 39 $T_1[x, \text{aux}] \xleftarrow{\\$} \mathbb{G}$ 40 return $T_1[x, \text{aux}]$ Oracle RO'₁(x, aux) 41 if $(x, \text{aux}) \notin T_1$ 42 $T_1[x, \text{aux}] \xleftarrow{\\$} \mathbb{G}$ 43 $T'_1[x, \text{aux}] \leftarrow 1$ 44 return $T_1[x, \text{aux}]$ Oracle RO₂(x, aux, γ) 45 if $T_{\text{aux}}[\text{aux}] \neq \perp \wedge T_{\text{aux}}[\text{aux}] \notin \mathcal{S}_{\text{cmp}} \wedge x = x_{C,i}$ 46 abort 47 if $(x, \text{aux}, \gamma) \notin T_2$ 48 $T_2[x, \text{aux}, \gamma] \xleftarrow{\\$} \{0, 1\}^{\text{kl}}$ 49 return $T_2[x, \text{aux}, \gamma]$ Oracle RO'₂(x, aux, γ) 50 if $(x, \text{aux}, \gamma) \notin T_2$ 51 $T_2[x, \text{aux}, \gamma] \xleftarrow{\\$} \{0, 1\}^{\text{kl}}$ 52 $T'_2[x, \text{aux}, \gamma] \xleftarrow{\\$} 1$ 53 return $T_2[x, \text{aux}, \gamma]$ Oracle CHECK(y, aux) 54 if $T_{\text{aux}}[\text{aux}] = \perp$ return 0 55 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 56 return $\llbracket y = \text{RO}_2(x_{C,i}, \text{aux}, \text{RO}_1(x_{C,i}, \text{aux})^{x_{S,i}}) \rrbracket \quad \parallel G_0$ 57 return $\llbracket y = \text{RO}'_2(x_{C,i}, \text{aux}, \text{RO}'_1(x_{C,i}, \text{aux})^{x_{S,i}}) \rrbracket \quad \parallel G_1-G_2$ </pre>
---	--

Fig. 23. Games $G_0 - G_2$ for the proof of Theorem 1.

Claim. It holds that

$$\Pr[G_1(\mathcal{A}) \Rightarrow 1] - \Pr[G_2(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{\mathcal{D}_C}^{(Q_{\text{REG}}, Q_{\text{RO}}, Q_{\text{COMP}})\text{-PW-Guess}}(\mathcal{B}).$$

Proof (Claim). The reduction is formally depicted in Figure 24. The oracles are simulated by querying the internal random oracles on some unique tag “chl” which acts as a placeholder and is considered to be a value outside of the valid random oracle input space. This allows the reduction $\mathcal{B}$ to consistently answer oracle queries. In case of a compromise, the random oracles must be patched on the respective values. This is possible because the client secret is known to the reduction upon compromise and done in Line 36 and Line 49. Note that this simulation is sound because queries on the correct client secret *before* compromise are captured by the newly introduced abort condition. To capture this condition, reduction $\mathcal{B}$ uses their TEST oracle and therefore knows whenever the abort condition is met. Moreover, in such a case $\mathcal{B}$ correctly

Adversary $\mathcal{B}^{\text{TEST}, \text{COMP}_B}$ <pre> 00 $\mathcal{S}_{\text{cmp}} \leftarrow \emptyset$ 01 $n_r \leftarrow 0$ 02 for $i \in [Q_{\text{REG}}]$ 03 $x_{C,i} \leftarrow \perp$ 04 $x_{S,i} \xleftarrow{\\$} \mathbb{Z}_q$ 05 $(y^*, \text{aux}^*) \xleftarrow{\\$} \mathcal{A}^{\text{REG}, \text{INIT}, \text{FIN}, \text{CHECK}, \text{COMP}, \text{RO}_1, \text{RO}_2}$ 06 return $\perp$ Oracle REG(aux) 07 require $T_{\text{aux}}[\text{aux}] = \perp$ 08 n_r++ 09 $T_{\text{aux}}[\text{aux}] \leftarrow n_r$ 10 for $x : (x, \text{aux}) \in T_1 \vee (x, \text{aux}, \cdot) \in T_2$ 11 if $\text{TEST}(n_r, x)$ 12 output $\perp$ 13 return x_{S,n_r} Oracle INIT(aux, j) 14 require $T_{\text{aux}}[\text{aux}] \neq \perp$ 15 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 16 $r \xleftarrow{\\$} \mathbb{Z}_q$ 17 $\alpha \leftarrow \text{RO}'_1(\text{"chl"}, \text{aux})^r$ 18 $T[i, j] \leftarrow (r, \text{aux})$ 19 return α Oracle FIN($\beta, i \in [n_r], j$) 20 if $T[i, j] = \perp$ return $\perp$ 21 $(r, \text{aux}) \leftarrow T[i, j]$ 22 $T[i, j] \leftarrow \perp$ 23 if $\beta \notin \mathbb{G}$ return $\perp$ 24 $\gamma \leftarrow \beta^{1/r}$ 25 $y \leftarrow \text{RO}'_2(\text{"chl"}, \text{aux}, \gamma)$ 26 if $y = \text{RO}'_2(\text{"chl"}, \text{aux}, \text{RO}'_1(\text{"chl"}, \text{aux})^{x_{S,i}})$ return T 27 return y </pre>	Oracle COMP($i \in [n_r]$) <pre> 28 $x_{C,i} \leftarrow \text{COMP}_B(i)$ 29 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 30 return $x_{C,i}$ Oracle RO₁(x, aux) 31 if $T_{\text{aux}}[\text{aux}] \neq \perp \wedge T_{\text{aux}}[\text{aux}] \notin \mathcal{S}_{\text{cmp}}$ 32 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 33 if $\text{TEST}(i, x)$ 34 output $\perp$ 35 else if $T_{\text{aux}}[\text{aux}] \in \mathcal{S}_{\text{cmp}} \wedge (\text{"chl"}, \text{aux}) \in T'_1 \wedge x = x_{C,i}$ 36 $T_1[x, \text{aux}] \leftarrow T_1[\text{"chl"}, \text{aux}]$ 37 if $(x, \text{aux}) \notin T_1$ 38 $T_1[x, \text{aux}] \xleftarrow{\\$} \mathbb{G}$ 39 return $T_1[x, \text{aux}]$ Oracle RO₁'(x, aux) 40 if $(x, \text{aux}) \notin T_1$ 41 $T_1[x, \text{aux}] \xleftarrow{\\$} \mathbb{G}$ 42 $T'_1[x, \text{aux}] \leftarrow 1$ 43 return $T_1[x, \text{aux}]$ Oracle RO₂(x, aux, γ) 44 if $T_{\text{aux}}[\text{aux}] \neq \perp \wedge T_{\text{aux}}[\text{aux}] \notin \mathcal{S}_{\text{cmp}}$ 45 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 46 if $\text{TEST}(i, x)$ 47 output $\perp$ 48 else if $T_{\text{aux}}[\text{aux}] \in \mathcal{S}_{\text{cmp}} \wedge (\text{"chl"}, \text{aux}, \gamma) \in T'_2 \wedge x = x_{C,i}$ 49 $T_2[x, \text{aux}, \gamma] \leftarrow T_2[\text{"chl"}, \text{aux}, \gamma]$ 50 if $(x, \text{aux}, \gamma) \notin T_2$ 51 $T_2[x, \text{aux}, \gamma] \xleftarrow{\\$} \{0, 1\}^{\text{kl}}$ 52 return $T_2[x, \text{aux}, \gamma]$ Oracle RO₂'(x, aux, γ) 53 if $(x, \text{aux}, \gamma) \notin T_2$ 54 $T_2[x, \text{aux}, \gamma] \xleftarrow{\\$} \{0, 1\}^{\text{kl}}$ 55 $T'_2[x, \text{aux}, \gamma] \xleftarrow{\\$} 1$ 56 return $T_2[x, \text{aux}, \gamma]$ Oracle CHECK(y, aux) 57 if $T_{\text{aux}}[\text{aux}] = \perp$ return 0 58 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 59 return $\llbracket y = \text{RO}'_2(\text{"chl"}, \text{aux}, \text{RO}'_1(\text{"chl"}, \text{aux})^{x_{S,i}}) \rrbracket$ </pre>
--	---

Fig. 24. Adversary $\mathcal{B}$ against PW-Guess having access to oracles TEST, COMP_B simulating $\mathcal{G}_1/\mathcal{G}_2$ for adversary $\mathcal{A}$.

guessed an uncompromised password and thus wins their game. Note that $\mathcal{B}$ issues at most Q_{RO} queries to TEST because queries from REG and RO_1/RO_2 are mutually exclusive. $\square$

Final game.

Claim. It holds that

$$\Pr[\mathcal{G}_2(\mathcal{A}) \Rightarrow 1] \leq \frac{Q_{\text{CHECK}} + 1}{2^{\text{kl}} - Q_{\text{CHECK}}}.$$

Proof (Claim). The random oracle was never queried directly on the correct input due to the abort introduced in the previous game. Further, the correct output y^* was also never output by FIN because FIN

outputs $\top$ instead of the actual value in such a case. Hence, adversary $\mathcal{A}$'s view is independent of the output of RO_2 with the correct client secret on input. The only possibility for $\mathcal{A}$ to win is to guess the correct output in the space 2^{kl} . This is possible once per query to CHECK and for the final output resulting in $Q_{\text{CHECK}} + 1$ many guesses. Since one guess rules out one possible value, we need to sum up $\sum_{i=0}^{Q_{\text{CHECK}}} 1/(2^{\text{kl}} - i)$ which can be upper bound to obtain the claim. $\square$

Collecting the probabilities completes the proof of Theorem 1. $\square$

B.2 Proof of Theorem 2

Theorem 2 (OUT-OW of 2HDH). *For any distribution $\mathcal{D}_{\mathcal{C}}$ and adversary $\mathcal{A}$ against OUT-OW of 2HDH $:= 2\text{HDH}[\mathbb{G}, q, g, \text{kl}]$, there exist a CT-OMCDH adversary $\mathcal{B}$ against $(\mathbb{G}, q, g)$, a CB-InvDH adversary $\mathcal{C}$ against $(\mathbb{G}, q, g)$ and a PW-Guess adversary $\mathcal{D}$ against $\mathcal{D}_{\mathcal{C}}$ with $t_{\mathcal{A}} \approx t_{\mathcal{B}} \approx t_{\mathcal{C}} - Q_{\text{FIN}} Q_{\text{RO}} \approx t_{\mathcal{D}}$ such that*

$$\begin{aligned} \text{Adv}_{2\text{HDH}, \mathcal{D}_{\mathcal{C}}}^{Q\text{-OUT-OW}}(\mathcal{A}) &\leq \text{Adv}_{\mathbb{G}, q, g}^{(Q_{\text{REG}}, Q_{\text{RO}}, Q_{\text{COMP}}, Q_{\text{RESP}}, Q_{\text{RO}} + Q_{\text{CHECK}})\text{-CT-OMCDH}}(\mathcal{B}) + \text{Adv}_{\mathbb{G}, q, g}^{(Q_{\text{INIT}}, Q_{\text{RO}}, 2Q_{\text{FIN}} Q_{\text{RO}})\text{-CB-InvDH}}(\mathcal{C}) \\ &\quad + \text{Adv}_{\mathcal{D}_{\mathcal{C}}}^{(Q_{\text{REG}}, Q_{\text{RESP}}, Q_{\text{COMP}})\text{-PW-Guess}}(\mathcal{D}) + \frac{Q_{\text{CHECK}} + 1}{2^{\text{kl}} - Q_{\text{CHECK}}} + \frac{Q_{\text{RO}}^2}{2^{\text{kl}}} \end{aligned}$$

with $Q = (Q_{\text{REG}}, Q_{\text{COMP}}, Q_{\text{INIT}}, Q_{\text{RESP}}, Q_{\text{FIN}}, Q_{\text{CHECK}}, Q_{\text{RO}})$.

Proof. Sequence of games described in Figure 25.

Game $\mathcal{G}_0$. This is the OUT-OW game for 2HDH. In the oracle INIT , we make explicit the group element h output by random oracle RO_1 .

Game $\mathcal{G}_1$. We make the outputs of RO_2 unique. By the birthday bound, we get

$$\Pr[\mathcal{G}_0(\mathcal{A}) \Rightarrow 1] - \Pr[\mathcal{G}_1(\mathcal{A}) \Rightarrow 1] \leq \frac{Q_{\text{RO}}^2}{2^{\text{kl}}}.$$

Game $\mathcal{G}_2$. We make a case distinction in FIN . First, we introduce additional book-keeping to keep track of matching transcripts, i.e., the oracle RESP is called on α which was previously output by INIT , and the output β is then forwarded to FIN . By perfect correctness, the check whether $y = y'$ will always succeed, and therefore, FIN makes no additional computations and just outputs $\top$. Next, if i is corrupted and β is correctly computed, then FIN also outputs $\top$.

For the remaining cases, we proceed similar to the previous game. Note that, due to the previous game hop, we now always have $y \neq y'$, therefore we do not compute y' anymore and directly output y .

These are only conceptual changes and we have

$$\Pr[\mathcal{G}_1(\mathcal{A}) \Rightarrow 1] = \Pr[\mathcal{G}_2(\mathcal{A}) \Rightarrow 1].$$

Game $\mathcal{G}_3$. In this game, we introduce several new variables. First, we count queries to RESP for each user i via ctr_i . We also introduce a set $\mathcal{S}_{\text{guess}}$ that is filled during queries to random oracle RO_2 , i.e., via a query by the adversary, whenever (x, aux) was previously also queried to RO_1 .¹⁰ We will use this set to count guessing attempts for the target user. So far, all changes were only about book-keeping. The actual difference to the previous game is now that this game aborts if the set $\mathcal{S}'_{\text{guess}}$ is larger than the counter ctr_{i^*} .

Claim. There exists an adversary $\mathcal{B}$ against CT-OMCDH such that

$$\Pr[\mathcal{G}_2(\mathcal{A}) \Rightarrow 1] - \Pr[\mathcal{G}_3(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{\mathbb{G}, p, g}^{(Q_{\text{REG}}, Q_{\text{RO}}, Q_{\text{COMP}}, Q_{\text{RESP}}, Q_{\text{RO}} + Q_{\text{CHECK}})\text{-CT-OMCDH}}(\mathcal{B}).$$

Game $G_0 - G_6$		
00 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_{\text{resp}}, \mathcal{S}_{\text{fin}} \leftarrow \emptyset$		
01 $n_r \leftarrow 0$		
02 $(x_{C,1}, \dots, x_{C,Q_{\text{REG}}}) \xleftarrow{\$} \mathcal{D}_C$		
03 for $i \in [Q_{\text{REG}}]$		
04 $x_{S,i} \xleftarrow{\$} \mathbb{Z}_q$		
05 $\mathcal{S}_{\text{guess},i} \leftarrow \emptyset$	$\parallel G_3-G_6$	
06 $(y^*, \text{aux}^*) \xleftarrow{\$} \mathcal{A}^{\text{INIT,RESP,FIN,COMP,CHECK,RO}_1,\text{RO}_2}$		
07 if $T_{\text{aux}}[\text{aux}^*] = \perp$ return 0		
08 $i^* \leftarrow T_{\text{aux}}[\text{aux}^*]$		
09 if $i^* \in \mathcal{S}_{\text{cmp}}$ return 0		
10 return $\llbracket y^* = \text{RO}_2(x_{C,i^*}, \text{aux}^*, \text{RO}_1(x_{C,i^*}, \text{aux}^*)^{x_{S,i^*}}) \rrbracket$	$\parallel G_0-G_5$	
11 return $\llbracket y^* = \text{RO}_2'[\text{aux}^*] \rrbracket$	$\parallel G_6$	
Oracle REG(aux)		
12 require $T_{\text{aux}}[\text{aux}] = \perp$		
13 n_r++		
14 $T_{\text{aux}}[\text{aux}] \leftarrow n_r$		
15 $\text{ctr}_{n_r} \leftarrow 0$	$\parallel G_3-G_6$	
16 if $(x_{C,i}, \text{aux}, \cdot) \in T_2$	$\parallel G_6$	
17 abort	$\parallel G_6$	
Oracle INIT(aux, j)		
18 require $T_{\text{aux}}[\text{aux}] \neq \perp$		
19 $i \leftarrow T_{\text{aux}}[\text{aux}]$		
20 $r \xleftarrow{\$} \mathbb{Z}_q$	$\parallel G_0-G_3$	
21 $h \leftarrow \text{RO}_1(x_{C,i}, \text{aux})$	$\parallel G_0-G_3$	
22 $\alpha \leftarrow h^r$	$\parallel G_0-G_3$	
23 $\hat{r} \xleftarrow{\$} \mathbb{Z}_q$	$\parallel G_4-G_6$	
24 $\alpha \leftarrow g^{\hat{r}}$	$\parallel G_4-G_6$	
25 $T[i, j] \leftarrow (r, \text{aux})$	$\parallel G_0-G_3$	
26 $T[i, j] \leftarrow (\hat{r}, \text{aux})$	$\parallel G_4-G_6$	
27 $T_\alpha[i, j] \leftarrow \alpha$	$\parallel G_2-G_6$	
28 return α		
Oracle FIN($\beta, i \in [n], j$)		
29 if $T[i, j] = \perp$ return $\perp$		
30 $(r, \text{aux}) \leftarrow T[i, j]$	$\parallel G_0-G_3$	
31 $(\hat{r}, \text{aux}) \leftarrow T[i, j]$	$\parallel G_4-G_6$	
32 $\alpha \leftarrow T_\alpha[i, j]$	$\parallel G_2-G_6$	
33 $T[i, j] \leftarrow \perp$		
34 if $\beta \notin \mathbb{G}$ return $\perp$		
35 if $(\alpha, i, \beta) \in \mathcal{S}_{\text{resp}}$ return $\top$	$\parallel G_2-G_6$	
36 if $i \in \mathcal{S}_{\text{cmp}} \wedge \beta = \alpha^{x_{S,i}}$ return $\top$	$\parallel G_2-G_6$	
37 if $\exists x : (x, \text{aux}) \in T_1 \wedge (x, \text{aux}, \beta^{T_u[x, \text{aux}]/\hat{r}}) \in T_2$	$\parallel G_5-G_6$	
38 abort	$\parallel G_5-G_6$	
39 $h \leftarrow \text{RO}_1(x_{C,i}, \text{aux})$	$\parallel G_4$	
40 $u \leftarrow T_u[x_{C,i}, \text{aux}]$	$\parallel G_4$	
41 $r \leftarrow \hat{r}/u$	$\parallel G_4$	
42 $y \leftarrow \text{RO}_2(x_{C,i}, \text{aux}, \beta^{1/r})$	$\parallel G_0-G_4$	
43 $y' \leftarrow \text{RO}_2(x_{C,i}, \text{aux}, \text{RO}_1(x_{C,i}, \text{aux})^{x_{S,i}})$	$\parallel G_0-G_1$	
44 if $y = y'$ return $\top$	$\parallel G_0-G_1$	
45 $y \xleftarrow{\$} \{0, 1\}^{\text{kl}}$	$\parallel G_5-G_6$	
46 $\mathcal{S}_{\text{fin}} \leftarrow \mathcal{S}_{\text{fin}} \cup \{(i, \beta, \hat{r})\}$	$\parallel G_5-G_6$	
47 return y		
Oracle RESP($\alpha, i \in [n_r]$)		
48 if $\alpha \notin \mathbb{G}$ return $\perp$		
49 ctr_i++	$\parallel G_3-G_6$	
50 $\beta \leftarrow \alpha^{x_{S,i}}$		
51 $\mathcal{S}_{\text{resp}} \leftarrow \mathcal{S}_{\text{resp}} \cup \{(\alpha, i, \beta)\}$	$\parallel G_2-G_6$	
52 return β		
Oracle RO₁(x, aux)		
53 if $(x, \text{aux}) \notin T_1$		
54 $T_1[x, \text{aux}] \xleftarrow{\$} \mathbb{G}$	$\parallel G_0-G_3$	
55 $u \xleftarrow{\$} \mathbb{Z}_q$	$\parallel G_4-G_6$	
56 $T_1[x, \text{aux}] \leftarrow g^u$	$\parallel G_4-G_6$	
57 $T_u[x, \text{aux}] \leftarrow u$	$\parallel G_4-G_6$	
58 return $T_1[x, \text{aux}]$		
Oracle RO₂(x, aux, γ)		
59 if $(x, \text{aux}) \in T_1 \wedge T_{\text{aux}}[\text{aux}] \neq \perp$	$\parallel G_3-G_6$	
60 $h \leftarrow T_1[x, \text{aux}]$	$\parallel G_3-G_6$	
61 $i \leftarrow T_{\text{aux}}[\text{aux}]$	$\parallel G_3-G_6$	
62 if $\gamma = h^{x_{S,i}} \wedge i \notin \mathcal{S}_{\text{cmp}}$	$\parallel G_3-G_6$	
63 $\mathcal{S}_{\text{guess},i} \leftarrow \mathcal{S}_{\text{guess},i} \cup \{(x, \text{aux}, h, \gamma)\}$	$\parallel G_3-G_6$	
64 if $ \mathcal{S}_{\text{guess},i} > \text{ctr}_i$ abort	$\parallel G_3-G_6$	
65 if $x = x_{C,i}$ abort	$\parallel G_6$	
66 if $\gamma \neq h^{x_{S,i}}$	$\parallel G_5-G_6$	
67 if $\exists (\beta, \hat{r}) : (i, \beta, \hat{r}) \in \mathcal{S}_{\text{fin}} \wedge \gamma = \beta^{T_u[x, \text{aux}]/\hat{r}}$	$\parallel G_5-G_6$	
68 abort	$\parallel G_5-G_6$	
69 if $(x, \text{aux}, \gamma) \notin T_2$		
70 $y \xleftarrow{\$} \{0, 1\}^{\text{kl}}$		
71 if $\exists (x', \text{aux}', \gamma') \in T_2 : T_2[x', \text{aux}', \gamma'] = y$	$\parallel G_1-G_6$	
72 abort	$\parallel G_1-G_6$	
73 $T_2[x, \text{aux}, \gamma] \leftarrow y$		
74 return $T_2[x, \text{aux}, \gamma]$		
Oracle COMP($i \in [n_r]$)		
75 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$		
76 return $(x_{C,i}, x_{S,i})$		
Oracle CHECK(y, aux)		
77 if $T_{\text{aux}}[\text{aux}] = \perp$ return 0		
78 $i \leftarrow T_{\text{aux}}[\text{aux}]$		
79 return $\llbracket y = \text{RO}_2(x_{C,i}, \text{aux}, \text{RO}_1(x_{C,i}, \text{aux})^{x_{S,i}}) \rrbracket$		

Fig. 25. Games $G_0 - G_6$ for the proof of Theorem 2.

Note in particular that we only call the EXP oracle Q_{RESP} times. This later translates into the number of password guesses.

Adversary $\mathcal{B}^{\text{EXP}, \text{CHECK}_B, \text{COMP}_B}(V_1, \dots, V_{Q_{\text{REG}}}, U_1, \dots, U_{Q_{\text{RO}}})$ 00 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_{\text{resp}} \leftarrow \emptyset$ 01 $n_r, n_{\text{RO}} \leftarrow 0$ 02 $(x_{C,1}, \dots, x_{C,Q_{\text{REG}}}) \xleftarrow{\$} \mathcal{D}_C$ 03 for $i \in [Q_{\text{REG}}]$ 04 $x_{S,i} \leftarrow \perp$ $\parallel x_{S,i} = v_i = \text{dlog}(V_i)$ 05 $\mathcal{S}_{\text{guess},i} \leftarrow \emptyset$ 06 $(y^*, \text{aux}^*) \xleftarrow{\$} \mathcal{A}^{\text{INIT}, \text{RESP}, \text{FIN}, \text{COMP}, \text{CHECK}, \text{RO}_1, \text{RO}_2}$ 07 return $\perp$	Oracle $\text{RESP}(\alpha, i \in [n_r])$ 32 if $\alpha \notin \mathbb{G}$ return $\perp$ 33 $\beta \leftarrow \text{EXP}(i, \alpha)$ $\parallel \text{sets } \beta = \alpha^{v_i}$ 34 ctr_i++ 35 $\mathcal{S}_{\text{resp}} \leftarrow \mathcal{S}_{\text{resp}} \cup \{(\alpha, i, \beta)\}$ 36 return β
Oracle $\text{REG}(\text{aux})$ 08 require $T_{\text{aux}}[\text{aux}] = \perp$ $\parallel \text{enforce unique aux}$ 09 n_r++ 10 $T_{\text{aux}}[\text{aux}] \leftarrow n_r$ 11 $\text{ctr}_{n_r} \leftarrow 0$	Oracle $\text{RO}_1(x, \text{aux})$ 37 if $(x, \text{aux}) \notin T_1$ 38 $n_{\text{RO}}++$ 39 $T_1[x, \text{aux}] \leftarrow U_{n_{\text{RO}}}$ $\parallel \text{embed challenge}$ 40 return $T_1[x, \text{aux}]$
Oracle $\text{INIT}(\text{aux}, j)$ 12 require $T_{\text{aux}}[\text{aux}] \neq \perp$ 13 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 14 $r \xleftarrow{\$} \mathbb{Z}_q$ 15 $h \leftarrow \text{RO}_1(x_{C,i}, \text{aux})$ 16 $\alpha \leftarrow h^r$ 17 $T[i, j] \leftarrow (r, \text{aux})$ 18 $T_\alpha[i, j] \leftarrow \alpha$ 19 return α	Oracle $\text{RO}_2(x, \text{aux}, \gamma)$ 41 if $(x, \text{aux}) \in T_1$ 42 $h \leftarrow T_1[x, \text{aux}]$ 43 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 44 if $i \notin \mathcal{S}_{\text{cmp}} \wedge \text{CHECK}_B(i, h, \gamma)$ $\parallel \text{check } \gamma = h^{v_i}$ 45 $\mathcal{S}_{\text{guess},i} \leftarrow \mathcal{S}_{\text{guess},i} \cup \{(h, \gamma)\}$ 46 if $ \mathcal{S}_{\text{guess},i} > \text{ctr}_i$ 47 output $(i, \mathcal{S}_{\text{guess},i})$ $\parallel \text{win}$ 48 if $T_2[x, \text{aux}, \text{"chl"}] \neq \perp \wedge \text{CHECK}_B(i, h, \gamma)$ 49 $T_2[x, \text{aux}, \gamma] \leftarrow T_2[x, \text{aux}, \text{"chl"}]$ $\parallel \text{patch RO}$ 50 if $(x, \text{aux}, \gamma) \notin T_2$ 51 $T_2[x, \text{aux}, \gamma] \xleftarrow{\$} \{0, 1\}^k$ 52 return $T_2[x, \text{aux}, \gamma]$
Oracle $\text{FIN}(\beta, i \in [n], j)$ 20 if $T[i, j] = \perp$ return $\perp$ 21 $(r, \text{aux}) \leftarrow T[i, j]$ 22 $\alpha \leftarrow T_\alpha[i, j]$ 23 $T[i, j] \leftarrow \perp$ 24 if $\beta \notin \mathbb{G}$ return $\perp$ 25 if $(\alpha, i, \beta) \in \mathcal{S}_{\text{resp}}$ return $\top$ 26 if $i \in \mathcal{S}_{\text{cmp}} \wedge \beta = \alpha^{x_{S,i}}$ return $\top$ 27 $y \leftarrow \text{RO}_2(x_{C,i}, \text{aux}, \beta^{1/r})$ 28 return y	Oracle $\text{CHECK}(y, \text{aux})$ 53 if $T_{\text{aux}}[\text{aux}] = \perp$ return 0 54 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 55 if $\exists \gamma \in T_2 : T_2[x_{C,i}, \text{aux}, \gamma] = y$ 56 $h \leftarrow \text{RO}_1(x_{C,i}, \text{aux})$ 57 return $\text{CHECK}_B(i, h, \gamma)$ 58 else 59 $y' \xleftarrow{\$} \{0, 1\}^k$ 60 $T_2[x_{C,i}, \text{aux}, \text{"chl"}] \leftarrow y'$ 61 return $\llbracket y = y' \rrbracket$
Oracle $\text{COMP}(i \in [n_r])$ 29 $x_{S,i} \leftarrow \text{COMP}_B(i)$ $\parallel x_{S,i} = v_i$ 30 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 31 return $(x_{C,i}, x_{S,i})$	

Fig. 26. Adversary $\mathcal{B}$ against CT-OMCDH having access to oracles EXP , CHECK_B , and COMP_B simulating $\mathcal{G}_2/\mathcal{G}_3$ for adversary $\mathcal{A}$.

Proof (Claim). The reduction $\mathcal{B}$ is presented in Figure 26. It gets as input $(V_1, \dots, V_{Q_{\text{REG}}}, U_1, \dots, U_{Q_{\text{RO}}})$, where we embed the exponent of V_i as the server secret for registered user i and U_j is embedded as the j -th output of RO_1 . $\mathcal{B}$ simulates the RESP oracle by calling its EXP oracle. The CHECK oracle can be simulated via $\mathcal{B}$'s CHECK_B oracle in case RO_2 was already queried on the checked value and the output y is correct. In case, the random oracle was not queried yet, a randomly chosen value is used and the random oracle is patched upon the correct random oracle query (Line 44). To simulate the random oracle RO_2 and to decide whether an element should be added to set $\mathcal{S}_{\text{guess}}$, $\mathcal{B}$ queries its CHECK_B oracle. If the oracle returns true, then the adversary has found a valid CDH solution for values V_i and U_j . In case the list of valid solutions is

¹⁰ We could already use the server secret here to only identify relevant queries, but this would require to iterate over all indices $i \in [n]$, which unnecessarily increases the runtime (of the game, but also of the reduction later).

Adversary $\mathcal{C}^{\text{CHECK}_C}(A_1, \dots, A_{Q_{\text{INIT}}}, U_1, \dots, U_{Q_{\text{RO}}})$ <pre> 00 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_{\text{resp}} \leftarrow \emptyset$ 01 $n_r, n_a, n_{\text{RO}} \leftarrow 0$ 02 $(x_{C,1}, \dots, x_{C,Q_{\text{REG}}}) \xleftarrow{\\$} \mathcal{D}_C$ 03 for $i \in [Q_{\text{REG}}]$ 04 $x_{S,i} \leftarrow \mathbb{Z}_q$ 05 $(y^*, \text{aux}^*) \xleftarrow{\\$} \mathcal{A}^{\text{INIT,RESP,FIN,COMP,CHECK,RO}_1,\text{RO}_2}$ 06 return $\perp$ Oracle $\text{REG}(\text{aux})$ 07 require $\mathsf{T}_{\text{aux}}[\text{aux}] = \perp$ $\parallel$ enforce unique aux 08 n_r++ 09 $\mathsf{T}_{\text{aux}}[\text{aux}] \leftarrow n_r$ Oracle $\text{INIT}(\text{aux}, j)$ 10 require $\mathsf{T}_{\text{aux}}[\text{aux}] \neq \perp$ 11 $i \leftarrow \mathsf{T}_{\text{aux}}[\text{aux}]$ 12 n_a++ 13 $\alpha \leftarrow A_{n_a}$ $\parallel$ embed challenge 14 $\mathsf{T}[i, j] \leftarrow (\perp, \text{aux})$ 15 $\mathsf{T}_\alpha[i, j] \leftarrow \alpha$ 16 return α Oracle $\text{FIN}(\beta, i \in [n], j)$ 17 if $\mathsf{T}[i, j] = \perp$ return $\perp$ 18 $(\perp, \text{aux}) \leftarrow \mathsf{T}[i, j]$ 19 $\alpha \leftarrow \mathsf{T}_\alpha[i, j]$ 20 $\mathsf{T}[i, j] \leftarrow \perp$ 21 if $\beta \notin \mathbb{G}$ return $\perp$ 22 if $(\alpha, i, \beta) \in \mathcal{S}_{\text{resp}}$ return $\top$ 23 if $i \in \mathcal{S}_{\text{cmp}} \wedge \beta = \alpha^{x_{S,i}}$ return $\top$ 24 if $\exists x, \gamma, \ell_u, \ell_v : (x, \text{aux}) \in \mathsf{T}_1 \wedge (x, \text{aux}, \gamma) \in \mathsf{T}_2 \wedge \alpha = A_{\ell_u}$ $\wedge \mathsf{T}_1[x, \text{aux}] = U_{\ell_v} \wedge \text{CHECK}_C(\ell_u, \ell_v, \beta, \gamma)$ output $(\ell_u, \ell_v, \beta, \gamma)$ $\parallel$ win 25 $y \xleftarrow{\\$} \{0, 1\}^{\text{kl}}$ 26 $\mathcal{S}_{\text{fin}} \leftarrow \mathcal{S}_{\text{fin}} \cup \{(i, j, \beta, \perp)\}$ 27 return y Oracle $\text{COMP}(i \in [n_r])$ 29 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 30 return $(x_{C,i}, x_{S,i})$ </pre>	Oracle $\text{RESP}(\alpha, i \in [n_r])$ <pre> 31 if $\alpha \notin \mathbb{G}$ return $\perp$ 32 ctr_i++ 33 $\beta \leftarrow \alpha^{x_{S,i}}$ 34 $\mathcal{S}_{\text{resp}} \leftarrow \mathcal{S}_{\text{resp}} \cup \{(\alpha, i, \beta)\}$ 35 return β Oracle $\text{RO}_1(x, \text{aux})$ 36 if $(x, \text{aux}) \notin \mathsf{T}_1$ 37 $n_{\text{RO}}++$ 38 $\mathsf{T}_1[x, \text{aux}] \leftarrow U_{n_{\text{RO}}}$ $\parallel$ embed challenge 39 return $\mathsf{T}_1[x, \text{aux}]$ Oracle $\text{RO}_2(x, \text{aux}, \gamma)$ 40 if $(x, \text{aux}) \in \mathsf{T}_1 \wedge \mathsf{T}_{\text{aux}}[\text{aux}] \neq \perp$ 41 $h \leftarrow \mathsf{T}_1[x, \text{aux}]$ 42 $i \leftarrow \mathsf{T}_{\text{aux}}[\text{aux}]$ 43 if $\gamma = h^{x_{S,i}} \wedge i \notin \mathcal{S}_{\text{cmp}}$ 44 $\mathcal{S}_{\text{guess},i} \leftarrow \mathcal{S}_{\text{guess},i} \cup \{(x, \text{aux}, h, \gamma)\}$ 45 if $\mathcal{S}_{\text{guess},i} > \text{ctr}_i$ abort 46 if $\gamma \neq h^{x_{S,i}}$ 47 if $\exists j, \beta, \ell_u, \ell_v : (i, j, \beta, \perp) \in \mathcal{S}_{\text{fin}} \wedge \mathsf{T}_\alpha[i, j] = A_{\ell_u}$ 48 $\wedge h = V_{\ell_v} \wedge \text{CHECK}_C(\ell_u, \ell_v, \beta, \gamma)$ 49 output $(\ell_u, \ell_v, \beta, \gamma)$ $\parallel$ win 50 if $(x, \text{aux}, \gamma) \notin \mathsf{T}_2$ 51 $y \xleftarrow{\\$} \{0, 1\}^{\text{kl}}$ 52 if $\exists (x', \text{aux}', \gamma') \in \mathsf{T}_2 : \mathsf{T}_2[x', \text{aux}', \gamma'] = y$ 53 abort 54 $\mathsf{T}_2[x, \text{aux}, \gamma] \leftarrow y$ 55 return $\mathsf{T}_2[x, \text{aux}, \gamma]$ Oracle $\text{CHECK}(y, \text{aux})$ 56 if $\mathsf{T}_{\text{aux}}[\text{aux}] = \perp$ return 0 57 $i \leftarrow \mathsf{T}_{\text{aux}}[\text{aux}]$ 58 $h \leftarrow \text{RO}_1(x_{C,i}, \text{aux})$ 59 if $(x_{C,i}, \text{aux}, h^{x_{S,i}}) \in \mathsf{T}_2$ 60 return $\mathsf{T}_2[x_{C,i}, \text{aux}, h^{x_{S,i}}]$ 61 $y' \xleftarrow{\\$} \{0, 1\}^{\text{kl}}$ 62 $\mathsf{T}_2[x_{C,i}, \text{aux}, h^{x_{S,i}}] \leftarrow y'$ 63 return $\llbracket y = y' \rrbracket$ </pre>
---	---

Fig. 27. Adversary $\mathcal{C}$ against CB-InvDH having access to oracle CHECK_C simulating $\mathsf{G}_4/\mathsf{G}_5$ for adversary $\mathcal{A}$.

larger than the queries to RESP for an index i , denoted by ctr_i , the adversary outputs $(i, \mathcal{S}_{\text{guess},i})$ and wins their game. Hence, whenever the newly introduced abort condition triggers, $\mathcal{B}$ wins. $\square$

Game G_4 . We make the exponent u in the random oracle RO_1 explicit instead of sampling it at random from the group. At the same time, we delay the call of RO_1 for queries to INIT . More concretely, we do not call RO_1 during INIT and sample a random exponent $\hat{r}$ instead. Then, during FIN , we determine u by querying RO_1 and compute the correct r . We add corresponding changes for book-keeping. Since r perfectly blinds α , this change cannot be observed. We have

$$\Pr[\mathsf{G}_3(\mathcal{A}) \Rightarrow 1] = \Pr[\mathsf{G}_4(\mathcal{A}) \Rightarrow 1].$$

Game G_5 . We prepare for the next game hop, where we want to replace the output of FIN with a random y without querying the random oracle. We argue that whenever FIN does not output $\top$, then it is hard for

Adversary $\mathcal{D}^{\text{TEST}, \text{COMP}_{\mathcal{D}}}$ 00 $\mathcal{S}_{\text{cmp}} \leftarrow \emptyset$ 01 for $i \in [n]$ 02 $x_{\mathcal{C},i} \leftarrow \perp$ $\parallel$ unknown 03 $x_{\mathcal{S},i} \xleftarrow{\$} \mathbb{Z}_q$ 04 $(y^*, \text{aux}^*) \xleftarrow{\$} \mathcal{A}^{\text{INIT}, \text{RESP}, \text{FIN}, \text{COMP}, \text{CHECK}, \text{RO}_1, \text{RO}_2}$ 05 return $\perp$ Oracle REG(aux) 06 require $T_{\text{aux}}[\text{aux}] = \perp$ 07 n_r++ 08 $T_{\text{aux}}[\text{aux}] \leftarrow n_r$ 09 $\text{ctr}_i \leftarrow 0$ 10 for $x : (x, \text{aux}, \cdot) \in T_2$ 11 if $\text{TEST}(n_r, x)$ 12 output $\perp$ Oracle INIT(aux, j) 13 require $T_{\text{aux}}[\text{aux}] \neq \perp$ 14 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 15 $\hat{r} \xleftarrow{\$} \mathbb{Z}_q$ 16 $\alpha \leftarrow g^{\hat{r}}$ 17 $T[i, j] \leftarrow (\hat{r}, \text{aux})$ 18 $T_{\alpha}[i, j] \leftarrow \alpha$ 19 return α Oracle FIN($\beta, i \in [n], j$) 20 if $T[i, j] = \perp$ return $\perp$ 21 $(\hat{r}, \text{aux}) \leftarrow T[i, j]$ 22 $\alpha \leftarrow T_{\alpha}[i, j]$ 23 $T[i, j] \leftarrow \perp$ 24 if $\beta \notin \mathbb{G}$ return $\perp$ 25 if $(\alpha, i, \beta) \in \mathcal{S}_{\text{resp}}$ return $\top$ 26 if $i \in \mathcal{S}_{\text{cmp}} \wedge \beta = \alpha^{x_{\mathcal{S},i}}$ return $\top$ 27 if $\exists x : (x, \text{aux}) \in T_1 \wedge (x, \text{aux}, \beta^{T_u[x, \text{aux}]/\hat{r}}) \in T_2$ 28 abort 29 $y \xleftarrow{\$} \{0, 1\}^{\text{kl}}$ 30 $\mathcal{S}_{\text{fin}} \leftarrow \mathcal{S}_{\text{fin}} \cup \{(i, \beta, \hat{r})\}$ 31 return y Oracle CHECK($i \in [n], y, \text{aux}$) 32 if $T_{\text{aux}}[\text{aux}] = \perp$ return 0 33 return $\llbracket y = \text{RO}_2(\text{"chl", aux, "chl"}) \rrbracket$	Oracle RESP($\alpha, i \in [n]$) 34 if $\alpha \notin \mathbb{G}$ return $\perp$ 35 ctr_i++ 36 $\beta \leftarrow \alpha^{x_{\mathcal{S},i}}$ 37 $\mathcal{S}_{\text{resp}} \leftarrow \mathcal{S}_{\text{resp}} \cup \{(\alpha, i, \beta)\}$ 38 return β Oracle RO₁(x, aux) 39 if $(x, \text{aux}) \notin T_1$ 40 $u \xleftarrow{\$} \mathbb{Z}_q$ 41 $T_1[x, \text{aux}] \leftarrow g^u$ 42 $T_u[x, \text{aux}] \leftarrow u$ 43 return $T_1[x, \text{aux}]$ Oracle RO₂(x, aux, γ) 44 if $(x, \text{aux}) \in T_1 \wedge T_{\text{aux}}[\text{aux}] \neq \perp$ 45 $h \leftarrow T_1[x, \text{aux}]$ 46 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 47 if $\gamma = h^{x_{\mathcal{S},i}} \wedge i \notin \mathcal{S}_{\text{cmp}}$ 48 $\mathcal{S}_{\text{guess},i} \leftarrow \mathcal{S}_{\text{guess},i} \cup \{(x, \text{aux}, h, \gamma)\}$ 49 if $ \mathcal{S}_{\text{guess},i} > \text{ctr}_i$ abort 50 if $\text{TEST}(x)$ 51 output $\perp$ $\parallel$ win 52 else if $\gamma = h^{x_{\mathcal{S},i}} \wedge x = x_{\mathcal{C},i} \wedge T_2[\text{"chl", aux, "chl"}] \neq \perp$ 53 $T_2[x, \text{aux}, \gamma] \leftarrow T_2[\text{"chl", aux, "chl"}]$ $\parallel$ patch RO 54 if $\gamma \neq h^{x_{\mathcal{S},i}}$ 55 if $\exists (\beta, \hat{r}) : (i, \beta, \hat{r}) \in \mathcal{S}_{\text{fin}} \wedge \gamma = \beta^{T_u[x, \text{aux}]/\hat{r}}$ 56 abort 57 if $(x, \text{aux}, \gamma) \notin T_2$ 58 $y \xleftarrow{\$} \{0, 1\}^{\text{kl}}$ 59 if $\exists (x', \text{aux}', \gamma') \in T_2 : T_2[x', \text{aux}', \gamma'] = y$ 60 abort 61 $T_2[x, \text{aux}, \gamma] \leftarrow y$ 62 return $T_2[x, \text{aux}, \gamma]$ Oracle COMP($i \in [n]$) 63 $x_{\mathcal{C},i} \leftarrow \text{COMP}_{\mathcal{D}}(i)$ 64 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 65 return $(x_{\mathcal{C},i}, x_{\mathcal{S},i})$
--	--

Fig. 28. Adversary $\mathcal{D}$ against PW-Guess having access to oracle TEST and $\text{Comp}_{\mathcal{D}}$ simulating $\mathbf{G}_5/\mathbf{G}_6$.

$\mathcal{A}$ to query the random oracle on the value of γ that is used to determine the output y . In fact, this should even be hard if the adversary has corrupted the secrets.

In the previous game, the oracle outputs $\text{RO}_2(x_{\mathcal{C},i}, \text{aux}, \gamma)$, where $\gamma = \beta^{T_u[x_{\mathcal{C},i}, \text{aux}]/\hat{r}}$ for some β chosen by $\mathcal{A}$ such that $\beta \neq \alpha^{x_{\mathcal{S},i}}$. We will show that it is indeed hard to compute γ . Since we also do not want to rely on $x_{\mathcal{C},i}$ anymore, we show something stronger, namely that $\mathcal{A}$ cannot compute the correct γ for any x that it queried to the random oracle. We define an appropriate abort event in FIN (Line 37) and in random oracle RO_2 (Line 67). Any remaining computations in FIN are not necessary anymore and we just sample a random output.

Claim. There exists an adversary $\mathcal{D}$ against CB-InvDH such that

$$\Pr[\mathbf{G}_4(\mathcal{A}) \Rightarrow 1] - \Pr[\mathbf{G}_5(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{\mathbb{G}, p, g}^{(Q_{\text{INT}}, Q_{\text{RO}}, 2Q_{\text{FIN}}, Q_{\text{RO}})\text{-CB-InvDH}}(\mathcal{C}).$$

Proof (Claim). The formal reduction is depicted in Figure 27. Challenges are embedded in the INIT oracle and in RO_1 . To check if the abort conditions are satisfied, reduction $\mathcal{C}$ can use their own $\text{CHECK}_{\mathcal{C}}$ oracle

and the values that were stored in previous queries. If the condition is satisfied in the FIN oracle or in RO₂, $\mathcal{C}$ immediately wins their own game. $\square$

Game G₆. In this game, we abort when the client secret is contained in the set $\mathcal{S}_{\text{guess}}$ (Line 65) and if the correct client secret was input to RO₂ before registration (Line 17).

Claim. There exists an adversary $\mathcal{D}$ against PW-Guess such that

$$\Pr[\mathbf{G}_5(\mathcal{A}) \Rightarrow 1] - \Pr[\mathbf{G}_6(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{\mathcal{D}_C}^{(Q_{\text{REG}}, Q_{\text{RESP}}, Q_{\text{COMP}})\text{-PW-Guess}}(\mathcal{D}).$$

Proof (Claim). The reduction is formally constructed in Figure 28. Due to the changes in the previous games, the oracles are independent of the client secrets except for CHECK and RO₂. The abort condition can be checked by $\mathcal{D}$'s check oracle CHECK _{$\mathcal{D}$} and if it triggers, $\mathcal{D}$ wins their game. To consistently answer random oracle queries with respect to the CHECK oracle, the reduction has to potentially patch the random oracle RO₂ when queried on a compromised user (Line 53). Due to the previous condition, the set to be tested is of size at most ctr_i for each user i which can again be upper bounded by Q_{RESP} . Note that $\mathcal{D}$ issues at most Q_{RO} queries to TEST because queries from REG and RO₁/RO₂ are mutually exclusive. $\square$

Final game.

Claim. It holds that

$$\Pr[\mathbf{G}_6(\mathcal{A}) \Rightarrow 1] \leq \frac{Q_{\text{CHECK}} + 1}{2^{\text{kl}} - Q_{\text{CHECK}}}.$$

Proof (Claim). If the client secret is not in the set $\mathcal{S}'_{\text{guess}}$, then the random oracle has never been queried on the right input. Further, the correct output y^* was also never output by FIN because FIN outputs $\top$ instead of the actual value in such a case. Hence, adversary $\mathcal{A}$'s view is independent of the output of RO₂ with the correct client secret on input. The only possibility for $\mathcal{A}$ to win the game is to guess the correct output in the space 2^{kl} . This is possible once per query to CHECK and for the final output resulting in $Q_{\text{CHECK}} + 1$ many guesses. Since one guess rules out one possible value, we need to sum up $\sum_{i=0}^{Q_{\text{CHECK}}} 1/(2^{\text{kl}} - i)$ which can be upper bound to obtain the claim. $\square$

Collecting the probabilities completes the proof of Theorem 2. $\square$

B.3 Proof of Theorem 3

Theorem 3 (INS-OW of SHash). For any distribution $\mathcal{D}_C$ and adversary $\mathcal{A}$ against INS-OW of SHash := SHash[$\mathcal{D}_S, \text{kl}$], there exists an PW-Guess adversary $\mathcal{B}$ against $\mathcal{D}_C$ with $t_{\mathcal{A}} \approx t_{\mathcal{B}}$ such that

$$\text{Adv}_{\text{SHash}, \mathcal{D}_C}^{Q\text{-INS-OW}}(\mathcal{A}) \leq \text{Adv}_{\mathcal{D}_C}^{(Q_{\text{REG}}, Q_{\text{RO}}, Q_{\text{COMP}})\text{-PW-Guess}}(\mathcal{B}) + \frac{Q_{\text{CHECK}} + 1}{2^{\text{kl}} - Q_{\text{CHECK}}}$$

with $Q = (Q_{\text{REG}}, Q_{\text{COMP}}, Q_{\text{INIT}}, Q_{\text{FIN}}, Q_{\text{CHECK}}, Q_{\text{RO}})$.

Proof. We use the sequence of games described in Figure 29.

Game G₀. This is the INS-OW game for SHash and $\mathcal{D}_C$.

Game G₁. This is the same game as the previous one except that it introduces several conceptual changes. We introduce a new random oracle, RO', which behave exactly the same as RO with the only difference that it maintains an additional table T' which indicate that the query was made via the new oracle. The adversary still only has access to the original random oracle but the new random oracle is called from all other oracle queries in the game. Hence, we mark if a query was made directly by the attacker or internally.

Game $G_0 - G_2$ 00 $\mathcal{S}_{\text{cmp}} \leftarrow \emptyset$ 01 $n_r \leftarrow 0$ 02 $(x_{C,1}, \dots, x_{C,Q_{\text{REG}}}) \xleftarrow{\$} \mathcal{D}_C$ 03 for $i \in [Q_{\text{REG}}]$ 04 $x_{S,i} \xleftarrow{\$} \mathcal{D}_S$ 05 $(y^*, \text{aux}^*) \xleftarrow{\$} \mathcal{A}^{\text{INIT,FIN,CHECK,RO}}$ 06 if $T_{\text{aux}}[\text{aux}] = \perp$ return 0 07 $i^* \leftarrow T_{\text{aux}}[\text{aux}]$ 08 if $i^* \in \mathcal{S}_{\text{cmp}}$ return 0 09 return $\llbracket y^* = \text{RO}(x_{C,i^*}, \text{aux}^*, x_{S,i^*}) \rrbracket$ 10 return $\llbracket y^* = \text{RO}'(x_{C,i^*}, \text{aux}^*, x_{S,i^*}) \rrbracket$ Oracle REG(aux) 11 require $T_{\text{aux}}[\text{aux}] = \perp$ 12 n_r++ 13 $T_{\text{aux}}[\text{aux}] \leftarrow n_r$ 14 if $(x_{C,n_r}, \text{aux}, \cdot) \in \mathcal{T}$ 15 abort 16 return x_{S,n_r} Oracle INIT(aux, j) 17 require $T_{\text{aux}}[\text{aux}] \neq \perp$ 18 $\text{auth}_1 \leftarrow \text{aux}$ 19 $T[i, j] \leftarrow (\varepsilon, \text{aux})$ 20 return auth_1 Oracle CHECK(y, aux) 21 if $T_{\text{aux}}[\text{aux}] = \perp$ return 0 22 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 23 return $\llbracket y = \text{RO}(x_{C,i}, \text{aux}, x_{S,i}) \rrbracket$ 24 return $\llbracket y = \text{RO}'(x_{C,i}, \text{aux}, x_{S,i}) \rrbracket$	Oracle FIN(auth₂, i ∈ [n_r], j) 25 if $T[i, j] = \perp$ 26 return $\perp$ 27 $(\varepsilon, \text{aux}) \leftarrow T[i, j]$ 28 $T[i, j] \leftarrow \perp$ 29 $y \leftarrow \text{RO}(x_{C,i}, \text{aux}, \text{auth}_2)$ 30 $y \leftarrow \text{RO}'(x_{C,i}, \text{aux}, \text{auth}_2)$ 31 if $y = \text{RO}(x_{C,i}, \text{aux}, x_{S,i})$ return $\top$ 32 if $y = \text{RO}'(x_{C,i}, \text{aux}, x_{S,i})$ return $\top$ 33 return y Oracle COMP(i ∈ [n_r]) 34 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 35 return $x_{C,i}$ Oracle RO(x_C, aux, x_S) 36 if $T_{\text{aux}}[\text{aux}] \neq \perp \wedge T_{\text{aux}}[\text{aux}] \notin \mathcal{S}_{\text{cmp}} \wedge x_C = x_{C,i}$ 37 abort 38 if $(x_C, \text{aux}, x_S) \notin \mathcal{T}$ 39 $T[x_C, \text{aux}, x_S] \xleftarrow{\$} \{0, 1\}^{\text{kl}}$ 40 return $T[x_C, \text{aux}, x_S]$ Oracle RO'(x_C, aux, x_S) 41 if $(x_C, \text{aux}, x_S) \notin \mathcal{T}$ 42 $T[x_C, \text{aux}, x_S] \xleftarrow{\$} \{0, 1\}^{\text{kl}}$ 43 $T'[x_C, \text{aux}, x_S] \xleftarrow{\$} 1$ 44 return $T[x_C, \text{aux}, x_S]$
---	--

Fig. 29. Games $G_0 - G_2$ for the proof of Theorem 3.

Since the oracle still maintains the same list to answer queries, the change is only conceptual and it holds that

$$\Pr[G_0(\mathcal{A}) \Rightarrow 1] = \Pr[G_1(\mathcal{A}) \Rightarrow 1].$$

Game G_2 . This is the same game as the previous one except that it aborts if random oracle RO is queried on a registered and uncompromised auxiliary input aux and the input x is the correct client secret for that aux. The game further aborts in the registration oracle if there already exists a previous random oracle query on the correct client secret and the registered aux.

Claim. There exists an adversary $\mathcal{B}$ against PW-Guess such that

$$\Pr[G_1(\mathcal{A}) \Rightarrow 1] - \Pr[G_2(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{\mathcal{D}_C}^{(Q_{\text{REG}}, Q_{\text{RO}}, Q_{\text{COMP}})\text{-PW-Guess}}(\mathcal{B}).$$

Proof (Claim). The reduction is formally depicted in Figure 30. The oracles are simulated by querying the internal random oracles on some unique tag “chl” which acts as a placeholder and is considered to be a value outside of the valid random oracle input space. This allows the reduction $\mathcal{B}$ to consistently answer oracle queries. In case of a compromise, the random oracles must be patched on the respective values. This is possible because the client secret is know to the reduction upon compromise and done in Line 32. Note that this simulation is sound because queries on the correct client secret *before* compromise are captured by the newly introduced abort condition. To capture this condition, reduction $\mathcal{B}$ uses their TEST oracle and therefore knows whenever the abort condition is met. Moreover, in such a case $\mathcal{B}$ correctly guessed an

Adversary $\mathcal{B}^{\text{TEST}, \text{COMP}_{\mathcal{B}}}$ <pre> 00 $\mathcal{S}_{\text{cmp}} \leftarrow \emptyset$ 01 $n_r \leftarrow 0$ 02 for $i \in [Q_{\text{REG}}]$ 03 $x_{C,i} \leftarrow \perp$ 04 $x_{S,i} \xleftarrow{\\$} \mathbb{Z}_q$ 05 $(y^*, \text{aux}^*) \xleftarrow{\\$} \mathcal{A}^{\text{REG}, \text{INIT}, \text{FIN}, \text{CHECK}, \text{COMP}, \text{RO}_1, \text{RO}_2}$ 06 return $\perp$ Oracle REG(aux) 07 require $T_{\text{aux}}[\text{aux}] = \perp$ 08 n_r++ 09 $T_{\text{aux}}[\text{aux}] \leftarrow n_r$ 10 for $x_C : (x_C, \text{aux}, \cdot) \in \mathbb{T}$ 11 if $\text{TEST}(n_r, x_C)$ 12 output $\perp$ 13 return x_{S,n_r} Oracle INIT(aux, j) 14 require $T_{\text{aux}}[\text{aux}] \neq \perp$ 15 $\text{auth}_1 \leftarrow \text{aux}$ 16 $T[i, j] \leftarrow (\varepsilon, \text{aux})$ 17 return auth_1 Oracle FIN(auth₂, i ∈ [n_r], j) 18 if $T[i, j] = \perp$ return $\perp$ 19 $(\varepsilon, \text{aux}) \leftarrow T[i, j]$ 20 $T[i, j] \leftarrow \perp$ 21 $y \leftarrow \text{RO}'(\text{"chl"}, \text{aux}, \text{auth}_2)$ 22 if $y = \text{RO}'(\text{"chl"}, \text{aux}, x_{S,i})$ return $\top$ 23 return y </pre>	Oracle COMP($i \in [n_r]$) <pre> 24 $x_{C,i} \leftarrow \text{COMP}_{\mathcal{B}}(i)$ 25 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 26 return $x_{C,i}$ Oracle RO₂(x_C, aux, x_S) 27 if $T_{\text{aux}}[\text{aux}] \neq \perp \wedge T_{\text{aux}}[\text{aux}] \notin \mathcal{S}_{\text{cmp}}$ 28 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 29 if $\text{TEST}(i, x_C)$ 30 output $\perp$ 31 else if $T_{\text{aux}}[\text{aux}] \in \mathcal{S}_{\text{cmp}} \wedge (\text{"chl"}, \text{aux}, x_S) \in \mathbb{T}' \wedge x = x_{C,i}$ 32 $T[x_C, \text{aux}, x_S] \leftarrow T[\text{"chl"}, \text{aux}, x_S]$ 33 if $(x_C, \text{aux}, x_S) \notin \mathbb{T}$ 34 $T[x_C, \text{aux}, x_S] \xleftarrow{\\$} \{0, 1\}^{\text{kl}}$ 35 return $T[x_C, \text{aux}, x_S]$ Oracle RO'₂(x_C, aux, x_S) 36 if $(x_C, \text{aux}, x_S) \notin \mathbb{T}$ 37 $T[x_C, \text{aux}, x_S] \xleftarrow{\\$} \{0, 1\}^{\text{kl}}$ 38 $T'[x_C, \text{aux}, x_S] \xleftarrow{\\$} 1$ 39 return $T[x_C, \text{aux}, x_S]$ Oracle CHECK(y, aux) 40 if $T_{\text{aux}}[\text{aux}] = \perp$ return 0 41 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 42 return $\llbracket y = \text{RO}'(\text{"chl"}, \text{aux}, x_{S,i}) \rrbracket$ </pre>
--	--

Fig. 30. Adversary $\mathcal{B}$ against PW-Guess having access to oracles TEST and $\text{Comp}_{\mathcal{B}}$ simulating $\mathbf{G}_1/\mathbf{G}_2$

uncompromised password and thus wins their game. Note that $\mathcal{B}$ issues at most Q_{RO} queries to TEST because queries from REG and RO are mutually exclusive. $\square$

Final game.

Claim. It holds that

$$\Pr[\mathbf{G}_2(\mathcal{A}) \Rightarrow 1] \leq \frac{Q_{\text{CHECK}} + 1}{2^{\text{kl}} - Q_{\text{CHECK}}}.$$

Proof (Claim). The random oracle was never queried directly on the correct input due to the abort introduced in the previous game. Further, the correct output y^* was also never output by FIN because FIN outputs $\top$ instead of the actual value in such a case. Hence, adversary $\mathcal{A}$'s view is independent of the output of RO_2 . The only possibility for $\mathcal{A}$ to win is to guess the correct output in the space 2^{kl} . This possible once per query to CHECK and for the final output resulting in $Q_{\text{CHECK}} + 1$ many guesses. Since one guess rules out one possible value, we need to sum up $\sum_{i=0}^{Q_{\text{CHECK}}} 1/(2^{\text{kl}} - i)$ which can be upper bound to obtain the claim. $\square$

Collecting the probabilities completes the proof of Theorem 3. $\square$

C Theorems and Proofs from Section 5

C.1 Confidentiality

We restate Theorem 5 using concrete bounds and provide the full proof.

Theorem 10 (Confidentiality). *For any password distribution $\mathcal{PW}$ and adversary $\mathcal{A}$ against the Conf security of $\text{CS} := \text{CS}[\text{Auth}, \text{AEAD}, \text{MAC}]$ there exist an INS-OW adversary $\mathcal{B}$ against Auth, INT-CTXT adversaries $\mathcal{C}_1, \mathcal{C}_2$ against AEAD, OW-CPA adversaries $\mathcal{D}_1, \mathcal{D}_2$ against AEAD, and an IND-CCA adversary $\mathcal{E}$ against AEAD with $t_{\mathcal{A}} \approx t_{\mathcal{B}} \approx t_{\mathcal{C}_1} \approx t_{\mathcal{D}_1} \approx t_{\mathcal{C}_2} - Q_{\text{COMP}} \cdot Q_{\text{AUTH}_1} \approx t_{\mathcal{D}_2} - Q_{\text{COMP}} \cdot Q_{\text{AUTH}_1} \approx t_{\mathcal{E}}$ such that*

$$\begin{aligned} \text{Adv}_{\text{CS}, \mathcal{PW}}^{Q_{\mathcal{A}}\text{-Conf}}(\mathcal{A}) &\leq 2 \cdot \left(\text{Adv}_{\text{Auth}, \mathcal{PW}}^{Q_{\mathcal{B}}\text{-INS-OW}}(\mathcal{B}) + \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{C}_1}\text{-INT-CTXT}}(\mathcal{C}_1) + \text{Adv}_{\text{AEAD}, \text{kl}}^{Q_{\mathcal{D}_1}\text{-OW-CPA}}(\mathcal{D}_1) \right. \\ &\quad \left. + \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{C}_2}\text{-INT-CTXT}}(\mathcal{C}_2) + \text{Adv}_{\text{AEAD}, \text{kl}}^{Q_{\mathcal{D}_2}\text{-OW-CPA}}(\mathcal{D}_2) \right) + \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{E}}\text{-IND-CCA}}(\mathcal{E}) \\ &\quad + 2Q_{\text{AUTH}_1} \cdot \delta_{\text{Auth}} + \frac{(Q_{\text{PUT}_1} + Q_{\text{ACPT}_1})^2 + (Q_{\text{UPD}_1} + 1)^2}{2^{nl}} \end{aligned}$$

with

$$\begin{aligned} Q_{\mathcal{A}} &= (Q_{\text{STEP}}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1}, Q_{\text{PUT}_1}, Q_{\text{UPD}_1}, Q_{\text{GET}_1}, Q_{\text{SHARE}_1}, Q_{\text{ACPT}_1}, Q_{\text{CHALL}}, Q_{\text{COMP}}, Q_{\text{RO}}), \\ Q_{\mathcal{B}} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{AUTH}_1}, Q_{\text{AUTH}_1}, Q_{\text{REG}_1} + Q_{\text{RO}}, Q_{\text{RO}}), \\ Q_{\mathcal{C}_1} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1}), \\ Q_{\mathcal{D}_1} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, 0, Q_{\text{REG}_1}, Q_{\text{REG}_1} + Q_{\text{RO}}), \\ Q_{\mathcal{C}_2} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{PUT}_1} + Q_{\text{ACPT}_1} + Q_{\text{CHL}}, Q_{\text{UPD}_1} + Q_{\text{GET}_1} + Q_{\text{SHARE}_1} + Q_{\text{CHL}}), \\ Q_{\mathcal{D}_2} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{ACPT}_1}, Q_{\text{PUT}_1} + Q_{\text{ACPT}_1} + Q_{\text{CHL}}, Q_{\text{PUT}_1} + Q_{\text{UPD}_1} + Q_{\text{GET}_1} + Q_{\text{SHARE}_1} + Q_{\text{CHL}} + Q_{\text{RO}}), \\ Q_{\mathcal{E}} &= (Q_{\text{PUT}_1} + Q_{\text{CHL}}, Q_{\text{PUT}_1} + Q_{\text{CHL}}, Q_{\text{PUT}_1} + Q_{\text{UPD}_1}, Q_{\text{GET}_1}, Q_{\text{CHL}}). \end{aligned}$$

Proof. We prove the theorem by a sequence of games and depict only those oracles in which a change occurs, starting with Figure 31.

Game $\mathbf{G}_0$. This is the confidentiality game in which we plug in construction CS. To ease the depiction of later changes, we distinguish two random oracles. One can be called by the adversary, which is denoted by RO as usual, and one is an internal random oracle, denoted by H, which is only called from other oracles. The change is conceptual since both oracles represent the same function since they are maintaining the same table for lazy sampling. Additionally, we introduce a table $\mathbf{T}_{\text{H-int}}$ which indicates if a value was queried to the internal RO; this will be used in later changes. It holds that

$$\Pr[\mathbf{G}_0(\mathcal{A}) \Rightarrow 1] = \text{Adv}_{\text{CS}, \mathcal{PW}}^{Q_{\mathcal{A}}\text{-Conf}}(\mathcal{A}),$$

where $Q_{\mathcal{A}}$ is the same as in the theorem statement.

Game $\mathbf{G}_1$. This is the same game as the previous one except that the raw secret rw computed in the second step of the authentication protocol is replaced by the value computed during the registration oracle if the received value auth_2 is correct, i.e. is output by the response algorithm of Auth with the correct inputs. To this end, we introduce a table $\mathbf{T}_{\text{rw}}$ to store rw and server secret $\mathbf{k}$ for each aid during the call to REG_1 . The changes are depicted in Figure 31.

Claim. It holds that

$$\Pr[\mathbf{G}_0(\mathcal{A}) \Rightarrow 1] - \Pr[\mathbf{G}_1(\mathcal{A}) \Rightarrow 1] \leq Q_{\text{AUTH}_1} \cdot \delta_{\text{Auth}}.$$

Proof (Claim). Let us consider one query to the second step of the authentication protocol. The value rw is only replaced if auth_2 is in the support of $\text{Auth.Resp}(\text{auth}_1, \mathbf{k})$. Since auth_1 was computed during the first step of the authentication protocol by the game itself, it is the output of the initialization procedure of Auth. The games can only be distinguished if rw computed by Fin and Full (as done in REG_1) are different. This is

Oracle REG₁(aid) 00 require $T_{\text{uaid}}[\text{aid}] = \perp$ 01 n_r++ 02 $\text{pw} \leftarrow \text{pw}_{n_r}$ 03 $(\text{inc.aid}, \text{inc.pw}, T_{\text{pw}}[\text{aid}]) \leftarrow (\text{aid}, \text{pw}, \text{pw})$ 04 $T_{\text{uaid}}[\text{aid}] \leftarrow \text{false}$ 05 $k \xleftarrow{\$} \text{Auth}.\mathcal{D}_S$ 06 $\text{rw} \leftarrow \text{Auth.Full}[\text{H}_{\text{Auth}}](\text{pw}, \text{aid}, k)$ 07 $T_{\text{rw}}[\text{aid}] \leftarrow (\text{rw}, k)$ $\parallel G_1-G_4$ 08 if $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{rw}, \text{aid}, \cdot) \in T_H$ $\parallel G_2-G_4$ 09 abort $\parallel G_2-G_4$ 10 $k_{\text{ek}} \leftarrow H(\text{rw}, \text{aid}, \text{"ek"})$ 11 $k_{\text{mac}} \leftarrow H(\text{rw}, \text{aid}, \text{"mac"})$ 12 $(n_{\text{mk}}, \text{seed}_{\text{mk}}) \xleftarrow{\$} \{0, 1\}^{n_l} \times \{0, 1\}^{k_l}$ 13 if $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}_{\text{mk}}, \text{aid}, \cdot) \in T_H$ $\parallel G_4$ 14 abort $\parallel G_4$ 15 $k_{\text{mk}} \leftarrow H(\text{seed}_{\text{mk}}, \text{aid}, \text{"mk"})$ 16 $\text{ct}_{\text{mk}} \xleftarrow{\$} \text{AEAD}.\text{Enc}(k_{\text{ek}}, n_{\text{mk}}, \text{seed}_{\text{mk}}, (\text{aid}, \text{"mk"}))$ 17 $T_{\text{ct-mk}}[\text{ct}_{\text{mk}}, n_{\text{mk}}, \text{aid}] \leftarrow \text{seed}_{\text{mk}}$ $\parallel G_3-G_4$ 18 $m_C \leftarrow (\text{aid}, k, n_{\text{mk}}, \text{ct}_{\text{mk}}, k_{\text{mac}})$ 19 n_p++ 20 $i_p \leftarrow n_p$ 21 $T_p[(\text{areg}, i_p)] \leftarrow (\varepsilon, \text{st}_C^{\text{tmp}}, \text{inc}, 2)$ 22 return $\text{Chk}(\text{areg}, i_p, \text{out}_C, m_C)$ Oracle COMP(aid) 23 require $T_{\text{uaid}}[\text{aid}] \neq \perp$ 24 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{\text{aid}\}$ 25 return $(T_{\text{pw}}[\text{aid}], \{T_{\text{stc}}[i_c] \mid T_u[i_c] = \text{aid}\}, \{T_{\text{oob}}[(\text{aid}, \cdot)]\})$ Oracle H(seed, aid, label) 26 if $(\text{seed}, \text{aid}, \text{label}) \notin T_H$ 27 $T_H[\text{seed}, \text{aid}, \text{label}] \xleftarrow{\$} \{0, 1\}^{k_l}$ 28 $T_{H\text{-int}}[\text{seed}, \text{aid}, \text{label}] \leftarrow 1$ 29 return $T_H[\text{seed}, \text{aid}, \text{label}]$	Oracle STEP(auth, i_p, m) $\parallel$ only for round $r = 2$ 30 $(i_c, \text{st}_C^{\text{tmp}}, \text{inc}, 2) \leftarrow T_p[(\text{auth}, i_p)]$ 31 $(\text{aid}, \text{pw}, \text{st}, m'_C) \leftarrow \text{st}_C^{\text{tmp}}$ 32 $(\cdot, \text{auth}_1) \leftarrow m'_C$ 33 $(\text{sid}, \text{auth}_2) \leftarrow m$ 34 $\text{rw} \leftarrow \text{Auth.Fin}[\text{H}_{\text{Auth}}](\text{auth}_2, \text{pw}, \text{aid}, \text{st})$ 35 $(\text{rw}', k) \leftarrow T_{\text{rw}}[\text{aid}]$ $\parallel G_1-G_4$ 36 if $\text{auth}_2 = \text{Auth.Resp}(\text{auth}_1, k)$ $\parallel G_1-G_4$ 37 $\text{rw} \leftarrow \text{rw}'$ $\parallel G_1-G_4$ 38 $\text{st}_C^{\text{tmp}}.k_{\text{ek}} \leftarrow H(\text{rw}, \text{aid}, \text{"ek"})$ 39 $k_{\text{mac}} \leftarrow H(\text{rw}, \text{aid}, \text{"mac"})$ 40 $m_C \leftarrow \text{MAC.Tag}(k_{\text{mac}}, (m'_C, m_S))$ 41 $T_p[(\text{auth}, i_p)] \leftarrow (i_c, \text{st}_C^{\text{tmp}}, \text{inc}, r + 1)$ 42 return $\text{Chk}(\text{auth}, i_p, \varepsilon, m_C)$ Oracle STEP(auth, i_p, m) $\parallel$ only for round $r = 3$ 43 $(i_c, \text{st}_C^{\text{tmp}}, \text{inc}, 3) \leftarrow T_p[(\text{auth}, i_p)]$ 44 $\text{aid} \leftarrow \text{st}_C^{\text{tmp}}.\text{aid}$ 45 $(n_{\text{mk}}, \text{ct}_{\text{mk}}) \leftarrow m_S$ 46 $\text{seed}_{\text{mk}} \leftarrow \text{AEAD}.\text{Dec}(\text{st}_C^{\text{tmp}}.k_{\text{ek}}, n_{\text{mk}}, \text{ct}_{\text{mk}}, (\text{aid}, \text{"mk"}))$ 47 if $T_u[i_c] \notin \mathcal{S}_{\text{cmp}} \wedge T_{\text{ct-mk}}[\text{ct}_{\text{mk}}, n_{\text{mk}}, \text{aid}] = \perp$ $\parallel G_3-G_4$ 48 $\text{seed}_{\text{mk}} \leftarrow \perp$ $\parallel G_3-G_4$ 49 $k_{\text{mk}} \leftarrow H(\text{seed}_{\text{mk}}, \text{aid}, \text{"mk"})$ 50 $T_{\text{stc}}[i_c] \leftarrow (\text{aid}, \text{st}_C^{\text{tmp}}.\text{sid}, k_{\text{mk}})$ 51 $\text{out}_C.\text{decision} \leftarrow \text{true}$ 52 $m_C \leftarrow \top$ 53 $T_p[(\text{auth}, i_p)] \leftarrow (i_c, \text{st}_C^{\text{tmp}}, \text{inc}, r + 1)$ 54 return $\text{Chk}(\text{auth}, i_p, \text{out}_C, m_C)$ Oracle RO(seed, aid, label) 55 if $T_{\text{uaid}}[\text{aid}] \neq \perp \wedge \text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}, \text{aid}, \text{label}) \in T_{H\text{-int}}$ 56 if $\text{label} \in \{\text{"ek"}, \text{"mac"}\}$ $\parallel G_2-G_4$ 57 abort $\parallel G_2-G_4$ 58 if $\text{label} = \text{"mk"}$ $\parallel G_4$ 59 abort $\parallel G_4$ 60 if $(\text{seed}, \text{aid}, \text{label}) \notin T_H$ 61 $T_H[\text{seed}, \text{aid}, \text{label}] \xleftarrow{\$} \{0, 1\}^{k_l}$ 62 return $T_H[\text{seed}, \text{aid}, \text{label}]$ Oracle RO_{Auth}(x) 63 return $H_{\text{Auth}}(x)$
--	--

Fig. 31. Games $G_0 - G_4$ for the proof of Theorem 10. H is an internal random oracle and cannot be queried by $\mathcal{A}$.

exactly the correctness error of Auth . Since the second step of the authentication protocol can be called only if the first step was executed, the change appears at most Q_{AUTH_1} times resulting in the claimed bound. $\square$

Game G_2 . This is the same game as the previous one except that it aborts if there exists a random oracle query on a registered and uncompromised user such that the same input was previously queried to the internal RO H and the label is either “ek” or “mac”. The game also aborts in the registration oracle if there already exists a random oracle query on the same rw and aid . The changes are depicted in Figure 31.

Claim. There exists an adversary $\mathcal{B}$ against INS-OW such that

$$\Pr[G_1(\mathcal{A}) \Rightarrow 1] - \Pr[G_2(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{\text{Auth}, \mathcal{P}\mathcal{W}}^{Q\text{-INS-OW}}(\mathcal{B})$$

with $Q = (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{AUTH}_1}, Q_{\text{AUTH}_1}, Q_{\text{REG}_1} + Q_{\text{RO}}, Q_{\text{RO}})$.

Proof (Claim). We formally construct adversary $\mathcal{B}$ in Figure 32 simulating the game for $\mathcal{A}$. First, we show that $\mathcal{B}$'s simulation is sound. The registration oracle is simulated with the reduction's registration oracle and by choosing a unique challenge tag "chl" (which is not in the valid input space of the random oracle) in case a user is not compromised (yet) and the internal random oracle is queried on that tag instead of the actual rw (Line 14). This is sound because later queries to the random oracle are answered consistently:

1. In the last step of the authentication procedure, i.e. in oracle STEP in Figure 32, the finalize algorithm is simulated by using $\mathcal{B}$'s oracle FIN. Note that FIN outputs $\top$ in case it is queried on the correct value, i.e. the value of the full execution of Auth. In this way, $\mathcal{B}$ recognizes when to use the previously programmed RO and answers the query with the same value. This works because of the changes in $\mathbf{G}_1$.
2. Whenever adversary $\mathcal{A}$ queries the random oracle RO on an input that corresponds to a previous registration query, $\mathcal{B}$ can just abort the game winning their INS-OW game. $\mathcal{B}$ can recognize that by using their oracle CHECK. The reduction must also simulate an abort in the registration oracle which can also be done by using the check oracle analogously (Line 12). The other oracles, i.e. AUTH₁ and STEP, can be simulated by using $\mathcal{B}$'s oracles INIT and FIN without the need of any client secrets.

The next challenge is to handle compromises in a sound way. More precisely, after a user is compromised, random oracle queries must still be consistent. This is done by patching the random oracle on values that were previously unknown and marked with "chl". Since the reduction is querying their own compromise oracle $\text{COMP}_{\mathcal{B}}$, they learn the client secret for that user and thus can patch the oracle later (Line 65). Note that this works because the random oracle was not queried on that input before because the game would have been aborted in such a case.

Finally, note that as soon as a newly introduced abort occurs, $\mathcal{B}$ wins their INS-OW game. $\square$

Game $\mathbf{G}_3$. This is the same game as the previous one except that the decryption of every ciphertext in the third round of the authentication procedure on an uncompromised user is set to $\perp$ if the ciphertext was not created by the game. More precisely, if the tuple $(\text{ct}_{\text{mk}}, n_{\text{mk}}, \text{aid})$ is not in table $\text{T}_{\text{ct-mk}}$. The changes are depicted in Figure 31.

Claim. There exists an adversary $\mathcal{C}_1$ against INT-CTXT such that

$$\Pr[\mathbf{G}_2(\mathcal{A}) \Rightarrow 1] - \Pr[\mathbf{G}_3(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{\text{AEAD}}^{Q\text{-INT-CTXT}}(\mathcal{C}_1)$$

with $Q = (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1})$.

Proof (Claim). The reduction $\mathcal{C}_1$ is formally constructed in Figure 33 showing all oracles that are not trivially simulatable, i.e. where the reduction cannot just execute the oracle as is. The registration oracle can be simulated by using the reduction's encryption oracle on a fresh instance and the ciphertext is stored to answer decryption queries in Line 58. In the last step of the authentication procedure (oracle STEP on the authentication sub protocol in the third step, i.e. $r = 3$), the reduction can either use the value that was encrypted during the registration process as described before or if the ciphertext is new, it can check if it is a valid ciphertext using oracle CHECK. This is done only for uncompromised users. If it is a valid ciphertext, i.e. decrypting to a message $\text{seed}_{\text{mk}} \neq \perp$, $\mathcal{C}_1$ can win their INT-CTXT game by outputting the valid tuple. Since the ciphertext is new because it is not in table $\text{T}_{\text{ct-mk}}$, the winning condition of $\mathcal{C}_1$ holds.

It remains to show that the simulation of $\mathcal{C}_1$ is sound with respect to consistent random oracle outputs. The keys for the AEAD is not known to $\mathcal{C}_1$ until a party is compromised. This also means that the output of the random oracle with input $(\text{rw}, \text{aid}, \text{"ek"})$, where rw is the correct Auth output computed during registration, is not known to the reduction. However, due to the abort introduced in the previous game (Line 71), the game aborts whenever the adversary makes such a query on an uncompromised user. That allows $\mathcal{C}_1$ to patch the random oracle upon a corruption query because they learn the secret k_Q .

An additional challenge in this situation is that the reduction does not only have to make the random oracle consistent but also potential states or temporary states. The only state that contains the value k_{ek} is the temporary client state $\text{st}_{\mathcal{C}}^{\text{tmp}}$ during the authentication protocol. More specifically, in the second step of

Adversary $\mathcal{B}^{\text{REG,INIT,FIN,COMP}_B,\text{CHECK}}(k_1, \dots, k_{Q_{\text{REG}}})$ 00 $(n_c, n_p, n_r) \leftarrow (0, 0, 0)$ 01 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_{\text{ch}}, \mathcal{S}_{\text{chOwn}} \leftarrow \emptyset$ 02 $b \xleftarrow{\$} \{0, 1\}$ 03 $b' \leftarrow \mathcal{A}^O$ 04 if $\exists \text{aid} \in \mathcal{S}_{\text{chOwn}} : \text{aid} \in \mathcal{S}_{\text{cmp}} \vee T_{\text{uaid}}[\text{aid}] = \perp$ 05 return 0 06 return $\perp$ Oracle $\text{REG}_1(\text{aid})$ 07 require $T_{\text{uaid}}[\text{aid}] = \perp$ 08 n_r++ 09 $\text{pw} \leftarrow \perp$ $\parallel$ unknown 10 $k \leftarrow \text{REG}(\text{aid})$ $\parallel$ register client 11 $(\text{inc.aid}, \text{inc.pw}, T_{\text{pw}}[\text{aid}]) \leftarrow (\text{aid}, \text{pw}, n_r)$ $\parallel$ store instance 12 if $\exists \text{rw} : (\text{rw}, \text{aid}, \cdot) \in T_H \wedge \text{CHECK}(\text{rw}, \text{aid})$ 13 output (rw, aid) $\parallel$ win 14 $\text{rw} \leftarrow \text{"chl"}$ $\parallel$ mark internal RO query 15 $k_{\text{ek}} \leftarrow H(\text{rw}, \text{aid}, \text{"ek"})$ 16 $k_{\text{mac}} \leftarrow H(\text{rw}, \text{aid}, \text{"mac"})$ 17 $(n_{\text{mk}}, \text{seed}_{\text{mk}}) \xleftarrow{\$} \{0, 1\}^{n_l} \times \{0, 1\}^{k_l}$ 18 $k_{\text{mk}} \leftarrow H(\text{seed}_{\text{mk}}, \text{aid}, \text{"mk"})$ 19 $\text{ct}_{\text{mk}} \xleftarrow{\$} \text{AEAD.Enc}(k_{\text{ek}}, n_{\text{mk}}, \text{seed}_{\text{mk}}, (\text{aid}, \text{"mk"}))$ 20 $m_C \leftarrow (\text{aid}, k, n_{\text{mk}}, \text{ct}_{\text{mk}}, k_{\text{mac}})$ 21 $\text{outc.decision} \leftarrow \text{true}$ 22 n_p++ 23 $i_p \leftarrow n_p$ 24 $T_p[(\text{areg}, i_p)] \leftarrow (\varepsilon, \text{st}_C^{\text{tmp}}, \text{inc}, 2)$ 25 return $\text{Chk}(\text{areg}, i_p, \text{outc}, m_C)$ Oracle $\text{COMP}(\text{aid})$ 26 require $T_{\text{uaid}}[\text{aid}] \neq \perp$ 27 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{\text{aid}\}$ 28 $n'_r \leftarrow T_{\text{pw}}[\text{aid}]$ 29 $\text{pw}_{n'_r} \leftarrow \text{COMP}_B(n'_r)$ $\parallel$ Comp oracle 30 return $(\text{pw}_{Q'}, \{T_{\text{stc}}[\text{ic}] \mid T_u[\text{ic}] = \text{aid}\}, \{T_{\text{oob}}[(\text{aid}, \cdot)]\})$ Oracle $H(\text{seed}, \text{aid}, \text{label})$ 31 if $(\text{seed}, \text{aid}, \text{label}) \notin T_H$ 32 $T_H[\text{seed}, \text{aid}, \text{label}] \xleftarrow{\$} \{0, 1\}^{k_l}$ 33 $T_{H\text{-int}}[\text{seed}, \text{aid}, \text{label}] \leftarrow 1$ 34 return $T_H[\text{seed}, \text{aid}, \text{label}]$	Oracle $\text{AUTH}_1(\text{aid})$ 35 require $T_{\text{uaid}}[\text{aid}] = \text{true}$ 36 $\text{inc.aid} \leftarrow \text{aid}$ 37 $\text{inc.pw} \leftarrow \perp$ $\parallel$ real pw not used 38 n_c++ 39 $i_c \leftarrow n_c$ 40 $(T_u[\text{ic}], T_{\text{stc}}[\text{ic}]) \leftarrow (\text{aid}, \varepsilon)$ 41 n_p++ 42 $i_p \leftarrow n_p$ 43 $\text{auth}_1 \leftarrow \text{INIT}(\text{aid}, i_p)$ $\parallel$ Init oracle 44 $m_C \leftarrow (\text{aid}, \text{inc.pw}, m_C)$ 45 $\text{st}_C^{\text{tmp}} \leftarrow (\text{aid}, \text{inc.pw}, m_C)$ 46 $T_p[(\text{auth}, i_p)] \leftarrow (i_c, \varepsilon, \text{st}_C^{\text{tmp}}, \text{inc}, 2)$ 47 return $\text{Chk}(\text{auth}, i_p, \varepsilon, m_C)$ Oracle $\text{STEP}(\text{auth}, i_p, m)$ $\parallel$ only for round $r = 2$ 48 $(i_c, \text{st}_C^{\text{tmp}}, \text{inc}, 2) \leftarrow T_p[(\text{auth}, i_p)]$ 49 $(\text{aid}, \perp, m'_C) \leftarrow \text{st}_C^{\text{tmp}}$ 50 $(\text{aid}, \text{auth}_2) \leftarrow m$ 51 $\text{rw} \leftarrow \text{FIN}(\text{auth}_2, T_{\text{pw}}[\text{aid}], i_p)$ $\parallel$ Fin oracle 52 if $\text{rw} = \text{T}$ $\parallel$ correct execution detected 53 $\text{rw} \leftarrow \text{"chl"}$ $\parallel$ use consistent RO output 54 $\text{st}_C^{\text{tmp}}.k_{\text{ek}} \leftarrow H(\text{rw}, \text{aid}, \text{"ek"})$ 55 $k_{\text{mac}} \leftarrow H(\text{rw}, \text{aid}, \text{"mac"})$ 56 $m_C \leftarrow \text{MAC.Tag}(k_{\text{mac}}, (m'_C, m_S))$ 57 $T_p[(\text{auth}, i_p)] \leftarrow (i_c, \text{st}_C^{\text{tmp}}, \text{inc}, r + 1)$ 58 return $\text{Chk}(\text{auth}, i_p, \varepsilon, m_C)$ Oracle $\text{RO}(\text{seed}, \text{aid}, \text{label})$ 59 if $T_{\text{uaid}}[\text{aid}] \neq \perp \wedge \text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{"chl"}, \text{aid}, \text{label}) \in T_{H\text{-int}}$ 60 if $\text{label} \in \{\text{"ek"}, \text{"mac"}\} \wedge \text{CHECK}(\text{seed}, \text{aid})$ $\parallel$ win 61 output $(\text{seed}, \text{aid})$ 62 else if $T_{\text{uaid}}[\text{aid}] = \text{true}, \text{aid} \in \mathcal{S}_{\text{cmp}}$ 63 $\wedge (\text{"chl"}, \text{aid}, \text{label}) \in T_{H\text{-int}}$ 64 $n'_r \leftarrow T_{\text{pw}}[\text{aid}]$ 65 if $\text{seed} = \text{Auth.Full}[H_{\text{Auth}}](\text{pw}_{n'_r}, \text{aid}, k_{n'_r})$ 66 $T_H[\text{seed}, \text{aid}, \text{label}] \leftarrow T_{H\text{-int}}[(\text{"chl"}, \text{aid}, \text{label})]$ $\parallel$ patch RO 67 if $(\text{seed}, \text{aid}, \text{label}) \notin T_H$ 68 $T_H[\text{seed}, \text{aid}, \text{label}] \xleftarrow{\$} \{0, 1\}^{k_l}$ 69 return $T_H[\text{seed}, \text{aid}, \text{label}]$
--	---

Fig. 32. Adversary $\mathcal{B}$ against INS-OW security of Auth having access to oracles REG, INIT, FIN, COMP $_B$, and CHECK simulating $\mathcal{G}_1/\mathcal{G}_2$ for adversary $\mathcal{A}$.

the authentication protocol k_{ek} is computed and stored in the temporary state (Line 43) and in the third step it is used for decryption (Line 54). To make the third step consistent, the reduction updates the state before it is used. This can be done by storing the value rw during the second step (Line 42) and query the random oracle again on the same values in the third step (Line 51). This captures all cases since for the real rw , the random oracle was reprogrammed during a compromise. $\square$

Game $\mathcal{G}_4$. This is the same game as the previous one except that it aborts if there exists a random oracle query on a registered and uncompromised user such that the same input was previously queried to the internal RO H and the tag is “mk”. The game also aborts in the registration oracle if the queried user is not

Adversary $\mathcal{C}_1^{\text{ENC,COMP}_C,\text{CHECK}}$ 00 $(n_c, n_p, n_r, Q) \leftarrow (0, 0, 0, 0)$ 01 $(\text{pw}_1, \dots, \text{pw}_{Q_{\text{REG}_1}}) \xleftarrow{\$} \mathcal{PW}$ 02 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_{\text{ch}}, \mathcal{S}_{\text{chOwn}} \leftarrow \emptyset$ 03 $\mathbf{b} \xleftarrow{\$} \{0, 1\}$ 04 $\mathbf{b}^* \leftarrow \mathcal{A}^O$ 05 if $\exists \text{aid} \in \mathcal{S}_{\text{chOwn}} : \text{aid} \in \mathcal{S}_{\text{cmp}} \vee \mathbf{T}_{\text{uaid}}[\text{aid}] = \perp$ 06 return 0 07 return $\llbracket \mathbf{b}^* = \mathbf{b} \rrbracket$ Oracle $\text{REG}_1(\text{aid})$ 08 require $\mathbf{T}_{\text{uaid}}[\text{aid}] = \perp$ 09 n_r++ 10 $\text{inc.pw} \leftarrow \text{pw}_{n_r}$ 11 $\mathbf{T}_{\text{pw}}[\text{aid}] \leftarrow \text{inc.pw}$ 12 $\text{inc.aid} \leftarrow \text{aid}$ 13 $\mathbf{T}_{\text{uaid}}[\text{aid}] \leftarrow \text{false}$ 14 $\mathbf{k} \xleftarrow{\$} \text{Auth.DS}$ 15 $\text{rw} \leftarrow \text{Auth.Full}[\text{H}_{\text{Auth}}](\text{pw}, \text{aid}, \mathbf{k})$ 16 $\mathbf{T}_{\text{rw}}[\text{aid}] \leftarrow (\text{rw}, \mathbf{k})$ 17 if $(\text{rw}, \text{aid}, \cdot) \in \mathbf{T}_{\text{H}}$ 18 abort 19 $\mathbf{k}_{\text{mac}} \leftarrow \text{H}(\text{rw}, \text{aid}, \text{"mac"})$ 20 $(\mathbf{n}_{\text{mk}}, \text{seed}_{\text{mk}}) \xleftarrow{\$} \{0, 1\}^{n_l} \times \{0, 1\}^{k_l}$ 21 $\mathbf{k}_{\text{mk}} \leftarrow \text{H}(\text{seed}_{\text{mk}}, \text{aid}, \text{"mk"})$ 22 $Q++$ $\ll$ new instance 23 $\mathbf{T}_{\text{key}}[\text{aid}] \leftarrow Q$ 24 $\mathbf{ct}_{\text{mk}} \xleftarrow{\$} \text{ENC}(Q, \mathbf{n}_{\text{mk}}, \text{seed}_{\text{mk}}, (\text{aid}, \text{"mk"}))$ $\ll$ Enc oracle 25 $\mathbf{T}_{\text{ct-mk}}[\mathbf{ct}_{\text{mk}}, \mathbf{n}_{\text{mk}}, \text{aid}] \leftarrow \text{seed}_{\text{mk}}$ $\ll$ store plaintext 26 $\mathbf{m}_{\text{C}} \leftarrow (\text{aid}, \mathbf{k}, \mathbf{n}_{\text{mk}}, \mathbf{ct}_{\text{mk}}, \mathbf{k}_{\text{mac}})$ 27 n_p++ 28 $i_p \leftarrow n_p$ 29 $\mathbf{T}_p[(\text{areg}, i_p)] \leftarrow (\varepsilon, \text{st}_{\text{C}}^{\text{tmp}}, \text{inc}, 2)$ 30 return $\text{Chk}(\text{areg}, i_p, \text{out}_{\text{C}}, \mathbf{m}_{\text{C}})$ Oracle $\text{COMP}(\text{aid})$ 31 require $\mathbf{T}_{\text{uaid}}[\text{aid}] \neq \perp$ 32 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{\text{aid}\}$ 33 $Q' \leftarrow \mathbf{T}_{\text{key}}[\text{aid}]$ 34 $\mathbf{k}_{Q'} \leftarrow \text{COMP}_C(Q')$ $\ll$ Comp oracle 35 $(\text{rw}, \cdot) \leftarrow \mathbf{T}_{\text{rw}}[\text{aid}]$ 36 $\mathbf{T}_{\text{H}}[\text{rw}, \text{aid}, \text{"ek"}] \leftarrow \mathbf{k}_{Q'}$ $\ll$ patch RO 37 return $(\mathbf{T}_{\text{pw}}[\text{aid}], \{\mathbf{T}_{\text{stc}}[\text{ic}] \mid \mathbf{T}_{\text{u}}[\text{ic}] = \text{aid}\}, \{\mathbf{T}_{\text{oob}}[(\text{aid}, \cdot)]\})$	Oracle $\text{STEP}(\text{auth}, i_p, \mathbf{m})$ $\ll$ only for round $r = 2$ 38 $(\text{ic}, \text{st}_{\text{C}}^{\text{tmp}}, \text{inc}, 2) \leftarrow \mathbf{T}_p[(\Pi, i_p)]$ 39 $(\text{aid}, \text{pw}, \text{st}, \mathbf{m}_{\text{C}}) \leftarrow \text{st}_{\text{C}}^{\text{tmp}}$ 40 $(\text{sid}, \text{auth}_2) \leftarrow \mathbf{m}$ 41 $\text{rw} \leftarrow \text{Auth.Fin}[\text{H}_{\text{Auth}}](\text{auth}_2, \text{pw}, \text{aid}, \text{st})$ 42 $\mathbf{T}_{\text{auth}_2}[i_p] \leftarrow \text{rw}$ $\ll$ store info for potential patching 43 $\text{st}_{\text{C}}^{\text{tmp}}.\mathbf{k}_{\text{ek}} \leftarrow \text{H}(\text{rw}, \text{aid}, \text{"ek"})$ 44 $\mathbf{k}_{\text{mac}} \leftarrow \text{H}(\text{rw}, \text{aid}, \text{"mac"})$ 45 $\mathbf{m}_{\text{C}} \leftarrow \text{MAC.Tag}(\mathbf{k}_{\text{mac}}, (\mathbf{m}_{\text{C}}', \mathbf{m}_{\text{S}}))$ 46 $\mathbf{T}_p[(\Pi, i_p)] \leftarrow (\text{ic}, \text{st}_{\text{C}}^{\text{tmp}}, \text{inc}, r + 1)$ 47 return $\text{Chk}(\Pi, i_p, \varepsilon, \mathbf{m}_{\text{C}})$ Oracle $\text{STEP}(\text{auth}, i_p, \mathbf{m})$ $\ll$ only for round $r = 3$ 48 $(\text{ic}, \text{st}_{\text{C}}^{\text{tmp}}, \text{inc}, 3) \leftarrow \mathbf{T}_p[(\text{auth}, i_p)]$ 49 if $\mathbf{T}_{\text{u}}[\text{ic}] \in \mathcal{S}_{\text{cmp}}$ 50 $\text{rw} \leftarrow \mathbf{T}_{\text{auth}_2}[i_p]$ $\ll$ recover rw used 51 $\text{st}_{\text{C}}^{\text{tmp}}.\mathbf{k}_{\text{ek}} \leftarrow \text{H}(\text{rw}, \text{aid}, \text{"ek"})$ $\ll$ patch tmp state 52 $\text{aid} \leftarrow \text{st}_{\text{C}}^{\text{tmp}}.\text{aid}$ 53 $(\mathbf{n}_{\text{mk}}, \mathbf{ct}_{\text{mk}}) \leftarrow \mathbf{m}_{\text{S}}$ 54 $\text{seed}_{\text{mk}} \leftarrow \text{AEAD.Dec}(\text{st}_{\text{C}}^{\text{tmp}}.\mathbf{k}_{\text{ek}}, \mathbf{n}_{\text{mk}}, \mathbf{ct}_{\text{mk}}, (\text{aid}, \text{"mk"}))$ 55 if $\mathbf{T}_{\text{u}}[\text{ic}] \notin \mathcal{S}_{\text{cmp}} \wedge \mathbf{T}_{\text{ct-mk}}[\mathbf{ct}_{\text{mk}}, \mathbf{n}_{\text{mk}}, \text{aid}] = \perp$ 56 $\wedge \text{CHECK}(\mathbf{T}_{\text{key}}[\mathbf{T}_{\text{u}}[\text{ic}]], \mathbf{n}_{\text{mk}}, \mathbf{ct}_{\text{mk}}, (\text{aid}, \text{"mk"})) = 1$ 57 output $(\mathbf{T}_{\text{key}}[\mathbf{T}_{\text{u}}[\text{ic}]], \mathbf{n}_{\text{mk}}, \mathbf{ct}_{\text{mk}}, (\text{aid}, \text{"mk"}))$ $\ll$ win 58 else if $\mathbf{T}_{\text{u}}[\text{ic}] \notin \mathcal{S}_{\text{cmp}}$ 59 $\text{seed}_{\text{mk}} \leftarrow \mathbf{T}_{\text{ct-mk}}[\mathbf{ct}_{\text{mk}}, \mathbf{n}_{\text{mk}}, \text{aid}]$ $\ll$ plaintext is known 60 $\mathbf{k}_{\text{mk}} \leftarrow \text{H}(\text{seed}_{\text{mk}}, \text{aid}, \text{"mk"})$ 61 $\mathbf{T}_{\text{stc}}[\text{ic}] \leftarrow (\text{aid}, \text{st}_{\text{C}}^{\text{tmp}}.\text{sid}, \mathbf{k}_{\text{mk}})$ 62 $\text{out}_{\text{C}}.\text{decision} \leftarrow \text{true}$ 63 $\mathbf{m}_{\text{C}} \leftarrow \mathbf{T}$ 64 $\mathbf{T}_p[(\text{auth}, i_p)] \leftarrow (\text{ic}, \text{st}_{\text{C}}^{\text{tmp}}, \text{inc}, r + 1)$ 65 return $\text{Chk}(\text{auth}, i_p, \text{out}_{\text{C}}, \mathbf{m}_{\text{C}})$ Oracle $\text{H}(\text{seed}, \text{aid}, \text{label})$ 65 if $(\text{seed}, \text{aid}, \text{label}) \notin \mathbf{T}_{\text{H}}$ 66 $\mathbf{T}_{\text{H}}[\text{seed}, \text{aid}, \text{label}] \xleftarrow{\$} \{0, 1\}^{k_l}$ 67 $\mathbf{T}_{\text{H-int}}[\text{seed}, \text{aid}, \text{label}] \leftarrow 1$ 68 return $\mathbf{T}_{\text{H}}[\text{seed}, \text{aid}, \text{label}]$ Oracle $\text{RO}(\text{seed}, \text{aid}, \text{label})$ 69 if $\mathbf{T}_{\text{uaid}}[\text{aid}] \neq \perp \wedge \text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}, \text{aid}, \text{label}) \in \mathbf{T}_{\text{H-int}}$ 70 if $\text{label} \in \{\text{"ek"}, \text{"mac"}\}$ 71 abort 72 if $(\text{seed}, \text{aid}, \text{label}) \notin \mathbf{T}_{\text{H}}$ 73 $\mathbf{T}_{\text{H}}[\text{seed}, \text{aid}, \text{label}] \xleftarrow{\$} \{0, 1\}^{k_l}$ 74 return $\mathbf{T}_{\text{H}}[\text{seed}, \text{aid}, \text{label}]$
---	---

Fig. 33. Adversary $\mathcal{C}_1$ against INT-CTXT security of AEAD having access to oracles ENC, COMP_C, and CHECK simulating $\mathcal{G}_2/\mathcal{G}_3$ for adversary $\mathcal{A}$.

compromised and there already exists a random oracle query on the same seed_{mk} and aid. The changes are depicted in Figure 31.

Claim. There exists an adversary $\mathcal{D}_1$ against OW-CPA such that

$$\Pr[\mathcal{G}_3(\mathcal{A}) \Rightarrow 1] - \Pr[\mathcal{G}_4(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{\text{AEAD,kl}}^{\text{Q-OW-CPA}}(\mathcal{D}_1),$$

with $Q = (Q_{\text{REG}_1}, Q_{\text{COMP}}, 0, Q_{\text{REG}_1}, Q_{\text{REG}_1} + Q_{\text{RO}})$.

Adversary $\mathcal{D}_1^{\text{ENC, CHL, COMP}_{\mathcal{D}}, \text{CHECK}}$ <pre> 00 $(n_c, n_p, n_r, Q) \leftarrow (0, 0, 0, 0)$ 01 $(\text{pw}_1, \dots, \text{pw}_{Q_{\text{REG}_1}}) \xleftarrow{\\$} \mathcal{PW}$ 02 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_{\text{ch}}, \mathcal{S}_{\text{chOwn}} \leftarrow \emptyset$ 03 $\mathbf{b} \xleftarrow{\\$} \{0, 1\}$ 04 $\mathbf{b}^* \leftarrow \mathcal{A}^0$ 05 if $\exists \text{aid} \in \mathcal{S}_{\text{chOwn}} : \text{aid} \in \mathcal{S}_{\text{cmp}} \vee \mathbf{T}_{\text{uaid}}[\text{aid}] = \perp$ 06 return 0 07 return $\llbracket \mathbf{b}^* = \mathbf{b} \rrbracket$ Oracle $\text{REG}_1(\text{aid})$ 08 require $\mathbf{T}_{\text{uaid}}[\text{aid}] = \perp$ 09 n_r++ 10 $\text{inc.pw} \leftarrow \text{pw}_{n_r}$ 11 $\mathbf{T}_{\text{pw}}[\text{aid}] \leftarrow \text{inc.pw}$ 12 $\text{inc.aid} \leftarrow \text{aid}$ 13 $\mathbf{T}_{\text{uaid}}[\text{aid}] \leftarrow \text{false}$ 14 $\mathbf{k} \xleftarrow{\\$} \text{Auth.DS}$ 15 $\text{rw} \leftarrow \text{Auth.Full}[\text{H}_{\text{Auth}}](\text{pw}, \text{aid}, \mathbf{k})$ 16 $\mathbf{T}_{\text{rw}}[\text{aid}] \leftarrow (\text{rw}, \mathbf{k})$ 17 if $(\text{rw}, \text{aid}, \cdot) \in \mathbf{T}_{\text{H}}$ 18 abort 19 $\mathbf{k}_{\text{ek}} \leftarrow \text{H}(\text{rw}, \text{aid}, \text{"ek"})$ 20 $\mathbf{k}_{\text{mac}} \leftarrow \text{H}(\text{rw}, \text{aid}, \text{"mac"})$ 21 $\mathbf{n}_{\text{mk}} \xleftarrow{\\$} \{0, 1\}^{n_l}$ 22 $\text{seed}_{\text{mk}} \xleftarrow{\\$} \{0, 1\}^{k_l}$ 23 $Q++$ // new instance 24 $\mathbf{T}_{\text{key}}[\text{aid}] \leftarrow Q$ 25 if $\text{aid} \notin \mathcal{S}_{\text{cmp}}$ 26 $\text{ct}_{\text{mk}} \xleftarrow{\\$} \text{CHL}(Q, \mathbf{n}_{\text{mk}}, Q, (\text{aid}, \text{"mk"}))$ // CHL oracle 27 $\mathbf{T}_{\text{Enc}}[\text{aid}] \leftarrow (\text{ct}_{\text{mk}}, \mathbf{n}_{\text{mk}})$ 28 if $\exists \text{seed} : (\text{seed}, \text{aid}, \text{"mk"}) \in \mathbf{T}_{\text{H}}$ 29 $\wedge \text{CHECK}(Q, \mathbf{n}_{\text{mk}}, \text{ct}_{\text{mk}}, (\text{aid}, \text{"mk"}), \text{seed})$ 30 output $(Q, \mathbf{n}_{\text{mk}}, \text{ct}_{\text{mk}}, (\text{aid}, \text{"mk"}), \text{seed})$ // win 31 $\text{seed}_{\text{mk}} \leftarrow \text{"chl"}$ // mark RO query 32 else 33 $\text{ct}_{\text{mk}} \leftarrow \text{AEAD.Enc}(\mathbf{k}_i, \mathbf{n}_{\text{mk}}, \text{seed}_{\text{mk}}, (\text{aid}, \text{"mk"}))$ 34 $\mathbf{k}_{\text{mk}} \leftarrow \text{H}(\text{seed}_{\text{mk}}, \text{aid}, \text{"mk"})$ 35 $\mathbf{T}_{\text{ct-mk}}[\text{ct}_{\text{mk}}, \mathbf{n}_{\text{mk}}, \text{aid}] \leftarrow \text{seed}_{\text{mk}}$ 36 $\mathbf{m}_{\text{C}} \leftarrow (\text{aid}, \mathbf{k}, \mathbf{n}_{\text{mk}}, \text{ct}_{\text{mk}}, \mathbf{k}_{\text{mac}})$ 37 n_p++ 38 $\mathbf{i}_p \leftarrow n_p$ 39 $\mathbf{T}_p[(\Pi, \mathbf{i}_p)] \leftarrow (\varepsilon, \text{st}_{\text{C}}^{\text{tmp}}, \text{inc}, 2)$ 40 return $\text{Chk}(\Pi, \mathbf{i}_p, \text{out}_{\text{C}}, \mathbf{m}_{\text{C}})$ Oracle $\text{COMP}(\text{aid})$ 40 require $\mathbf{T}_{\text{uaid}}[\text{aid}] \neq \perp$ 41 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{\text{aid}\}$ 42 $Q' \leftarrow \mathbf{T}_{\text{key}}[\text{aid}]$ 43 $\mathbf{k}_{Q'} \leftarrow \text{COMP}_{\text{C}}(Q')$ // Comp oracle 44 $(\text{rw}, \cdot) \leftarrow \mathbf{T}_{\text{rw}}[\text{aid}]$ 45 $\mathbf{T}_{\text{H}}[\text{rw}, \text{aid}, \text{"ek"}] \leftarrow \mathbf{k}_{Q'}$ // patch RO 46 return $(\mathbf{T}_{\text{pw}}[\text{aid}], \{\mathbf{T}_{\text{st}_{\text{C}}}[\mathbf{i}_{\text{C}}] \mid \mathbf{T}_{\text{u}}[\mathbf{i}_{\text{C}}] = \text{aid}\}, \{\mathbf{T}_{\text{ob}}[\text{aid}, \cdot]\})$ </pre>	Oracle $\text{STEP}(\text{auth}, \mathbf{i}_p, \mathbf{m})$ // only for round $r = 2$ <pre> 47 $(\mathbf{i}_{\text{C}}, \text{st}_{\text{C}}^{\text{tmp}}, \text{inc}, 2) \leftarrow \mathbf{T}_p[(\Pi, \mathbf{i}_p)]$ 48 $(\text{aid}, \text{pw}, \text{st}, \mathbf{m}_{\text{C}}) \leftarrow \text{st}_{\text{C}}^{\text{tmp}}$ 49 $(\text{sid}, \text{auth}_2) \leftarrow \mathbf{m}$ 50 $\text{rw} \leftarrow \text{Auth.Fin}[\text{H}_{\text{Auth}}](\text{auth}_2, \text{pw}, \text{aid}, \text{st})$ 51 $\mathbf{T}_{\text{auth}_2}[\mathbf{i}_p] \leftarrow \text{rw}$ // store info for potential patching 52 $\text{st}_{\text{C}}^{\text{tmp}}.\mathbf{k}_{\text{ek}} \leftarrow \text{H}(\text{rw}, \text{aid}, \text{"ek"})$ 53 $\mathbf{k}_{\text{mac}} \leftarrow \text{H}(\text{rw}, \text{aid}, \text{"mac"})$ 54 $\mathbf{m}_{\text{C}} \leftarrow \text{MAC.Tag}(\mathbf{k}_{\text{mac}}, (\mathbf{m}_{\text{C}}', \mathbf{m}_{\text{S}}))$ 55 $\mathbf{T}_p[(\Pi, \mathbf{i}_p)] \leftarrow (\mathbf{i}_{\text{C}}, \text{st}_{\text{C}}^{\text{tmp}}, \text{inc}, r + 1)$ 56 return $\text{Chk}(\Pi, \mathbf{i}_p, \varepsilon, \mathbf{m}_{\text{C}})$ Oracle $\text{STEP}(\text{auth}, \mathbf{i}_p, \mathbf{m})$ // only for round $r = 3$ 57 $(\mathbf{i}_{\text{C}}, \text{st}_{\text{C}}^{\text{tmp}}, \text{inc}, 3) \leftarrow \mathbf{T}_p[(\Pi, \mathbf{i}_p)]$ 58 if $\mathbf{T}_{\text{u}}[\mathbf{i}_{\text{C}}] \in \mathcal{S}_{\text{cmp}}$ 59 $\text{rw} \leftarrow \mathbf{T}_{\text{auth}_2}[\mathbf{i}_p]$ // recover rw used 60 $\text{st}_{\text{C}}^{\text{tmp}}.\mathbf{k}_{\text{ek}} \leftarrow \text{H}(\text{rw}, \text{aid}, \text{"ek"})$ // patch tmp state 61 $\text{aid} \leftarrow \text{st}_{\text{C}}^{\text{tmp}}.\text{aid}$ 62 $(\mathbf{n}_{\text{mk}}, \text{ct}_{\text{mk}}) \leftarrow \mathbf{m}_{\text{S}}$ 63 if $\mathbf{T}_{\text{u}}[\mathbf{i}_{\text{C}}] \notin \mathcal{S}_{\text{cmp}}$ 64 $\text{seed}_{\text{mk}} \leftarrow \mathbf{T}_{\text{ct-mk}}[\text{ct}_{\text{mk}}, \mathbf{n}_{\text{mk}}, \text{aid}]$ // potentially $\perp$ 65 else 66 $\text{seed}_{\text{mk}} \leftarrow \text{AEAD.Dec}(\text{st}_{\text{C}}^{\text{tmp}}.\mathbf{k}_{\text{ek}}, \mathbf{n}_{\text{mk}}, \text{ct}_{\text{mk}}, (\text{aid}, \text{"mk"}))$ 67 $\mathbf{k}_{\text{mk}} \leftarrow \text{H}(\text{seed}_{\text{mk}}, \text{aid}, \text{"mk"})$ 68 $\mathbf{T}_{\text{st}_{\text{C}}}[\mathbf{i}_{\text{C}}] \leftarrow (\text{aid}, \text{st}_{\text{C}}^{\text{tmp}}.\text{sid}, \mathbf{k}_{\text{mk}})$ 69 $\text{out}_{\text{C}}.\text{decision} \leftarrow \text{true}$ 70 $\mathbf{m}_{\text{C}} \leftarrow \mathbf{T}$ 71 $\mathbf{T}_p[(\Pi, \mathbf{i}_p)] \leftarrow (\mathbf{i}_{\text{C}}, \text{st}_{\text{C}}^{\text{tmp}}, \text{inc}, r + 1)$ 72 return $\text{Chk}(\Pi, \mathbf{i}_p, \text{out}_{\text{C}}, \mathbf{m}_{\text{C}})$ Oracle $\text{H}(\text{seed}, \text{aid}, \text{label})$ 73 if $(\text{seed}, \text{aid}, \text{label}) \notin \mathbf{T}_{\text{H}}$ 74 $\mathbf{T}_{\text{H}}[\text{seed}, \text{aid}, \text{label}] \xleftarrow{\\$} \{0, 1\}^{k_l}$ 75 $\mathbf{T}_{\text{H-int}}[\text{seed}, \text{aid}, \text{label}] \leftarrow 1$ 76 return $\mathbf{T}_{\text{H}}[\text{seed}, \text{aid}, \text{label}]$ Oracle $\text{RO}(\text{seed}, \text{aid}, \text{label})$ 77 if $\mathbf{T}_{\text{uaid}}[\text{aid}] \neq \perp \wedge \text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge ((\text{"chl"}, \text{aid}, \text{label}) \in \mathbf{T}_{\text{H}} \vee (\text{seed}, \text{aid}, \text{label}) \in \mathbf{T}_{\text{H}})$ 78 if $\text{label} \in \{\text{"ek"}, \text{"mac"}\} \wedge (\text{seed}, \text{aid}, \text{label}) \in \mathbf{T}_{\text{H}}$ 79 abort 80 if $\text{label} = \text{"mk"} \wedge (\text{"chl"}, \text{aid}, \text{label}) \in \mathbf{T}_{\text{H}}$ 81 $(\text{ct}_{\text{mk}}, \mathbf{n}_{\text{mk}}) \leftarrow \mathbf{T}_{\text{Enc}}[\text{aid}]$ 82 $Q' \leftarrow \mathbf{T}_{\text{key}}[\text{aid}]$ 83 if $\text{CHECK}(Q', \mathbf{n}_{\text{mk}}, \text{ct}_{\text{mk}}, (\text{aid}, \text{"mk"}), \text{seed})$ 84 output $(Q', \mathbf{n}_{\text{mk}}, \text{ct}_{\text{mk}}, (\text{aid}, \text{"mk"}), \text{seed})$ // win 85 if $(\text{seed}, \text{aid}, \text{label}) \notin \mathbf{T}_{\text{H}}$ 86 $\mathbf{T}_{\text{H}}[\text{seed}, \text{aid}, \text{label}] \xleftarrow{\\$} \{0, 1\}^{k_l}$ 87 return $\mathbf{T}_{\text{H}}[\text{seed}, \text{aid}, \text{label}]$ </pre>
--	---

Fig. 34. Adversary $\mathcal{D}_1$ against OW-CPA security of AEAD having access to oracles ENC, CHL, $\text{COMP}_{\mathcal{D}}$, and CHECK simulating $\mathbf{G}_3/\mathbf{G}_4$ for adversary $\mathcal{A}$.

Proof (Claim). Adversary $\mathcal{D}_1$ is formally constructed in Figure 34. They simulate their registration oracle by calling CHL for uncompromised users. This works since seed_{mk} is a uniformly random value from the message set $\{0, 1\}^{kl}$. If the adversary queried the random oracle on a value seed which is by chance the encrypted value seed_{mk} , the reduction can recognize that by using oracle CHECK and win the OW-CPA game. As for the previous reductions, the random oracle is queried on a unique symbol “chl” (not being in the valid input space of the random oracle) and programmed later. The authentication protocol can be simulated as follows. Due to the changes in the previous game, $\mathcal{D}_1$ can answer all queries for uncompromised users: either by returning $\perp$ if the ciphertext is new or by querying the random oracle on “chl” if the ciphertext came from a previous query to the registration oracle. For the random oracle, if adversary $\mathcal{A}$ queries it on a value corresponding to a challenge ciphertext, which can be recognized by a query to CHECK again, the reduction wins their one-wayness game. The simulation of compromises can be handled identical to the reduction in Figure 33 from the proof of $\mathbf{G}_3$. $\square$

Game $\mathbf{G}_5$. This is the same game as the previous one except that it aborts if there is a collision in the nonces for the same user or file during queries to the put oracle, the update oracle, or the accept oracle. In more detail, we use set $\mathcal{S}_{\text{keys}, \text{aid}}$ to store the nonces used by user aid for key encryption and $\mathcal{S}_{\text{files}, \text{fid}}$ to store the nonces used to encrypt files with id fid . If the game chooses a nonce already present in the set, it aborts. The changes are depicted in Figure 35.

Claim. It hold that

$$\Pr[\mathbf{G}_4(\mathcal{A}) \Rightarrow 1] - \Pr[\mathbf{G}_5(\mathcal{A}) \Rightarrow 1] \leq \frac{(Q_{\text{PUT}_1} + Q_{\text{ACPT}_1})^2 + (Q_{\text{UPD}_1} + 1)^2}{2^{nl+1}}.$$

Proof (Claim). For $\mathcal{S}_{\text{keys}, \text{aid}}$, there is one nonce chosen per query to PUT_1 or ACPT_1 for a particular user aid . Since nonces have length nl , the first part of the bound is the probability of a collision for the respective number of queries. For a given file id fid , there is one encryption in the call to PUT_1 and at most Q_{UPD_1} encryptions in update oracle calls resulting in the presented bound. $\square$

Game $\mathbf{G}_6$. This is the same game as the previous one except that the decryption of every file key ciphertext owned by uncompromised user is set to $\perp$ if the ciphertext was not created by the game. More precisely, if the tuple $(\text{aid}, n_k, \text{fid}, \text{ct}_k)$ is not in table $\mathbf{T}_{\text{ct-k}}$. A decryption of file key ciphertexts occurs in the update, get, and share sub protocol. To simplify the depiction, we use subroutine `getHeader` which is then queried from the three oracles. The changes are depicted in Figure 35.

Claim. There exists an adversary $\mathcal{C}_2$ against INT-CTXT such that

$$\Pr[\mathbf{G}_5(\mathcal{A}) \Rightarrow 1] - \Pr[\mathbf{G}_6(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{\text{AEAD}}^{\text{Q-INT-CTXT}}(\mathcal{C}_2)$$

with $Q = (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{PUT}_1} + Q_{\text{ACPT}_1} + Q_{\text{CHL}}, Q_{\text{UPD}_1} + Q_{\text{GET}_1} + Q_{\text{SHARE}_1} + Q_{\text{CHL}})$.

Proof (Claim). The reduction $\mathcal{C}_2$ is formally constructed in Figure 36 showing all oracles that are not trivially simulatable, i.e. by just executing the oracle as is. The put and accept oracle can be simulated by using the reduction’s encryption oracle and the ciphertext is stored to answer decryption queries. We start a new game instance if there was no instance started for aid yet. The instances are stored in table $\mathbf{T}_{\text{key}}$. Note that even though the counter Q is incremented in several oracles, there can be at most Q_{REG_1} many instances since this is an upper bound on the number of aid ’s registered.

The decryption of a file key ciphertext occurs during the update, get, and share protocol but we use the abstraction of the `getHeader` sub protocol to simplify the depiction. Whenever the reduction needs to simulate such a decryption, they can either use the value that was encrypted by the game before or if the ciphertext is new, it can check if it is a valid ciphertext using oracle CHECK. This is done only for uncompromised users. If it is a valid ciphertext, i.e. decrypting to a message $\text{seed}_k \neq \perp$, $\mathcal{C}_2$ can win their INT-CTXT game by

Oracle PUT₁($i_c \in [n_c]$, fid, file)

```

00 require  $T_f[\text{fid}] = \perp$ 
01  $T_f[\text{fid}] \leftarrow \{T_u[i_c]\}$ 
02  $(\text{inc.fid}, \text{inc.file}) \leftarrow (\text{fid}, \text{file})$ 
03  $\text{aid} \leftarrow T_u[i_c]$ 
04  $\text{stc} \leftarrow T_{\text{stc}}[i_c]$ 
05  $n_k \xleftarrow{\$} \{0, 1\}^{n_l}$ 
06  $n_f \xleftarrow{\$} \{0, 1\}^{n_l}$ 
07 if  $n_k \in \mathcal{S}_{\text{keys}, \text{aid}} \vee n_f \in \mathcal{S}_{\text{files}, \text{fid}}$ 
08   abort
09  $\mathcal{S}_{\text{keys}, \text{aid}} \leftarrow \mathcal{S}_{\text{keys}, \text{aid}} \cup \{n_k\}$ 
10  $\mathcal{S}_{\text{files}, \text{fid}} \leftarrow \mathcal{S}_{\text{files}, \text{fid}} \cup \{n_f\}$ 
11  $\text{ad} \leftarrow (\text{stc.aid}, \text{fid}, \text{"fk"})$ 
12  $\text{seed}_k \xleftarrow{\$} \{0, 1\}^{k_l}$ 
13  $\text{ct}_k \xleftarrow{\$} \text{AEAD.Enc}(\text{stc.k}_{\text{mk}}, n_k, \text{seed}_k, \text{ad})$ 
14  $T_{\text{ct-k}}[\text{aid}, n_k, \text{fid}, \text{ct}_k] \leftarrow \text{seed}_k$ 
15 if  $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}_k, \text{fid}, \cdot) \in T_H$ 
16   abort
17  $k_f \xleftarrow{\$} H(\text{seed}_k, \text{stc.aid}, \text{"fk"})$ 
18  $\text{ct}_f \xleftarrow{\$} \text{AEAD.Enc}(k_f, n_f, \text{file}, (\text{fid}, \text{"fk"}))$ 
19  $\text{mc} \leftarrow (\text{stc.sid}, \text{fid}, n_k, n_f, \text{ct}_k, \text{ct}_f)$ 
20  $\text{outc.decision} \leftarrow \text{true}$ 
21  $n_p++$ 
22  $i_p \leftarrow n_p$ 
23  $T_p[(\Pi, i_p)] \leftarrow (i_c, \varepsilon, \text{inc}, 2)$ 
24 return  $\text{Chk}(\Pi, i_p, \text{outc}, \text{mc})$ 

```

Oracle STEP(update, $i_p \in [n_p]$, m) $\parallel$ only for round $r = 2$

```

25  $(i_c, \text{stc}^{\text{tmp}}, \text{inc}, 2) \leftarrow T_p[(\Pi, i_p)]$ 
26  $\text{aid} \leftarrow T_u[i_c]$ 
27  $\text{fid} \leftarrow \text{stc}^{\text{tmp}}.\text{fid}$ 
28  $\text{seed}_k \leftarrow \text{getHeader}^{C:2}(\text{stc}, \text{fid}, \text{m}_s)$ 
29  $k_f \leftarrow H(\text{seed}_k, \text{fid}, \text{"fk"})$ 
30  $n \xleftarrow{\$} \{0, 1\}^{n_l}$ 
31 if  $n \in \mathcal{S}_{\text{files}, \text{fid}}$ 
32   abort
33  $\mathcal{S}_{\text{files}, \text{fid}} \leftarrow \mathcal{S}_{\text{files}, \text{fid}} \cup \{n\}$ 
34  $\text{ct}_f \xleftarrow{\$} \text{AEAD.Enc}(k_f, n, \text{stc}^{\text{tmp}}.\text{file}, (\text{fid}, \text{"file"}))$ 
35  $\text{mc} \leftarrow (\text{stc.sid}, \text{fid}, n, \text{ct}_k, \text{ct}_f)$ 
36  $\text{outc.decision} \leftarrow \text{true}$ 
37  $T_p[(\Pi, i_p)] \leftarrow (i_c, \text{stc}^{\text{tmp}}, \text{inc}, 3)$ 
38 return  $\text{Chk}(\Pi, i_p, \text{outc}, \text{mc})$ 

```

Oracle H(seed, aid, label)

```

39 if  $(\text{seed}, \text{aid}, \text{label}) \notin T_H$ 
40    $T_H[\text{seed}, \text{aid}, \text{label}] \xleftarrow{\$} \{0, 1\}^{k_l}$ 
41    $T_{H\text{-int}}[\text{seed}, \text{aid}, \text{label}] \leftarrow 1$ 
42 return  $T_H[\text{seed}, \text{aid}, \text{label}]$ 

```

Oracle COMP(aid)

```

43 require  $T_{\text{uaid}}[\text{aid}] \neq \perp$ 
44  $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{\text{aid}\}$ 
45 return  $(T_{\text{pw}}[\text{aid}], \{T_{\text{stc}}[i_c] \mid T_u[i_c] = \text{aid}\}, \{T_{\text{oob}}[(\text{aid}, \cdot)]\})$ 

```

Oracle ACPT₁($i_c \in [n_c]$, fid, oob)

```

46  $\text{aid} \leftarrow T_u[i_c]$ 
47 require  $\text{oob} \neq \perp \vee T_{\text{oob}}[(\text{aid}, \text{fid})] \neq \perp$ 
48  $T_f[\text{fid}] \leftarrow T_f[\text{fid}] \cup \{\text{aid}\}$ 
49 if  $\text{oob} = \perp$ 
50    $\text{inc.oob} \leftarrow T_{\text{oob}}[(\text{aid}, \text{fid})]$ 
51 else
52    $T_f[\text{fid}] \leftarrow T_f[\text{fid}] \cup \{i_{\text{mal}}\}$ 
53    $\text{inc.oob} \leftarrow \text{oob}$ 
54 if  $\text{fid} \in \mathcal{S}_{\text{ch}}$ 
55    $\mathcal{S}_{\text{chOwn}} \leftarrow \mathcal{S}_{\text{chOwn}} \cup T_f[\text{fid}]$ 
56  $\text{stc} \leftarrow T_{\text{stc}}[i_c]$ 
57  $\text{ad} \leftarrow (\text{stc.aid}, \text{inc.fid}, \text{"fk"})$ 
58  $n \xleftarrow{\$} \{0, 1\}^{n_l}$ 
59 if  $n \in \mathcal{S}_{\text{keys}, \text{aid}}$ 
60   abort
61  $\mathcal{S}_{\text{keys}, \text{aid}} \leftarrow \mathcal{S}_{\text{keys}, \text{aid}} \cup \{n\}$ 
62  $\text{seed}_k \leftarrow \text{inc.oob}$ 
63  $\text{ct}_k \xleftarrow{\$} \text{AEAD.Enc}(\text{stc.k}_{\text{mk}}, n, \text{seed}_k, \text{ad})$ 
64  $T_{\text{ct-k}}[\text{aid}, n, \text{fid}, \text{ct}_k] \leftarrow \text{seed}_k$ 
65  $\text{mc} \leftarrow (\text{stc.sid}, \text{inc.fid}, n, \text{ct}_k)$ 
66  $\text{outc.decision} \leftarrow \text{true}$ 
67  $n_p++$ 
68  $i_p \leftarrow n_p$ 
69  $T_p[(\Pi, i_p)] \leftarrow (i_c, \varepsilon, \text{inc}, 2)$ 
70 return  $\text{Chk}(\Pi, i_p, \text{outc}, \text{mc})$ 

```

Oracle RO(seed, id, label)

```

71 if  $T_{\text{uaid}}[\text{id}] = \text{true}, \text{id} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}, \text{id}, \text{label}) \in T_{H\text{-int}}$ 
72   if  $\text{label} \in \{\text{"ek"}, \text{"mac"}\}$ 
73     abort
74   if  $\text{label} = \text{"mk"}$ 
75     abort
76 if  $T_f[\text{id}] \cap \mathcal{S}_{\text{cmp}} = \emptyset \wedge (\text{seed}, \text{id}, \text{label}) \in T_{H\text{-int}}$ 
77   if  $\text{label} = \text{"fk"}$ 
78     abort
79 if  $(\text{seed}, \text{id}, \text{label}) \notin T_H$ 
80    $T_H[\text{seed}, \text{id}, \text{label}] \xleftarrow{\$} \{0, 1\}^{k_l}$ 
81 return  $T_H[\text{seed}, \text{id}, \text{label}]$ 

```

Oracle CHALL($i_c \in [n_c]$, fid, file₀, file₁, prot)

```

82 require  $\text{prot} \in \{\text{put}, \text{update}\}$ 
83  $\mathcal{S}_{\text{ch}} \leftarrow \mathcal{S}_{\text{ch}} \cup \{\text{fid}\}$ 
84  $\mathcal{S}_{\text{chOwn}} \leftarrow \mathcal{S}_{\text{chOwn}} \cup \{T_u[i_c]\} \cup T_f[\text{fid}]$ 
85 return  $\text{prot}(i_c, \text{fid}, \text{file}_b)$ 

```

getHeader^{C:2}(stc, fid, m)

```

86  $(n_k, \text{ct}_k) \leftarrow m$ 
87  $\text{aid} \leftarrow \text{stc.aid}$ 
88  $\text{ad} \leftarrow (\text{aid}, \text{fid}, \text{"fk"})$ 
89  $\text{seed}_k \leftarrow \text{AEAD.Dec}(\text{stc.k}_{\text{mk}}, n_k, \text{ct}_k, \text{ad})$ 
90 if  $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge T_{\text{ct-k}}[\text{aid}, n_k, \text{fid}, \text{ct}_k] = \perp$ 
91    $\text{seed}_k \leftarrow \perp$ 
92 if  $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}_k, \text{fid}, \cdot) \in T_H$ 
93   abort
94 return  $\text{seed}_k$ 

```

Fig. 35. Games $G_4 - G_7$ for the proof of Theorem 10.

Adversary $\mathcal{C}_2^{\text{ENC, COMP}_C, \text{CHECK}}$

```

00  $n_c, n_p, Q \leftarrow 0$ 
01  $(\text{pw}_1, \dots, \text{pw}_{Q_{\text{REG}}}) \xleftarrow{\$} \mathcal{PW}$ 
02  $\mathcal{S}_{\text{cmp}}, \mathcal{S}_{\text{ch}}, \mathcal{S}_{\text{chOwn}} \leftarrow \emptyset$ 
03  $\mathbf{b} \xleftarrow{\$} \{0, 1\}$ 
04  $\mathbf{b}^* \leftarrow \mathcal{A}^0$ 
05 if  $\exists \text{aid} \in \mathcal{S}_{\text{chOwn}} : \text{aid} \in \mathcal{S}_{\text{cmp}} \vee \text{T}_{\text{uaid}}[\text{aid}] = \perp$ 
06   return 0
07 return  $\llbracket \mathbf{b}^* = \mathbf{b} \rrbracket$ 

```

Oracle $\text{PUT}_1(i_c \in [n_c], \text{fid}, \text{file})$

```

08 require  $\text{T}_f[\text{fid}] = \perp$ 
09  $\text{aid} \leftarrow \text{T}_u[i_c]$ 
10  $\text{stc} \leftarrow \text{T}_{\text{stc}}[i_c]$ 
11  $(\text{inc.fid}, \text{inc.file}) \leftarrow (\text{fid}, \text{file})$ 
12  $\text{T}_f[\text{fid}] \leftarrow \{\text{T}_u[i_c]\}$ 
13  $\mathbf{n}_k \xleftarrow{\$} \{0, 1\}^{\text{nl}}$ 
14  $\mathbf{n}_f \xleftarrow{\$} \{0, 1\}^{\text{nl}}$ 
15 if  $\mathbf{n}_k \in \mathcal{S}_{\text{keys,aid}} \vee \mathbf{n}_f \in \mathcal{S}_{\text{files,fid}}$ 
16   abort
17  $\mathcal{S}_{\text{keys,aid}} \leftarrow \mathcal{S}_{\text{keys,aid}} \cup \{\mathbf{n}_k\}$ 
18  $\mathcal{S}_{\text{files,fid}} \leftarrow \mathcal{S}_{\text{files,fid}} \cup \{\mathbf{n}_f\}$ 
19  $\text{ad} \leftarrow (\text{aid}, \text{fid}, \text{"fk"})$ 
20  $\text{seed}_k \xleftarrow{\$} \{0, 1\}^{\text{kl}}$ 
21 if  $\text{T}_{\text{key}}[\text{aid}] = \perp$  // check if user key used before
22    $Q++$  // new instance
23    $\text{T}_{\text{key}}[\text{aid}] \leftarrow Q$ 
24    $Q' \leftarrow \text{T}_{\text{key}}[\text{aid}]$ 
25    $\text{ct}_k \xleftarrow{\$} \text{ENC}(Q', \mathbf{n}_k, \text{seed}_k, \text{ad})$  // Enc oracle
26    $\text{T}_{\text{ct-k}}[\text{aid}, \mathbf{n}_k, \text{fid}, \text{ct}_k] \leftarrow \text{seed}_k$  // store ciphertext
27    $\mathbf{k}_f \xleftarrow{\$} \text{H}(\text{seed}_k, \text{stc.aid}, \text{"fk"})$ 
28    $\text{ct}_f \xleftarrow{\$} \text{AEAD.Enc}(\mathbf{k}_f, \mathbf{n}_f, \text{file}, (\text{fid}, \text{"fk"}))$ 
29    $\mathbf{m}_c \leftarrow (\text{stc.sid}, \text{fid}, \mathbf{n}_k, \mathbf{n}_f, \text{ct}_k, \text{ct}_f)$ 
30    $\text{outc.decision} \leftarrow \text{true}$ 
31    $n_p++$ 
32    $\mathbf{i}_p \leftarrow n_p$ 
33    $\text{T}_p[(\Pi, \mathbf{i}_p)] \leftarrow (i_c, \varepsilon, \text{inc}, 2)$ 
34   return  $\text{Chk}(\Pi, \mathbf{i}_p, \text{outc}, \mathbf{m}_c)$ 

```

Oracle $\text{COMP}(\text{aid})$

```

35 require  $\text{T}_{\text{uaid}}[\text{aid}] \neq \perp$ 
36  $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{\text{aid}\}$ 
37 if  $\text{T}_{\text{key}}[\text{aid}] = \perp$  // check if user key used before
38    $Q++$  // new instance
39    $\text{T}_{\text{key}}[\text{aid}] \leftarrow Q$ 
40    $Q' \leftarrow \text{T}_{\text{key}}[\text{aid}]$ 
41    $\mathbf{k}_{Q'} \leftarrow \text{COMP}_C(Q')$  // Comp oracle
42    $\text{seed}_{\text{mk}} \leftarrow \text{get seed from previous queries}$ 
43    $\text{T}_H[\text{seed}_{\text{mk}}, \text{aid}, \text{"mk"}] \leftarrow \mathbf{k}_{Q'}$  // patch random oracle
44   for  $i_c : \text{T}_u[i_c] = \text{aid}$ 
45      $\text{T}_{\text{stc}}[i_c].\mathbf{k}_{\text{mk}} \leftarrow \mathbf{k}_{Q'}$  // patch state
46   return  $(\text{T}_{\text{pw}}[\text{aid}], \{\text{T}_{\text{stc}}[i_c] \mid \text{T}_u[i_c] = \text{aid}\}, \{\text{T}_{\text{oob}}[\text{aid}, \cdot]\})$ 

```

Oracle $\text{ACPT}_1(i_c \in [n_c], \text{fid}, \text{oob})$

```

47  $\text{aid} \leftarrow \text{T}_u[i_c]$ 
48 require  $\text{oob} \neq \perp \vee \text{T}_{\text{oob}}[(\text{aid}, \text{fid})] \neq \perp$ 
49  $\text{T}_f[\text{fid}] \leftarrow \text{T}_f[\text{fid}] \cup \{\text{aid}\}$ 
50 if  $\text{oob} = \perp$ :
51    $\text{inc.oob} \leftarrow \text{T}_{\text{oob}}[(\text{aid}, \text{fid})]$ 
52 else
53    $\text{T}_f[\text{fid}] \leftarrow \text{T}_f[\text{fid}] \cup \{i_{\text{mal}}\}$ 
54    $\text{inc.oob} \leftarrow \text{oob}$ 
55 if  $\text{fid} \in \mathcal{S}_{\text{ch}}$ :
56    $\mathcal{S}_{\text{chOwn}} \leftarrow \mathcal{S}_{\text{chOwn}} \cup \text{T}_f[\text{fid}]$ 
57    $\text{stc} \leftarrow \text{T}_{\text{stc}}[i_c]$ 
58    $\text{ad} \leftarrow (\text{aid}, \text{inc.fid}, \text{"fk"})$ 
59    $\mathbf{n} \xleftarrow{\$} \{0, 1\}^{\text{nl}}$ 
60   if  $\mathbf{n} \in \mathcal{S}_{\text{keys,aid}}$ 
61     abort
62    $\mathcal{S}_{\text{keys,aid}} \leftarrow \mathcal{S}_{\text{keys,aid}} \cup \{\mathbf{n}\}$ 
63    $\text{seed}_k \leftarrow \text{inc.oob}$ 
64   if  $\text{T}_{\text{key}}[\text{aid}] = \perp$  // check if user key used before
65      $Q++$  // new instance
66      $\text{T}_{\text{key}}[\text{aid}] \leftarrow Q$ 
67      $Q' \leftarrow \text{T}_{\text{key}}[\text{aid}]$ 
68      $\text{ct}_k \xleftarrow{\$} \text{ENC}(Q', \mathbf{n}_k, \text{seed}_k, \text{ad})$  // Enc oracle
69      $\text{T}_{\text{ct-k}}[\text{aid}, \mathbf{n}_k, \text{fid}, \text{ct}_k] \leftarrow \text{seed}_k$  // store ciphertext
70      $\mathbf{m}_c \leftarrow (\text{stc.sid}, \text{inc.fid}, \mathbf{n}, \text{ct}_k)$ 
71      $\text{outc.decision} \leftarrow \text{true}$ 
72      $n_p++$ 
73      $\mathbf{i}_p \leftarrow n_p$ 
74      $\text{T}_p[(\Pi, \mathbf{i}_p)] \leftarrow (i_c, \varepsilon, \text{inc}, 2)$ 
75     return  $\text{Chk}(\Pi, \mathbf{i}_p, \text{outc}, \mathbf{m}_c)$ 

```

getHeader $^{C:2}(\text{stc}, \text{fid}, \mathbf{m})$

```

76  $(\mathbf{n}_k, \text{ct}_k) \leftarrow \mathbf{m}$ 
77  $\text{aid} \leftarrow \text{stc.aid}$ 
78  $\text{ad} \leftarrow (\text{aid}, \text{fid}, \text{"fk"})$ 
79 if  $\text{T}_{\text{key}}[\text{aid}] = \perp$  // check if user key used before
80    $Q++$  // new instance
81    $\text{T}_{\text{key}}[\text{aid}] \leftarrow Q$ 
82    $Q' \leftarrow \text{T}_{\text{key}}[\text{aid}]$ 
83    $\text{seed}_k \leftarrow \text{AEAD.Dec}(\text{stc.k}_{\text{mk}}, \mathbf{n}_k, \text{ct}_k, \text{ad})$ 
84   if  $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge \text{T}_{\text{ct-k}}[\text{aid}, \mathbf{n}_k, \text{fid}, \text{ct}_k] = \perp \wedge \text{CHECK}(Q', \mathbf{n}_k, \text{ct}_k, \text{ad}) = 1$ 
85     output  $(Q', \mathbf{n}_k, \text{ct}_k, \text{ad})$  // win
86   else if  $i \notin \mathcal{S}_{\text{cmp}}$ 
87      $\text{seed}_k \leftarrow \text{T}_{\text{ct-k}}[i, \mathbf{n}_k, \text{fid}, \text{ct}_k]$  // potentially  $\perp$ 
88   return  $\text{seed}_k$ 

```

Oracle $\text{RO}(\text{seed}, \text{id}, \text{label})$

```

89 if  $\text{T}_{\text{uaid}}[\text{aid}] \neq \perp \wedge \text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}, \text{id}, \text{label}) \in \text{T}_{\text{H-int}}$ 
90   if  $\text{label} \in \{\text{"ek"}, \text{"mac"}\}$ 
91     abort
92   if  $\text{label} = \text{"mk"}$ 
93     abort
94   if  $(\text{seed}, \text{id}, \text{label}) \notin \text{T}_H$ 
95      $\text{T}_H[\text{seed}, \text{id}, \text{label}] \xleftarrow{\$} \{0, 1\}^{\text{kl}}$ 
96   return  $\text{T}_H[\text{seed}, \text{id}, \text{label}]$ 

```

Fig. 36. Adversary $\mathcal{C}_2$ against INT-CTXT security of AEAD having access to oracles ENC, COMP_C , and CHECK simulating $\mathcal{G}_5/\mathcal{G}_6$ for adversary $\mathcal{A}$.

outputting the valid tuple. Since the ciphertext is new because it is not in table $T_{\text{ct-mk}}$, the winning condition of $\mathcal{C}_2$ holds.

It remains to show that the simulation of $\mathcal{C}_2$ is sound with respect to consistent random oracle outputs. The keys for the AEAD are not known to $\mathcal{C}_2$ until a party is compromised. This also means that the output of the random oracle with input $(\text{seed}_{\text{mk}}, \text{aid}, \text{"mk"})$, where seed_{mk} is the correct seed chosen during the registration process for that user, is not known to the reduction. However, due to the abort introduced in $\mathbf{G}_4$ (Line 93), the game aborts whenever the adversary makes such a query for an uncompromised user. That allows $\mathcal{C}_2$ to patch the random oracle upon a corruption query because they learn the secret $k_{Q'}$.

An additional challenge in this situation is that the reduction does not only have to make the random oracle consistent but also potential states. This holds for all states of user aid because the persistent states include the master key. Since the states are directly output by the compromise oracle, they must be patched immediately. Hence, the reduction must go through all protocol states for user aid and add $k_{Q'}$ (Line 45). $\square$

Game $\mathbf{G}_7$. This is the same game as the previous one except that it aborts if there exists a random oracle query on a file id for which the file is only owned by uncompromised users, the same input was previously queried to the internal RO H , and the tag is "fk" (Line 78). The game also aborts in the `getHeader` routine if the queried user is not compromised and there already exists a random oracle query on the same seed_{mk} and aid . The changes are depicted in Figure 35.

Claim. There exists an adversary $\mathcal{D}_2$ against OW-CPA such that

$$\Pr[\mathbf{G}_6(\mathcal{A}) \Rightarrow 1] - \Pr[\mathbf{G}_7(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{\text{AEAD,kl}}^{Q\text{-OW-CPA}}(\mathcal{D}_2)$$

with $Q = (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{ACPT}_1}, Q_{\text{PUT}_1} + Q_{\text{ACPT}_1} + Q_{\text{CHL}}, Q_{\text{PUT}_1} + Q_{\text{UPD}_1} + Q_{\text{GET}_1} + Q_{\text{SHARE}_1} + Q_{\text{CHL}} + Q_{\text{RO}})$.

Proof (Claim). Adversary $\mathcal{D}_2$ is formally constructed in Figure 37. For ease depiction, we use subroutine `getHeader`^{C:2} and model the aborts occurring in the update, get, and share oracle within that subroutine. The game for adversary $\mathcal{A}$ can be simulated by the reduction using their own oracles because due to the changes in $\mathbf{G}_5$ we have unique nonces which is a requirement for the challenge oracle in the OW-CPA game. In more detail, the put oracle can be simulated for uncompromised users using $\mathcal{D}_2$'s challenge oracle with a fresh message. This represents choosing a new (unknown) seed seed_k and encrypting it. If there was a previous random oracle query such that the input is the unknown message of the just created ciphertext, adversary $\mathcal{D}_2$ wins their game. This can be checked using oracle `CHECK`. If the random oracle was not queried on a correct value, it is queried on challenge tag "chl" to patch it later. The ciphertext together with some relevant information like the message tag Q_m is stored for later use (Line 27). The main challenge for the reduction compared to previous games is that files can be shared by users. More specifically, the seeds for the file keys are shared. Since these seeds are encrypted by each user (who is an owner of some file), the reduction needs to simulate ciphertexts for different users that encrypt the same plaintext. This is why we need the challenge oracle `CHL` to have the message tag as an input, i.e. adversary $\mathcal{D}_2$ can specify to either obtain the encryption of a new message by querying the challenge on a fresh index Q_m or obtain an encryption of a previous query by querying the challenge on a previously used Q'_m . In the accept oracle, this is how $\mathcal{D}_2$ is simulating a query to an uncompromised user in case the seed for a file with id fid was already encrypted.

The aborts in the update, get, and share oracles are applied in the subroutine `getHeader` and follow the same structure as the previous reductions since $\mathcal{D}_2$ can check correct seeds via their check oracle. The same holds for the abort in the random oracle.

For consistent simulation, the reduction needs to patch the random oracle upon corruption. To this end, $\mathcal{D}_2$ recovers the master key seed for user aid and patches the random oracle on the user's master key $k_{Q'_u}$ which is obtained by $\mathcal{D}_2$'s own compromise oracle. Note that the simulation is sound because the random oracle was not queried before on the same inputs due to the abort in Line 112 introduced in $\mathbf{G}_4$. Patching the states is similar to the reduction for $\mathbf{G}_6$. $\square$

Final reduction. Game $\mathbf{G}_7$ is now in a state where we can reduce it to the IND-CCA security of AEAD.

Adversary $\mathcal{D}_2^{\text{ENC, CHL, COMP}_{\mathcal{D}}, \text{CHECK}}$ <pre> 00 $n_c, n_p, Q_u, Q_m \leftarrow \perp$ 01 $(\text{pw}_1, \dots, \text{pw}_{Q_{\text{REG}}}) \xleftarrow{\\$} \mathcal{PW}$ 02 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_{\text{ch}}, \mathcal{S}_{\text{chOwn}} \leftarrow \emptyset$ 03 $\mathbf{b} \xleftarrow{\\$} \{0, 1\}$ 04 $\mathbf{b}^* \leftarrow \mathcal{A}^0$ 05 if $\exists \text{aid} \in \mathcal{S}_{\text{chOwn}} : \text{aid} \in \mathcal{S}_{\text{cmp}} \vee \text{T}_{\text{uaid}}[\text{aid}] = \perp$ 06 return 0 07 return $\llbracket \mathbf{b}^* = \mathbf{b} \rrbracket$ </pre> Oracle $\text{PUT}_1(\text{ic} \in [n_c], \text{fid}, \text{file})$ <pre> 08 require $\text{T}_f[\text{fid}] = \perp$ 09 $\text{aid} \leftarrow \text{T}_u[\text{ic}]$ 10 $\text{stc} \leftarrow \text{T}_{\text{stc}}[\text{ic}]$ 11 $(\text{inc.fid}, \text{inc.file}) \leftarrow (\text{fid}, \text{file})$ 12 $\text{T}_f[\text{fid}] \leftarrow \{\text{T}_u[\text{ic}]\}$ 13 $\mathbf{n}_k \xleftarrow{\\$} \{0, 1\}^{n_l}$ 14 $\mathbf{n}_f \xleftarrow{\\$} \{0, 1\}^{n_l}$ 15 if $\mathbf{n}_k \in \mathcal{S}_{\text{keys,aid}} \vee \mathbf{n}_f \in \mathcal{S}_{\text{files,fid}}$ 16 abort 17 $\mathcal{S}_{\text{keys,aid}} \leftarrow \mathcal{S}_{\text{keys,aid}} \cup \{\mathbf{n}_k\}$ 18 $\mathcal{S}_{\text{files,fid}} \leftarrow \mathcal{S}_{\text{files,fid}} \cup \{\mathbf{n}_f\}$ 19 $\text{ad} \leftarrow (\text{stc.aid}, \text{fid}, \text{"fk"})$ 20 if $\text{aid} \notin \mathcal{S}_{\text{cmp}}$ 21 if $\text{T}_{\text{key}}[\text{aid}] = \perp$ \(\backslash\backslash\) check if user key used before 22 Q_u++ \(\backslash\backslash\) new instance 23 $\text{T}_{\text{key}}[\text{aid}] \leftarrow Q_u$ 24 $Q'_u \leftarrow \text{T}_{\text{key}}[\text{aid}]$ 25 Q_m++ \(\backslash\backslash\) new seed 26 $\text{ctk} \xleftarrow{\\$} \text{CHL}(Q'_u, \mathbf{n}_k, Q_m, \text{ad})$ \(\backslash\backslash\) CHL oracle 27 $\text{T}_{\text{Enc}}[\text{fid}] \leftarrow \text{T}_{\text{Enc}}[\text{fid}] \cup \{(\text{aid}, \mathbf{n}_k, Q_m, \text{ctk})\}$ \(\backslash\backslash\) store ciphertext 28 if $\exists \text{seed} : (\text{seed}, \text{fid}, \text{"fk"}) \in \text{T}_H$ 29 $\wedge \text{CHECK}(Q'_u, \mathbf{n}_k, \text{ctk}, \text{ad}, \text{seed})$ 30 output $(Q'_u, \mathbf{n}_k, \text{ctk}, \text{ad}, \text{seed})$ \(\backslash\backslash\) win 31 $\text{seed}_k \leftarrow \text{"chl"}$ 32 else 33 $\text{seed}_k \xleftarrow{\\$} \{0, 1\}^{k_l}$ 34 $\text{ctk} \xleftarrow{\\$} \text{AEAD.Enc}(\text{stc.kmk}, \mathbf{n}_k, \text{seed}_k, \text{ad})$ 35 $\mathbf{k}_f \xleftarrow{\\$} \text{H}(\text{seed}_k, \text{stc.aid}, \text{"fk"})$ 36 $\text{ctr} \xleftarrow{\\$} \text{AEAD.Enc}(\mathbf{k}_f, \mathbf{n}_f, \text{file}, (\text{fid}, \text{"fk"}))$ 37 $\text{mc} \leftarrow (\text{stc.sid}, \text{fid}, \mathbf{n}_k, \mathbf{n}_f, \text{ctr}, \text{ctr})$ 38 $\text{outc.decision} \leftarrow \text{true}$ 39 n_p++ 40 $\text{T}_p[(\Pi, \text{i}_p)] \leftarrow (\text{ic}, \varepsilon, \text{inc}, 2)$ 41 return $\text{Chk}(\Pi, \text{i}_p, \text{outc}, \text{mc})$ </pre> Oracle $\text{H}(\text{seed}, \text{aid}, \text{label})$ <pre> 42 if $(\text{seed}, \text{aid}, \text{label}) \notin \text{T}_H$ 43 $\text{T}_H[\text{seed}, \text{aid}, \text{label}] \xleftarrow{\\$} \{0, 1\}^{k_l}$ 44 $\text{T}_{H\text{-int}}[\text{seed}, \text{aid}, \text{label}] \leftarrow 1$ 45 return $\text{T}_H[\text{seed}, \text{aid}, \text{label}]$ </pre> Oracle $\text{COMP}(\text{aid})$ <pre> 46 require $\text{T}_{\text{uaid}}[\text{aid}] \neq \perp$ 47 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{\text{aid}\}$ 48 if $\text{T}_{\text{key}}[\text{aid}] = \perp$ \(\backslash\backslash\) check if user key used before 49 Q_u++ \(\backslash\backslash\) new instance 50 $\text{T}_{\text{key}}[\text{aid}] \leftarrow Q_u$ 51 $Q'_u \leftarrow \text{T}_{\text{key}}[\text{aid}]$ 52 $\mathbf{k}_{Q'_u} \leftarrow \text{COMP}_{\mathcal{C}}(Q'_u)$ \(\backslash\backslash\) Comp oracle 53 $\text{seed}_{\text{mk}} \leftarrow \text{get seed from previous queries}$ 54 $\text{T}_H[\text{seed}_{\text{mk}}, \text{aid}, \text{"mk"}] \leftarrow \mathbf{k}_{Q'_u}$ \(\backslash\backslash\) patch random oracle 55 for $\text{ic} : \text{T}_u[\text{ic}] = \text{aid}$ 56 $\text{T}_{\text{stc}}[\text{ic}].\text{kmk} \leftarrow \mathbf{k}_{Q'_u}$ \(\backslash\backslash\) patch state 57 return $(\text{T}_{\text{pw}}[\text{aid}], \{\text{T}_{\text{stc}}[\text{ic}] \mid \text{T}_u[\text{ic}] = \text{aid}\}, \{\text{T}_{\text{oob}}[(\text{aid}, \cdot)]\})$ </pre>	Oracle $\text{ACPT}_1(\text{ic} \in [n_c], \text{fid}, \text{oob})$ <pre> 58 $\text{aid} \leftarrow \text{T}_u[\text{ic}]$ 59 require $\text{oob} \neq \perp \vee \text{T}_{\text{oob}}[(\text{aid}, \text{fid})] \neq \perp$ 60 $\text{T}_f[\text{fid}] \leftarrow \text{T}_f[\text{fid}] \cup \{\text{aid}\}$ 61 if $\text{oob} = \perp$: 62 $\text{inc.oob} \leftarrow \text{T}_{\text{oob}}[(\text{aid}, \text{fid})]$ 63 else 64 $\text{T}_f[\text{fid}] \leftarrow \text{T}_f[\text{fid}] \cup \{\text{i}_{\text{mal}}\}$ 65 $\text{inc.oob} \leftarrow \text{oob}$ 66 if $\text{fid} \in \mathcal{S}_{\text{ch}}$: 67 $\mathcal{S}_{\text{chOwn}} \leftarrow \mathcal{S}_{\text{chOwn}} \cup \text{T}_f[\text{fid}]$ 68 $\text{stc} \leftarrow \text{T}_{\text{stc}}[\text{ic}]$ 69 $\text{ad} \leftarrow (\text{aid}, \text{inc.fid}, \text{"fk"})$ 70 $\mathbf{n} \xleftarrow{\\$} \{0, 1\}^{n_l}$ 71 if $\mathbf{n} \in \mathcal{S}_{\text{keys,aid}}$ 72 abort 73 $\mathcal{S}_{\text{keys,aid}} \leftarrow \mathcal{S}_{\text{keys,aid}} \cup \{\mathbf{n}\}$ 74 $\text{seed}_k \leftarrow \text{inc.oob}$ 75 $\text{ctk} \xleftarrow{\\$} \text{AEAD.Enc}(\text{stc.kmk}, \mathbf{n}, \text{seed}_k, \text{ad})$ 76 if $\text{aid} \notin \mathcal{S}_{\text{cmp}}$ 77 if $\text{T}_{\text{key}}[\text{aid}] = \perp$ \(\backslash\backslash\) check if user key used before 78 Q_u++ \(\backslash\backslash\) new instance 79 $\text{T}_{\text{key}}[\text{aid}] \leftarrow Q_u$ 80 $Q'_u \leftarrow \text{T}_{\text{key}}[\text{aid}]$ 81 if $\exists (\cdot, \cdot, Q'_m, \cdot) \in \text{T}_{\text{Enc}}[\text{fid}]$ 82 $\text{ctk} \xleftarrow{\\$} \text{CHL}(Q'_u, \mathbf{n}, Q'_m, \text{ad})$ \(\backslash\backslash\) CHL oracle 83 $\text{T}_{\text{Enc}}[\text{fid}] \leftarrow \text{T}_{\text{Enc}}[\text{fid}] \cup \{(\text{aid}, \mathbf{n}, Q'_m, \text{ctk})\}$ \(\backslash\backslash\) store ct 84 else 85 $\text{ctk} \xleftarrow{\\$} \text{ENC}(Q'_u, \mathbf{n}, \text{seed}_k, \text{ad})$ \(\backslash\backslash\) Enc oracle 86 $\text{mc} \leftarrow (\text{stc.sid}, \text{inc.fid}, \mathbf{n}, \text{ctk})$ 87 $\text{outc.decision} \leftarrow \text{true}$ 88 n_p++ 89 $\text{i}_p \leftarrow n_p$ 90 $\text{T}_p[(\Pi, \text{i}_p)] \leftarrow (\text{ic}, \varepsilon, \text{inc}, 2)$ 91 return $\text{Chk}(\Pi, \text{i}_p, \text{outc}, \text{mc})$ </pre> $\text{getHeader}^{C:2}(\text{stc}, \text{fid}, \text{m})$ <pre> 92 $(\mathbf{n}_k, \text{ctk}) \leftarrow \text{m}$ 93 $\text{aid} \leftarrow \text{stc.aid}$ 94 $\text{ad} \leftarrow (\text{aid}, \text{fid}, \text{"fk"})$ 95 $\text{seed}_k \leftarrow \text{AEAD.Dec}(\text{stc.kmk}, \mathbf{n}_k, \text{ctk}, \text{ad})$ 96 if $\text{aid} \notin \mathcal{S}_{\text{cmp}}$ 97 if $\text{T}_{\text{key}}[\text{aid}] = \perp$ \(\backslash\backslash\) check if user key used before 98 Q_u++ \(\backslash\backslash\) new instance 99 $\text{T}_{\text{key}}[\text{aid}] \leftarrow Q_u$ 100 $Q'_u \leftarrow \text{T}_{\text{key}}[\text{aid}]$ 101 if $\exists \text{seed} : (\text{seed}, \text{fid}, \text{"fk"}) \in \text{T}_H \wedge \text{CHECK}(Q'_u, \mathbf{n}_k, \text{ctk}, \text{ad}, \text{seed})$ 102 output $(Q'_u, \mathbf{n}_k, \text{ctk}, \text{ad}, \text{seed})$ \(\backslash\backslash\) win 103 else if $\exists Q'_m : (Q'_u, \mathbf{n}_k, Q'_m, \text{ctk}) \in \text{T}_{\text{Enc}}[\text{fid}]$ \(\backslash\backslash\) check challenge ct 104 $\text{seed}_k \leftarrow \text{"chl"}$ 105 else 106 $\text{seed}_k \leftarrow \perp$ \(\backslash\backslash\) new ciphertext 107 return seed_k </pre> Oracle $\text{RO}(\text{seed}, \text{id}, \text{label})$ <pre> 108 if $\text{T}_{\text{uaid}}[\text{id}] \neq \perp \wedge \text{id} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}, \text{id}, \text{label}) \in \text{T}_{H\text{-int}}$ 109 if $\text{label} \in \{\text{"ek"}, \text{"mac"}\}$ 110 abort 111 if $\text{label} = \text{"mk"}$ 112 abort 113 if $\text{T}_f[\text{id}] \cap \mathcal{S}_{\text{cmp}} = \emptyset \wedge (\text{"chl"}, \text{id}, \text{label}) \in \text{T}_{H\text{-int}} \wedge \text{label} = \text{"fk"}$ 114 $(Q'_u, \mathbf{n}, \cdot, \text{ctk}) \xleftarrow{\\$} \text{T}_{\text{Enc}}[\text{id}]$ \(\backslash\backslash\) arbitrary challenge ct 115 $\text{find aid} : \text{T}_{\text{key}}[\text{aid}] = Q'_u$ 116 if $\text{CHECK}(Q'_u, \mathbf{n}, \text{ctk}, (\text{aid}, \text{id}, \text{"fk"}), \text{seed})$ 117 output $(Q'_u, \mathbf{n}, \text{ctk}, (\text{aid}, \text{id}, \text{"fk"}), \text{seed})$ \(\backslash\backslash\) win 118 if $(\text{seed}, \text{id}, \text{label}) \notin \text{T}_H$ 119 $\text{T}_H[\text{seed}, \text{id}, \text{label}] \xleftarrow{\\$} \{0, 1\}^{k_l}$ 120 return $\text{T}_H[\text{seed}, \text{id}, \text{label}]$ </pre>
---	---

Fig. 37. Adversary $\mathcal{D}_2$ against OW-CPA security of AEAD having access to oracles ENC , $\text{COMP}_{\mathcal{D}}$, and CHECK simulating $\mathcal{G}_6/\mathcal{G}_7$ for adversary $\mathcal{A}$.

Claim. There exists an adversary $\mathcal{E}$ against IND-CCA such that

$$2 \Pr[\mathbf{G}_7(\mathcal{A}) \Rightarrow 1] - 1 \leq \text{Adv}_{\text{AEAD}}^{Q\text{-IND-CCA}}(\mathcal{E})$$

with $Q = (Q_{\text{PUT}_1} + Q_{\text{CHL}}, Q_{\text{PUT}_1} + Q_{\text{CHL}}, Q_{\text{PUT}_1} + Q_{\text{UPD}_1}, Q_{\text{GET}_1}, Q_{\text{CHL}})$.

Proof (Claim). Adversary $\mathcal{E}$ is formally constructed in Figure 38. $\mathcal{E}$ is not sampling file keys themselves but using their own IND-CCA security game against AEAD. For each query to PUT_1 , they increase a counter Q and use their own encryption oracle ENC with a new instance to encrypt the file. If there is a challenge query implicitly querying the put oracle but on file file_b , $\mathcal{E}$ queries their challenge oracle CHL instead. This is done using both files, file_0 and file_1 , from $\mathcal{A}$'s challenge query. To simplify the depiction in Figure 38, we combine direct put queries and put queries called via the challenge oracle and use the same lines of code. The differences explained before are highlighted with comment PUT_1C . Further, the instance number Q is stored together with the seed seed_k and the file id fid in table T_{fid} such that the reduction can later query their oracles on the correct instance. This happens in the second step of the get oracle where $\mathcal{E}$ needs to potentially decrypt a ciphertext with respect to a previously used file key (Line 66) which is the only situation in which a file decryption is necessary.

In the second step of the update procedure, $\mathcal{E}$ might require an encryption with respect to a unknown file key again. In case the update oracle was queried directly, i.e. not via a challenge to CHL, the reduction can recognize previously used instances via table T_{fid} and use oracle ENC. Analogously to the put challenge, challenges for the update routine can be answered via a query to $\text{CHL}_{\mathcal{E}}$.

Note that this simulation is sound because the adversary cannot learn file keys that belong to any uncompromised user due to the abort introduced in $\mathbf{G}_7$. Further, due to the changes in $\mathbf{G}_5$, we do not have any colliding nonces such that $\mathcal{E}$'s oracles always output a solution.

The remaining issue is to handle compromise queries. A compromise query of $\mathcal{A}$ is with respect to a user while $\mathcal{E}$ is playing a game with several AEAD instances corresponding to files instead of users. Hence, upon a corruption on user aid we could check all files which aid has access to and compromise the corresponding file keys for all these files. To obtain a tighter reduction, we deploy the reverse check for any query to the random oracle (either explicit ones to RO or implicit ones by the game to H. A reverse check means that we check if the file for a query to the random oracle (if there exists a file id id) is owned by a compromised user. If this is the case, we look for the correct instance which is stored in T_{fid} and compromise the key of said instance if this was not done already. The random oracle can then be patched on this value. Note that the number of compromises is still not the number of random oracle queries since there are at most $Q_{\text{PUT}_1} + Q_{\text{CHL}}$ many file keys that can be corrupted. $\square$

Collecting the probabilities completes the proof of Theorem 10. $\square$

C.2 Integrity

We restate Theorem 6 using concrete bounds and provide the full proof.

Theorem 11 (Integrity). *For any password distribution $\mathcal{PW}$ and adversary $\mathcal{A}$ against the Int security of $\text{CS} := \text{CS}[\text{Auth}, \text{AEAD}, \text{MAC}]$ there exist an INS-OW adversary $\mathcal{B}$ against Auth, INT-CTXT adversaries $\mathcal{C}_1, \mathcal{C}_2$ against AEAD, OW-CPA adversaries $\mathcal{D}_1, \mathcal{D}_2$ against AEAD, and an INT-CTXT adversary $\mathcal{E}$ against AEAD with $t_{\mathcal{A}} \approx t_{\mathcal{B}} \approx t_{\mathcal{C}_1} \approx t_{\mathcal{D}_1} \approx t_{\mathcal{C}_2} - Q_{\text{COMP}} \cdot Q_{\text{AUTH}_1} \approx t_{\mathcal{D}_2} - Q_{\text{COMP}} \cdot Q_{\text{AUTH}_1} \approx t_{\mathcal{E}}$ such that*

$$\begin{aligned} \text{Adv}_{\text{CS}, \mathcal{PW}}^{Q_{\mathcal{A}}\text{-Int}}(\mathcal{A}) &\leq \text{Adv}_{\text{Auth}, \mathcal{PW}}^{Q_{\mathcal{B}}\text{-INS-OW}}(\mathcal{B}) + \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{C}_1}\text{-INT-CTXT}}(\mathcal{C}_1) + \text{Adv}_{\text{AEAD}, \text{kl}}^{Q_{\mathcal{D}_1}\text{-OW-CPA}}(\mathcal{D}_1) \\ &\quad + \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{C}_2}\text{-INT-CTXT}}(\mathcal{C}_2) + \text{Adv}_{\text{AEAD}, \text{kl}}^{Q_{\mathcal{D}_2}\text{-OW-CPA}}(\mathcal{D}_2) + \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{E}}\text{-INT-CTXT}}(\mathcal{E}) \\ &\quad + Q_{\text{AUTH}_1} \cdot \delta_{\text{Auth}} + \frac{(Q_{\text{PUT}_1} + Q_{\text{ACPT}_1})^2 + (Q_{\text{UPD}_1} + 1)^2}{2^{nl+1}} \end{aligned}$$

Adversary $\mathcal{E}^{\text{ENC,DEC,CHL,COMP}}_{\mathcal{E}}$	Oracle $\text{STEP}(\text{get}, i_p, m)$ \only for round $r = 2$
00 $k_1, \dots, k_{Q_{\text{PUT}_1} + Q_{\text{CHL}}} \leftarrow \perp$ \keys unknown 01 $n_c, n_p, Q \leftarrow 0$ 02 $(\text{pw}_1, \dots, \text{pw}_{Q_{\text{REG}_1}}) \xleftarrow{\$} \mathcal{PW}$ 03 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_{\text{ch}}, \mathcal{S}_{\text{chOwn}} \leftarrow \emptyset$ 04 $\mathbf{b} \xleftarrow{\$} \{0, 1\}$ 05 $\mathbf{b}^* \leftarrow \mathcal{A}^O$ 06 if $\exists \text{aid} \in \mathcal{S}_{\text{chOwn}} : \text{aid} \in \mathcal{S}_{\text{cmp}} \vee \text{T}_{\text{uaid}}[\text{aid}] = \perp$ 07 return 0 08 return $\llbracket \mathbf{b}^* = \mathbf{b} \rrbracket$	51 $(i_c, \text{st}_c^{\text{tmp}}, \text{inc}, 2) \leftarrow \text{T}_p[(\Pi, i_p)]$ 52 $\text{aid} \leftarrow \text{T}_u[i_c]$ 53 $\text{st}_c \leftarrow \text{T}_{\text{st}_c}[i_c]$ 54 $\text{fid} \leftarrow \text{st}_c^{\text{tmp}}.\text{fid}$ 55 $((n_k, \text{ct}_k), (n_f, \text{ctr})) \leftarrow m$ 56 $\text{ad} \leftarrow (\text{st}_c.\text{aid}, \text{fid}, \text{"fk"})$ 57 $\text{seed}_k \leftarrow \text{AEAD.Dec}(\text{st}_c.k_{\text{mk}}, n_k, \text{ct}_k, \text{ad})$ 58 if $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge \text{T}_{\text{ct-k}}[\text{aid}, n_k, \text{fid}, \text{ct}_k] = \perp$ 59 $\text{seed}_k \leftarrow \perp$ 60 if $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}_k, \text{fid}, \cdot) \in \text{T}_H$ 61 abort 62 $k_f \leftarrow \text{H}(\text{seed}_k, \text{fid}, \text{"fk"})$ 63 $\text{out}_c.\text{file} \leftarrow \text{AEAD.Dec}(k_f, n_f, \text{ctr}, (\text{fid}, \text{"file"}))$ 64 if $\text{T}_{\text{fid}}[\text{seed}_k, \text{fid}] \neq \perp$ 65 $Q' \leftarrow \text{T}_{\text{fid}}[\text{seed}_k, \text{fid}]$ 66 $\text{out}_c.\text{file} \leftarrow \text{DEC}(Q', n_f, \text{ctr}, (\text{fid}, \text{"file"}))$ \dec oracle 67 $\text{out}_c.\text{decision} \leftarrow \text{true}$ 68 $\text{T}_p[(\Pi, i_p)] \leftarrow (i_c, \text{st}_c^{\text{tmp}}, \text{inc}, 3)$ 69 return $\text{Chk}(\Pi, i_p, \text{out}_c, \text{mc})$
Oracle $\text{PUT}_1(i_c \in [n_c], \text{fid}, \text{file})$	Oracle $\text{UPD}_1 C(i_c, \text{fid}, \text{file}_0, \text{file}_1)$ \store both files
Oracle $\text{PUT}_1 C(i_c \in [n_c], \text{fid}, \text{file}_0, \text{file}_1)$ \queried from CHALL 09 require $\text{T}_{\text{uaid}}[\text{aid}] = \text{true}$ 10 $\text{aid} \leftarrow \text{T}_u[i_c]$ 11 $\text{st}_c \leftarrow \text{T}_{\text{st}_c}[i_c]$ 12 $\text{T}_f[\text{fid}] \leftarrow \{\text{T}_u[i_c]\}$ 13 $n_k \xleftarrow{\$} \{0, 1\}^{n_l}$ 14 $n_f \xleftarrow{\$} \{0, 1\}^{n_l}$ 15 if $n_k \in \mathcal{S}_{\text{keys,aid}} \vee n_f \in \mathcal{S}_{\text{files,fid}}$ 16 abort 17 $\mathcal{S}_{\text{keys,aid}} \leftarrow \mathcal{S}_{\text{keys,aid}} \cup \{n_k\}$ 18 $\mathcal{S}_{\text{files,fid}} \leftarrow \mathcal{S}_{\text{files,fid}} \cup \{n_f\}$ 19 $\text{ad} \leftarrow (\text{aid}, \text{fid}, \text{"fk"})$ 20 $\text{seed}_k \xleftarrow{\$} \{0, 1\}^{k_l}$ 21 $\text{ct}_k \xleftarrow{\$} \text{AEAD.Enc}(\text{st}_c.k_{\text{mk}}, n_k, \text{seed}_k, \text{ad})$ 22 $\text{T}_{\text{ct-k}}[\text{aid}, n_k, \text{fid}, \text{ct}_k] \leftarrow \text{seed}_k$ 23 if $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}_k, \text{fid}, \cdot) \in \text{T}_H$ 24 abort 25 $k_f \xleftarrow{\$} \text{H}(\text{seed}_k, \text{fid}, \text{"fk"})$ 26 $Q \leftarrow Q + 1$ \new instance 27 $\text{T}_{\text{fid}}[\text{seed}_k, \text{fid}] \leftarrow Q$ 28 $\text{ctr} \xleftarrow{\$} \text{ENC}(Q, n_f, \text{file}, (\text{fid}, \text{"fk"}))$ \enc oracle 29 $\text{ctr} \xleftarrow{\$} \text{CHL}(Q, n_f, \text{file}_0, \text{file}_1, (\text{fid}, \text{"fk"}))$ \chl oracle, only PUT1C 30 $\text{mc} \leftarrow (\text{st}_c.\text{sid}, \text{fid}, n_k, n_f, \text{ct}_k, \text{ctr})$ 31 $\text{out}_c.\text{decision} \leftarrow \text{true}$ 32 n_p++ 33 $i_p \leftarrow n_p$ 34 $\text{T}_p[(\Pi, i_p)] \leftarrow (i_c, \varepsilon, \text{inc}, 2)$ 35 return $\text{Chk}(\Pi, i_p, \text{out}_c, \text{mc})$	70 $\text{st}_c \leftarrow \text{T}_{\text{st}_c}[i_c]$ 71 $\text{mc} \leftarrow \text{getHeader}^{C:1}(\text{st}_c, \text{inc}, \text{fid})$ 72 $(\text{st}_c^{\text{tmp}}.\text{fid}, \text{st}_c^{\text{tmp}}.\text{file}_0, \text{st}_c^{\text{tmp}}.\text{file}_1) \leftarrow (\text{fid}, \text{file}_0, \text{file}_1)$ 73 n_p++ 74 $i_p \leftarrow n_p$ 75 $\text{T}_p[(\text{update}, i_p)] \leftarrow (i_c, \text{st}_c^{\text{tmp}}, \text{inc}, 2)$ 76 return $\text{Chk}(\text{update}, i_p, \text{out}_c, \text{mc})$
Oracle $\text{H}(\text{seed}, \text{id}, \text{label})$	Oracle $\text{STEP}(\text{update}, i_p, m)$ \only for round $r = 2$
36 if $\text{T}_f[\text{id}] \cap \mathcal{S}_{\text{cmp}} \neq \emptyset \wedge \text{T}_{\text{fid}}[\text{seed}, \text{id}] \neq \perp$ 37 $Q' \leftarrow \text{T}_{\text{fid}}[\text{seed}, \text{id}]$ 38 if $k_{Q'} = \perp$ 39 $k_{Q'} \xleftarrow{\$} \text{COMP}_{\mathcal{E}}(Q')$ \comp oracle 40 $\text{T}_H[\text{seed}, \text{id}, \text{label}] \leftarrow k_{Q'}$ \patch RO 41 if $(\text{seed}, \text{aid}, \text{label}) \notin \text{T}_H$ 42 $\text{T}_H[\text{seed}, \text{aid}, \text{label}] \xleftarrow{\$} \{0, 1\}^{k_l}$ 43 $\text{T}_{H\text{-int}}[\text{seed}, \text{aid}, \text{label}] \leftarrow 1$ 44 return $\text{T}_H[\text{seed}, \text{aid}, \text{label}]$	77 $(i_c, \text{st}_c^{\text{tmp}}, \text{inc}, 2) \leftarrow \text{T}_p[(\Pi, i_p)]$ 78 $\text{aid} \leftarrow \text{T}_u[i_c]$ 79 $\text{fid} \leftarrow \text{st}_c^{\text{tmp}}.\text{fid}$ 80 $\text{seed}_k \leftarrow \text{getHeader}^{C:2}(\text{st}_c, \text{fid}, m)$ 81 $k_f \leftarrow \text{H}(\text{seed}_k, \text{fid}, \text{"fk"})$ 82 $n \xleftarrow{\$} \{0, 1\}^{n_l}$ 83 if $n \in \mathcal{S}_{\text{files,fid}}$ 84 abort 85 $\mathcal{S}_{\text{files,fid}} \leftarrow \mathcal{S}_{\text{files,fid}} \cup \{n\}$ 86 $\text{ctr}_f \xleftarrow{\$} \text{AEAD.Enc}(k_f, n, \text{st}_c^{\text{tmp}}.\text{file}, (\text{fid}, \text{"file"}))$ 87 if $\text{T}_{\text{fid}}[\text{seed}_k, \text{fid}] \neq \perp$ 88 $Q' \leftarrow \text{T}_{\text{fid}}[\text{seed}_k, \text{fid}]$ 89 if $\text{st}_c^{\text{tmp}}.\text{file}_0 \neq \perp$ \check{check} challenge case 90 $\text{ctr}_f \xleftarrow{\$} \text{CHL}_{\mathcal{E}}(Q', n, \text{st}_c^{\text{tmp}}.\text{file}_0, \text{st}_c^{\text{tmp}}.\text{file}_1, (\text{fid}, \text{"file"}))$ \chl oracle 91 else 92 $\text{ctr}_f \xleftarrow{\$} \text{ENC}(Q, n, \text{st}_c^{\text{tmp}}.\text{file}, (\text{fid}, \text{"file"}))$ \enc oracle 93 $\text{mc} \leftarrow (\text{st}_c.\text{sid}, \text{fid}, n, \text{ct}_k, \text{ctr}_f)$ 94 $\text{out}_c.\text{decision} \leftarrow \text{true}$ 95 $\text{T}_p[(\Pi, i_p)] \leftarrow (i_c, \text{st}_c^{\text{tmp}}, \text{inc}, 3)$ 96 return $\text{Chk}(\Pi, i_p, \text{out}_c, \text{mc})$
Oracle $\text{COMP}(\text{aid})$	Oracle $\text{RO}(\text{seed}, \text{id}, \text{label})$
45 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{\text{aid}\}$ 46 return $(\text{T}_{\text{pw}}[\text{aid}], \{\text{T}_{\text{st}_c}[i_c] \mid \text{T}_u[i_c] = \text{aid}\}, \{\text{T}_{\text{oob}}(\text{aid}, \cdot)\})$	97 if $\text{T}_{\text{uaid}}[\text{id}] \neq \perp \wedge \text{id} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}, \text{id}, \text{label}) \in \text{T}_{H\text{-int}}$ 98 if $\text{label} \in \{\text{"ek"}, \text{"mac"}\}$ abort 99 if $\text{label} = \text{"mk"}$ abort 100 if $\text{T}_f[\text{id}] \cap \mathcal{S}_{\text{cmp}} = \emptyset \wedge (\text{seed}, \text{id}, \text{label}) \in \text{T}_{H\text{-int}}$ 101 if $\text{label} = \text{"fk"}$ abort 102 else if $\text{T}_f[\text{id}] \cap \mathcal{S}_{\text{cmp}} \neq \emptyset \wedge \text{T}_{\text{fid}}[\text{seed}, \text{id}] \neq \perp$ 103 $Q' \leftarrow \text{T}_{\text{fid}}[\text{seed}, \text{id}]$ 104 if $k_{Q'} = \perp$ 105 $k_{Q'} \xleftarrow{\$} \text{COMP}_{\mathcal{E}}(Q')$ \comp oracle 106 $\text{T}_H[\text{seed}, \text{id}, \text{label}] \leftarrow k_{Q'}$ \patch RO 107 if $(\text{seed}, \text{id}, \text{label}) \notin \text{T}_H$ 108 $\text{T}_H[\text{seed}, \text{id}, \text{label}] \xleftarrow{\$} \{0, 1\}^{k_l}$ 109 return $\text{T}_H[\text{seed}, \text{id}, \text{label}]$
Oracle $\text{CHALL}(i_c \in [n_c], \text{fid}, \text{file}_0, \text{file}_1, \text{prot})$	
47 require $\text{prot} \in \{\text{PUT}_1, \text{UPD}_1\}$ 48 $\mathcal{S}_{\text{ch}} \leftarrow \mathcal{S}_{\text{ch}} \cup \{\text{fid}\}$ 49 $\mathcal{S}_{\text{chOwn}} \leftarrow \mathcal{S}_{\text{chOwn}} \cup \{\text{T}_u[i_c]\} \cup \text{T}_f[\text{fid}]$ 50 return $\text{protC}(i_c, \text{fid}, \text{file}_0, \text{file}_1)$	

Fig. 38. Adversary $\mathcal{E}$ against IND-CCA security of AEAD having access to oracles ENC, DEC, CHL, and CHECK simulating $\mathcal{G}_7$ for adversary $\mathcal{A}$.

with

$$\begin{aligned}
Q_{\mathcal{A}} &= (Q_{\text{STEP}}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1}, Q_{\text{PUT}_1}, Q_{\text{UPD}_1}, Q_{\text{GET}_1}, Q_{\text{SHARE}_1}, Q_{\text{ACPT}_1}, Q_{\text{COMP}}, Q_{\text{RO}}), \\
Q_{\mathcal{B}} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{AUTH}_1}, Q_{\text{AUTH}_1}, Q_{\text{REG}_1} + Q_{\text{RO}}, Q_{\text{RO}}), \\
Q_{\mathcal{C}_1} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1}), \\
Q_{\mathcal{D}_1} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, 0, Q_{\text{REG}_1}, Q_{\text{REG}_1} + Q_{\text{RO}}), \\
Q_{\mathcal{C}_2} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{PUT}_1} + Q_{\text{ACPT}_1}, Q_{\text{UPD}_1} + Q_{\text{GET}_1} + Q_{\text{SHARE}_1}), \\
Q_{\mathcal{D}_2} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{ACPT}_1}, Q_{\text{PUT}_1} + Q_{\text{ACPT}_1}, Q_{\text{PUT}_1} + Q_{\text{UPD}_1} + Q_{\text{GET}_1} + Q_{\text{SHARE}_1} + Q_{\text{RO}}), \\
Q_{\mathcal{E}} &= (Q_{\text{PUT}_1}, Q_{\text{COMP}}, Q_{\text{PUT}_1} + Q_{\text{UPD}_1}, Q_{\text{GET}_1}).
\end{aligned}$$

Proof. We prove the theorem by a sequence of games.

Game $\mathbf{G}_0$. This is the integrity game in which we plug in construction CS. As in the proof of Theorem 10, we distinguish two random oracles. One can be called by the adversary, which is denoted by RO as usual, and one is an internal random oracle, denoted by H, which is only called from other oracles. The change is conceptual since both oracles represent the same function since they are maintaining the same table for lazy sampling. Additionally, we introduce a table $\mathbf{T}_{\text{H-int}}$ which indicates if a value was queried to the internal RO; this will be used in later changes. It holds that

$$\Pr[\mathbf{G}_0(\mathcal{A}) \Rightarrow 1] = \text{Adv}_{\text{CS}, \mathcal{PW}}^{Q_{\mathcal{A}}\text{-Int}}(\mathcal{A}),$$

where $Q_{\mathcal{A}}$ is the same as in the theorem statement.

Game $\mathbf{G}_1$. This is game $\mathbf{G}_0$ with the same changes as in Game $\mathbf{G}_7$ from the proof of Theorem 10 as depicted in Figure 35. In particular, the game aborts if there are any direct random oracle queries corresponding to file keys which are only owned by uncompromised users and include the correct seed, i.e. the seed to extract the file key. This means that at this point all master keys of uncompromised users are uniformly random. The same holds for file keys which are exclusively owned by uncompromised users. Since the changes are exactly the same as in the proof of Theorem 10, we obtain the same bounds:

$$\begin{aligned}
\Pr[\mathbf{G}_0(\mathcal{A}) \Rightarrow 1] - \Pr[\mathbf{G}_1(\mathcal{A}) \Rightarrow 1] &\leq \left(\text{Adv}_{\text{Auth}, \mathcal{PW}}^{Q_{\mathcal{B}}\text{-INS-OW}}(\mathcal{B}) + \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{C}_1}\text{-INT-CTXT}}(\mathcal{C}_1) \right. \\
&\quad \left. + \text{Adv}_{\text{AEAD}, \text{kl}}^{Q_{\mathcal{D}_1}\text{-OW-CPA}}(\mathcal{D}_1) + \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{C}_2}\text{-INT-CTXT}}(\mathcal{C}_2) + \text{Adv}_{\text{AEAD}, \text{kl}}^{Q_{\mathcal{D}_2}\text{-OW-CPA}}(\mathcal{D}_2) \right) \\
&\quad + Q_{\text{AUTH}_1} \cdot \delta_{\text{Auth}} + \frac{(Q_{\text{PUT}_1} + Q_{\text{ACPT}_1})^2 + (Q_{\text{UPD}_1} + 1)^2}{2^{nl+1}}
\end{aligned}$$

with number of queries denoted in the theorem bound.

Reduction to INT-CTXT of AEAD.

Claim. There exists an adversary $\mathcal{E}$ against INT-CTXT such that

$$\Pr[\mathbf{G}_1(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{\text{AEAD}}^{Q\text{-INT-CTXT}}(\mathcal{E})$$

with $Q = (Q_{\text{PUT}_1}, Q_{\text{COMP}}, Q_{\text{PUT}_1} + Q_{\text{UPD}_1}, Q_{\text{CHECK}})$.

Proof (Claim). The reduction is formalized in Figure 39. Note that due to the changes in the previous game, all file keys corresponding to files solely owned by uncompromised users are uniformly random. Hence, the reduction can use their own oracles to simulate encryption queries with respect to these keys. These occur in oracle PUT_1 and UPD_1 . While $\mathcal{E}$ chooses a new instance for a PUT_1 query since the file key is generated in that query, an old instance is used in oracle UPD_1 . All instances are stored together with their seed seed_k and their id fid in table $\mathbf{T}_{\text{fid}}$ to be recognized later. Further, every file encryption is stored in table $\mathbf{T}_{\text{ct-f}}$, to be able to completely answer get queries; more specifically the AEAD decryption in the second step of the get procedure (Line 69). If this is not the case, i.e. the file is new, the reduction outputs a solution if the queried

user is uncompromised, the file is only owned by uncompromised users and the file ciphertext is valid, i.e. does not decrypt to $\perp$. Note that these conditions imply the winning conditions of adversary $\mathcal{A}$: the only condition that is not syntactically the same is $\text{file} \notin T_f[\text{fid}].F$. Analyzing this condition, we can observe two things. First, in case $T_f[\text{fid}].F$ is empty, the file id is not tracked by the game and hence the corresponding file key was never used. This means that there cannot be a matching entry in table $T_{\text{ct-k}}$ which means that the seed seed_k was set to $\perp$ only yielding a $\perp$ output for the oracle which does not fulfill $\mathcal{A}$'s winning condition (or the CHECK of $\mathcal{E}$). Second, if the challenge file is in $T_f[\text{fid}].F$, then there must be a matching element in $T_{\text{ct-f}}$ and hence $\mathcal{E}$ does not reach this line but can just answer the oracle query instead.

To preserve consistent outputs of the random oracle it must be patched upon queries to file keys which correspond to files owned by at least one compromised user. To this end, the instance is restored from table T_{fid} and $\mathcal{E}$ queries their own compromise oracle to obtain the key of that instance. The entry of the random oracle table is then patched accordingly (Line 86).

It remains to show that the output tuple of $\mathcal{E}$ is actually a valid solution for their INT-CTXT game. The decryption of the challenge is not $\perp$ due to the check via CHECK. The instance of the AEAD also must not be compromised which is ensured by the queried user not being compromised and the file not being owned by any compromised user because the compromise oracle of $\mathcal{E}$ is only queried if at least one compromised user owns the file. $\square$

Collecting the probabilities completes the proof of Theorem 11. $\square$

D Additional Auth Notion and Construction for External Adversaries

For completeness, we give a security notion which can be used in the external adversary model sketched by BDGHP and described above. We also provide a construction that is much simpler than 2HDH.

Definition 19 (Weak One-Wayness). *Output one-wayness for weak adversaries is defined via the game in Figure 40. We define the advantage of an adversary $\mathcal{A}$ and a client's secrets distribution $\mathcal{D}_C$ as*

$$\text{Adv}_{\text{Auth}, \mathcal{D}_C}^{Q\text{-Weak-OW}}(\mathcal{A}) := \Pr [Q\text{-Weak-OW}_{\text{Auth}, \mathcal{D}_C}(\mathcal{A}) \Rightarrow 1]$$

with $Q = (Q_{\text{REG}}, Q_{\text{COMP}}, Q_{\text{INIT}}, Q_{\text{FIN}}, Q_{\text{CHECK}}, Q_{\text{RO}})$.

TRIPLE HASH. In addition to the two authentication schemes from Section 3, we give a third instantiation, called THash in Figure 41. It achieves the same security bound for INS-OW as the other two instantiations and cannot improve its security bound for OUT-OW as in the case of 2HDH. We still state the construction because it achieves a better bound for Weak-OW (compared to SHash, see Table 2) which is needed for the external adversary model from [BDG⁺24] as discussed in Section 6.

E Theorems and Proofs from Section 6

E.1 Confidentiality

We restate Theorem 7 using concrete bounds and provide the full proof.

Theorem 12 (Net-Conf). *For any password distribution PW and adversary $\mathcal{A}$ against the Net-Conf security of $\text{CS} := \text{CS}[\text{Auth}, \text{AEAD}, \text{MAC}]$ there exist an OUT-OW adversary $\mathcal{B}$ against Auth, INT-CTXT adversaries $\mathcal{C}_1, \mathcal{C}_2$ against AEAD, OW-CPA adversaries $\mathcal{D}_1, \mathcal{D}_2$ against AEAD, and an IND-CCA adversary $\mathcal{E}$ against*

Adversary $\mathcal{E}^{\text{ENC}, \text{COMP}_E, \text{CHECK}}$ 00 $H \xleftarrow{\$} \mathcal{OS}$ 01 $k_1, \dots, k_{Q_{\text{PUT}_1}} \xleftarrow{\$} \perp$ 02 $n_c, n_p, n_r, Q \leftarrow 0$ 03 $(pw_1, \dots, pw_{Q_{\text{REG}_1}}) \xleftarrow{\$} \mathcal{PW}$ 04 $\mathcal{S}_{\text{cmp}} \leftarrow \{i_{\text{mal}}\}$ 05 $\text{win} \leftarrow \text{false}$ 06 $\mathcal{A}^{\mathcal{O}}$ 07 return win Oracle $\text{PUT}_1(i_c \in [n_c], \text{fid}, \text{file})$ 08 require $T_f[\text{fid}] = \perp$ 09 $U \leftarrow \{T_u[i_c]\}$ 10 $F \leftarrow \{\text{file}\}$ 11 $\text{aid} \leftarrow T_u[i_c]$ 12 $\text{stc} \leftarrow T_{\text{stc}}[i_c]$ 13 $(\text{inc.fid}, \text{inc.file}) \leftarrow (\text{fid}, \text{file})$ 14 $T_f[\text{fid}] \leftarrow (U, F)$ 15 $(n_k, n_f) \xleftarrow{\$} \{0, 1\}^{2n_l}$ 16 if $n_k \in \mathcal{S}_{\text{keys}, i} \vee n_f \in \mathcal{S}_{\text{files}, \text{fid}}$ abort 17 $\mathcal{S}_{\text{keys}, \text{aid}} \leftarrow \mathcal{S}_{\text{keys}, \text{aid}} \cup \{n_k\}$ 18 $\mathcal{S}_{\text{files}, \text{fid}} \leftarrow \mathcal{S}_{\text{files}, \text{fid}} \cup \{n_f\}$ 19 $\text{ad} \leftarrow (\text{aid}, \text{fid}, \text{"fk"})$ 20 $\text{seed}_k \xleftarrow{\$} \{0, 1\}^{k_l}$ 21 $\text{ct}_k \xleftarrow{\$} \text{AEAD}.\text{Enc}(\text{stc}.k_{\text{mk}}, n_k, \text{seed}_k, \text{ad})$ 22 $T_{\text{ct-k}}[\text{aid}, n_k, \text{fid}, \text{ct}_k] \leftarrow \text{seed}_k$ 23 if $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}_k, \text{fid}, \cdot) \in T_H$ abort 24 $k_f \xleftarrow{\$} H(\text{seed}_k, \text{fid}, \text{"fk"})$ 25 $Q++$ // new instance 26 $T_{\text{fid}}[\text{seed}_k, \text{fid}] \leftarrow Q$ 27 $\text{ctr} \xleftarrow{\$} \text{ENC}(Q, n_f, \text{file}, (\text{fid}, \text{"fk"}))$ // enc oracle 28 $T_{\text{ct-f}}[\text{seed}_k, n_f, \text{fid}, \text{ctr}] \leftarrow \text{file}$ // store file 29 $\text{mc} \leftarrow (\text{stc}.sid, \text{fid}, n_k, n_f, \text{ct}_k, \text{ctr})$ 30 $\text{outc}.decision \leftarrow \text{true}$ 31 n_p++ 32 $i_p \leftarrow n_p$ 33 $T_p[(\Pi, i_p)] \leftarrow (i_c, \varepsilon, \text{inc}, 2)$ 34 return $\text{Chk}(\Pi, i_p, \text{outc}, \text{mc})$ Oracle $\text{STEP}(\text{update}, i_p \in [n_p], m)$ // only for round $r = 2$ 35 $(i_c, \text{stc}^{\text{tmp}}, \text{inc}, 2) \leftarrow T_p[(\Pi, i_p)]$ 36 $\text{aid} \leftarrow T_u[i_c]$ 37 $\text{fid} \leftarrow \text{stc}^{\text{tmp}}.\text{fid}$ 38 $\text{seed}_k \leftarrow \text{getHeader}^{C:2}(\text{stc}, \text{fid}, m_s)$ 39 if $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge T_{\text{ct-k}}[\text{aid}, n_k, \text{fid}, \text{ct}_k] = \perp$ 40 $\text{seed}_k \leftarrow \perp$ 41 if $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}_k, \text{fid}, \cdot) \in T_H$ abort 42 $k_f \leftarrow H(\text{seed}_k, \text{fid}, \text{"fk"})$ 43 $n \xleftarrow{\$} \{0, 1\}^{n_l}$ 44 if $n \in \mathcal{S}_{\text{files}, \text{fid}}$ abort 45 $\mathcal{S}_{\text{files}, \text{fid}} \leftarrow \mathcal{S}_{\text{files}, \text{fid}} \cup \{n\}$ 46 $\text{ctr}_f \xleftarrow{\$} \text{AEAD}.\text{Enc}(k_f, n, \text{stc}^{\text{tmp}}.\text{file}, (\text{fid}, \text{"file"}))$ 47 if $\exists Q' : T_{\text{fid}}[\text{seed}_k, \text{fid}] = Q'$ 48 $\text{ctr} \xleftarrow{\$} \text{ENC}(Q', n, \text{stc}^{\text{tmp}}.\text{file}, (\text{fid}, \text{"file"}))$ // enc oracle 49 $T_{\text{ct-f}}[\text{seed}_k, n, \text{fid}, \text{ctr}] \leftarrow \text{stc}^{\text{tmp}}.\text{file}$ // store file 50 $\text{mc} \leftarrow (\text{stc}.sid, \text{fid}, n, \text{ct}_k, \text{ctr})$ 51 $\text{outc}.decision \leftarrow \text{true}$ 52 $T_p[(\Pi, i_p)] \leftarrow (i_c, \text{stc}^{\text{tmp}}, \text{inc}, 3)$ 53 return $\text{Chk}(\Pi, i_p, \text{outc}, \text{mc})$ Oracle $\text{COMP}(\text{aid})$ 54 require $T_{\text{uaid}}[\text{aid}] \neq \perp$ 55 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{\text{aid}\}$ 56 return $(T_{\text{pw}}[\text{aid}], \{T_{\text{stc}}[i_c] \mid T_u[i_c] = \text{aid}\}, \{T_{\text{oob}}[(\text{aid}, \cdot)]\})$	Oracle $\text{STEP}(\text{get}, i_p \in [n_p], m)$ // only for round $r = 2$ 57 $(i_c, \text{stc}^{\text{tmp}}, \text{inc}, 2) \leftarrow T_p[(\Pi, i_p)]$ 58 $(\text{aid}, \text{stc}) \leftarrow (T_u[i_c], T_{\text{stc}}[i_c])$ 59 $\text{fid} \leftarrow \text{stc}^{\text{tmp}}.\text{fid}$ 60 $((n_k, \text{ct}_k), (n_f, \text{ctr})) \leftarrow m$ 61 $\text{ad} \leftarrow (\text{aid}, \text{fid}, \text{"fk"})$ 62 $\text{seed}_k \leftarrow \text{AEAD}.\text{Dec}(\text{stc}.k_{\text{mk}}, n_k, \text{ct}_k, \text{ad})$ 63 if $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge T_{\text{ct-k}}[\text{aid}, n_k, \text{fid}, \text{ct}_k] = \perp$ 64 $\text{seed}_k \leftarrow \perp$ 65 if $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}_k, \text{fid}, \cdot) \in T_H$ abort 66 $k_f \leftarrow H(\text{seed}_k, \text{fid}, \text{"fk"})$ 67 $Q' \leftarrow T_{\text{fid}}[\text{seed}_k, \text{fid}]$ // extract instance 68 if $T_{\text{ct-f}}[\text{seed}_k, n_f, \text{fid}, \text{ctr}] \neq \perp$ // previous query 69 $\text{outc}.file \leftarrow T_{\text{ct-f}}[\text{seed}_k, n_f, \text{fid}, \text{ctr}]$ 70 else if $\text{aid} \notin \mathcal{S}_{\text{cmp}} \wedge T_f[\text{fid}] \cup \mathcal{S}_{\text{cmp}} = \emptyset \wedge \text{CHECK}(Q', n_f, \text{ctr}, (\text{fid}, \text{"file"}))$ // check oracle 71 $\text{output}(Q', n_f, \text{ctr}, (\text{fid}, \text{"file"}))$ // win 72 else 73 $\text{outc}.file \leftarrow \text{AEAD}.\text{Dec}(k_f, n_f, \text{ctr}, (\text{fid}, \text{"file"}))$ 74 $\text{outc}.decision \leftarrow \text{true}$ 75 $T_p[(\Pi, i_p)] \leftarrow (i_c, \text{stc}^{\text{tmp}}, \text{inc}, 3)$ 76 return $\text{Chk}(\Pi, i_p, \text{outc}, \text{mc})$ Oracle $\text{RO}(\text{seed}, \text{id}, \text{label})$ 77 if $T_{\text{uaid}}[\text{id}] \neq \perp, \text{id} \notin \mathcal{S}_{\text{cmp}} \wedge (\text{seed}, \text{id}, \text{label}) \in T_{H\text{-int}}$ 78 if $\text{label} \in \{\text{"ek"}, \text{"mac"}\}$ abort 79 if $\text{label} = \text{"mk"}:$ abort 80 if $T_f[\text{id}] \cup \mathcal{S}_{\text{cmp}} = \emptyset \wedge (\text{seed}, \text{id}, \text{label}) \in T_{H\text{-int}}$ 81 if $\text{label} = \text{"fk"}:$ abort 82 else if $T_f[\text{id}] \cup \mathcal{S}_{\text{cmp}} \neq \emptyset \wedge T_{\text{fid}}[\text{seed}, \text{id}] \neq \perp$ 83 $Q' \leftarrow T_{\text{fid}}[\text{seed}, \text{id}]$ 84 if $k_{Q'} = \perp$ 85 $k_{Q'} \xleftarrow{\$} \text{COMP}_E(Q')$ // comp oracle 86 $T_H[\text{seed}, \text{id}, \text{label}] \leftarrow k_{Q'}$ // patch RO 87 if $(\text{seed}, \text{id}, \text{label}) \notin T_H$ 88 $T_H[\text{seed}, \text{id}, \text{label}] \xleftarrow{\$} \{0, 1\}^{k_l}$ 89 return $T_H[\text{seed}, \text{id}, \text{label}]$ Procedure $\text{Chk}(\text{prot}, i_p, \text{outc}, m^*)$ 90 $(i_c, \text{stc}^{\text{tmp}}, \text{inc}, r) \leftarrow T_p[(\text{prot}, i_p)]$ 91 $\text{fid} \leftarrow \text{inc.fid}$ 92 if $\text{prot} = \text{areg}$ 93 if $\text{outc}.decision$ 94 $T_{\text{uaid}}[\text{inc.aid}] \leftarrow \text{true}$ 95 else 96 $T_{\text{uaid}}[\text{inc.aid}] \leftarrow \perp$ 97 if $\text{prot} = \text{get}:$ 98 if $T_u[i_c] \notin \mathcal{S}_{\text{cmp}} \wedge T_f[\text{fid}] \cup \mathcal{S}_{\text{cmp}} = \emptyset \wedge \text{win} = \text{false}$ 99 $\text{file} \leftarrow \text{outc}.file$ 100 if $\text{outc}.decision \wedge \text{file} \neq \perp \wedge \text{file} \notin T_f[\text{fid}].F$ 101 $\text{win} \leftarrow \text{true}$ 102 return $(\text{outc}.f, m^*)$ 103 if $\text{prot} = \text{share} \wedge \text{outc}.decision:$ 104 $T_{\text{oob}}[(\text{inc}.raid, \text{fid})] \leftarrow \text{outc}.oob$ 105 if $\text{prot} = \text{accept} \wedge \text{outc}.decision:$ 106 $T_{\text{oob}}[(T_u[i_c], \text{fid})] \leftarrow \perp$ 107 return m^* Oracle $H(\text{seed}, \text{id}, \text{label})$ 108 if $(\text{seed}, \text{id}, \text{label}) \notin T_H$ 109 $T_H[\text{seed}, \text{id}, \text{label}] \xleftarrow{\$} \{0, 1\}^{k_l}$ 110 $T_{H\text{-int}}[\text{seed}, \text{id}, \text{label}] \leftarrow 1$ 111 return $T_H[\text{seed}, \text{id}, \text{label}]$
---	--

Fig. 39. Adversary $\mathcal{E}$ against INT-CTXT security of AEAD having access to oracles ENC , COMP_E , and CHECK simulating G_1 for adversary $\mathcal{A}$.

Game $Q\text{-Weak-OW}_{\text{Auth}, \mathcal{D}_C}(\mathcal{A})$ 00 $H \xleftarrow{\$} \mathcal{OS}$ 01 $\mathcal{S}_{\text{cmp}} \leftarrow \emptyset$ 02 $n_r \leftarrow 0$ 03 $(x_{C,1}, \dots, x_{C,Q_{\text{REG}}}) \xleftarrow{\$} \mathcal{D}_C$ 04 for $i \in [Q_{\text{REG}}]$ 05 $x_{S,i} \xleftarrow{\$} \mathcal{D}_S$ 06 $(y^*, \text{aux}^*) \xleftarrow{\$} \mathcal{A}^{\text{REG,RESP,COMP,CHECK,RO}}$ 07 if $T_{\text{aux}}[\text{aux}^*] = \perp$ return 0 08 $i^* \leftarrow T_{\text{aux}}[\text{aux}^*]$ 09 return $\llbracket y^* = \text{Full}[H](x_{C,i^*}, \text{aux}^*, x_{S,i^*}) \wedge i^* \notin \mathcal{S}_{\text{cmp}} \rrbracket$ Oracle REG(aux) 10 require $T_{\text{aux}}[\text{aux}] = \perp$ $\llcorner$ enforce unique aux 11 n_r++ 12 $T_{\text{aux}}[\text{aux}] \leftarrow n_r$	Oracle RESP(auth₁, $i \in [n_r]$) 13 $\text{auth}_2 \leftarrow \text{Resp}[H](\text{auth}_1, x_{S,i})$ 14 return auth_2 Oracle COMP($i \in [n]$) 15 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{i\}$ 16 return $(x_{C,i}, x_{S,i})$ Oracle CHECK(y, aux) 17 if $T_{\text{aux}}[\text{aux}] = \perp$ return 0 18 $i \leftarrow T_{\text{aux}}[\text{aux}]$ 19 return $\llbracket y = \text{Full}[H](x_{C,i}, \text{aux}, x_{S,i}) \rrbracket$ Oracle RO(x) 20 return $H(x)$
--	---

Fig. 40. Game defining Weak-OW security for a two-message authentication scheme $\text{Auth} := (\mathcal{OS}, \mathcal{D}_S, Y, \text{Init}, \text{Resp}, \text{Fin}, \text{Full})$ for distribution $\mathcal{D}_C$, adversary $\mathcal{A}$, and $Q = (Q_{\text{REG}}, Q_{\text{COMP}}, Q_{\text{RESP}}, Q_{\text{CHECK}}, Q_{\text{RO}})$ making at most Q_O queries to oracle O .

THash.Init(x_C, aux) 00 $\text{auth}_1 \leftarrow H_1(x_C, \text{aux})$ 01 return $(\text{auth}_1, \varepsilon)$	THash.Resp(auth₁, x_S) 04 $\text{auth}_2 \leftarrow H_3(x_S, \text{auth}_1)$ 05 return x_S
THash.Fin(auth₂, $x_C, \text{aux}, \text{st}$) 02 $y \leftarrow H_2(x_C, \text{aux}, \text{auth}_2)$ 03 return y	THash.Full(x_C, aux, x_S) 06 $y \leftarrow H_2(x_C, \text{aux}, H_3(x_S, H_1(x_C, \text{aux})))$ 07 return y

Fig. 41. The authentication protocol $\text{THash} := (\mathcal{OS}, \mathcal{D}_S, \{0, 1\}^{\text{kl}}, \text{Init}, \text{Resp}, \text{Fin}, \text{Full})$ using three hash functions, where $\mathcal{D}_S$ is an arbitrary distribution.

AEAD with $t_A \approx t_B \approx t_{C_1} \approx t_{D_1} \approx t_{C_2} - Q_{\text{COMP}} \cdot Q_{\text{AUTH}_1} \approx t_{D_2} - Q_{\text{COMP}} \cdot Q_{\text{AUTH}_1} \approx t_E$ such that

$$\begin{aligned}
\text{Adv}_{\text{CS}, \mathcal{PW}}^{Q_A\text{-Net-Conf}}(\mathcal{A}) &\leq 2 \cdot \left(\text{Adv}_{\text{Auth}, \mathcal{PW}}^{Q_B\text{-OUT-OW}}(\mathcal{B}) + \text{Adv}_{\text{AEAD}}^{Q_{C_1}\text{-INT-CTXT}}(\mathcal{C}_1) + \text{Adv}_{\text{AEAD}, \text{kl}}^{Q_{D_1}\text{-OW-CPA}}(\mathcal{D}_1) \right. \\
&\quad \left. + \text{Adv}_{\text{AEAD}}^{Q_{C_2}\text{-INT-CTXT}}(\mathcal{C}_2) + \text{Adv}_{\text{AEAD}, \text{kl}}^{Q_{D_2}\text{-OW-CPA}}(\mathcal{D}_2) \right) + \text{Adv}_{\text{AEAD}}^{Q_E\text{-IND-CCA}}(\mathcal{E}) \\
&\quad + Q_{\text{AUTH}_1} \cdot \delta_{\text{Auth}} + \frac{(Q_{\text{PUT}_1} + Q_{\text{ACPT}_1})^2 + (Q_{\text{UPD}_1} + 1)^2}{2^{nl+1}}
\end{aligned}$$

with

$$\begin{aligned}
Q_A &= (Q_{\text{STEP}}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1}, Q_{\text{PUT}_1}, Q_{\text{UPD}_1}, Q_{\text{GET}_1}, Q_{\text{SHARE}_1}, Q_{\text{ACPT}_1}, Q_{\text{CHALL}}, Q_{\text{COMP}}, Q_{\text{RO}}), \\
Q_B &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{AUTH}_1}, Q_{\text{AUTH}_1}, Q_{\text{AUTH}_1}, Q_{\text{REG}_1} + Q_{\text{RO}}), \\
Q_{C_1} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1}), \\
Q_{D_1} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, 0, Q_{\text{REG}_1}, Q_{\text{REG}_1} + Q_{\text{RO}}), \\
Q_{C_2} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{PUT}_1} + Q_{\text{ACPT}_1} + Q_{\text{CHL}}, Q_{\text{UPD}_1} + Q_{\text{GET}_1} + Q_{\text{SHARE}_1} + Q_{\text{CHL}}), \\
Q_{D_2} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{ACPT}_1}, Q_{\text{PUT}_1} + Q_{\text{ACPT}_1} + Q_{\text{CHL}}, Q_{\text{PUT}_1} + Q_{\text{UPD}_1} + Q_{\text{GET}_1} + Q_{\text{SHARE}_1} + Q_{\text{CHL}} + Q_{\text{RO}}), \\
Q_E &= (Q_{\text{PUT}_1} + Q_{\text{CHL}}, Q_{\text{PUT}_1} + Q_{\text{CHL}}, Q_{\text{PUT}_1} + Q_{\text{UPD}_1}, Q_{\text{GET}_1}, Q_{\text{CHL}}).
\end{aligned}$$

Proof. We prove the theorem by a sequence of games and depict only those oracles in which a change occurs, starting with Figure 42.

Game G_0 . This is the **Net-Conf** game in which we plug in construction **CS**. As for the proof of Theorem 10, we distinguish two random oracles. One can be called by the adversary, which is denoted by **RO** as usual, and one is an internal random oracle, denoted by **H**, which is only called from other oracles. The change is conceptual since both oracles represent the same function since they are maintaining the same table for lazy sampling. Additionally, we introduce a table T_{H-int} which indicates if a value was queried to the internal **RO**; this will be used in later changes. It holds that

$$\Pr[G_0(\mathcal{A}) \Rightarrow 1] = \text{Adv}_{CS, \mathcal{PW}}^{Q_{\mathcal{A}}\text{-Net-Conf}}(\mathcal{A}),$$

where $Q_{\mathcal{A}}$ is the same as in the theorem statement.

Game G_1 . This is the same game as the previous one except that the raw secret **rw** computed in the second step of the authentication protocol is replaced by the value computed during the registration oracle if the received value **auth**₂ is correct, i.e. is in the support of the response algorithm of **Auth** with the correct inputs. To this end, we introduce a table T_{rw} to store **rw** and server secret **k** for each **aid** during the call to **REG**₁. Changes are depicted in Figure 42.

Claim. It holds that

$$\Pr[G_0(\mathcal{A}) \Rightarrow 1] - \Pr[G_1(\mathcal{A}) \Rightarrow 1] \leq Q_{AUTH_1} \cdot \delta_{Auth}.$$

Proof (Claim). The step is analogous to the proof of Theorem 10. □

Game G_2 . This is the same game as the previous one except that it aborts if there exists a random oracle query on a registered and uncompromised user such that the same input was previously queried to the internal **RO** **H** and the tag is either “ek” or “mac”. The game also aborts in the registration oracle if the queried user is not compromised and there already exists a random oracle query on the same **rw** and **aid**. The changes are depicted in Figure 42.

Claim. There exists an adversary $\mathcal{B}$ against **OUT-OW** such that

$$\Pr[G_1(\mathcal{A}) \Rightarrow 1] - \Pr[G_2(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{Auth, \mathcal{PW}}^{Q\text{-OUT-OW}}(\mathcal{B})$$

with $Q = (Q_{REG_1}, Q_{COMP}, Q_{AUTH_1}, Q_{AUTH_1}, Q_{AUTH_1}, Q_{REG_1} + Q_{RO})$.

Proof (Claim). The reduction follows a similar approach as the reduction to **Auth** in the proof of Theorem 10. The differences are as follows. $\mathcal{B}$ needs to simulate a server oracle for the authentication sub protocol which is done via $\mathcal{B}$ ’s response oracle **RESP**. Further, the reduction cannot make use of the server secret because compared to **INS-OW**, $\mathcal{B}$ does not have access to these values in the **OUT-OW** game for uncompromised users. Since $\mathcal{B}$ can rely on the server response oracle, they do not need the values until a user is compromised. Upon compromise, the server secret **k** is known to the reduction (Line 31) which is necessary to patch the random oracle correctly. In particular, if the **RO** is queried on a compromised user, the reduction can check if it is the correct value by executing **Auth.Full** (Line 81) which is only possible knowing client and server secret. If this is the case, the oracle can be patched on the previously sampled value stored in T_{H-int} . As shown in the proof of Theorem 10, the simulation is sound and in all cases in which the newly introduced abort condition triggers, adversary $\mathcal{B}$ wins their **OUT-OW** game. For the number of queries, note that a step of some procedure can only be executed if the previous step was executed and further this is only possible once, i.e. it is not possible to branch a protocol and execute the server step of the authentication twice for one client step. Hence, we can upper bound all authentication sub protocol steps by the number of queries to the initiation oracle **AUTH**₁. □

Final reduction. Finally, we can apply the same changes as applied in the proof of Theorem 10 between G_2 and the last game and also apply the final reduction. Note that G_2 is exactly in the same state as G_2 of

Oracle REG₁(aid) <pre> 00 require T_{uaid}[aid] = ⊥ 01 n_r++ 02 inc.pw ← pw_{n_r} 03 T_{pw}[aid] ← inc.pw 04 inc.aid ← aid 05 T_{uaid}[aid] ← false 06 k $\xleftarrow{\\$}$ Auth.D_S 07 rw ← Auth.Full(pw, aid, k) 08 T_{rw}[aid] ← (rw, k) 09 if aid ∉ S_{cmp} ∧ (rw, aid, ·) ∈ T_H 10 abort 11 k_{ek} ← H(rw, aid, "ek") 12 k_{mac} ← H(rw, aid, "mac") 13 seed_{mk} $\xleftarrow{\\$}$ {0, 1}^{kl} 14 k_{mk} ← H(seed_{mk}, aid, "mk") 15 n_{mk} $\xleftarrow{\\$}$ {0, 1}^{nl} 16 ct_{mk} $\xleftarrow{\\$}$ AEAD.Enc(k_{ek}, n_{mk}, seed_{mk}, (aid, "mk")) 17 m_C ← (aid, k, n_{mk}, ct_{mk}, k_{mac}) 18 n_p++ 19 i_p ← n_p 20 T_p[(Π, C, i_p)] ← (i, ε, inc, 2) 21 return Chk(Π, i_p, outc, m_C) </pre> Oracle COMP(aid) <pre> 22 require T_{uaid}[aid] ≠ ⊥ 23 S_{cmp} ← S_{cmp} ∪ {aid} 24 return (T_{pw}[aid], {T_{stc}[i_c] T_u[i_c] = aid}, {T_{oob}[(aid, ·)]}) </pre> Oracle STEP(auth, S, i_p, m) $\parallel$ only for server round $r = 1$ <pre> 25 (i_c, st_C^{tmp}, in, 1) ← T_p[(prot, S, i_p)] 26 (aid, auth₁) ← m 27 if aid ∉ st_S.acc 28 fails 29 sid $\xleftarrow{\\$}$ NewID 30 (k, ..., k_{mac}) ← st_S.acc[aid] 31 auth₂ $\xleftarrow{\\$}$ Auth.Resp(auth₁, k) 32 m_S ← (sid, auth₂) 33 st_S^{tmp} ← (aid, k_{mac}, m, m_S) 34 T_p[(auth, S, i_p)] ← (i_c, st_C^{tmp}, in, 2) 35 return m_S </pre>	Oracle AUTH₁(aid) <pre> 36 require T_{uaid}[aid] = true 37 inc.aid ← aid 38 inc.pw ← T_{pw}[aid] 39 n_c++ 40 i_c ← n_c 41 T_u[i_c] ← aid 42 T_{stc}[i_c] ← ε 43 (auth₁, st) $\xleftarrow{\\$}$ Auth.Init(inc.pw, aid) 44 m_C ← (aid, auth₁) 45 st_C^{tmp} ← (aid, inc.pw, st, m_C) 46 n_p++ 47 i_p ← n_p 48 T_p[(auth, C, i_p)] ← (i_c, st_C^{tmp}, inc, 2) 49 return Chk(prot, i_p, ε, m_C) </pre> Oracle STEP(auth, C, i_p, m) $\parallel$ only for client round $r = 2$ <pre> 50 (i_c, st_C^{tmp}, inc, 2) ← T_p[(Π, C, i_p)] 51 (aid, pw, st, m_C) ← st_C^{tmp} 52 (·, auth₁) ← m_C 53 (sid, auth₂) ← m 54 rw ← Auth.Fin(auth₂, pw, aid, st) 55 (rw', k) ← T_{rw}[aid] 56 if auth₂ ∈ Auth.Resp(auth₁, k) 57 rw ← rw' 58 st_C^{tmp}.k_{ek} ← H(rw, aid, "ek") 59 k_{mac} ← H(rw, aid, "mac") 60 m_C ← MAC.Tag(k_{mac}, (m_C, m_S)) 61 T_p[(Π, C, i_p)] ← (i_c, st_C^{tmp}, inc, 3) 62 return Chk(Π, i_p, ε, m_C) </pre> Oracle H(seed, aid, label) <pre> 63 if (seed, aid, label) ∉ T_H 64 T_H[seed, aid, label] $\xleftarrow{\\$}$ {0, 1}^{kl} 65 T_{H-int}[seed, aid, label] ← 1 66 return T_H[seed, aid, label] </pre> Oracle RO(seed, id, label) <pre> 67 if T_{uaid}[id] ≠ ⊥ ∧ id ∉ S_{cmp} ∧ (seed, id, label) ∈ T_{H-int} 68 if label ∈ {"ek", "mac"} abort 69 if (seed, id, label) ∉ T_H 70 T_H[seed, id, label] $\xleftarrow{\\$}$ {0, 1}^{kl} 71 return T_H[seed, id, label] </pre>
---	---

Fig. 42. Games $G_0 - G_2$ for the proof of Theorem 12.

the proof of Theorem 10 except for the server oracles, i.e. queries to STEP with $P = S$. The server oracles can be simulated trivially because the server is not involved in any nonce sampling or AEAD operations but only stores and forwards ciphertexts. Hence, we can apply the same arguments and obtain the same bound:

$$\begin{aligned}
2 \Pr[G_2(\mathcal{A}) \Rightarrow 1] - 1 &\leq 2 \left(\text{Adv}_{\text{AEAD}}^{Q_{C_1}\text{-INT-CTXT}}(\mathcal{C}_1) + \text{Adv}_{\text{AEAD}, \text{kl}}^{Q_{D_1}\text{-OW-CPA}}(\mathcal{D}_1) \right. \\
&\quad \left. + \text{Adv}_{\text{AEAD}}^{Q_{C_2}\text{-INT-CTXT}}(\mathcal{C}_2) + \text{Adv}_{\text{AEAD}, \text{kl}}^{Q_{D_2}\text{-OW-CPA}}(\mathcal{D}_2) \right) + \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{E}}\text{-IND-CCA}}(\mathcal{E}) \\
&\quad + \frac{(Q_{\text{PUT}_1} + Q_{\text{ACPT}_1})^2 + (Q_{\text{UPD}_1} + 1)^2}{2^{nl+1}}
\end{aligned}$$

Adversary $\mathcal{B}^{\text{REG,INIT,RESP,FIN,COMP}_\mathcal{B},\text{CHECK}}$ 00 $k_1, \dots, k_{Q_{\text{REG}_1}} \leftarrow \perp$ $\parallel$ not known 01 $n_c, n_p, n_r \leftarrow 0$ 02 $\mathcal{S}_{\text{cmp}}, \mathcal{S}_{\text{ch}}, \mathcal{S}_{\text{chOwn}} \leftarrow \emptyset$ 03 $b \xleftarrow{\$} \{0, 1\}$ 04 $b' \leftarrow \mathcal{A}^0$ 05 if $\exists \text{aid} \in \mathcal{S}_{\text{chOwn}} : \text{aid} \in \mathcal{S}_{\text{cmp}} \vee T_{\text{uaid}}[\text{aid}] = \perp$ 06 return 0 07 return $\llbracket b' = b \rrbracket$	Oracle $\text{AUTH}_1(\text{aid})$ 45 require $T_{\text{uaid}}[\text{aid}] = \text{true}$ 46 $\text{inc.aid} \leftarrow \text{aid}$ 47 $\text{inc.pw} \leftarrow T_{\text{pw}}[\text{aid}]$ 48 n_c++ 49 $i_c \leftarrow n_c$ 50 n_p++ 51 $i_p \leftarrow n_p$ 52 $T_u[i_c] \leftarrow \text{aid}$ 53 $T_{\text{stc}}[i_c] \leftarrow \varepsilon$ 54 $\text{auth}_1 \xleftarrow{\$} \text{INIT}(\text{aid}, i_p)$ $\parallel$ init oracle 55 $m_c \leftarrow (\text{aid}, \text{auth}_1)$ 56 $\text{st}_c^{\text{tmp}} \leftarrow (\text{aid}, \text{inc.pw}, \text{st}, m_c)$ 57 $T_p[(\text{auth}, C, i_p)] \leftarrow (i_c, \text{st}_c^{\text{tmp}}, \text{inc}, 2)$ 58 return $\text{Chk}(\text{prot}, i_p, \varepsilon, m_c)$
Oracle $\text{REG}_1(\text{aid})$ 08 require $T_{\text{uaid}}[\text{aid}] = \perp$ 09 n_r++ 10 $T_{\text{pw}}[\text{aid}] \leftarrow (\perp, n_r)$ 11 $\text{inc.aid} \leftarrow \text{aid}$ 12 $T_{\text{uaid}}[\text{aid}] \leftarrow \text{false}$ 13 REG (aid) $\parallel$ register client 14 if $\exists rw : (rw, \text{aid}, \cdot) \in T_H \wedge \text{CHECK}(rw, \text{aid})$ 15 output (rw, aid) $\parallel$ win 16 $rw \leftarrow \text{"chl"}$ $\parallel$ mark internal RO query 17 $k_{\text{ek}} \leftarrow H(rw, \text{aid}, \text{"ek"})$ 18 $k_{\text{mac}} \leftarrow H(rw, \text{aid}, \text{"mac"})$ 19 $\text{seed}_{\text{mk}} \xleftarrow{\$} \{0, 1\}^{kl}$ 20 $k_{\text{mk}} \leftarrow H(\text{seed}_{\text{mk}}, \text{aid}, \text{"mk"})$ 21 $n_{\text{mk}} \xleftarrow{\$} \{0, 1\}^{nl}$ 22 $\text{ct}_{\text{mk}} \xleftarrow{\$} \text{AEAD.Enc}(k_{\text{ek}}, n_{\text{mk}}, \text{seed}_{\text{mk}}, (\text{aid}, \text{"mk"}))$ 23 $m_c \leftarrow (\text{aid}, k, n_{\text{mk}}, \text{ct}_{\text{mk}}, k_{\text{mac}})$ 24 n_p++ 25 $i_p \leftarrow n_p$ 26 $T_p[(\Pi, C, i_p)] \leftarrow (i, \varepsilon, \text{inc}, 2)$ 27 return $\text{Chk}(\Pi, i_p, \text{out}_c, m_c)$	Oracle $\text{STEP}(\text{auth}, C, i_p, m)$ $\parallel$ only for client round $r = 2$ 59 $(i_c, \text{st}_c^{\text{tmp}}, \text{inc}, 2) \leftarrow T_p[(\text{auth}, C, i_p)]$ 60 $(\text{aid}, \text{pw}, \text{st}, m'_c) \leftarrow \text{st}_c^{\text{tmp}}$ 61 $(\cdot, \text{auth}_1) \leftarrow m'_c$ 62 $(\text{sid}, \text{auth}_2) \leftarrow m$ 63 $(\cdot, n'_r) \leftarrow T_{\text{pw}}[\text{aid}]$ $\parallel$ recover instance 64 $rw \xleftarrow{\$} \text{FIN}(\text{auth}_2, n'_r, i_p)$ $\parallel$ fin oracle 65 if $rw = \top$ $\parallel$ correct execution detected 66 $rw \leftarrow \text{"chl"}$ $\parallel$ use consistent RO output 67 $\text{st}_c^{\text{tmp}}.k_{\text{ek}} \leftarrow H(rw, \text{aid}, \text{"ek"})$ 68 $k_{\text{mac}} \leftarrow H(rw, \text{aid}, \text{"mac"})$ 69 $m_c \leftarrow \text{MAC.Tag}(k_{\text{mac}}, (m'_c, \text{ms}))$ 70 $T_p[(\text{auth}, C, i_p)] \leftarrow (i_c, \text{st}_c^{\text{tmp}}, \text{inc}, 3)$ 71 return $\text{Chk}(\text{auth}, i_p, \varepsilon, m_c)$
Oracle $\text{COMP}(\text{aid})$ 28 require $T_{\text{uaid}}[\text{aid}] \neq \perp$ 29 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{\text{aid}\}$ 30 $(\cdot, n'_r) \leftarrow T_{\text{pw}}[\text{aid}]$ 31 $(\text{pw}, k_{n'_r}) \xleftarrow{\$} \text{COMP}_\mathcal{B}(n'_r)$ $\parallel$ comp oracle 32 $T_{\text{pw}}[\text{aid}] \leftarrow (\text{pw}, n'_r)$ $\parallel$ store pw to patch 33 return $(\text{pw}, \{T_{\text{stc}}[i_c] \mid T_u[i_c] = \text{aid}\}, \{T_{\text{oob}}[(\text{aid}, \cdot)]\})$	Oracle $H(\text{seed}, \text{aid}, \text{label})$ 72 if $(\text{seed}, \text{aid}, \text{label}) \notin T_H$ 73 $T_H[\text{seed}, \text{aid}, \text{label}] \xleftarrow{\$} \{0, 1\}^{kl}$ 74 $T_{H\text{-int}}[\text{seed}, \text{aid}, \text{label}] \leftarrow 1$ 75 return $T_H[\text{seed}, \text{aid}, \text{label}]$
Oracle $\text{STEP}(\text{auth}, S, i_p, m)$ $\parallel$ only for server round $r = 1$ 34 $(i_c, \text{st}^{\text{tmp}}, \text{in}, 1) \leftarrow T_p[(\text{prot}, S, i_p)]$ 35 $(\text{aid}, \text{auth}_1) \leftarrow m$ 36 if $\text{aid} \notin \text{st}_S.\text{acc}$: fails 37 $\text{sid} \xleftarrow{\$} \text{NewID}$ 38 $(k, \dots, k_{\text{mac}}) \leftarrow \text{st}_S.\text{acc}[\text{aid}]$ 39 $(\cdot, n'_r) \leftarrow T_{\text{pw}}[\text{aid}]$ $\parallel$ recover instance 40 $\text{auth}_2 \xleftarrow{\$} \text{RESP}(\text{auth}_1, n'_r)$ $\parallel$ resp oracle 41 $m_S \leftarrow (\text{sid}, \text{auth}_2)$ 42 $\text{st}_S^{\text{tmp}} \leftarrow (\text{aid}, k_{\text{mac}}, m, m_S)$ 43 $T_p[(\text{auth}, S, i_p)] \leftarrow (i_c, \text{st}_S^{\text{tmp}}, \text{in}, 2)$ 44 return m_S	Oracle $\text{RO}(\text{seed}, \text{id}, \text{label})$ 76 if $T_{\text{uaid}}[\text{id}] \neq \perp \wedge (\text{"chl"}, \text{id}, \text{label}) \in T_{H\text{-int}}$ 77 if $\text{id} \notin \mathcal{S}_{\text{cmp}} \wedge \text{label} \in \{\text{"ek"}, \text{"mac"}\}$ 78 output (seed, id) $\parallel$ win 79 else if $\text{id} \in \mathcal{S}_{\text{cmp}}$ 80 $(\text{pw}, n'_r) \leftarrow T_{\text{pw}}[\text{id}]$ 81 if $\text{seed} = \text{Auth.Full}(\text{pw}, \text{id}, k_{n'_r})$ 82 $T_H[\text{seed}, \text{id}, \text{label}] \leftarrow T_{H\text{-int}}[\text{"chl"}, \text{id}, \text{label}]$ $\parallel$ patch RO 83 if $T_H[\text{seed}, \text{id}, \text{label}] = \perp$ 84 $T_H[\text{seed}, \text{id}, \text{label}] \xleftarrow{\$} \{0, 1\}^{kl}$ 85 return $T_H[\text{seed}, \text{id}, \text{label}]$

Fig. 43. Adversary $\mathcal{B}$ against OUT-OW security of Auth having access to oracles REG, INIT, RESP, FIN, and CHECK simulating $\mathcal{G}_1/\mathcal{G}_2$ for adversary $\mathcal{A}$.

with

$$\begin{aligned}
Q_{C_1} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1}), \\
Q_{D_1} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, 0, Q_{\text{REG}_1}, Q_{\text{REG}_1} + Q_{\text{RO}}), \\
Q_{C_2} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{PUT}_1} + Q_{\text{ACPT}_1} + Q_{\text{CHL}}, Q_{\text{UPD}_1} + Q_{\text{GET}_1} + Q_{\text{SHARE}_1} + Q_{\text{CHL}}), \\
Q_{D_2} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{ACPT}_1}, Q_{\text{PUT}_1} + Q_{\text{ACPT}_1} + Q_{\text{CHL}}, Q_{\text{PUT}_1} + Q_{\text{UPD}_1} + Q_{\text{GET}_1} + Q_{\text{SHARE}_1} + Q_{\text{CHL}} + Q_{\text{RO}}), \\
Q_{\mathcal{E}} &= (Q_{\text{PUT}_1} + Q_{\text{CHL}}, Q_{\text{PUT}_1} + Q_{\text{CHL}}, Q_{\text{PUT}_1} + Q_{\text{UPD}_1}, Q_{\text{GET}_1}, Q_{\text{CHL}}).
\end{aligned}$$

Collecting the probabilities completes the proof of Theorem 12. $\square$

E.2 Integrity

We restate Theorem 8 using concrete bounds. The proof follows from the combination of other theorems, as shown in Section 6.2.

Theorem 13 (Net-Int). *For any password distribution $\mathcal{PW}$ and adversary $\mathcal{A}$ against the Net-Int security of $\text{CS} := \text{CS}[\text{Auth}, \text{AEAD}, \text{MAC}]$ there exist an OUT-OW adversary $\mathcal{B}$ against Auth, INT-CTXT adversaries $\mathcal{C}_1, \mathcal{C}_2$ against AEAD, OW-CPA adversaries $\mathcal{D}_1, \mathcal{D}_2$ against AEAD, and an INT-CTXT adversary $\mathcal{E}$ against AEAD with $t_{\mathcal{A}} \approx t_{\mathcal{B}} \approx t_{\mathcal{C}_1} \approx t_{\mathcal{D}_1} \approx t_{\mathcal{C}_2} - Q_{\text{COMP}} \cdot Q_{\text{AUTH}_1} \approx t_{\mathcal{D}_2} - Q_{\text{COMP}} \cdot Q_{\text{AUTH}_1} \approx t_{\mathcal{E}}$ such that*

$$\begin{aligned}
\text{Adv}_{\text{CS}, \mathcal{PW}}^{Q_{\mathcal{A}}\text{-Net-Int}}(\mathcal{A}) &\leq \text{Adv}_{\text{Auth}, \mathcal{PW}}^{Q_{\mathcal{B}}\text{-OUT-OW}}(\mathcal{B}) + \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{C}_1}\text{-INT-CTXT}}(\mathcal{C}_1) + \text{Adv}_{\text{AEAD}, \text{kl}}^{Q_{\mathcal{D}_1}\text{-OW-CPA}}(\mathcal{D}_1) \\
&\quad + \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{C}_2}\text{-INT-CTXT}}(\mathcal{C}_2) + \text{Adv}_{\text{AEAD}, \text{kl}}^{Q_{\mathcal{D}_2}\text{-OW-CPA}}(\mathcal{D}_2) + \text{Adv}_{\text{AEAD}}^{Q_{\mathcal{E}}\text{-INT-CTXT}}(\mathcal{E}) \\
&\quad + Q_{\text{AUTH}_1} \cdot \delta_{\text{Auth}} + \frac{(Q_{\text{PUT}_1} + Q_{\text{ACPT}_1})^2 + (Q_{\text{UPD}_1} + 1)^2}{2^{nl+1}}
\end{aligned}$$

with

$$\begin{aligned}
Q_{\mathcal{A}} &= (Q_{\text{STEP}}, Q_{\text{STEPS}_1}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1}, Q_{\text{PUT}_1}, Q_{\text{UPD}_1}, Q_{\text{GET}_1}, Q_{\text{SHARE}_1}, Q_{\text{ACPT}_1}, Q_{\text{COMP}}, Q_{\text{RO}}), \\
Q_{\mathcal{B}} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{AUTH}_1}, Q_{\text{AUTH}_1}, Q_{\text{AUTH}_1}, Q_{\text{REG}_1} + Q_{\text{RO}}), \\
Q_{C_1} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1}), \\
Q_{D_1} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, 0, Q_{\text{REG}_1}, Q_{\text{REG}_1} + Q_{\text{RO}}), \\
Q_{C_2} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{PUT}_1} + Q_{\text{ACPT}_1}, Q_{\text{UPD}_1} + Q_{\text{GET}_1} + Q_{\text{SHARE}_1}), \\
Q_{D_2} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{ACPT}_1}, Q_{\text{PUT}_1} + Q_{\text{ACPT}_1}, Q_{\text{PUT}_1} + Q_{\text{UPD}_1} + Q_{\text{GET}_1} + Q_{\text{SHARE}_1} + Q_{\text{RO}}), \\
Q_{\mathcal{E}} &= (Q_{\text{PUT}_1}, Q_{\text{COMP}}, Q_{\text{PUT}_1} + Q_{\text{UPD}_1}, Q_{\text{GET}_1}).
\end{aligned}$$

E.3 Explicit Authentication

We restate Theorem 9 using concrete bounds and provide the full proof.

Theorem 14 (Explicit Authentication). *For any password distribution $\mathcal{PW}$ and adversary $\mathcal{A}$ against the Exp-Auth security of $\text{CS} := \text{CS}[\text{Auth}, \text{AEAD}, \text{MAC}]$ there exist an OUT-OW adversary $\mathcal{B}$ against Auth and an SUF adversary $\mathcal{C}$ against MAC with $t_{\mathcal{A}} \approx t_{\mathcal{B}} \approx t_{\mathcal{C}}$ such that*

$$\text{Adv}_{\text{CS}, \mathcal{PW}}^{Q_{\mathcal{A}}\text{-Exp-Auth}}(\mathcal{A}) \leq \text{Adv}_{\text{Auth}, \mathcal{PW}}^{Q_{\mathcal{B}}\text{-OUT-OW}}(\mathcal{B}) + \text{Adv}_{\text{MAC}}^{Q_{\mathcal{C}}\text{-SUF}}(\mathcal{C}) + Q_{\text{AUTH}_1} \cdot \delta_{\text{Auth}}$$

with

$$\begin{aligned}
Q_{\mathcal{A}} &= (Q_{\text{STEP}}, Q_{\text{STEPS}_1}, Q_{\text{REG}_1}, Q_{\text{AUTH}_1}, Q_{\text{PUT}_1}, Q_{\text{UPD}_1}, Q_{\text{GET}_1}, Q_{\text{SHARE}_1}, Q_{\text{ACPT}_1}, Q_{\text{COMP}}, Q_{\text{RO}}), \\
Q_{\mathcal{B}} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{AUTH}_1}, Q_{\text{AUTH}_1}, Q_{\text{AUTH}_1}, Q_{\text{REG}_1} + Q_{\text{RO}}), \\
Q_{\mathcal{C}} &= (Q_{\text{REG}_1}, Q_{\text{COMP}}, Q_{\text{AUTH}_1}, Q_{\text{AUTH}_1}).
\end{aligned}$$

Proof. We prove the theorem by a sequence of games.

Game G_0 . This is the explicit authentication game in which we plug in construction CS. As in the proof of Theorem 10, we distinguish two random oracles. One can be called by the adversary, which is denoted by RO as usual, and one is an internal random oracle, denoted by H, which is only called from other oracles. The change is conceptual since both oracles represent the same function since they are maintaining the same table for lazy sampling. Additionally, we introduce a table T_{H-int} which indicates if a value was queried to the internal RO; this will be used in later changes. It holds that

$$\Pr[G_0(\mathcal{A}) \Rightarrow 1] = \text{Adv}_{CS, \mathcal{PW}}^{Q_{\mathcal{A}}\text{-Exp-Auth}}(\mathcal{A}),$$

where $Q_{\mathcal{A}}$ is the same as in the theorem statement.

Game G_1 . This is the same as game G_2 from the proof of Theorem 12 as depicted in Figure 42. In particular, the game aborts if there are any direct random oracle queries corresponding to master key or mac key of uncompromised users and include the correct raw secret rw . This means that at this point all encryption keys and mac keys of uncompromised users are uniformly random. Since the changes are exactly the same as in the proof of Theorem 12, we obtain the same bound:

$$\Pr[G_0(\mathcal{A}) \Rightarrow 1] - \Pr[G_1(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{Auth, \mathcal{PW}}^{Q_{\mathcal{B}}\text{-OUT-OW}}(\mathcal{B}) + Q_{Auth_1} \cdot \delta_{Auth}$$

with number of queries denoted in the theorem bound.

Final reduction. We can reduce G_1 to the SUF of MAC.

Claim. There exists an adversary $\mathcal{C}$ against SUF such that

$$\Pr[G_1(\mathcal{A}) \Rightarrow 1] \leq \text{Adv}_{MAC}^{Q\text{-SUF}}(\mathcal{C}),$$

with $Q = (Q_{REG_1}, Q_{COMP}, Q_{AUTH_1}, Q_{AUTH_1})$.

Proof (Claim). The formal reduction can be found in Figure 44 showing only the oracles that cannot trivially (by just executing as they are) be simulated. In the registration oracle, $\mathcal{C}$ starts a new instance for their SUF game and stores them for the queried aid. The output of the oracle is the length of the message which can be simulated since all values of the message are known to the reduction except for the MAC key k_{mac} which has a fixed length of kl (Line 32). The server's side of the registration protocol is simple to simulate since the server does not output any value. The reduction can therefore store the relevant instance Q' in the server state representing the MAC key.

If the authentication protocol is queried for the second step of the client, the reduction can look up the MAC instance and compute the correct tag using their tag oracle (Line 58). This is only done for the correct raw secret which can be checked via table T_{rw} and since the Auth computation is perfectly correct due to the previous changes. Otherwise $\mathcal{C}$ can just compute the tag themselves because the MAC key does not correspond to one of their challenge instances. The second server step can be simulated by using the reduction's verification oracle (Line 69).

It remains to show that the simulation of $\mathcal{C}$ is sound with respect to consistent random oracle outputs. The keys for the MAC are not known to $\mathcal{C}$ until a party is compromised. This also means that the output of the random oracle with input $(rw, aid, "mac")$, where rw is the correct Auth output computed during registration, is not known to the reduction. However, due to the abort in the RO introduced in the previous games, G_1 aborts whenever the adversary makes such a query on an uncompromised user. That allows $\mathcal{C}$ to patch the random oracle upon a corruption query because they learn the secret $k_{Q'}$. An additional challenge could be that the reduction does not only have to make the random oracle consistent but also potential states or temporary states. However, the persistent or temporary client states never contain MAC keys and the server states are not revealed upon compromise.

Eventually, we argue why $\mathcal{C}$ wins their SUF game if $\mathcal{A}$ wins. Note that the Chk subroutine is only accessed if the verification oracle in Line 69 returns 1 implying the first winning condition of $\mathcal{C}$. Further, the flag win

Adversary $\mathcal{C}^{\text{TAG,VER,COMP}_C}$ <pre> 00 $H \xleftarrow{\\$} \mathcal{OS}$ 01 $(n_c, n_p, n_r) \leftarrow (0, 0, 0)$ 02 $(\text{pw}_1, \dots, \text{pw}_{Q_{\text{REG}_1}}) \xleftarrow{\\$} \mathcal{PW}$ 03 $\mathcal{S}_{\text{cmp}} \leftarrow \emptyset$ 04 $\text{win} \leftarrow \text{false}$ 05 $\mathcal{A}^\mathcal{O}$ 06 return win Procedure Step₁(prot, ic, ir, inc) 07 $(T_{\text{stc}}[ic], \text{st}_c^{\text{tmp}}, \text{out}_c, m^*) \leftarrow \Pi_{\text{prot}}^{C:1}[H](T_{\text{stc}}[ic], \varepsilon, \text{inc}, \varepsilon)$ 08 n_p++ 09 $i_p \leftarrow n_p$ 10 $T_p[(\text{prot}, C, i_p)] \leftarrow (i_c, \text{st}_c^{\text{tmp}}, \text{inc}, 2)$ 11 if prot = auth: $T_{\text{Client}}[i_p] \leftarrow (m^*)$ 12 return m^* Oracle REG₁(aid) 13 require $T_{\text{uaid}}[\text{aid}] = \perp$ 14 n_r++ 15 $\text{inc.pw} \leftarrow \text{pw}_{n_r}$ 16 $T_{\text{pw}}[\text{aid}] \leftarrow \text{inc.pw}$ 17 $\text{inc.aid} \leftarrow \text{aid}$ 18 $k \xleftarrow{\\$} \text{Auth.D}_S$ 19 $\text{rw} \leftarrow \text{Auth.Full}[H_{\text{Auth}}](\text{inc.pw}, \text{aid}, k)$ 20 $T_{\text{rw}}[\text{aid}] \leftarrow (\text{rw}, k)$ 21 if $(\text{rw}, \text{aid}, \cdot) \in T_H$: abort 22 $Q++$ // new instance 23 $T_{\text{key}}[\text{aid}] \leftarrow Q$ // store instance 24 $(n_{\text{mk}}, \text{seed}_{\text{mk}}) \xleftarrow{\\$} \{0, 1\}^{nl} \times \{0, 1\}^{kl}$ 25 $k_{\text{mk}} \leftarrow H(\text{seed}_{\text{mk}}, \text{aid}, \text{"mk"})$ 26 $\text{ct}_{\text{mk}} \xleftarrow{\\$} \text{AEAD.Enc}(k_{\text{mk}}, n_{\text{mk}}, \text{seed}_{\text{mk}}, (\text{aid}, \text{"mk"}))$ 27 n_p++ 28 $i_p \leftarrow n_p$ 29 $T_p[(\text{areg}, i_p)] \leftarrow (\varepsilon, \text{st}_c^{\text{tmp}}, \text{inc}, 2)$ 30 $T_{\text{uaid}}[\text{inc.aid}] \leftarrow \text{true}$ // reg completed 31 $T_{\text{areg}}[i_p] \leftarrow (\text{aid}, k, n_{\text{mk}}, \text{ct}_{\text{mk}}, Q')$ 32 $m_c \leftarrow \text{aid} + k + nl + \text{ct}_{\text{mk}} + kl$ // simulate message length 33 return m_c Oracle STEPS₁(areg, $i_p \in [n_p]$, m_c, ins) // only areg 34 $(\text{aid}, k, n_{\text{mk}}, \text{ct}_{\text{mk}}, Q') \leftarrow T_{\text{areg}}[i_p]$ 35 $\text{sts.acc}[\text{aid}] \leftarrow (k, n_{\text{mk}}, \text{ct}_{\text{mk}}, Q')$ 36 $T_p[(\text{areg}, S, i_p)] \leftarrow (\varepsilon, \varepsilon, \text{ins}, 2)$ 37 success_S Oracle COMP(aid) 38 require $T_{\text{uaid}}[\text{aid}] \neq \perp$ 39 $\mathcal{S}_{\text{cmp}} \leftarrow \mathcal{S}_{\text{cmp}} \cup \{\text{aid}\}$ 40 $Q' \leftarrow T_{\text{key}}[\text{aid}]$ 41 $k_{Q'} \leftarrow \text{COMP}_C(Q')$ // Comp oracle 42 $(\text{rw}, \cdot) \leftarrow T_{\text{rw}}[\text{aid}]$ 43 $T_H[\text{rw}, \text{aid}, \text{"mac"}] \leftarrow k_{Q'}$ // patch RO 44 return $(T_{\text{pw}}[\text{aid}], \{T_{\text{stc}}[ic] \mid T_u[ic] = \text{aid}\}, \{T_{\text{oob}}[\text{aid}, \cdot]\})$ </pre>	Oracle STEP_{S1}(auth, $i_p \in [n_p]$, m, ins) // only auth <pre> 45 $(\text{sts}, \text{st}_c^{\text{tmp}}, \text{out}, m^*) \leftarrow \Pi_{\text{auth}}^{S:1}[H](\text{sts}, \varepsilon, \text{ins}, m)$ 46 $T_p[(\text{auth}, S, i_p)] \leftarrow (\varepsilon, \text{st}_c^{\text{tmp}}, \text{ins}, 2)$ 47 $T_{\text{Server}}[i_p] \leftarrow (m, m^*)$ 48 return m^* Oracle STEP(auth, $C, i_p \in [n_p]$, m_S) // only for $C, r = 2$ 49 require $T_p[(\text{auth}, C, i_p)] \neq \perp$ 50 $(i_c, \text{st}_c^{\text{tmp}}, \text{in}, 2) \leftarrow T_p[(\text{auth}, C, i_p)]$ 51 $(\text{aid}, \text{pw}, \text{st}, m'_c) \leftarrow \text{st}_c^{\text{tmp}}$ 52 $(\text{sid}, \text{auth}_2) \leftarrow m_S$ 53 $\text{rw} \leftarrow \text{Auth.Fin}[H_{\text{Auth}}](\text{auth}_2, \text{pw}, \text{aid}, \text{st})$ 54 $\text{st}_c^{\text{tmp}}.k_{\text{ek}} \leftarrow H(\text{rw}, \text{aid}, \text{"ek"})$ 55 $(\text{rw}', \cdot) \leftarrow T_{\text{rw}}[\text{aid}]$ 56 if $\text{rw} = \text{rw}'$ // correct raw secret 57 $Q' \leftarrow T_{\text{key}}[\text{aid}]$ // recover instance 58 $m^* \xleftarrow{\\$} \text{TAG}(Q', (m'_c, m_S))$ // tag oracle 59 else 60 $k_{\text{mac}} \leftarrow H(\text{rw}, \text{aid}, \text{"mac"})$ 61 $m^* \leftarrow \text{MAC.Tag}(k_{\text{mac}}, (m'_c, m_S))$ 62 $T_p[(\text{auth}, C, i_p)] \leftarrow (i_c, \text{st}_c^{\text{tmp}}, \text{in}, 3)$ 63 $T_{\text{Client}}[i_p].\text{append}(m_S, m^*)$ 64 return m^* Oracle STEP(auth, $S, i_p \in [n_p]$, m_C) // only for $S, r = 2$ 65 require $T_p[(\text{auth}, S, i_p)] \neq \perp$ 66 $(i_c, \text{st}_c^{\text{tmp}}, \text{in}, 2) \leftarrow T_p[(\text{auth}, S, i_p)]$ 67 $(\text{sts}, \text{st}_c^{\text{tmp}}, \text{outs}, m^*) \leftarrow \Pi_{\text{auth}}^{S:2}[H](\text{sts}, \text{st}_c^{\text{tmp}}, m)$ 68 $(\text{aid}, Q', n_{\text{mk}}, \text{ct}_{\text{mk}}, m'_c, m_S) \leftarrow \text{st}_c^{\text{tmp}}$ 69 $b \leftarrow \text{VER}(Q', (m'_c, m_S), m_C)$ // ver oracle 70 if $b = 0$ 71 fail_S 72 $T_p[(\text{auth}, S, i_p)] \leftarrow (i_c, \text{st}_c^{\text{tmp}}, \text{in}, 3)$ 73 $T_{\text{Server}}[i_p].\text{append}(m_C)$ 74 $m^* \leftarrow (n_{\text{mk}}, \text{ct}_{\text{mk}})$ 75 return $\text{Chk}(i_p, \text{outs}, m^*)$ Procedure Chk(i_p, outs, m^*) 76 if $\text{outs.decision} \wedge m^* \neq \perp$ // execution succeeded 77 if $T_{\text{Client}}[i_p] \neq T_{\text{Server}}[i_p]$ // messages fresh 78 $(i_c, \cdot, \cdot, \cdot) \leftarrow T_p[(\text{auth}, C, i_p)]$ 79 if $T_u[i_c] \notin \mathcal{S}_{\text{cmp}}$ // client uncompromised 80 win $\leftarrow \text{true}$ 81 $T_{\text{Server}}[i_p].\text{append}(m^*)$ 82 return m^* Oracle STEP(auth, $C, i_p \in [n_p]$, m_S) // only $C, r = 3$ 83 require $T_p[(\text{auth}, C, i_p)] \neq \perp$ 84 $(i_c, \text{st}_c^{\text{tmp}}, \text{in}, 3) \leftarrow T_p[(\text{auth}, C, i_p)]$ 85 $(n_{\text{mk}}, \text{ct}_{\text{mk}}) \leftarrow m_S$ 86 $k_{\text{mk}} \leftarrow \text{AEAD.Dec}(\text{st}_c^{\text{tmp}}.k_{\text{ek}}, n_{\text{mk}}, \text{ct}_{\text{mk}}, (\text{st}_c^{\text{tmp}}.\text{aid}, \text{"mk"}))$ 87 $T_{\text{stc}}[ic] \leftarrow (\text{st}_c^{\text{tmp}}.\text{aid}, \text{st}_c^{\text{tmp}}.\text{sid}, k_{\text{mk}})$ 88 $T_p[(\text{auth}, C, i_p)] \leftarrow (i_c, \text{st}_c^{\text{tmp}}, \text{in}, 4)$ 89 success_C </pre>
--	--

Fig. 44. Adversary $\mathcal{C}$ against SUF security of MAC having access to oracles TAG, VER, COMP_C simulating $\mathbf{G}_1$ for adversary $\mathcal{A}$.

of $\mathcal{A}$ is only set if the corresponding user was not compromised implying the second winning condition of $\mathcal{C}$. For the third condition, we need to show that the message/tag pair was never queried for that key to the tag oracle which output the same tag.

For a proof by contradiction, assume that $\mathcal{C}$ queried the message, i.e. (m'_C, m_S) , to their oracle TAG and obtained output m_C . Note that these messages must belong to the same protocol session i_p because they are taken from the temporary client's and server's state which the adversary cannot modify for uncompromised parties. Hence, the client's message m'_C was added to T_{Client} in AUTH_1 (more precisely in STEP_1 , Line 11). Since we assumed that m'_C also occurs as the first part of the message to VER, it originates from the server's temporary state and was therefore added in STEPS_1 which implies that it was also added to T_{Server} (Line 47). The same holds for m_S which was added in the same query (Line 47). By assumption, this is again the same value as the second part of the message queried to TAG and was also added to T_{Client} (Line 63) together with the output tag m_C . Lastly, the tag m_C is also added to T_{Server} (Line 73). This means it holds $T_{\text{Client}} = T_{\text{Server}}$ and $\mathcal{A}$ cannot win their Exp-Auth game concluding the contradiction argument. $\square$

Collecting the probabilities completes the proof of Theorem 14. $\square$